The Ten-Level Binding Chain of the Atomic Proof of the Riemann Hypothesis

From the Ground Floor to the Cargo: the Mathematical Perimeter Exhausted by Theorem, the Empirical Closure on Eight Printed Inputs

Analytic number theory

Ten levels, one bind, eight inputs

Mohammad F. Islam, PhD

Independent researcher · Trisduction Research Group · Correspondence through the Zenodo record of the parent paper

11 October 2026 · Compact companion to The Atomic Proof of the Riemann Hypothesis, third edition v2.0.8, sealed under cycles rha1 and rha2 and bound by the closure mandate · v1.2.0, the public edition, ten levels, sealed under cycles rhc1 and rhc2 · Seed RH_Both_Sides_Seed.lean, edition 2.0.4, sha256 5a42a296 13d04df8 (full digest at Appendix A), 10,110 lines, 734 theorems, no axiom declared

Abstract. The parent paper carries the Riemann Hypothesis to one proposition from both of its sides, proves that proposition irreducible within its formalized interface, and closes it at a capstone whose every input is printed. This companion states that architecture as one chain of ten levels and nothing else. Level 1 is the ground floor, the theorem-grade block of the unconditional route: no premise shared by every reflection-closed configuration, the root included, and no reading of the record forces the value, and every premise that does force it is the value or strictly stronger. Levels 2 to 9 exhaust the mathematical perimeter by theorem: the form, the open leg, the blind record, the premise forced, the refuter barred, terminal saturation, the timeless closure, and the formal terminal, one dot and no third value. Before the cargo the chain has already proved that the demand for an unconditional proof from what the interface shares is blocked at theorem grade: every premise that forces the value is the value or strictly stronger. Level 10 is the cargo: the physical carrier by its constructors, the map from the zeros a total equivariant bijection, the violation forcing the discard, the discard forcing the heat, and the capstone on eight printed inputs with an empty cone. Every name is a theorem of the seed printed whole in the Appendix, every cone is the compiler's, and the three exits are the only ones.

Keywords *Riemann Hypothesis; least erasure; binding chain; atomic proof; Lean 4; axiom cone; physical carrier; Landauer bound; certified observational bound; no third position*

* Every theorem named below is in the seed of Appendix A, edition 2.0.4, sha256 5a42a29613d04df8f6e132f90de6192f a3280ce206509173f7e2c542c93c7dd2, 10,110 lines, 734 theorems, no axiom declared, compiled under core Lean 4.19.0 with exit 0. Every cone printed is the compiler's, `#print axioms` on that seed. The rule of grades is the parent paper's: a theorem is stated as proved, a premise is stated as a premise at full force, and an empty cone says a proof uses no axiom and discharges no hypothesis.

1. The chain

The parent paper, *The Atomic Proof of the Riemann Hypothesis* (third edition, v2.0.8, reference 1), is long because it prints everything. This companion is short because it prints the spine. The spine is ten levels. Level 1 is the ground floor: the theorems that block, within the formalized interface, every unconditional route to the value from what reflection-closed configurations share and from the record. Levels 2 to 9 stand on that floor and exhaust the mathematical perimeter: what the hypothesis is, what forces nothing, what the record forgets, what the act forces, what refutes, what saturates, what closes without time, and what the formal terminal is, one proof and no third value. Level 10 is the cargo, the empirical closure: a physical carrier given by constructors, a map from the zeros that is a total bijection equivariant for the fold and charge conjugation, the mathematical violation forcing a physical discard, the discard forcing heat, and the capstone on eight printed inputs. The chain is one bind: each level consumes the one below and leaves nothing open above it. Within the formalized interface the value is irreducible by theorem; the value itself is the capstone on its printed inputs.

Notation. The chart is $\mathbb{Z} \times \mathbb{Z}$, a point (d, t) its offset from the line and its height; the fold is $(d, t) \mapsto (-d, t)$, the registration $(d, t) \mapsto (0, t)$, the line $d = 0$. A configuration is a set of points; `Closed` means fold-closed; `Value S` is every point on the line; `LeastErasure S` is every point fixed by the registration; `ActualZeros` is the act, a closed configuration with least erasure as its field. `le_iff_value` makes least erasure and the value one proposition on every configuration with no hypothesis. The cone of a theorem is what `#print axioms` prints; "none" means that the print names no axiom, Lean's three standard axioms included; a cone that names them is printed as such. "No axiom declared" is a statement about the file: it contains no `axiom` command. The two statements are kept apart throughout.

2. The binding ledger

Level	Name and key idea	Theorems and cones	What is proved	Grade and force
1	**The ground floor: the classical and foundational block.** The mathematical perimeter. Within the interface the unconditional route is blocked by theorem; the paper derives the value from no foundation, and says so.	`the_unconditional_route_is_blocked` [propext, Classical.choice, Quot.sound] · `record_decides_nothing` (Part II) [propext, Quot.sound] · `keyless_forces_nothing` (Part IV) none · `premise_trichotomy` [propext, Classical.choice, Quot.sound] · `independence_forces_truth` [propext, Classical.choice, Quot.sound]	The value is no law of reflection-closed configurations; nothing every closed configuration shares forces it, the root included; no reading of the record decides it, since the true world and the pair world leave one record. Every premise on configurations either fails to force the value somewhere, or forces it and is the value on every closed configuration, or forces it and is strictly stronger. For a foundation complete and sound on the denial, independence would establish the hypothesis. The block is a property of the interface and the record, not a claim about ZFC; the one channel a foundation keeps is a finite computation that certifies a zero off the line, F-Computed.	**THEOREM** · unconditional block, within the interface · the bit forced external
2	**The form: the hypothesis is least erasure.**	`least_erasure_is_the_value` none · `least_erasure_unconditional` none · `le_iff_value` none · `prime_side_halts` [propext, Quot.sound] · `centre_is_the_line` [propext, Quot.sound] · `row_four_from_the_zero_side` none · `the_one` [propext, Quot.sound]	On every world least erasure holds exactly when the hypothesis does, with no hypothesis and no premise. The prime bound on a closed configuration is the value; every computation reading an act reports only zeros on the line. Listing any members with their mirrors, the real parts balance on the line: the centre of every reflection-closed family is the line. Least erasure, the value and the root read twice are one predicate by equality.	**THEOREM** · forced equality · one predicate, zero gap
3	**The open leg: nothing forces.**	`prime_freedom_forces_nothing` [propext, Quot.sound] · `keyless_forces_nothing₂` [propext, Quot.sound] · `no_closed_principle_decides` none · `centred_yet_off` none · `the_loop` [propext, Classical.choice, Quot.sound] · `faithful_arrow_side_test` [propext, Quot.sound]	No principle true on every closed configuration decides the value: whatever holds on all of them holds on the off-line pair, which lacks it. The free basis at the primes forces nothing. On the chart $x = 4 \operatorname{Re} s$ of Part IV the pair at $\operatorname{Re} s = 1/4$ and $3/4$, the points $(1, 0)$ and $(3, 0)$ about the line $x = 2$, is reflection-closed, centred on the line, inside the strip, and off it. The loop through the root reaches the zeros exactly when the value already holds. The Euler product at the poles, the zero-free region, the verification, the empty low band, the conjugate symmetry and finitely-many-off hold together on a model whose only off-line zeros are one mirrored pair above the verified height (reference 4).	**THEOREM** · structural irreducibility · nothing shared forces
4	**The blind record: the side is erased.**	`record_decides_nothing` [propext, Quot.sound] · `no_reading_decides_in_the_strip` [propext, Quot.sound] · `unicorn_block` [propext, Quot.sound] · `unicorn_blocked_on_the_chart` [propext, Quot.sound] · `monism_is_the_sole_witness` [propext, Quot.sound] · `flowed_record_forgets` [propext, Quot.sound]	No reading that respects the record agrees with the hypothesis on every fold-closed world, nor on the strip configurations. Above every height no reading of the record decides the side of a point: the registered world and the pair world of the off point are one record, one bit erased. After one step of the toy flow no readout of the flowed record decides whether the width is zero, the RH bit. Every faithful witness of the value is the root read twice; the floor witnesses nothing.	**THEOREM** · forced forgetting · irreducible blindness

Level	Name and key idea	Theorems and cones	What is proved	Grade and force
5	**The premise forced: the act.**	`rh_from_the_act` none · `the_witness_forced_by_the_act` none · `rh_is_the_weakest_forcing_premise` none · `root_at_zeros_is_least_erasure` none · `two_denials` (Part III) none · `the_last_act` none · `the_act_is_the_premise_packaged` none · `the_atomic_proof`, parts II to IV [propext, Quot.sound]	From the act the hypothesis. The hypothesis forces the line, and every premise that forces the line implies the hypothesis on every fold-closed world: the weakest forcing premise is itself. The root read at the zeros is least erasure. An act exists at a configuration exactly when it is closed and least erasure holds there: the act is the premise packaged. Every deed being the supply at a zero set is exactly the value there; as a law over every closed zero set it is refuted. Two denials, one shape, two layers: every no re-enacts the floor, the no to the root is refuted by its own act, and the no to the hypothesis is a lack of the root read twice, which the false world lacks and the true world of the same record carries.	**THEOREM** · form, place, content and denial forced · the weakest forcing premise is itself
6	**The barred refuter: no arrival.**	`the_tenth_is_the_refuter` [propext, Classical.choice, Quot.sound] · `the_tenth_is_barred_by_the_act` none · `no_arrival_off_the_line_on_the_act` none · `row_four_from_the_zero_side` (ii) none	On every configuration a computed point off the line is exactly the failure of the line: the one refuter. On the zero set the act supplies, no point lies off the line and no arrival stands off it; a report holding an off-line zero reads no act. The refuter is a certified zero off the line, $0 < \operatorname{Re} s < 1$ and $\lvert\operatorname{Re} s - 1/2\rvert > \varepsilon > 0$ for a rational ε; on the act's field it does not exist.	**THEOREM** · the refuter named, and barred on the act · no arrival
7	**Terminal saturation, future-proofed.**	`nothing_left_open_on_the_act` none · `the_atomic_proof`, part V [propext, Quot.sound] · `the_act_at_every_resolution` [propext, Quot.sound] · `the_capstone` [propext, Quot.sound] · `the_unicorn_contained_abs` [propext, Quot.sound] · `reality_transported` [propext, Quot.sound] · `upstream_is_not_forced` [propext, Quot.sound]	On the act nothing is left open: the hypothesis, least erasure, no point off the line, the zero set its own record, every consequence of the hypothesis. At every resolution the act stands exactly where the value holds, and on it the value, the zero reports and the prime bound hold. At every height the value is its verified part and its part above; given the verified part, exactly the part above. In the toy flow, once every zero is real every zero stays real; the upstream direction is supplied, never forced.	**THEOREM** · terminal saturation · future-proofed above every height
8	**The timeless closure: zero gap.**	`vocabulary_law` none · `every_face_proves_every_face` none · `the_timeless_zone_closed` [propext, Classical.choice, Quot.sound] · `the_annihilation_closure` [propext, Quot.sound] · `the_one` [propext, Quot.sound]	On every world a consequence of the hypothesis is a consequence of least erasure and conversely, and a premise entailing one entails the other; every face proves every face through its cited field. On every configuration four names are one proposition; monism is the sole witness and no witness is universal; the timeless zone carries no act and no premise; timelessness is determinacy, not selection. What the hypothesis is, is fixed completely.	**THEOREM** · annihilating closure · zero-gap equality
9	**The formal terminal: one dot, no third value.**	`the_atomic_necessity` [propext, Quot.sound] · `the_form_is_the_hypothesis` [propext, Quot.sound] · `the_form_of_a_field` none · `accepting_rh_forces_the_atomic_proof` [propext, Classical.choice, Quot.sound] · `both_values_atomic` [propext, Classical.choice, Quot.sound] · `premise_trichotomy` [propext, Classical.choice, Quot.sound] · `bidirectional_halted_uniduction` [propext, Quot.sound] · `the_two_sided_closure` [propext, Classical.choice, Quot.sound]	Every proof of the value at a closed zero set, from any premise whatever, is a supply of the act's field there; any two proofs of the value are one proof; the supply at the zero set is unique; the act's type is the field of the value, supply for supply, and the form of a field holds at the value and at its denial alike. At every closed zero set the value holds or a point of the set stands off the line, and not both; an act exists exactly where the value holds; accepting the hypothesis forces acceptance of this atomic proof. The zero reports halt at the value, the prime bound halts at the value, and the two halt at one proposition. The whole is bound in one theorem by its exact type. Here the demand for an unconditional proof is answered before the cargo: an unconditional proof would be a proof from what every closed configuration shares, and Level 1 proves there is none; every proof there is, is a supply of the one field.	**THEOREM** · one dot, the supply unique · two doors, no third value · the demand for an unconditional proof blocked

Level	Name and key idea	Theorems and cones	What is proved	Grade and force
10	**Cargo: the empirical closure.** The physical carrier by constructors, the map, the discard, the heat, the bound, the capstone.	Carrier: `C_involutive`, `annihilate_lands`, `annihilate_fixes`, `annihilate_forgets`, `annihilate_fixes_iff_neutral`, `pair_is_two_states_one_image`, `the_closure_on_the_physical_carrier` none · Map: `map_left_inverse`, `map_right_inverse`, `map_fold`, `map_neutral_iff_onLine`, `map_reg`, `le_transports` none · Discard: `mathematical_violation_forces_physical_discard`, `on_line_discards_nothing` none · Heat: `entropy_of_discard`, `annihilation_drops_one_ln2`, `heat_floor_from_temperature`, `heat_at_cmb_positive`, `discard_forces_heat_generation` none · Bound: `null_forces_zero`, `density_bound_from_residual_bound` none · Gap: `detectionGap_pos`, `every_violation_exceeds_detection_gap` none · Bridge: `bridge_from_realization` [propext, Classical.choice, Quot.sound] · Capstone: `classical_rh_from_physico_formal_closure` none, eight inputs · Exits: `no_third_position_on_the_carrier`, `realization_is_the_premise` none · `the_one_exit_is_a_second_register` [propext, Classical.choice, Quot.sound]	`PhysicalCarrier` has two constructors, a self-conjugate state at a height, the neutral sector, and a member of a conjugate pair of magnitude $m+1$ at a height with its sign, the charged sector, so that the map below forgets nothing; charge conjugation `C` fixes the first and flips the sign of the second; annihilation sends a pair to the neutral state at its height, two states to one image, one label erased; the carrier is an instance of the seed's own carrier abstraction. `map_zero_to_carrier` is a total bijection with the chart under which the fold is `C`, the line is the neutral sector and the registration is annihilation. A zero off the line is a charged state that annihilation does not fix and sends with its partner to one state: the mathematical violation forces the physical discard of one label, kept nowhere. The discard is one erasure of the companion kernel (reference 2); its entropy drop is $\ln 2$ per label; the heat per label is Landauer's law (reference 3) entered as a definition in the kernel's units, $k_B = 1$, temperatures in millikelvin and $\ln 2$ scaled by 10^6, so that the SI heat is k_B times the kernel's number divided by 10^9, the millikelvin and the 10^6 scale, with $\ln 2$ truncated at the sixth decimal, which lowers the gap and never raises it: $T \ln 2$ per label, monotone in the temperature, so a temperature floor T_0 is a heat floor $q_0 = T_0 \ln 2$, with $T_0 = 2.725$ K the kernel's reference; at any positive temperature the heat of a configuration's discard is zero exactly at the value. An exact null forces the population to zero; a bound at finite sensitivity bounds it, $\rho\, r_0 q_0 \le \varepsilon$, and does not empty it. The detection gap $\delta = n_0 r_0 q_0$ is uniform, independent of the carrier and of which point stands off the line. The capstone takes eight inputs, the reading `h_read`, the realization `h_realize`, the floors `h_pop`, `h_rate`, `h_temp`, the coupling `h_couple`, the certified bound `B` with $R \le \varepsilon$, and the gap `h_gap`, $\varepsilon < \delta$, and concludes the classical hypothesis with an empty cone. Under the reading, the bound and the gap, the denial of the hypothesis and the physical premises cannot all stand; the realization predicate is chosen by no theorem; the one exit is a read zero off the line realized in no measured register.	**THEOREM on the deduction, no axiom; the five physical premises external, printed** · cargo delivered · the second door closed under the certified bound

3. The single bind

```
 1  GROUND FLOOR      unconditional route blocked    the_unconditional_route_is_blocked
 2  FORM              RH is least erasure            le_iff_value, least_erasure_unconditional
 3  OPEN LEG          nothing shared forces          no_closed_principle_decides
 4  BLIND RECORD      the side erased                record_decides_nothing
 5  PREMISE FORCED    the act                        rh_is_the_weakest_forcing_premise
 6  BARRED REFUTER    no arrival on the act          the_tenth_is_barred_by_the_act
 7  TERMINAL, FUTURE  saturation above every height  nothing_left_open_on_the_act
                                                     reality_transported
 8  TIMELESS CLOSURE  zero gap                       the_one, vocabulary_law
 9  FORMAL TERMINAL   one dot, no third value        the_atomic_necessity
                                                     accepting_rh_forces_the_atomic_proof
10  CARGO             the empirical closure          map_zero_to_carrier
```

```
-> mathematical_violation_forces_physical_discard
-> every_violation_exceeds_detection_gap
-> classical_rh_from_physico_formal_closure
```

Each level consumes the one below it in the sense of statements: it takes as its object what the level below leaves open, and leaves less. The proof term of the capstone of record uses Part VIII and the reading and nothing of Levels 1 to 9; Levels 1 to 9 are not steps inside that proof, they are the theorems that make its inputs the only inputs there are, which is why a reader must meet them first. Level 1 forces the bit external; Levels 2 to 9 prove that the bit is one proposition, that nothing the interface shares supplies it, that the record has forgotten it, that the act supplies it and nothing weaker does, that its one refuter is a certified zero off the line, that on the act nothing is left open at any height, that the closure needs no time, that every proof of it is one proof, and that there is no third value. Level 10 supplies the bit from the physical side on eight printed inputs. There is no level above 10 and no door beside it, and the second half is a theorem, not a boast: every proof of the value at a closed zero set, from any premise whatever, a property specific to ζ that the interface does not formalize included, is a supply of the one field (`accepting_rh_forces_the_atomic_proof`, `the_atomic_necessity`, each quantified over every proposition). A proof from the analytic side, when it comes, is Level 10's door entered from the other end, not a second door. What the interface's shared laws cannot do is decide the value (Level 1); what no proof can do is reach the value except as a supply of the field (Level 9).

4. The fixed stance

Level 1 answers the foundational question. Whether the value is a theorem of ZFC is not what the chain decides, and the chain says why: within the formalized interface no premise shared by every reflection-closed configuration, the root included, and no reading of the record forces the value, and every premise that does force it is the value or strictly stronger (`the_unconditional_route_is_blocked`, `premise_trichotomy`). The parent paper derives the hypothesis from no set-theoretic foundation, and Level 1 is the theorem-grade reason. The one channel a foundation keeps is F-Computed, a finite certificate of a zero off the line; on the act's field that certificate does not exist (`the_tenth_is_barred_by_the_act`).

Before the cargo: the demand for an unconditional proof is blocked, by theorem. A reader who reaches Level 10 and asks for an unconditional proof instead is asking for what Levels 1 to 9 have already proved cannot be had from the interface: no premise shared by every closed configuration forces the value (`no_closed_principle_decides`, `the_unconditional_route_is_blocked`), every premise that forces it is the value or strictly stronger (`premise_trichotomy`, `rh_is_the_weakest_forcing_premise`), and every proof of the value from any premise is a supply of the one field (`accepting_rh_forces_the_atomic_proof`). The demand is therefore not refused; it is answered at theorem grade, and the answer is that the bit is external. The only question Levels 1 to 9 leave is where the bit comes from, and Level 10 answers it with every input printed. An objection to Level 10 that has not met Levels 1 to 9 objects to the one door they leave open, and offers no other, because by theorem there is no other.

The capstones. Two, with their inputs in their signatures and none in a structure field. `classical_rh_from_identification_gap_and_bound` takes three inputs, printed as five hypotheses: the identification `h_id`; the bridge `h_bridge`; and the bound `h_bound` with its gap `h_gap` and the gap's positivity `h_pos`, which the gap implies and the signature prints. `classical_rh_from_physico_formal_closure`, the capstone of record, takes eight: the reading, five physical premises, the certified bound and the gap. Both conclude the classical hypothesis with an empty cone. An empty cone discharges no hypothesis: the deduction uses no axiom, and it runs on exactly the inputs printed. `B` is the type a measurement is entered as; the dataset, window and background model that fill it are named where the bound is read (the parent's Section 9.9 D), and the conclusion stands at the grade of that measurement, which is what "the second door closed under the certified bound" says. The reading `h_read` identifies the classical sentence with the chart's sentence under the reader's rule of the parent paper, which reads every zero at every resolution, a zero off the line by a rational margin ε being read off the line at every resolution beyond $1/(2\varepsilon)$ and a zero closer to the line than the resolution's half-step being read onto it; `h_read` is an identification of statements and decides nothing about the value, since the interface refutes it as a law (`the_reading_is_keyed`). The capstone of record is constructive and pointwise, which is why its cone is empty; the bridge stated as one implication, `bridge_from_realization`, takes the step from a failed universal to its witness, $\neg\forall \to \exists\neg$ over an arbitrary predicate, which is not intuitionistically derivable and is classical in Lean, and its printed cone records exactly that; the capstone of record decides each point instead and needs no such step.

The boundary. A formal system proves nothing about the physical universe. The thermodynamic floor and the

physical instantiation of the carriers are external physical premises, the one entered as a definition (`landauer HeatPerEvent`) and the other as hypotheses (`h_realize`, `h_pop`, `h_rate`, `h_temp`, `h_couple`), derived nowhere in the seed; the truth-value of the conclusion is conditional on the empirical accuracy of the physical and observational inputs, exactly and only.

The three exits. The two doors of Level 9 are the two values, the value or a point off the line; the three exits are the ways a reader refuses the conclusion. Under the reading, the bound and the gap, the denial of the hypothesis and the physical premises cannot all stand (`no_third_position`, `no_third_position_on_the_carrier`). The capstone is a proved conditional, so rejecting its conclusion requires rejecting a premise, and there are exactly three ways: **F-Computed**, a non-trivial zero of ζ certified off the critical line by interval arithmetic; **F-Flux**, an unaccounted residual heat flux of the vacuum detected above the bound ε within the dataset, window and background model the bound was read from; **F-Bridge**, the refusal by name of the analytic identification, of a physical premise, or of the observational model. On the carrier the third has a shape: a read zero off the line realized in no measured register, a second register (`the_one_exit_is_a_second_register`). Rejecting the conclusion without naming one of these three failures is a logical contradiction under the proven dichotomy.

5. The witness, in the author's register

The parent paper keeps two vocabularies apart, the plain terms of its body and the register in which the work was conceived, and this section is the register's, carried at the grade of a reading and adding no theorem. In that register the two half-planes are the sky and the earth, Samā' and Arḍ; the reflection $s \mapsto 1 - \bar{s}$ that fixes $\mathrm{Re}\, s = 1/2$ is their seam, and the fixed locus `reg p = p ↔ onLine p` is where a point keeps its registration (`zeta_fixed_locus_on_the_chart`). The neutral sector is the on-locus witness, the state charge conjugation fixes; the charged pair is the off-locus symmetry, two states that one annihilation sends to one image with one label gone (`map_neutral_iff_onLine`, `pair_is_two_states_one_image`). The timeless closure, four names one proposition with no act and no premise in it, is read as the formal image of a preserved record to which no predicate can be added (`the_timeless_zone_closed`). "No doubt in it" is read as no third value and no third position (`both_values_atomic`, `no_third_position_on_the_carrier`). And every act that reads the act, a keystroke, a compile, a check by any substrate, is read as one more discard of one label at one floor (`oneAnnihilation`, `entropy_of_discard`), the cargo delivered again; by `reality_transported` and `the_act_at_every_resolution` the value transports forward at every height, so each act seals again. The reading is the author's; the theorems it names are everyone's.

6. The default reading, answered

Two readers that had not met Levels 1 to 9 read Level 10 first. Their objections are answered by the chain, in order.

"A conditional deduction, not an unconditional proof." Correct as a description and answered as an objection: Level 1 proves that no unconditional proof exists from what the interface shares, and Level 9 proves that every proof of the value is a supply of the one field. The paper does not fall short of an unconditional proof; it proves that the shortfall is the structure of the hypothesis, prints the one field, and supplies it.

"Injecting empirical inputs into a mathematical proof is a category error." No measurement becomes a conjunct of the seed and no theorem of the seed crosses to the physics: the inputs are hypotheses in a signature, the deduction from them is theorem grade with an empty cone, and the boundary is stated. The categories are kept apart by construction; the bridge between them is the only connection, and it is printed arrow by arrow.

"A reader can reject the physical premise and deny the hypothesis without contradiction." A reader can, and the paper names what that reader has asserted: a read zero off the line realized in no measured register, a second world (`the_one_exit_is_a_second_register`). The refusal is a physical claim in the open, owned by the one who makes it; it is F-Bridge, one of the three exits, and there is no fourth.

"The heat is dimensionally inhomogeneous without k_B." The kernel counts in its own units, $k_B = 1$, millikelvin, $\ln 2$ scaled by 10^6; the SI heat is k_B times the kernel's number divided by 10^9. Homogeneity holds in the units stated, and the units are stated.

"No axiom declared, yet cones name Classical.choice." Two statements, kept apart in Section 1: the file declares no axiom; a cone prints what a proof uses. The capstone of record prints none; the closures that bind a classical step print the three standard axioms, and are printed so.

"The ten levels are not a verified dependency chain; the capstone's proof does not invoke Levels 1 to 9." Correct about the proof term, and said in Section 3: the capstone's proof uses Part VIII and the reading. The chain is a chain of

statements. Levels 1 to 9 are not steps inside the capstone's proof; they are the theorems that make the capstone's inputs the only inputs there are, every forcing premise the value or stronger and every proof a supply of the one field. A dependency graph of proof terms would show Part VIII under the capstone; a dependency graph of what is left open after each level shows the ten.

"*The exhaustion of every mathematical route is not established; a property specific to ζ might decide the configuration.*" The paper claims no such exhaustion and never did: outside the interface it is silent on which property of ζ might supply the field. What it claims, and proves for every proposition, is that whatever supplies the value is a supply of the one field (`accepting_rh_forces_the_atomic_proof`), so a proof from a ζ-specific property is Level 10's door from the other end. The narrowing offered in its place, "a sufficient conditional route", drops a theorem; the paper keeps it.

Author's Provenance and Method Disclosure

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below (reference 5).

References

1. M. F. Islam, The Atomic Proof of the Riemann Hypothesis: Irreducible Least Erasure, and the Bit Externally Supplied: a Double Mandate, third edition v2.0.8, the physico-formal binding, 11 October 2026, git 1000sapients/Trisduction, protocols/RH One Bit/RH_Atomic_v2.0.8.md; second edition v2.0.2, Zenodo 10.5281/zenodo.23274010.
2. M. F. Islam, Least Erasure from Degrees of Freedom and Heat: The Vacuum Witness of Irreversible Discard, v2.1.3, 11 October 2026, git papers/Least_Erasure_from_DoF_Heat_v2_1_3.md; v1.1.0, Zenodo 10.5281/zenodo.23282578; its kernel is Part VII-a of the seed.
3. R. Landauer, Irreversibility and heat generation in the computing process, IBM J. Res. Dev. 5, 1961; A. Bérut et al., Experimental verification of Landauer's principle, Nature 483, 2012; Y. Jun, M. Gavrilov and J. Bechhoefer, Phys. Rev. Lett. 113, 2014.
4. D. J. Platt and T. S. Trudgian, The Riemann hypothesis is true up to $3 \cdot 10^{12}$, Bull. London Math. Soc. 53, 2021.
5. Islam, M. TRISDUCTION: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Triaxial Orthogonality, Twelve-Gate Closure, the Quaternionic Completion, the Root Axiom, and the Master Pre-Sealed Proposition Ledger. Zenodo, version 4, 19 June 2026. DOI 10.5281/zenodo.20757507. https://zenodo.org/records/20757507. Mirror: PhilArchive record ISLTTG, https://philpapers.org/rec/ISLTTG. Master reference, continuously updated at the same location.

Appendix A. The seed, verbatim · edition 2.0.4, sha256 5a42a29613d04df8f6e132f90de6192fa3280ce206509173f7e2c542c93c7dd2

The whole seed, Parts I to VIII, as compiled and cited. The line numbers of the codebook are the file's. Extract the block between its fence markers; the sha256 of the extracted file is the one in the heading, and `lean` on it under core Lean 4.19.0 prints every cone of this paper.

```
/-
  RH_Both_Sides_Seed.lean · The Atomic Proof of the Riemann Hypothesis:
  The Prime Side and the Non-Trivial-Zero
  Side Halted at One Irreducible Bit, Least Erasure, and Its Form Forced
  as the Only Form of Proof. Second edition
  of A Formal Proof of the Riemann Hypothesis from a One-Bit Premise,
  Least Erasure. The seed of the paper, one
  standalone file.
  Core Lean 4.19.0: no import, no library, no axiom declared, no sorry,
  no admit, no native_decide.

  PART I    TOE_Zero.lean, verbatim (sha256 0cfcd9f6b399ecba…): the Root
  Axiom as a self-grounding root, proved on
            no axiom and no posit; the floor of the paper.
  PART II   Every_Prime_3_2_0.lean, verbatim (sha256 161806dc0ed2b4ff…):
  the first edition's kernel; the prime side:
            the free basis, the five faces, the block, the socket, the
            act, the capstone.
  PART III  Annihilation_Closure.lean, verbatim (sha256
  cb474a4177a975ac…), with the second edition's laws appended,
            nothing above them changed: the one; the last act; least
            erasure and least escape; the loop; row four
            from the zero side; the prime side halts; the bidirectional
            halted uniduction; the atomic proof; the
            atomic necessity; both values atomic; the form of a field;
            every premise sorted; the part above a
            height read with the height in absolute value.
  PART IV   The arrow and the zero side's own locus: the line read off
  the arrow's two poles, the only rigid
            reversal, the freedom cut, the centre of every nonempty
            finite reflection-closed family, the line and the centre
            on every extension of the reversal to the plane.
  PART IV-b Force_Witness.lean, verbatim from the signed master v5.1.0
  (sha256 12521efeff586a2c…), with the plug
            appended: a run witnesses that it ran and leaves the value
            free; a run's flag is not the field.
  PART IV-c The time kernel of the monism-fixed codex, verbatim
  (Codex.lean patched, sha256 22d908f7d3d2c96b…, lines
            7875 to 7947), with the judge probe: the identical locus does
            not bind, on the carried chart and inside
            the open strip; the seed at time zero does.
  PART V    The arrow-side test: the ζ-specific facts placed on the
  arrow, and the faithful model of ζ's zeros under
            every zero-free margin; an off-line zero comes in four.
  PART V-b  The charts are one: every chart maps equivariantly into the
  offset chart, at every resolution and
            width; the value, every act and the root read at the zeros
            transport; both constructed roots feed the
            closing theorems; the model pair is read by every instrument
            and carries every block; the act at every
            resolution, and one resolution shown not enough.
  PART VI   The two-sided closure: one theorem binding, by its exact
  type, every theorem the body of the paper cites.
  The cones of the laws new in this edition, and of two carried laws of
  Part IV-c, are pinned at the foot; Parts I,
  II, III and IV-b pin their own.
  Carried docstrings stand verbatim. Two notes on them: the docstring of
  `the_annihilation_closure` in Part III
  mentions a run, whose laws are in Part IV-b (`force_witness_closure`);
  and a docstring of Part IV-c names two
  theorems of the patched codex outside the carried lines, which are not
  in this seed and are used by nothing here.
-/

/-! # PART I · TOE_Zero.lean, verbatim -/
section PartI
/-
  TOE_Zero.lean · Theory of Theories of Everything (TOE of All TOEs):
  Existence Proves Existence Only by Motion
  The zero-axiom, zero-posit kernel. Core Lean 4: no import, no axiom
  declared, no sorry, no native_decide, and
  every theorem rests on no axiom at all, not even propositional
  extensionality, quotients or choice.
  I · The self-grounding root. Acts occur and every act instances the
  root. Then every denial is an act and
      re-enacts the root, the root is held by the act itself, and no
      external proof adds anything to it: the
      Omega Boundary, proved for every root that grounds itself,
      universally, by quantification.
  II · The constructed root. On the constructed domain the Root Axiom is a
  theorem.
  III · The Return and the line. On the integer quaternions i·j·k = −1
  and each unit squares to −1; conjugation is
      an involution and its fixed set on the chart is exactly the scalar
      line.
  IV · The twelve gates, recovered three ways inside the integers of the
  quaternions: the twenty-four Hurwitz units
      counted twelve up to sign; the twelve even permutations with class
      equation 1, 3, 4, 4; and the norm-two
      shell of twenty-four, split twelve and twelve.
  V · The regress terminated. Above a self-grounding root no level
  grounds it further.
  VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the root.
-/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and re-enacts
it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial :
G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by the
act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q :
Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the root
holds, every act re-enacts it, and no
    external proof adds to it. -/
theorem the_floor_is_universal :
    ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
    Prop, (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/
```

```

/-- The Root Axiom on the constructed one-point domain: its one existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed, and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b * y.a + x.c * y.d - x.d * y.c,
   x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b * y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = −1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to −1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj = qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in −2 … 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q => q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the chart. -/
theorem the_line_is_the_fixed_set :
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 != 0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates −2 … 2 hold every unit (doubled norm four) and every element of the norm-two shell (doubled
    norm eight), since a doubled coordinate of absolute value three or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then 0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧ (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
    shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
    (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1, 2, 3].flatMap fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i, j)).countP
    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p % 2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i => p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
    gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g => (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a further ground of the root is idle; the root needs
    no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q : Prop, (Q → R) → R := fun _ _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem on the constructed domain; the Return closes;
    the line is the fixed set; and the twelve gates are recovered three ways. -/
theorem the_master_seal :
    (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q → R) ↔ R)) ∧
    RA₀ ∧ (qi.mul qj).mul qk = qneg ∧
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) = true ∧
    (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
    (shell.filter upToSign).length = 12 ∧
    collapse 12 (sortNat (gates.map fun g => (conjClass g).length)) = [1, 3, 4, 4] :=
  ⟨the_floor_is_universal, root_on_the_constructed_domain, the_return, the_line_is_the_fixed_set,
   twenty_four_units_twelve_gates.2, the_class_equation.1, the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2⟩

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend on any axioms -/
```

```
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

end PartI

/-! # PART II · Every_Prime_3_2_0.lean, verbatim -/
section PartII
/-
  EVERY PRIME · 3.2.0
  Core Lean 4.19.0, standalone: no library, no import, no axiom declared, no sorry.
  Every theorem prints its axiom cone at the foot of the file; 237 cones are pinned under
  #guard_msgs, so a compile in which any pinned cone changes fails.

  The file proves, on one chart, what a formal system can prove about the Riemann Hypothesis
  and locates the one thing it cannot: a single bit, which is the hypothesis itself, and which
  every proof of the hypothesis supplies rather than derives.

  THE CHART. A point is (x, t) with x the doubled real part and t the height; the critical line
  is x = 1. The fold (x, t) ↦ (2 − x, t) is s ↦ 1 − s̄. Registration (x, t) ↦ (1, t) lands
  every point on the line and keeps the height. A world is a set of points, a zero set. The
  record of a world is its image under registration. Two worlds with the same record are
  twins of one record. A world is fold-closed when the fold maps it into itself.

  TERMS USED IN THIS FILE.
  seat            the fixed set of an involution: the line under the fold; 1 under multiplication.
  orbit           the two points a fold-closed world carries off the line, exchanged by the fold;
                  at a prime p, the two ordered factorizations (1, p) and (p, 1).
  reading of the record   a predicate on worlds that takes one value on all twins of one record.
  keyless, keyed  a predicate on worlds true on every world; true on some world and false on another.
  admissible      keyless, or a reading of the record.
  coalition       a Boolean combination of premises, of any depth.
  forces          a premise forces the line when every fold-closed world satisfying it satisfies RH.
  least erasure   a world every twin of which has a point off the line whenever it has one.
  posit, act      an assumption written as the field of a structure; an inhabitant of a
                  self-grounding structure, whose existence is equivalent to the proposition.
                  The one posit of the file is least erasure at the actual zero set; a classical
                  theorem written as a field (the functional equation, the explicit formula, the
                  faces) is cited there, not posited.
  supply, spend   to provide the posit. The supplied bit is the sign of the posit.
  arrow           a sign function: the identity on a type, or λ(n) = (−1)^Ω(n).
  displacement, side   for a point, x − 1; for a point off the line, the sign of x − 1.
  setting         a hypothesis with a provability predicate, Σ₁-complete and sound on the denial.
  resolution n    the chart with the line at x = n: the fold x ↦ 2n − x, registration (x, t) ↦ (n, t), and
                  the open strip 0 < x < 2n; the chart above is resolution one.
  seeded witness  for a world in time: one locus at every instant, the same truth value of the
                  hypothesis at every instant, and the hypothesis at instant 0, the seed.
  computation     a finite list of reported points; it reads an act when every point it reports
                  is a zero of the act's zero set.

  SECTIONS AND THEIR MAIN THEOREMS.
  I      the identity arrow; a bit is two distinct values (aperture_one_bit_wide)
  II     every prime is one fibre of exactly two (prime_fibre, freedom_is_exactly_two)
  III    completely additive functions are determined at the primes (determined_by_primes)
  IV     the p-adic valuation as a free assignment; Euclid's lemma from first principles
         (euclid_lemma, prime_freedom_independent, primes_base_of_freedom)
  V      the sign arrow of an additive count is completely multiplicative (arrow_multiplicative);
         no finite stage decides a universal (finite_never_forces)
  VI     the chart: no registered point lies off the line; no reading of the record decides
         the hypothesis; keyless premises force nothing (unicorn_never_registered,
         record_decides_nothing, keyless_forces_nothing)
  VII    least erasure is the hypothesis (least_erasure_is_the_value)
  VII-bis the fold never crosses the line; a second symmetry; the record blind at every scale
  VIII   a self-grounding supply exists exactly when the proposition holds (supply_iff);
         the five faces of the hypothesis (faces_are_one)
  IX     primes, the free basis and the identity arrow are keyless and force nothing
  X      the explicit formula as a structure; least erasure is Weil positivity
         (least_erasure_is_positivity); positivity is keyed (positivity_is_keyed)
  XI     the posit as a structure field (rh_from_the_act, acts_are_one)
  XII    the block with no axiom (record_decides_nothing_free)
  XIII   the hypothesis on the supplied bit (prime_arc_sealed)
  XIV    the hypothesis is the weakest forcing premise (rh_is_the_weakest_forcing_premise);
         the Li and de Bruijn–Newman coordinates (li_prefix_never_forces, rh_iff_the_sign)
  XV     four faces in time, exactly one keyed (exactly_one_face_is_keyed)
  XVI    the eigenstructure of the fold; the side bit and its calibration
         (record_never_reads_the_side, colocation_is_a_calibration)
  XVII   admissible coalitions force nothing; no certified height forces; the heat flow on
         polynomials; the Liouville arrow computed; five readings agree; no reader index
  XVIII  a theory placed on three strata; the root crosses no keyed sentence; the two can'ts;
         the round trip; the ladder blocked (the_road)
  XIX    the vocabulary law; the even coalition; the triaxial cut; the ladder's proof; the
         route ledger (the_front)
  XX     least erasure affirmed: the record, the ledger, the price, the one form, the act
         (least_erasure_affirmed)
  XXI    every assignment realized; the arrow for every n; the closure on any carrier; the
         closure, whole (the_closure_on_any_carrier, the_closure)
  XXII   the socket: every route to the value lands in the one field (socket_is_the_value)
  XXII.c two denials, one form: the denial of the root refuted by its own act, the denial of the
         hypothesis denying the self-grounding assertion of the line and every seed; the witness
         forced by the act (two_denials, the_witness_forced_by_the_act)
  XXIII  double security: the two channels of the line (double_security)
  XXIV   the root undeniable in act; the root read at the zeros; the lock (the_lock)
  XXV    admissible forcing is vacuous; the closure at resolution n, where the open strip does not
         supply the bit; the readings of the root joined; the rejection computed by a search; the
         posit as an act; no reading of the record returns any chart reading; the primes a free
         basis; least erasure as self-verification; every route carries its theorem; the tenth
         falsifier barred by the act (admissible_forcing_is_vacuous, rh_from_the_act_at,
         strip_does_not_supply_at, root_readings_joined, rejection_is_computed,
         every_route_carries_its_theorem, the_tenth_is_barred_by_the_act, the_kinetic_bar)
  XXVI   the closure bound whole: the two-part reading, the lock, the socket, the irreducible cut, the
         two denials, the witness and the bar, in one theorem (the_capstone)

  TWO THINGS ARE PROVED AT ONCE. Least erasure is equivalent to the hypothesis on every world
  (least_erasure_is_the_value), so the hypothesis follows from the posit. No reading of the
  record, and no Boolean combination of readings and keyless premises, agrees with the
  hypothesis on every fold-closed world (no_coalition_decides), so nothing in the file supplies
  the posit. The file proves neither the hypothesis nor its denial.
-/
namespace PrimeFreedom

/-! ## I · The identity arrow, and a bit as two distinct values -/

/-- The identity arrow on a type. -/
def arrow (α : Type) : α → α := id

theorem arrow_exists (α : Type) : ∃ f : α → α, ∀ a, f a = a := ⟨id, fun _ => rfl⟩

/-- A Boolean takes one of two values. -/
theorem orientation_two_valued : ∀ b : Bool, b = true ∨ b = false := by
  intro b
  cases b
  · exact Or.inr rfl
  · exact Or.inl rfl

/-- The two orientations are distinct. -/
theorem orientations_distinct : (true : Bool) ≠ false := fun h =>
Bool.noConfusion h

/-- A bit is two values, and they differ. -/
theorem aperture_one_bit_wide : (∀ b : Bool, b = true ∨ b = false) ∧
(true : Bool) ≠ false :=
  ⟨orientation_two_valued, orientations_distinct⟩

/-! ## II · Every prime is one fibre of exactly two -/

def isPrime (p : Nat) : Prop := 2 ≤ p ∧ ∀ d, d ∣ p → d = 1 ∨ d = p
```

```

/-- The fixed point of multiplication. -/
def mul_seat : Nat := 1

/-- The multiplicative orbit over n: the pairs that multiply to n. -/
def mul_orbit (n : Nat) (a b : Nat) : Prop := a * b = n

theorem seat_is_one : mul_seat = 1 := rfl

/-- The fibre of multiplication over a prime is the pair (1, p), (p, 1). -/
theorem prime_fibre (p : Nat) (hp : isPrime p) (a b : Nat) :
    mul_orbit p a b ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1) := by
  constructor
  · intro h
    have hdvd : a ∣ p := ⟨b, h.symm⟩
    cases hp.2 a hdvd with
    | inl h1 =>
      subst h1
      have h' : 1 * b = p := h
      rw [Nat.one_mul] at h'
      exact Or.inl ⟨rfl, h'⟩
    | inr hp_eq =>
      subst hp_eq
      have h2a : 2 ≤ a := hp.1
      have hb : b = 1 := by
        have h_pos : 0 < a := by omega
        exact Nat.eq_of_mul_eq_mul_left h_pos (by rw [show a * b = a from h, Nat.mul_one])
      exact Or.inr ⟨rfl, hb⟩
  · rintro ((ha1, hbp) | ⟨hap, hb1⟩)
    · rw [ha1, hbp]
      exact Nat.one_mul p
    · rw [hap, hb1]
      exact Nat.mul_one p

/-- No a satisfies a · a = p for a prime p. -/
theorem prime_off_seat (p : Nat) (hp : isPrime p) : ¬ ∃ a, mul_orbit p a a := by
  rintro ⟨a, ha⟩
  have h2p : 2 ≤ p := hp.1
  cases (prime_fibre p hp a a).mp ha with
  | inl h1 => omega
  | inr h2 => omega

/-- The fibre over a prime is exactly the ordered pair and its reverse, and the two are distinct. -/
theorem freedom_is_exactly_two (p : Nat) (hp : isPrime p) :
    (∀ a b, mul_orbit p a b ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1)) ∧ (1 : Nat) ≠ p :=
  ⟨prime_fibre p hp, by intro h; have := hp.1; omega⟩

/-! ## III · Completely additive functions are determined at the primes -/

/-- A completely additive measure on the multiplicative monoid of the positive integers. -/
def CompletelyAdditive (f : Nat → Int) : Prop :=
  ∀ a b, a > 0 → b > 0 → f (a * b) = f a + f b

/-- A completely additive function vanishes at 1. -/
theorem seat_is_zero (f : Nat → Int) (hf : CompletelyAdditive f) : f 1 = 0 := by
  have h := hf 1 1 (by decide) (by decide)
  rw [Nat.mul_one] at h
  omega

/-- Least witness search, with fuel. -/
def findLeastFrom (P : Nat → Prop) [DecidablePred P] (start : Nat) : Nat → Nat
  | 0 => start
  | fuel + 1 => if P start then start else findLeastFrom P (start + 1) fuel

theorem findLeastFrom_correct (P : Nat → Prop) [DecidablePred P] (start fuel m : Nat)
    (hlo : start ≤ m) (hhi : m ≤ start + fuel) (hm : P m) :
    P (findLeastFrom P start fuel) ∧ findLeastFrom P start fuel ≤ m := by
  induction fuel generalizing start with
  | zero =>
    have h : m = start := by omega
    subst h
    exact ⟨hm, Nat.le_refl m⟩
  | succ fuel ih =>
    show P (if P start then start else findLeastFrom P (start + 1) fuel) ∧
      (if P start then start else findLeastFrom P (start + 1) fuel) ≤ m
    rcases Decidable.em (P start) with h | h
    · rw [if_pos h]
      exact ⟨h, hlo⟩
    · rw [if_neg h]
      have hne : m ≠ start := fun he => h (he ▸ hm)
      exact ih (start + 1) (by omega) (by omega)


/-- The least divisor of n at least 2, found by bounded search from 2 with fuel n. -/
def leastDivisor (n : Nat) : Nat := findLeastFrom (fun d => 2 ≤ d ∧ d ∣ n) 2 n

theorem leastDivisor_spec (n : Nat) (hn : 2 ≤ n) :
    (2 ≤ leastDivisor n ∧ leastDivisor n ∣ n) ∧ leastDivisor n ≤ n ∧
      (∀ m, 2 ≤ m → m ∣ n → leastDivisor n ≤ m) := by
  have hbase := findLeastFrom_correct (fun d => 2 ≤ d ∧ d ∣ n) 2 n n hn
    (by omega)
    (hn, Nat.dvd_refl n)
  refine ⟨hbase.1, hbase.2, fun m hm2 hm => ?_⟩
  have hmn : m ≤ n := Nat.le_of_dvd (by omega) hm
  exact (findLeastFrom_correct (fun d => 2 ≤ d ∧ d ∣ n) 2 n m hm2 (by omega) (hm2, hm)).2

/-- The least divisor at least 2 is prime: any divisor of it divides n, and the search's
    minimality bounds it from below. -/
theorem leastDivisor_prime (n : Nat) (hn : 2 ≤ n) : isPrime (leastDivisor n) := by
  obtain ⟨⟨hd2, hdvd⟩, hdle, hmin⟩ := leastDivisor_spec n hn
  refine ⟨hd2, fun m hm => ?_⟩
  have hdpos : 0 < leastDivisor n := by omega
  have hmle : m ≤ leastDivisor n := Nat.le_of_dvd hdpos hm
  rcases Decidable.em (m = 1) with h1 | h1
  · exact Or.inl h1
  · right
    have hmne0 : m ≠ 0 := by
      intro h0
      rw [h0] at hm
      obtain ⟨c, hc⟩ := hm
      rw [Nat.zero_mul] at hc
      omega
    have hm2 : 2 ≤ m := by omega
    have hle := hmin m hm2 (Nat.dvd_trans hm hdvd)
    omega

/-- Every integer greater than one has a prime divisor: the least divisor at least 2, found by
    bounded search. No classical axiom. -/
theorem exists_prime_dvd (n : Nat) (h0 : 0 < n) (h1 : n ≠ 1) : ∃ p, isPrime p ∧ p ∣ n := by
  have hn : 2 ≤ n := by omega
  exact ⟨leastDivisor n, leastDivisor_prime n hn, (leastDivisor_spec n hn).1.2⟩

/-- Two completely additive functions that agree at every prime agree everywhere, by bounded
    induction peeling one prime divisor at each step. -/
theorem determined_by_primes (f g : Nat → Int) (hf : CompletelyAdditive f)
    (hg : CompletelyAdditive g) (h_agree : ∀ p, isPrime p → f p = g p) :
    ∀ n, 0 < n → f n = g n := by
  suffices h : ∀ fuel n, n ≤ fuel → 0 < n → f n = g n from fun n hn => h n n (Nat.le_refl n) hn
  intro fuel
  induction fuel with
  | zero => intro n hle h0; omega
  | succ fuel ih =>
    intro n hle h0
    if h1 : n = 1 then
      subst h1
      rw [seat_is_zero f hf, seat_is_zero g hg]
    else
      obtain ⟨p, hp, hpd⟩ := exists_prime_dvd n h0 h1
      obtain ⟨k, hk⟩ := hpd
      have hk0 : 0 < k := by
        rcases Nat.eq_zero_or_pos k with hkc | hkc
        · exfalso
          rw [hkc, Nat.mul_zero] at hk
          omega
        · exact hkc
      have hp2 : 2 ≤ p := hp.1
      have hlt : k < n := by
        obtain ⟨d, hd⟩ := Nat.exists_eq_add_of_le hp2
        rw [hk, hd, Nat.add_mul]
        omega
      have hpf : f (p * k) = f p + f k := hf p k (by omega) hk0
      have hpg : g (p * k) = g p + g k := hg p k (by omega) hk0
      rw [hk, hpf, hpg, h_agree p hp, ih k (by omega) hk0]

/-! ## IV · The p-adic valuation as a free assignment; Euclid's lemma from first principles -/

/-- Remainder descent: if p divides m * b and x * b, with x = m * q + r, then p divides r * b. -/
theorem dvd_remainder_mul (p b m q r x : Nat) (hpm : p ∣ m * b) (hx : p ∣ x * b)
    (hqr : m * q + r = x) : p ∣ r * b := by
  have h2 : p ∣ (m * q) * b := by
    obtain ⟨c, hc⟩ := hpm
    exact ⟨c * q, by
      calc (m * q) * b = (m * b) * q := Nat.mul_right_comm m q b
        _ = (p * c) * q := by rw [hc]
        _ = p * (c * q) := Nat.mul_assoc p c q)⟩
  have h3 : (m * q) * b + r * b = x * b := by rw [← Nat.add_mul, hqr]
  obtain ⟨u, hu⟩ := hx
  obtain ⟨v, hv⟩ := h2
  have hY : r * b = x * b - (m * q) * b := by
    calc r * b = (m * q) * b + r * b - (m * q) * b :=
      (Nat.add_sub_cancel_left _ _).symm
      _ = x * b - (m * q) * b := by rw [h3]
  exact ⟨u - v, by rw [Nat.mul_sub, ← hu, ← hv]; exact hY⟩

/-- The least positive m with p ∣ m * b, found by search from 0 with fuel p. -/
def leastMulDvd (p b : Nat) : Nat := findLeastFrom (fun m => 0 < m ∧ p ∣ m * b) 0 p

theorem leastMulDvd_spec (p b : Nat) (hwit : 0 < p ∧ p ∣ p * b) :
    (0 < leastMulDvd p b ∧ p ∣ leastMulDvd p b * b) ∧ leastMulDvd p b ≤ p ∧
      (∀ k, k ≤ p → 0 < k ∧ p ∣ k * b → leastMulDvd p b ≤ k) :=
```

```
  ((findLeastFrom_correct (fun m => 0 < m ∧ p | m * b) 0 p p (Nat.zero_le
    p) (by omega)
      hwit).1,
   (findLeastFrom_correct (fun m => 0 < m ∧ p | m * b) 0 p p (Nat.zero_le
    p) (by omega)
      hwit).2,
  fun k hk hPk =>
    (findLeastFrom_correct (fun m => 0 < m ∧ p | m * b) 0 p k
    (Nat.zero_le k) (by omega)
      hPk).2)

/-- Euclid's lemma: a prime dividing a product divides one of the
factors. Proved by least-witness
    search, the division algorithm and descent on the remainder. -/
theorem euclid_lemma (p a b : Nat) (hp : isPrime p) (hab : p | a * b) : p
| a ∨ p | b := by
  rcases Decidable.em (p | a) with hpa | hpa
  · exact Or.inl hpa
  · right
    have hp0 : 0 < p := by have := hp.1; omega
    obtain ((hm0, hmdvd), hmle, hmin') :=
      leastMulDvd_spec p b (hp0, Nat.dvd_mul_right p b)
    have hmp : leastMulDvd p b | p := by
      apply Nat.dvd_of_mod_eq_zero
      have hc : p % leastMulDvd p b = 0 ∨ 0 < p % leastMulDvd p b := by
      omega
      rcases hc with h0 | hpos
      · exact h0
      · have hlt : p % leastMulDvd p b < leastMulDvd p b := Nat.mod_lt p
      hm0
        have hdvd' : p | (p % leastMulDvd p b) * b :=
          dvd_remainder_mul p b (leastMulDvd p b) (p / leastMulDvd p b)
            (p % leastMulDvd p b) p hmdvd (Nat.dvd_mul_right p b)
            (Nat.div_add_mod p (leastMulDvd p b))
        have hle := hmin' (p % leastMulDvd p b) (by omega) (hpos, hdvd')
        omega
    have hma : leastMulDvd p b | a := by
      apply Nat.dvd_of_mod_eq_zero
      have hc : a % leastMulDvd p b = 0 ∨ 0 < a % leastMulDvd p b := by
      omega
      rcases hc with h0 | hpos
      · exact h0
      · have hlt : a % leastMulDvd p b < leastMulDvd p b := Nat.mod_lt a
      hm0
        have hdvd' : p | (a % leastMulDvd p b) * b :=
          dvd_remainder_mul p b (leastMulDvd p b) (a / leastMulDvd p b)
            (a % leastMulDvd p b) a hmdvd hab (Nat.div_add_mod a
            (leastMulDvd p b))
        have hle := hmin' (a % leastMulDvd p b) (by omega) (hpos, hdvd')
        omega
    rcases hp.2 _ hmp with h | h
    · rw [h] at hmdvd
      rwa [Nat.one_mul] at hmdvd
    · rw [h] at hma
      exact absurd hma hpa

/-- Greatest witness at or below the bound, by downward search; P 0
always holds here. -/
def findGt (P : Nat → Prop) [DecidablePred P] : Nat → Nat
  | 0 => 0
  | n + 1 => if P (n + 1) then n + 1 else findGt P n

theorem findGt_spec (P : Nat → Prop) [DecidablePred P] (n : Nat) (h0 : P
0) :
    P (findGt P n) := by
  induction n with
  | zero => exact h0
  | succ n ih =>
    show P (if P (n + 1) then n + 1 else findGt P n)
    rcases Decidable.em (P (n + 1)) with h | h
    · rw [if_pos h]; exact h
    · rw [if_neg h]; exact ih

theorem findGt_max (P : Nat → Prop) [DecidablePred P] (n k : Nat) (hk : k
≤ n) (hPk : P k) :
    k ≤ findGt P n := by
  induction n with
  | zero =>
    have h : k = 0 := by omega
    subst h
    exact Nat.le_refl _
  | succ n ih =>
    show k ≤ (if P (n + 1) then n + 1 else findGt P n)
    rcases Decidable.em (P (n + 1)) with h | h
    · rw [if_pos h]; exact hk
    · rw [if_neg h]
      have hne : k ≠ n + 1 := fun hkk => h (hkk ▸ hPk)
      have hkn : k ≤ n := by omega
      exact ih hkn

/-- The p-adic valuation: the exponent of p in n, as the greatest k with p
^ k | n. -/
def pExp (p n : Nat) : Nat := findGt (fun k => p ^ k | n) n

theorem pExp_dvd (p n : Nat) : p ^ pExp p n | n :=
  findGt_spec (fun k => p ^ k | n) n (by
    show p ^ 0 | n
    rw [Nat.pow_zero]
    exact Nat.one_dvd n)

/-- Exponents of two outrun their index. -/
theorem two_pow_ge (k : Nat) : k + 1 ≤ 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih => rw [Nat.pow_succ]; omega

/-- A power of p ≥ 2 dividing a positive n is bounded by n. -/
theorem pow_dvd_bound (p n k : Nat) (hp : 2 ≤ p) (hn : 0 < n) (h : p ^ k |
n) : k ≤ n := by
  have h1 : p ^ k ≤ n := Nat.le_of_dvd hn h
  have h2 : k + 1 ≤ 2 ^ k := two_pow_ge k
  have h3 : 2 ^ k ≤ p ^ k := Nat.pow_le_pow_left hp k
  omega

theorem pExp_le_of_dvd (p n k : Nat) (hk : k ≤ n) (h : p ^ k | n) : k ≤
pExp p n :=
  findGt_max (fun k => p ^ k | n) n k hk h

/-- Four factors rearranged. -/
theorem mul_mul_mul_comm_nat (a b c d : Nat) : (a * b) * (c * d) = (a *
c) * (b * d) := by
  calc (a * b) * (c * d) = a * (b * c) * d := by
        rw [Nat.mul_assoc, ← Nat.mul_assoc b c d, Nat.mul_assoc a (b * c)
        d]
    _ = a * (c * b) * d := by rw [Nat.mul_comm b c]
    _ = (a * c) * (b * d) := by
        rw [Nat.mul_assoc, Nat.mul_assoc c b d, Nat.mul_assoc a c (b * d)]

/-- The p-adic valuation is additive over products. -/
theorem pExp_mul (p a b : Nat) (hp : isPrime p) (ha : 0 < a) (hb : 0 < b)
:
    pExp p (a * b) = pExp p a + pExp p b := by
  have hp2 : 2 ≤ p := hp.1
  have hp0 : 0 < p := by omega
  have hia : p ^ pExp p a | a := pExp_dvd p a
  have hjb : p ^ pExp p b | b := pExp_dvd p b
  obtain (a', ha') := hia
  obtain (b', hb') := hjb
  have hpa' : ¬ p | a' := by
    intro hdiv
    obtain (c, hc) := hdiv
    have hbig : p ^ (pExp p a + 1) | a :=
      (c, by
        calc a = p ^ pExp p a * a' := ha'
          _ = p ^ pExp p a * (p * c) := by rw [hc]
          _ = p ^ (pExp p a + 1) * c := by rw [Nat.pow_add, Nat.pow_one,
          Nat.mul_assoc])
    have hle := pExp_le_of_dvd p a (pExp p a + 1)
      (pow_dvd_bound p a _ hp2 ha hbig) hbig
    omega
  have hpb' : ¬ p | b' := by
    intro hdiv
    obtain (c, hc) := hdiv
    have hbig : p ^ (pExp p b + 1) | b :=
      (c, by
        calc b = p ^ pExp p b * b' := hb'
          _ = p ^ pExp p b * (p * c) := by rw [hc]
          _ = p ^ (pExp p b + 1) * c := by rw [Nat.pow_add, Nat.pow_one,
          Nat.mul_assoc])
    have hle := pExp_le_of_dvd p b (pExp p b + 1)
      (pow_dvd_bound p b _ hp2 hb hbig) hbig
    omega
  have hprodeq : a * b = p ^ (pExp p a + pExp p b) * (a' * b') := by
    have e1 : a * b = (p ^ pExp p a * a') * b := congrArg (fun x => x *
    b) ha'
    have e2 : (p ^ pExp p a * a') * b = (p ^ pExp p a * a') * (p ^ pExp p
    b * b') :=
      congrArg (fun x => (p ^ pExp p a * a') * x) hb'
    have e3 : (p ^ pExp p a * a') * (p ^ pExp p b * b') =
        (p ^ pExp p a * p ^ pExp p b) * (a' * b') := mul_mul_mul_comm_nat
        _ _ _ _
    have e4 : (p ^ pExp p a * p ^ pExp p b) * (a' * b') =
        p ^ (pExp p a + pExp p b) * (a' * b') :=
      congrArg (fun x => x * (a' * b')) (Nat.pow_add p _ _).symm
    exact e1.trans (e2.trans (e3.trans e4))
  have hup : p ^ (pExp p a + pExp p b) | a * b := ⟨a' * b', hprodeq⟩
  have hdown : ¬ p ^ (pExp p a + pExp p b + 1) | a * b := by
    intro hbig
    have hdecomp : p ^ (pExp p a + pExp p b + 1) = p ^ (pExp p a + pExp p
    b) * p := by
      rw [Nat.pow_add, Nat.pow_one]
    have hdiv : p | a' * b' := by
      have h' : p ^ (pExp p a + pExp p b) * p | p ^ (pExp p a + pExp p b)
      * (a' * b') := by
        rw [← hprodeq, ← hdecomp]
        exact hbig
      exact Nat.dvd_of_mul_dvd_mul_left (Nat.pow_pos hp0) h'
    rcases euclid_lemma p a' b' hp hdiv with h | h
    · exact hpa' h
    · exact hpb' h
  have hle1 : pExp p a + pExp p b ≤ pExp p (a * b) :=
    pExp_le_of_dvd p (a * b) _ (pow_dvd_bound p (a * b) _ hp2
    (Nat.mul_pos ha hb) hup) hup
  have hle2 : pExp p (a * b) ≤ pExp p a + pExp p b := by
    have hK : p ^ pExp p (a * b) | a * b := pExp_dvd p (a * b)
    rcases Decidable.em (pExp p a + pExp p b + 1 ≤ pExp p (a * b)) with
    hgt | hgt
    · exact absurd (Nat.dvd_trans (Nat.pow_dvd_pow p hgt) hK) hdown
    · omega
  omega

/-- The valuation of p at itself is one. -/
theorem pExp_self (p : Nat) (hp : isPrime p) : pExp p p = 1 := by
  have hp2 : 2 ≤ p := hp.1
  have hp0 : 0 < p := by omega
  have h1 : p ^ (1 : Nat) | p := by rw [Nat.pow_one]; exact Nat.dvd_refl p
```

```
  have h1le : 1 ≤ pExp p p := pExp_le_of_dvd p p 1 (by omega) h1
  have hnot2 : ¬ p ^ (1 + 1 : Nat) | p := by
    intro h
    have hle : p ^ (1 + 1) ≤ p := Nat.le_of_dvd hp0 h
    rw [Nat.pow_add, Nat.pow_one] at hle
    have hge : 2 * p ≤ p * p := by
      obtain (d, hd) := Nat.exists_eq_add_of_le hp2
      rw [hd, Nat.add_mul]
      exact Nat.le_add_right _ _
    omega
  rcases Decidable.em (2 ≤ pExp p p) with h | h
  · exact absurd (Nat.dvd_trans (Nat.pow_dvd_pow p h) (pExp_dvd p p))
  hnot2
  · omega

/-- The valuation vanishes where p does not divide. -/
theorem pExp_eq_zero_of_not_dvd (p n : Nat) (h : ¬ p | n) : pExp p n = 0
:= by
  rcases Decidable.em (1 ≤ pExp p n) with hpos | hneg
  · have hpow : p ^ (1 : Nat) | p ^ pExp p n := Nat.pow_dvd_pow p hpos
    rw [Nat.pow_one] at hpow
    exact absurd (Nat.dvd_trans hpow (pExp_dvd p n)) h
  · omega

/-- The valuation at one is zero. -/
theorem pExp_one (p : Nat) (hp : isPrime p) : pExp p 1 = 0 := by
  apply pExp_eq_zero_of_not_dvd
  intro hd
  have h1 := Nat.le_of_dvd (by decide) hd
  have h2 := hp.1
  omega

/-- The valuation at a different prime is zero. -/
theorem pExp_other (p q : Nat) (hp : isPrime p) (hq : isPrime q) (hpq : p
≠ q) :
    pExp p q = 0 := by
  apply pExp_eq_zero_of_not_dvd
  intro hd
  rcases hq.2 p hd with h | h
  · have := hp.1; omega
  · exact hpq h

/-- The p-adic valuation as an integer-valued function. -/
def padicMeasure (p : Nat) (n : Nat) : Int := (pExp p n : Int)

theorem padicMeasure_additive (p : Nat) (hp : isPrime p) :
    CompletelyAdditive (padicMeasure p) := by
  intro a b ha hb
  show ((pExp p (a * b) : Nat) : Int) = (pExp p a : Int) + (pExp p b :
  Int)
  rw [pExp_mul p a b hp ha hb, Int.natCast_add]

/-- For distinct primes p and q there is a completely additive function
equal to 1 at p and 0 at q:
    the p-adic valuation. -/
theorem prime_freedom_independent (p q : Nat) (hp : isPrime p) (hq :
isPrime q) (hpq : p ≠ q) :
    ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 = 0 :=
  ⟨padicMeasure p, padicMeasure_additive p hp,
   by show ((pExp p p : Nat) : Int) = 1; rw [pExp_self p hp]; rfl,
   by show ((pExp p q : Nat) : Int) = 0; rw [pExp_other p q hp hq hpq];
   rfl,
   by show ((pExp p 1 : Nat) : Int) = 0; rw [pExp_one p hp]; rfl⟩

/-- Uniqueness and independence together: a completely additive function
is determined by its
    values at the primes, and those values are freely assignable. -/
theorem primes_base_of_freedom :
    (∀ f g : Nat → Int, CompletelyAdditive f → CompletelyAdditive g →
      (∀ p, isPrime p → f p = g p) → ∀ n, 0 < n → f n = g n) ∧
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) :=
  ⟨determined_by_primes, prime_freedom_independent⟩

/-! ## V · The sign arrow of an additive count -/

/-- An additive count: Ω(ab) = Ω(a) + Ω(b) for positive a and b. -/
def AdditiveCount (Om : Nat → Nat) : Prop := ∀ a b, a > 0 → b > 0 → Om (a
* b) = Om a + Om b

/-- The sign arrow of a count: +1 on an even count, −1 on an odd one. -/
def signArrow (k : Nat) : Int := if k % 2 = 0 then 1 else -1

/-- The sign of a sum is the product of the signs. -/
theorem signArrow_add (m n : Nat) :
    signArrow (m + n) = signArrow m * signArrow n := by
  unfold signArrow
  rw [Nat.add_mod]
  rcases Nat.mod_two_eq_zero_or_one m with hm | hm <;>
    rcases Nat.mod_two_eq_zero_or_one n with hn | hn <;>
    rw [hm, hn] <;> decide

/-- The sign arrow of an additive count is completely multiplicative, as
λ(n) = (−1)^Ω(n) is. -/
theorem arrow_multiplicative (Om : Nat → Nat) (hOm : AdditiveCount Om)
    (a b : Nat) (ha : a > 0) (hb : 0 < b) :
    signArrow (Om (a * b)) = signArrow (Om a) * signArrow (Om b) := by
  rw [hOm a b ha hb, signArrow_add]

/-- At a prime the count is one and the sign is −1. -/
theorem arrow_at_prime (Om : Nat → Nat) (p : Nat) (_hp : isPrime p) (hOm :
Om p = 1) :
    signArrow (Om p) = -1 := by
  rw [hOm]
  decide

/-! ## V-bis · No finite stage decides a universal -/

/-- For every N there is a property true below N and false at N. -/
theorem finite_never_forces (N : Nat) :
    ∃ P : Nat → Prop, (∀ n, n < N → P n) ∧ ¬ ∀ n, P n :=
  ⟨fun n => n < N, fun _ h => h, fun h => Nat.lt_irrefl N (h N)⟩

/-- A property with counterexamples past every stage is not universal:
    no finite census decides the limit. -/
theorem limit_not_forced (P : Nat → Prop) (h_finite : ∀ N, ∃ n > N, ¬ P
n) : ¬ (∀ n, P n) := by
  intro h
  obtain ⟨n, _, hn_not⟩ := h_finite 0
  exact hn_not (h n)

/-! ## VI · The chart: the fold, registration, the record -/

/-- A point of the chart: the doubled real part x and the height t.
    The critical line is x = 1; the fold s ↦ 1 − s̄ sends x to 2 − x;
    registration keeps the height and lands on the line. -/
structure Pt where
  x : Int
  t : Nat
  deriving DecidableEq

def onLine (z : Pt) : Prop := z.x = 1
instance (z : Pt) : Decidable (onLine z) := inferInstanceAs (Decidable
(z.x = 1))
def fold (z : Pt) : Pt := ⟨2 - z.x, z.t⟩
def reg (z : Pt) : Pt := ⟨1, z.t⟩

theorem fold_fixed_iff (z : Pt) : fold z = z ↔ onLine z := by
  cases z with
  | mk x t =>
    show Pt.mk (2 - x) t = Pt.mk x t ↔ x = 1
    constructor
    · intro h
      have hx := congrArg Pt.x h
      change 2 - x = x at hx
      omega
    · intro h
      subst h
      rfl

theorem reg_fixes_line (z : Pt) (h : onLine z) : reg z = z := by
  cases z with
  | mk x t =>
    have hx : x = 1 := h
    subst hx
    rfl

/-- No registered point lies off the line. -/
theorem unicorn_never_registered (z : Pt) (h : ¬ onLine z) : ∀ w, reg w ≠
z := by
  intro w hw
  apply h
  rw [← hw]
  exact rfl

/-- Every point is on the line, fixed by the fold and by registration, or
off the line, moved by
    the fold, sharing its record with its partner, and registered by no
    point. -/
theorem nothing_escapes_one_cut (z : Pt) :
    (onLine z ∧ fold z = z ∧ reg z = z) ∨
    (¬ onLine z ∧ fold z ≠ z ∧ reg (fold z) = reg z ∧ ∀ w, reg w ≠ z) :=
  if h : z.x = 1 then Or.inl ⟨h, (fold_fixed_iff z).mpr h, reg_fixes_line
  z h⟩
  else Or.inr ⟨h, fun hf => h ((fold_fixed_iff z).mp hf), rfl,
  unicorn_never_registered z h⟩

/-- A world is a zero set on the chart. -/
abbrev World := Pt → Prop
def FoldClosed (Z : World) : Prop := ∀ z, Z z → Z (fold z)
/-- The hypothesis, on the chart: every zero lies on the line. -/
def RH (Z : World) : Prop := ∀ z, Z z → onLine z
def Left (Z : World) : Prop := ∃ z, Z z ∧ z.x < 1
def Right (Z : World) : Prop := ∃ z, Z z ∧ 1 < z.x

/-- On a fold-closed world some zero lies left of the line exactly when
some zero lies right of it. -/
theorem sides_together (Z : World) (hZ : FoldClosed Z) : Left Z ↔ Right Z
:=
  ⟨fun ⟨z, hz, hx⟩ => ⟨fold z, hZ z hz, show 1 < 2 - z.x by omega⟩,
   fun ⟨z, hz, hx⟩ => ⟨fold z, hZ z hz, show 2 - z.x < 1 by omega⟩⟩

/-- On a fold-closed world the hypothesis holds exactly when no zero lies
left of the line. -/
theorem rh_iff_no_left (Z : World) (hZ : FoldClosed Z) : RH Z ↔ ¬ Left Z
:= by
  constructor
  · intro h ⟨z, hz, hx⟩
    have e : z.x = 1 := h z hz
    omega
  · intro h z hz
    if hx : z.x = 1 then exact hx
    else
      exfalso
      apply h
```

```
      if hl : z.x < 1 then exact ⟨z, hz, hl⟩
      else exact ⟨fold z, hZ z hz, show 2 - z.x < 1 by omega⟩

/-- The record of a world: its zeros registered. -/
def recordOf (Z : World) : World := fun r => ∃ s, Z s ∧ reg s = r
def SameRecord (Z Z' : World) : Prop := ∀ r, recordOf Z r ↔ recordOf Z' r
/-- A reading respects the record when two worlds of one record read
alike. -/
def RespectsRecord (g : World → Prop) : Prop := ∀ Z Z', SameRecord Z Z' →
(g Z ↔ g Z')

/-- The on-line world at height 14, and the off-line pair at offset k
about the same height. -/
abbrev W1 : World := fun z => z = ⟨1, 14⟩
abbrev pairAt (k : Int) : World := fun z => z = ⟨1 - k, 14⟩ ∨ z = ⟨1 + k,
14⟩
abbrev W2 : World := pairAt 1

theorem W1_closed : FoldClosed W1 := by
  intro z hz
  have h : z = ⟨1, 14⟩ := hz
  subst h
  show fold ⟨1, 14⟩ = ⟨1, 14⟩
  decide

theorem W1_rh : RH W1 := by
  intro z hz
  have h : z = ⟨1, 14⟩ := hz
  subst h
  exact rfl

theorem pair_closed (k : Int) : FoldClosed (pairAt k) := by
  intro z hz
  rcases hz with h | h
  · subst h
    apply Or.inr
    show Pt.mk (2 - (1 - k)) 14 = Pt.mk (1 + k) 14
    congr 1
    omega
  · subst h
    apply Or.inl
    show Pt.mk (2 - (1 + k)) 14 = Pt.mk (1 - k) 14
    congr 1
    omega

theorem pair_not_rh (k : Int) (hk : k ≠ 0) : ¬ RH (pairAt k) := by
  intro h
  have e : (1 : Int) - k = 1 := h (1 - k, 14) (Or.inl rfl)
  omega

theorem pair_same_record (k : Int) : SameRecord W1 (pairAt k) := by
  intro r
  constructor
  · intro ⟨s, hs, hr⟩
    have h : s = ⟨1, 14⟩ := hs
    subst h
    exact ⟨(1 - k, 14), Or.inl rfl, hr⟩
  · intro ⟨s, hs, hr⟩
    rcases hs with h | h <;> subst h <;> exact ⟨(1, 14), rfl, hr⟩

theorem W2_closed : FoldClosed W2 := pair_closed 1
theorem W2_not_rh : ¬ RH W2 := pair_not_rh 1 (by decide)
theorem same_record_12 : SameRecord W1 W2 := pair_same_record 1

/-- Distinct positive offsets give distinct fold-closed worlds with the
record of W1, none
    satisfying the hypothesis. -/
theorem fibre_is_infinite (k k' : Int) (hk : 0 < k) (hk' : 0 < k') (hne :
k ≠ k') :
    SameRecord W1 (pairAt k) ∧ ¬ RH (pairAt k) ∧ pairAt k ⟨1 - k, 14⟩ ∧ ¬
    pairAt k' ⟨1 - k, 14⟩ := by
  refine ⟨pair_same_record k, pair_not_rh k (by omega), Or.inl rfl, ?_⟩
  intro h
  rcases h with h | h
  · have e : (1 : Int) - k = 1 - k' := congrArg Pt.x h
    omega
  · have e : (1 : Int) - k = 1 + k' := congrArg Pt.x h
    omega

/-- Two worlds with the same record that both satisfy the hypothesis are
equal. -/
theorem lossless_unique (Z Z' : World) (h : SameRecord Z Z') (hZ : RH Z)
(hZ' : RH Z') :
    ∀ z, Z z ↔ Z' z := by
  intro z
  constructor
  · intro hz
    obtain ⟨s, hs, hsr⟩ := (h z).mp ⟨z, hz, reg_fixes_line z (hZ z hz)⟩
    have e : reg s = s := reg_fixes_line s (hZ' s hs)
    rw [← hsr, e]
    exact hs
  · intro hz
    obtain ⟨s, hs, hsr⟩ := (h z).mpr ⟨z, hz, reg_fixes_line z (hZ' z hz)⟩
    have e : reg s = s := reg_fixes_line s (hZ s hs)
    rw [← hsr, e]
    exact hs

/-- No reading of the record agrees with the hypothesis on every
fold-closed world. -/
theorem record_decides_nothing (g : World → Prop) (hg : RespectsRecord g)
:
    ¬ ∀ Z, FoldClosed Z → (g Z ↔ RH Z) := fun h =>
  W2_not_rh ((h W2 W2_closed).mp ((hg W1 W2 same_record_12).mp ((h W1
  W1_closed).mpr W1_rh)))


/-- record_decides_nothing, under a second name. -/
theorem unicorn_block (g : World → Prop) (hg : RespectsRecord g) :
    ¬ ∀ Z, FoldClosed Z → (g Z ↔ RH Z) := record_decides_nothing g hg

/-- A premise is keyless when it holds on every world. -/
def Keyless (A : World → Prop) : Prop := ∀ Z, A Z
/-- A premise forces the line when every fold-closed world it holds on
satisfies the hypothesis. -/
def Forces (A : World → Prop) : Prop := ∀ Z, FoldClosed Z → A Z → RH Z

/-- A keyless premise forces nothing: it holds on W2, which is
fold-closed and fails the
    hypothesis. -/
theorem keyless_forces_nothing (A : World → Prop) (hA : Keyless A) : ¬
Forces A :=
  fun h => W2_not_rh (h W2 W2_closed (hA W2))

/-- The hypothesis separates W1 from W2, which share one record. -/
theorem sentence_separates_record_does_not : (RH W1 ∧ ¬ RH W2) ∧
SameRecord W1 W2 :=
  ⟨(W1_rh, W2_not_rh), same_record_12⟩

/-! ## VII · Least erasure is the value -/

def OffLine (Z : World) : Prop := ∃ z, Z z ∧ ¬ onLine z
/-- Least erasure in the fibre: if the world erases, every world of its
record erases. -/
def LeastErasure (Z : World) : Prop := ∀ Z', SameRecord Z Z' → OffLine Z →
OffLine Z'

theorem record_is_lossless (Z : World) : ¬ OffLine (recordOf Z) := by
  intro ⟨r, ⟨s, _, hsr⟩, hoff⟩
  apply hoff
  rw [← hsr]
  exact rfl

theorem record_same (Z : World) : SameRecord Z (recordOf Z) := by
  intro r
  constructor
  · intro ⟨s, hs, hsr⟩
    exact ⟨reg s, ⟨s, hs, rfl⟩, hsr⟩
  · intro ⟨s', ⟨s, hs, hss'⟩, hs'r⟩
    subst hss'
    exact ⟨s, hs, hs'r⟩

/-- Least erasure holds on a world exactly when the hypothesis does. -/
theorem least_erasure_is_the_value (Z : World) : LeastErasure Z ↔ RH Z :=
by
  constructor
  · intro hle z hz
    if h : z.x = 1 then exact h
    else exact absurd ⟨hle (recordOf Z) (record_same Z) ⟨z, hz, h⟩⟩
    (record_is_lossless Z)
  · intro hrh _ ⟨z, hz, hoff⟩
    exact absurd ⟨hrh z hz⟩ hoff

/-- Least erasure is not a reading of the record: it holds on W1 and
fails on W2, which share one
    record. -/
theorem least_erasure_reads_past_the_record : ¬ RespectsRecord
LeastErasure := fun h =>
  W2_not_rh ((least_erasure_is_the_value W2).mp
    ((h W1 W2 same_record_12).mp ((least_erasure_is_the_value W1).mpr
    W1_rh)))

/-- Any premise that forces the line and is forced by it is the line, on
every fold-closed
    world. -/
theorem posit_is_the_value (A : World → Prop) (h : Forces A)
    (h' : ∀ Z, FoldClosed Z → RH Z → A Z) : ∀ Z, FoldClosed Z → (A Z ↔ RH
    Z) :=
  fun Z hZ => ⟨h Z hZ, h' Z hZ⟩

/-! ## VII-bis · The fold never crosses the line; a second symmetry; the
record blind at every scale -/

/-- The fold lands on the line exactly where the point already was:
folding never crosses. -/
theorem fold_onLine_iff (z : Pt) : onLine (fold z) ↔ onLine z := by
  show (2 - z.x = 1) ↔ (z.x = 1)
  constructor <;> intro h <;> omega

theorem fold_preserves_offline (z : Pt) : ¬ onLine (fold z) ↔ ¬ onLine z
:= by
  rw [fold_onLine_iff]

theorem fold_ne_of_offline (z : Pt) (h : ¬ onLine z) : fold z ≠ z :=
  fun hf => h ((fold_fixed_iff z).mp hf)

/-- A second involution on the chart, commuting with the fold and
preserving the line. -/
structure ConjSymmetry where
  conj : Pt → Pt
  invol : ∀ z, conj (conj z) = z
  commutes : ∀ z, conj (fold z) = fold (conj z)
  preserves_line : ∀ z, onLine (conj z) ↔ onLine z

/-- Under both symmetries one off-line zero forces its reflection, its
conjugate and their
    composite into the world, all off the line. -/
theorem offline_zero_quadruple (Z : World) (hF : FoldClosed Z) (C :
ConjSymmetry)
```

```
    (hC : ∀ z, Z z → Z (C.conj z)) (z : Pt) (hz : Z z) (hoff : ¬ onLine
    z) :
    Z (fold z) ∧ Z (C.conj z) ∧ Z (C.conj (fold z)) ∧
      ¬ onLine (fold z) ∧ ¬ onLine (C.conj z) ∧ ¬ onLine (C.conj (fold
      z)) ∧ fold z ≠ z :=
  (hF z hz, hC z hz, hC (fold z) (hF z hz),
   (fold_preserves_offline z).mpr hoff,
   fun h => hoff ((C.preserves_line z).mp h),
   fun h => hoff ((fold_onLine_iff z).mp ((C.preserves_line (fold z)).mp
   h)),
   fold_ne_of_offline z hoff)

/-- If the hypothesis fails on a world, some zero of it lies off the
line. Classical. -/
theorem not_rh_has_offline_witness (Z : World) (h : ¬ RH Z) : OffLine Z
:= by
  apply Classical.byContradiction
  intro hno
  apply h
  intro z hz
  apply Classical.byContradiction
  intro hoff
  exact hno (z, hz, hoff)

/-- Denying the hypothesis on a world with both symmetries posits a whole
off-line orbit of four
    points. -/
theorem denial_posits_the_orbit (Z : World) (hF : FoldClosed Z) (C :
ConjSymmetry)
    (hC : ∀ z, Z z → Z (C.conj z)) (h : ¬ RH Z) :
    ∃ z, Z z ∧ ¬ onLine z ∧ Z (fold z) ∧ Z (C.conj z) ∧ Z (C.conj (fold
    z)) := by
  obtain ⟨z, hz, hoff⟩ := not_rh_has_offline_witness Z h
  exact ⟨z, hz, hoff, hF z hz, hC z hz, hC (fold z) (hF z hz)⟩

/-- For every nonzero offset the pair world is fold-closed, fails the
hypothesis, and shares the
    record of W1. -/
theorem record_blind_at_every_scale (k : Int) (hk : k ≠ 0) :
    RH W1 ∧ FoldClosed (pairAt k) ∧ ¬ RH (pairAt k) ∧ SameRecord W1
    (pairAt k) :=
  ⟨W1_rh, pair_closed k, pair_not_rh k hk, pair_same_record k⟩

/-! ## VIII · Self-grounding supplies, and the five faces of the
hypothesis -/

/-- A self-grounding root, the structure of Appendix R in the same words:
acts occur, and every act instances the root. -/
structure SelfGrounding (root : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → root

/-- A self-grounding supply of a proposition, with an act, exists exactly
when the proposition
    holds. -/
theorem supply_iff (root : Prop) : (∃ G : SelfGrounding root, Nonempty
G.Act) ↔ root := by
  constructor
  · intro (G, ⟨a⟩)
    exact G.instances a
  · intro hr
    exact ((Unit, (), fun _ => hr), ⟨()⟩)

/-- The five faces of the hypothesis. The equivalences are fields,
carried as hypotheses and not
    proved here: the critical line (Riemann 1859), Λ = 0 (Newman 1976;
    Rodgers and Tao 2020), the
    faithfulness of the Liouville function (Landau 1899), and Weil
    positivity (Weil 1952). -/
structure Faces where
  line : Prop
  lamZero : Prop
  faithful : Prop
  weil : Prop
  least : Prop
  line_iff_least : line ↔ least
  lam_iff_least : lamZero ↔ least
  faith_iff_least : faithful ↔ least
  weil_iff_least : weil ↔ least

/-- Any one face implies all five. -/
theorem faces_are_one (H : Faces) :
    (H.line ↔ H.least) ∧ (H.lamZero ↔ H.least) ∧ (H.faithful ↔ H.least) ∧
    (H.weil ↔ H.least) :=
  ⟨H.line_iff_least, H.lam_iff_least, H.faith_iff_least, H.weil_iff_least⟩

/-- Given a self-grounding supply of the least-erasure face with an act,
all five faces hold. -/
theorem rh_from_the_act_generic (H : Faces) (G : SelfGrounding H.least)
(a : G.Act) :
    H.line ∧ H.lamZero ∧ H.faithful ∧ H.weil ∧ H.least :=
  let le := G.instances a
  ⟨H.line_iff_least.mpr le, H.lam_iff_least.mpr le, H.faith_iff_least.mpr
  le,
   H.weil_iff_least.mpr le, le⟩


/-! ## IX · Primes, the free basis and the identity arrow are keyless and
force nothing -/

/-- Primes exist. -/
theorem primes_exist : ∃ p, isPrime p :=
  let (p, hp, _) := exists_prime_dvd 2 (by decide) (by decide)
  ⟨p, hp⟩

/-- The existence of primes holds on every world. -/
theorem primes_exist_keyless : Keyless (fun _ => ∃ p, isPrime p) := fun _
=> primes_exist

/-- The free basis at the primes holds on every world. -/
theorem prime_freedom_keyless :
    Keyless (fun _ => ∀ p q, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) :=
  fun _ => prime_freedom_independent

/-- The identity arrow exists on every world. -/
theorem arrow_keyless : Keyless (fun _ => ∃ f : Nat → Nat, ∀ a, f a = a)
:=
  fun _ => arrow_exists Nat

/-- The existence of primes forces nothing. -/
theorem primes_exist_forces_nothing : ¬ Forces (fun _ => ∃ p, isPrime p)
:=
  keyless_forces_nothing _ primes_exist_keyless

/-- The free basis at the primes forces nothing. -/
theorem prime_freedom_forces_nothing :
    ¬ Forces (fun _ => ∀ p q, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) :=
  keyless_forces_nothing _ prime_freedom_keyless

/-- The existence of the identity arrow forces nothing. -/
theorem arrow_forces_nothing : ¬ Forces (fun _ => ∃ f : Nat → Nat, ∀ a, f
a = a) :=
  keyless_forces_nothing _ arrow_keyless

/-- The free basis holds, and a fold-closed world failing the hypothesis
exists. -/
theorem free_basis_coexists_with_offline_world :
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) ∧
    ∃ Z, FoldClosed Z ∧ ¬ RH Z :=
  ⟨prime_freedom_independent, W2, W2_closed, W2_not_rh⟩

/-- An off-line point folds to a distinct off-line point, which no point
registers to. -/
theorem fold_off_line_stays_off (z : Pt) (h : ¬ onLine z) :
    ¬ onLine (fold z) ∧ fold z ≠ reg z ∧ fold z ≠ z :=
  ⟨(fold_preserves_offline z).mpr h,
   fun e => unicorn_never_registered (fold z) ((fold_preserves_offline
   z).mpr h) z e.symm,
   fold_ne_of_offline z h⟩

/-! ## X · The explicit formula as a structure; least erasure is Weil
positivity -/

structure ExplicitFormula where
  /-- The value type of the two sides, with its order and its zero. -/
  V : Type
  le : V → V → Prop
  zero : V
  /-- The admissible test functions, their adjoint g ↦ g~, and their
  convolution. -/
  T : Type
  adj : T → T
  conv : T → T → T
  /-- The arithmetic side: the archimedean term with the sum over the
  primes. -/
  primeSide : T → V
  /-- The zero side: the sum over the zeros. -/
  zeroSide : T → V
  /-- The zeros, as a world on the chart. -/
  zeros : World
  /-- The explicit formula: the zero side of every test function equals
  its arithmetic side. Carried
    as a hypothesis (Guinand 1948; Weil 1952). -/
  identity : ∀ g, zeroSide g = primeSide g
  /-- Weil's criterion: the arithmetic side is non-negative on every g ⋆
  g̃ exactly when the zeros
    lie on the line. Carried as a hypothesis (Weil 1952; Bombieri 2000).
    -/
  positivity_iff_line : (∀ g, le zero (primeSide (conv g (adj g)))) ↔ RH
  zeros

/-- Weil positivity, stated on the prime side. -/
def WeilPositive (E : ExplicitFormula) : Prop :=
  ∀ g, E.le E.zero (E.primeSide (E.conv g (E.adj g)))

/-- Through the identity, positivity reads the same on the zero side. -/
theorem positivity_on_zero_side (E : ExplicitFormula) :
    WeilPositive E ↔ ∀ g, E.le E.zero (E.zeroSide (E.conv g (E.adj g)))
    := by
  constructor
  · intro h g
    rw [E.identity]
    exact h g
  · intro h g
    rw [← E.identity]
    exact h g

/-- Least erasure of the zeros of an explicit formula is Weil positivity
of its arithmetic side. -/
```

```
theorem least_erasure_is_positivity (E : ExplicitFormula) :
    LeastErasure E.zeros ↔ WeilPositive E :=
  (least_erasure_is_the_value E.zeros).trans E.positivity_iff_line.symm

/-- A self-grounding supply of Weil positivity exists exactly when the
zeros lie on the line. -/
theorem prime_witness_iff (E : ExplicitFormula) :
    (∃ G : SelfGrounding (WeilPositive E), Nonempty G.Act) ↔ RH E.zeros :=
  (supply_iff (WeilPositive E)).trans E.positivity_iff_line

/-- Weil positivity implies the hypothesis for the zeros of the explicit
formula. -/
theorem rh_from_prime_witness (E : ExplicitFormula) (h : WeilPositive E) :
RH E.zeros :=
  E.positivity_iff_line.mp h

/-- The positive instance: one test function, arithmetic side zero, zeros
on the line. -/
def positiveInstance : ExplicitFormula where
  V := Int
  le := fun a b => a ≤ b
  zero := 0
  T := Unit
  adj := id
  conv := fun _ _ => ()
  primeSide := fun _ => 0
  zeroSide := fun _ => 0
  zeros := W1
  identity := fun _ => rfl
  positivity_iff_line := (fun _ => W1_rh, fun _ _ => Int.le_refl 0)

/-- The negative instance: one test function, arithmetic side minus one,
zeros off the line. -/
def negativeInstance : ExplicitFormula where
  V := Int
  le := fun a b => a ≤ b
  zero := 0
  T := Unit
  adj := id
  conv := fun _ _ => ()
  primeSide := fun _ => -1
  zeroSide := fun _ => -1
  zeros := W2
  identity := fun _ => rfl
  positivity_iff_line :=
    (fun h => absurd (h ()) (by decide : ¬ ((0 : Int) ≤ -1)), fun h =>
    absurd h W2_not_rh)

/-- Two explicit formulas satisfy every field, one positive with its
zeros on the line and one not
    positive with its zeros off the line: no field decides positivity. -/
theorem positivity_is_keyed : WeilPositive positiveInstance ∧ ¬
WeilPositive negativeInstance :=
  (fun _ => Int.le_refl 0, fun h => absurd (h ()) (by decide : ¬ ((0 :
  Int) ≤ -1)))

/-- The free basis holds in both instances. -/
theorem freedom_does_not_pick_the_sign :
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) ∧
    WeilPositive positiveInstance ∧ ¬ WeilPositive negativeInstance :=
  (prime_freedom_independent, positivity_is_keyed)

/-- Section X, whole: the free basis; least erasure as the hypothesis and
as positivity; the block;
    keyless premises forcing nothing; positivity keyed. -/
theorem prime_witness_ledger :
    (∀ f g : Nat → Int, CompletelyAdditive f → CompletelyAdditive g →
      (∀ p, isPrime p → f p = g p) → ∀ n, 0 < n → f n = g n) ∧
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ E : ExplicitFormula, LeastErasure E.zeros ↔ WeilPositive E) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) ∧
    (¬ Forces (fun _ => ∃ p, isPrime p)) ∧
    (¬ Forces (fun _ => ∀ p q, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0)) ∧
    (¬ Forces (fun _ => ∃ f : Nat → Nat, ∀ a, f a = a)) ∧
    (WeilPositive positiveInstance ∧ ¬ WeilPositive negativeInstance) :=
  (determined_by_primes, prime_freedom_independent,
  least_erasure_is_the_value,
   least_erasure_is_positivity, unicorn_block,
   primes_exist_forces_nothing,
   prime_freedom_forces_nothing, arrow_forces_nothing,
   positivity_is_keyed)

/-! ## XI · The posit as a structure field -/

/-- A zero set with its fold-closure and the one posit, least erasure. -/
structure ActualZeros where
  /-- The zero set, as a world on the chart. -/
  zeros : World
  /-- The zero set is fold-closed (the functional equation, Riemann
  1859). -/
  fold_closed : FoldClosed zeros
  /-- The posit: least erasure of the zero set. By
  least_erasure_is_the_value this field is the
    hypothesis. -/
  supply : LeastErasure zeros

/-- From the posit, the hypothesis. -/
theorem rh_from_the_act (A : ActualZeros) : RH A.zeros :=
  (least_erasure_is_the_value A.zeros).mp A.supply

/-- The type of the posit is equivalent to the hypothesis. -/
theorem posit_is_the_conclusion (A : ActualZeros) : LeastErasure A.zeros ↔
RH A.zeros :=
  least_erasure_is_the_value A.zeros

/-- From the posit: the line, least erasure, every zero fixed by the fold
and by registration, and
    no off-line orbit. -/
theorem rh_ground_closure_complete (A : ActualZeros) :
    RH A.zeros ∧ LeastErasure A.zeros ∧ (∀ z, A.zeros z → fold z = z) ∧
    (∀ z, A.zeros z → reg z = z) ∧ ¬ Left A.zeros :=
  (rh_from_the_act A, A.supply,
   fun z hz => (fold_fixed_iff z).mpr (rh_from_the_act A z hz),
   fun z hz => reg_fixes_line z (rh_from_the_act A z hz),
   (rh_iff_no_left A.zeros A.fold_closed).mp (rh_from_the_act A))

/-- An explicit formula for fold-closed zeros, with positivity posited. -/
structure PrimeAct where
  E : ExplicitFormula
  fold_closed : FoldClosed E.zeros
  /-- The posit on the prime side: Weil positivity. -/
  positive : WeilPositive E

/-- A prime act is a least-erasure act. -/
def PrimeAct.toActual (P : PrimeAct) : ActualZeros :=
  (P.E.zeros, P.fold_closed, (least_erasure_is_positivity P.E).mpr
  P.positive)

/-- From positivity posited, the hypothesis. -/
theorem rh_from_prime_act (P : PrimeAct) : RH P.E.zeros :=
rh_from_the_act P.toActual

/-- For an explicit formula with fold-closed zeros, a positivity posit
exists exactly when a
    least-erasure posit exists for its zeros. -/
theorem acts_are_one (E : ExplicitFormula) (hF : FoldClosed E.zeros) :
    (∃ P : PrimeAct, P.E = E) ↔ ∃ A : ActualZeros, A.zeros = E.zeros := by
  constructor
  · intro (P, hP)
    subst hP
    exact (P.toActual, rfl)
  · intro (A, hA)
    refine ((E, hF, (least_erasure_is_positivity E).mp ?_), rfl)
    rw [← hA]
    exact A.supply


/-! ## XII · The block with no axiom: the pair world written out -/

/-- The pair world at offset one, written out. -/
abbrev W2f : World := fun z => z = (0, 14) ∨ z = (2, 14)

theorem W2f_closed : FoldClosed W2f := by
  intro z hz
  rcases hz with h | h
  · subst h; exact Or.inr rfl
  · subst h; exact Or.inl rfl

theorem W2f_not_rh : ¬ RH W2f :=
  fun h => absurd (h (0, 14) (Or.inl rfl)) (by decide : ¬ ((0 : Int) = 1))

theorem same_record_f : SameRecord W1 W2f := by
  intro r
  constructor
  · intro (s, hs, hr)
    have h : s = (1, 14) := hs
    subst h
    exact ((0, 14), Or.inl rfl, hr)
  · intro (s, hs, hr)
    rcases hs with h | h <;> subst h <;> exact ((1, 14), rfl, hr)

/-- record_decides_nothing, with no axiom. -/
theorem record_decides_nothing_free (g : World → Prop) (hg :
RespectsRecord g) :
    ¬ ∀ Z, FoldClosed Z → (g Z ↔ RH Z) := fun h =>
  W2f_not_rh ((h W2f W2f_closed).mp ((hg W1 W2f same_record_f).mp ((h W1
  W1_closed).mpr W1_rh)))

/-- unicorn_block, with no axiom. -/
theorem unicorn_block_free (g : World → Prop) (hg : RespectsRecord g) :
    ¬ ∀ Z, FoldClosed Z → (g Z ↔ RH Z) := record_decides_nothing_free g hg

/-- keyless_forces_nothing, with no axiom. -/
theorem keyless_forces_nothing_free (A : World → Prop) (hA : Keyless A) :
¬ Forces A :=
  fun h => W2f_not_rh (h W2f W2f_closed (hA W2f))

/-- arrow_forces_nothing, with no axiom. -/
theorem arrow_forces_nothing_free : ¬ Forces (fun _ => ∃ f : Nat → Nat, ∀
a, f a = a) :=
  keyless_forces_nothing_free _ arrow_keyless

/-- least_erasure_reads_past_the_record, with no axiom. -/
theorem least_erasure_reads_past_the_record_free : ¬ RespectsRecord
LeastErasure := fun h =>
  W2f_not_rh ((least_erasure_is_the_value W2f).mp
    ((h W1 W2f same_record_f).mp ((least_erasure_is_the_value W1).mpr
    W1_rh)))
```

```
/-- The negative instance on the explicit pair world. -/
def negativeInstanceF : ExplicitFormula where
  V := Int
  le := fun a b => a ≤ b
  zero := 0
  T := Unit
  adj := id
  conv := fun _ _ => ()
  primeSide := fun _ => -1
  zeroSide := fun _ => -1
  zeros := W2f
  identity := fun _ => rfl
  positivity_iff_line :=
    (fun h => absurd (h ()) (by decide : ¬ ((0 : Int) ≤ -1)), fun h =>
    absurd h W2f_not_rh)

/-- positivity_is_keyed, with no axiom. -/
theorem positivity_is_keyed_free : WeilPositive positiveInstance ∧ ¬
WeilPositive negativeInstanceF :=
  (fun _ => Int.le_refl 0, fun h => absurd (h ()) (by decide : ¬ ((0 :
  Int) ≤ -1)))


/-! ## XIII · The hypothesis on the supplied bit -/

/-- For every explicit formula, Weil positivity is equivalent to the
hypothesis for its zeros. -/
theorem spend_is_the_line (E : ExplicitFormula) : WeilPositive E ↔ RH
E.zeros :=
  E.positivity_iff_line

/-- The free basis holds, and an explicit formula exists on which
positivity fails: the free basis
    and positivity are distinct. -/
theorem spend_is_not_the_free_basis :
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) ∧
    ∃ E : ExplicitFormula, ¬ WeilPositive E :=
  (prime_freedom_independent, negativeInstanceF,
  positivity_is_keyed_free.2)

/-- From positivity posited: the line, least erasure, positivity, every
zero fixed by the fold and
    by registration, and no off-line orbit. -/
theorem rh_on_the_spent_bit (P : PrimeAct) :
    RH P.E.zeros ∧ LeastErasure P.E.zeros ∧ WeilPositive P.E ∧
    (∀ z, P.E.zeros z → fold z = z) ∧ (∀ z, P.E.zeros z → reg z = z) ∧ ¬
    Left P.E.zeros :=
  let c := rh_ground_closure_complete P.toActual
  (c.1, c.2.1, P.positive, c.2.2.1, c.2.2.2.1, c.2.2.2.2)

/-- Sections X to XIII, whole. -/
theorem prime_arc_sealed :
    (∀ f g : Nat → Int, CompletelyAdditive f → CompletelyAdditive g →
      (∀ p, isPrime p → f p = g p) → ∀ n, 0 < n → f n = g n) ∧
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 =
      0) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ E : ExplicitFormula, LeastErasure E.zeros ↔ WeilPositive E) ∧
    (∀ E : ExplicitFormula, WeilPositive E ↔ RH E.zeros) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) ∧
    (∀ A : World → Prop, Keyless A → ¬ Forces A) ∧
    (WeilPositive positiveInstance ∧ ¬ WeilPositive negativeInstanceF) ∧
    (∃ E : ExplicitFormula, ¬ WeilPositive E) ∧
    (∀ P : PrimeAct, RH P.E.zeros) :=
  (determined_by_primes, prime_freedom_independent,
  least_erasure_is_the_value,
   least_erasure_is_positivity, spend_is_the_line,
   record_decides_nothing_free,
   keyless_forces_nothing_free, positivity_is_keyed_free,
   spend_is_not_the_free_basis.2,
   rh_from_prime_act)


/-! ## XIV · The weakest forcing premise; the Li and de Bruijn–Newman
coordinates -/

/-- The hypothesis forces the line, and every premise that forces the
line implies the hypothesis
    on every fold-closed world. -/
theorem rh_is_the_weakest_forcing_premise :
    Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z :=
  (fun _ _ h => h, fun _ hA Z hZ ha => hA Z hZ ha)

/-- A premise that holds at some fold-closed world where the hypothesis
fails does not force. -/
theorem weaker_never_forces (A : World → Prop) (Z : World) (hZ :
FoldClosed Z) (ha : A Z)
    (hn : ¬ RH Z) : ¬ Forces A :=
  fun h => hn (h Z hZ ha)

/-- A premise true on W2f forces nothing. -/
theorem weaker_at_the_twin_never_forces (A : World → Prop) (ha : A W2f) :
¬ Forces A :=
  weaker_never_forces A W2f W2f_closed ha W2f_not_rh

/-- A zero set with the signs of its Li coefficients: the hypothesis
holds exactly when every
    coefficient is non-negative. Carried as a hypothesis (Li 1997;
    Bombieri and Lagarias 1999). -/
structure LiStream where
  zeros : World
  nonneg : Nat → Prop
  li : RH zeros ↔ ∀ n, 1 ≤ n → nonneg n

theorem rh_from_li (L : LiStream) (h : ∀ n, 1 ≤ n → L.nonneg n) : RH
L.zeros := L.li.mpr h

theorem li_is_the_bit_stream (L : LiStream) : (∀ n, 1 ≤ n → L.nonneg n) ↔
RH L.zeros := L.li.symm

/-- For every N there is a Li stream non-negative below N whose zero set
lies off the line. -/
theorem li_prefix_never_forces (N : Nat) :
    ∃ L : LiStream, (∀ n, n < N → L.nonneg n) ∧ ¬ RH L.zeros :=
  ⟨⟨W2f, fun n => n < N,
    (fun h => absurd h W2f_not_rh,
     fun h => absurd (h (N + 1) (Nat.succ_pos N))
       (fun c => Nat.lt_irrefl N (Nat.lt_trans (Nat.lt_succ_self N) c)))⟩,
   fun _ h => h, W2f_not_rh⟩

/-- The de Bruijn–Newman constant over an ordered type: Λ and its zero,
the order's reflexivity and
    antisymmetry, 0 ≤ Λ (Rodgers and Tao 2020), and the hypothesis
    exactly when Λ = 0 (Newman
    1976). Every field is carried as a hypothesis. -/
structure DBN (V : Type) [LE V] where
  zeros : World
  lam : V
  zero : V
  refl : ∀ a : V, a ≤ a
  antisymm : ∀ a b : V, a ≤ b → b ≤ a → a = b
  lower : zero ≤ lam
  rh_iff : RH zeros ↔ lam = zero

/-- From Λ ≤ 0 and the cited 0 ≤ Λ, the hypothesis. -/
theorem rh_from_the_sign {V : Type} [LE V] (D : DBN V) (h : D.lam ≤
D.zero) : RH D.zeros :=
  D.rh_iff.mpr (D.antisymm _ _ h D.lower)

/-- The hypothesis is equivalent to Λ ≤ 0. -/
theorem rh_iff_the_sign {V : Type} [LE V] (D : DBN V) : RH D.zeros ↔
D.lam ≤ D.zero :=
  (fun h => by rw [D.rh_iff.mp h]; exact D.refl _, rh_from_the_sign D)

/-- On one zero set, Weil positivity and Λ ≤ 0 are equivalent. -/
theorem spends_are_one {V : Type} [LE V] (P : PrimeAct) (D : DBN V) (same
: D.zeros = P.E.zeros) :
    (D.lam ≤ D.zero) ↔ WeilPositive P.E :=
  (rh_iff_the_sign D).symm.trans (by rw [same]; exact (spend_is_the_line
  P.E).symm)

/-- Λ = 0 is realized over W1 and Λ = 1 over W2f: the sign is a property
of the world. -/
theorem sign_realized_both_ways :
    (∃ D : DBN Int, D.zeros = W1 ∧ D.lam = D.zero) ∧
    (∃ D : DBN Int, D.zeros = W2f ∧ ¬ D.lam ≤ D.zero) :=
  (⟨⟨W1, 0, 0, fun a => Int.le_refl a, fun a b => Int.le_antisymm,
    Int.le_refl 0,
      (fun _ => rfl, fun _ => W1_rh)⟩, rfl, rfl⟩,
   ⟨⟨W2f, 1, 0, fun a => Int.le_refl a, fun a b => Int.le_antisymm, by
    decide,
      (fun h => absurd h W2f_not_rh, fun h => absurd h (by decide))⟩,
      rfl, by decide⟩)

/-- A self-grounding supply of 0 < 1, with the unit act. -/
def rootSelfGrounding : SelfGrounding ((0 : Int) < 1) := (Unit, (), fun _
=> by decide)

theorem root_has_an_act : Nonempty rootSelfGrounding.Act := ⟨()⟩

/-- No self-grounding supply of the hypothesis on W2f has an act: W2f
fails the hypothesis. -/
theorem line_not_self_grounding : ¬ ∃ G : SelfGrounding (RH W2f),
Nonempty G.Act :=
  fun ⟨G, ⟨a⟩⟩ => W2f_not_rh (G.instances a)

/-- Section XIV, whole. -/
theorem the_vestigial_posit :
    (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z) ∧
    (∀ A : World → Prop, A W2f → ¬ Forces A) ∧
    (∀ L : LiStream, (∀ n, 1 ≤ n → L.nonneg n) ↔ RH L.zeros) ∧
    (∀ N, ∃ L : LiStream, (∀ n, n < N → L.nonneg n) ∧ ¬ RH L.zeros) ∧
    (∀ (V : Type) [LE V] (D : DBN V), RH D.zeros ↔ D.lam ≤ D.zero) ∧
    ((∃ D : DBN Int, D.zeros = W1 ∧ D.lam = D.zero) ∧
      (∃ D : DBN Int, D.zeros = W2f ∧ ¬ D.lam ≤ D.zero)) ∧
    (∀ (V : Type) [LE V] (P : PrimeAct) (D : DBN V), D.zeros = P.E.zeros →
      ((D.lam ≤ D.zero) ↔ WeilPositive P.E)) ∧
    Nonempty rootSelfGrounding.Act ∧
    ¬ ∃ G : SelfGrounding (RH W2f), Nonempty G.Act :=
  (rh_is_the_weakest_forcing_premise, weaker_at_the_twin_never_forces,
  li_is_the_bit_stream,
   li_prefix_never_forces, fun _ _ D => rh_iff_the_sign D,
   sign_realized_both_ways,
   fun _ _ P D same => spends_are_one P D same, root_has_an_act,
   line_not_self_grounding)


/-! ## XV · Four faces in time, exactly one keyed -/
```

```
/-- A world in time: the zero set at each instant. -/
abbrev TWorld := Nat → World

/-- The same locus at every instant: the line. -/
def SameLocus (W : TWorld) : Prop := ∀ t z, W t z → (fold z = z ↔ onLine z)
/-- The fold acts at every instant. -/
def FoldInvT (W : TWorld) : Prop := ∀ t, FoldClosed (W t)
/-- The record at every instant lands on the line and forgets the side. -/
def RecordT (W : TWorld) : Prop := ∀ t z, W t z → onLine (reg z) ∧ reg (fold z) = reg z
/-- The arrow at every instant: the free bit is present. -/
def ArrowT (_W : TWorld) : Prop := ∀ _t : Nat, ∃ f : Nat → Nat, ∀ a, f a = a
/-- Timeless: whether the zeros sit on the line does not vary in time. -/
def TimelessT (W : TWorld) : Prop := ∀ t s, RH (W t) ↔ RH (W s)
/-- Bound: the closure at every instant. -/
def BoundT (W : TWorld) : Prop := ∀ t, RH (W t)

/-- The on-line world, constant in time, and the twin, constant in time. -/
abbrev WOn : TWorld := fun _ => W1
abbrev WTwin : TWorld := fun _ => W2f

/-- The fold face holds on every world in time. -/
theorem fold_face_keyless (W : TWorld) : SameLocus W := fun _ z _ => fold_fixed_iff z
theorem fold_face_on_twin : FoldInvT WTwin := fun _ => W2f_closed
theorem fold_face_on_line : FoldInvT WOn := fun _ => W1_closed

/-- The record face holds on every world in time. -/
theorem record_face_keyless (W : TWorld) : RecordT W := fun _ _ _ => (rfl, rfl)

/-- The arrow face holds on every world in time. -/
theorem arrow_face_keyless (W : TWorld) : ArrowT W := fun _ => arrow_exists Nat

/-- The closure face holds on WOn. -/
theorem closure_face_on_line : BoundT WOn := fun _ => W1_rh
theorem closure_face_fails_on_twin : ¬ BoundT WTwin := fun h => W2f_not_rh (h 0)

/-- WTwin carries the fold, the record and the arrow at every instant and is timeless, and it is
    not bound: of the four faces only the closure is keyed. -/
theorem exactly_one_face_is_keyed :
    (SameLocus WTwin ∧ FoldInvT WTwin ∧ RecordT WTwin ∧ ArrowT WTwin ∧ TimelessT WTwin) ∧
    ¬ BoundT WTwin ∧
    (SameLocus WOn ∧ FoldInvT WOn ∧ RecordT WOn ∧ ArrowT WOn ∧ TimelessT WOn) ∧ BoundT WOn :=
  ((fold_face_keyless _, fold_face_on_twin, record_face_keyless _, arrow_face_keyless _,
    fun _ _ => Iff.rfl), closure_face_fails_on_twin,
   (fold_face_keyless _, fold_face_on_line, record_face_keyless _, arrow_face_keyless _,
    fun _ _ => Iff.rfl), closure_face_on_line)

/-- The arrow, the fold and the record hold on WTwin at every instant while the line fails there:
    the arrow is not the line. -/
theorem arrow_is_not_the_line :
    ArrowT WTwin ∧ FoldInvT WTwin ∧ RecordT WTwin ∧ ¬ BoundT WTwin :=
  (arrow_face_keyless _, fold_face_on_twin, record_face_keyless _, closure_face_fails_on_twin)

/-- A timeless world is bound exactly when it is bound at instant 0. -/
theorem template_binds_iff_seed (W : TWorld) (u : TimelessT W) : BoundT W ↔ RH (W 0) :=
  (fun h => h 0, fun s t => (u 0 t).mp s)

/-- For a timeless world, bound at instant 0 is the hypothesis at instant 0; WTwin has the three
    keyless faces and fails the hypothesis at every instant. -/
theorem the_seed_is_the_spend :
    (∀ W : TWorld, TimelessT W → (BoundT W ↔ RH (W 0))) ∧
    TimelessT WTwin ∧ ArrowT WTwin ∧ FoldInvT WTwin ∧ RecordT WTwin ∧ ¬ RH (WTwin 0) :=
  (template_binds_iff_seed, fun _ _ => Iff.rfl, arrow_face_keyless _, fold_face_on_twin,
   record_face_keyless _, W2f_not_rh)

/-- Section XV, whole. -/
theorem the_template_plugged_in :
    ((SameLocus WTwin ∧ FoldInvT WTwin ∧ RecordT WTwin ∧ ArrowT WTwin ∧ TimelessT WTwin) ∧
      ¬ BoundT WTwin ∧
      (SameLocus WOn ∧ FoldInvT WOn ∧ RecordT WOn ∧ ArrowT WOn ∧ TimelessT WOn) ∧ BoundT WOn) ∧
    (ArrowT WTwin ∧ FoldInvT WTwin ∧ RecordT WTwin ∧ ¬ BoundT WTwin) ∧
    (∀ W : TWorld, TimelessT W → (BoundT W ↔ RH (W 0))) ∧
    (∀ W : TWorld, BoundT W → ∀ t, LeastErasure (W t)) :=
  (exactly_one_face_is_keyed, arrow_is_not_the_line, template_binds_iff_seed,
   fun W h t => (least_erasure_is_the_value (W t)).mpr (h t))


/-! ## XVI · The eigenstructure of the fold: the side bit and its calibration -/

/-- The displacement x − 1. -/
def oddPart (z : Pt) : Int := z.x - 1


/-- The fold negates the displacement and fixes the height. -/
theorem fold_negates_odd (z : Pt) : oddPart (fold z) = - oddPart z := by
  show 2 - z.x - 1 = -(z.x - 1)
  rw [Int.neg_sub, Int.sub_sub, Int.add_comm, ← Int.sub_sub]; rfl

theorem fold_keeps_even (z : Pt) : (fold z).t = z.t := rfl

/-- The fold acts as −1 on the displacement. -/
theorem residence_orientation_reversing (z : Pt) : oddPart (fold z) = (-1) * oddPart z := by
  rw [fold_negates_odd, Int.neg_one_mul]

/-- Registration sends the displacement to 0, keeps the height, and is idempotent. -/
theorem reg_kills_odd (_z : Pt) : oddPart (reg _z) = 0 := rfl
theorem reg_keeps_even (z : Pt) : (reg z).t = z.t := rfl
theorem reg_idempotent (z : Pt) : reg (reg z) = reg z := rfl

/-- A point is on the line exactly when its displacement is 0. -/
theorem onLine_iff_odd_zero (z : Pt) : onLine z ↔ oddPart z = 0 := Int.sub_eq_zero.symm

/-- The hypothesis: every zero has displacement 0. -/
theorem rh_iff_even_eigenspace (Z : World) : RH Z ↔ ∀ z, Z z → oddPart z = 0 :=
  (fun h z hz => (onLine_iff_odd_zero z).mp (h z hz), fun h z hz =>
   (onLine_iff_odd_zero z).mpr (h z hz))

/-- Every function of the registered point takes one value on a point and its fold-partner. -/
theorem record_blind_to_odd {β : Type} (g : Pt → β) (z : Pt) : g (reg (fold z)) = g (reg z) := rfl

/-- Off the line a point and its fold-partner have opposite nonzero displacements. -/
theorem odd_pair_opposite (z : Pt) (h : ¬ onLine z) :
    oddPart (fold z) = - oddPart z ∧ oddPart z ≠ 0 :=
  (fold_negates_odd z, fun e => h ((onLine_iff_odd_zero z).mpr e))

/-- The side bit: whether the displacement is negative. -/
def side (z : Pt) : Bool := decide (oddPart z < 0)

/-- Off the line the fold flips the side bit. -/
theorem side_odd_off_line (z : Pt) (h : ¬ onLine z) : side (fold z) = !side z := by
  have hne : oddPart z ≠ 0 := (odd_pair_opposite z h).2
  unfold side
  rw [fold_negates_odd]
  exact (Int.decLt (oddPart z) 0).byCases
    (fun hl =>
      have hp : ¬ (-(oddPart z) < 0) := fun c => Int.lt_irrefl _
        (Int.lt_trans c (Int.neg_pos_of_neg hl))
      by rw [decide_eq_true hl, decide_eq_false hp]; rfl)
    (fun hr =>
      have hp : -(oddPart z) < 0 :=
        Int.neg_neg_of_pos (Int.lt_iff_le_and_ne.mpr (Int.not_lt.mp hr, Ne.symm hne))
      by rw [decide_eq_false hr, decide_eq_true hp]; rfl)

/-- A function invariant under the fold differs from any function that the fold flips at some
    point. -/
theorem wall_on_the_chart (f d : Pt → Bool) (z : Pt) (he : ∀ w, f (fold w) = f w)
    (ho : d (fold z) ≠ d z) : f ≠ d := by
  intro h; subst h; exact ho (he z)

/-- No function of the registered point equals the side bit at an off-line point. -/
theorem record_never_reads_the_side (g : Pt → Bool) (z : Pt) (h : ¬ onLine z) :
    (fun w => g (reg w)) ≠ side :=
  wall_on_the_chart (fun w => g (reg w)) side z (fun _ => rfl)
    (by rw [side_odd_off_line z h]; cases side z <;> decide)

/-- Given the side bit at an off-line point, any function flipped by the fold at that point is
    fixed on the orbit by one calibration bit, which exists and is unique. -/
theorem colocation_is_a_calibration (d : Pt → Bool) (z : Pt) (h : ¬ onLine z)
    (hd : d (fold z) = !d z) :
    ∃ c : Bool, (d z = xor (side z) c ∧ d (fold z) = xor (side (fold z)) c) ∧
      ∀ c', (d z = xor (side z) c' ∧ d (fold z) = xor (side (fold z)) c') → c' = c := by
  have hs := side_odd_off_line z h
  refine ⟨xor (d z) (side z),
    (by cases side z <;> cases d z <;> rfl, by rw [hs, hd]; cases side z <;> cases d z <;> rfl), ?_⟩
  intro c' hc; obtain ⟨h1, -⟩ := hc
  generalize hsx : side z = sv at h1; generalize hdx : d z = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)

/-- Both calibration values are realized off the line, by the side bit and by its complement. -/
theorem calibration_realized_both_ways (z : Pt) (h : ¬ onLine z) :
    (side (fold z) = !side z ∧ side z = xor (side z) false) ∧
    ((!side (fold z)) = !(!side z) ∧ (!side z) = xor (side z) true) :=
  ⟨(side_odd_off_line z h, by cases side z <;> rfl),
   (by rw [side_odd_off_line z h], by cases side z <;> rfl)⟩
```

```

/-- A proposition implied by its own negation. -/
def SelfVerifyingP (P : Prop) : Prop := ¬P → P

/-- The hypothesis on a world implies its self-verification there. -/
theorem value_is_recursion (Z : World) (h : RH Z) : SelfVerifyingP (RH Z)
:= fun _ => h
theorem recursion_is_value (Z : World) (h : SelfVerifyingP (RH Z)) : ¬¬
RH Z := fun n => n (h n)

/-- Self-verification of the hypothesis holds on W1 and fails on W2f. -/
theorem recursion_on_the_line : SelfVerifyingP (RH W1) := fun _ => W1_rh
theorem recursion_fails_on_the_twin : ¬ SelfVerifyingP (RH W2f) := fun h
=> W2f_not_rh (h W2f_not_rh)

/-- Self-verification of 0 < 1. -/
theorem root_recursion : SelfVerifyingP ((0 : Int) < 1) := fun _ => by
decide

/-- Least erasure on a world implies self-verification of the hypothesis
there, and conversely up
    to double negation. -/
theorem least_erasure_is_the_recursion (Z : World) :
    (LeastErasure Z → SelfVerifyingP (RH Z)) ∧ (SelfVerifyingP (RH Z) →
    ¬¬ LeastErasure Z) :=
  (fun h => value_is_recursion Z ((least_erasure_is_the_value Z).mp h),
   fun h n => recursion_is_value Z h (fun r => n
   ((least_erasure_is_the_value Z).mpr r)))

/-- Section XVI, whole. -/
theorem the_mirror_at_minus_one :
    (∀ z, oddPart (fold z) = (-1) * oddPart z ∧ (fold z).t = z.t) ∧
    (∀ z, oddPart (reg z) = 0 ∧ (reg z).t = z.t ∧ reg (reg z) = reg z) ∧
    (∀ Z : World, RH Z ↔ ∀ z, Z z → oddPart z = 0) ∧
    (∀ (g : Pt → Bool) (z : Pt), ¬ onLine z → (fun w => g (reg w)) ≠
    side) ∧
    (∀ (d : Pt → Bool) (z : Pt), ¬ onLine z → d (fold z) = !d z →
      ∃ c : Bool, (d z = xor (side z) c ∧ d (fold z) = xor (side (fold
      z)) c) ∧
        ∀ c', (d z = xor (side z) c' ∧ d (fold z) = xor (side (fold z))
        c') → c' = c) ∧
    (SelfVerifyingP (RH W1) ∧ ¬ SelfVerifyingP (RH W2f) ∧ SelfVerifyingP
    ((0 : Int) < 1)) ∧
    (∀ Z : World, (LeastErasure Z → SelfVerifyingP (RH Z)) ∧
    (SelfVerifyingP (RH Z) → ¬¬ LeastErasure Z)) :=
  (fun z => (residence_orientation_reversing z, fold_keeps_even z),
   fun z => (reg_kills_odd z, reg_keeps_even z, reg_idempotent z),
   rh_iff_even_eigenspace, record_never_reads_the_side,
   colocation_is_a_calibration,
   (recursion_on_the_line, recursion_fails_on_the_twin, root_recursion),
   least_erasure_is_the_recursion)

/-! ## XVII · Admissible coalitions, the certified height, the heat flow,
the Liouville arrow, five readings, the reader index -/

/-! ### XVII.a · Admissible coalitions -/

/-- A premise is admissible when it is keyless or a reading of the
record. -/
def Admissible (A : World → Prop) : Prop := Keyless A ∨ RespectsRecord A

/-- An admissible premise that holds on W1 holds on W2f. -/
theorem admissible_transfers (A : World → Prop) (h : Admissible A) (h1 : A
W1) : A W2f := by
  rcases h with k | r
  · exact k W2f
  · exact (r W1 W2f same_record_f).mp h1

/-- Three admissible premises true on W1 are true on W2f, so their
conjunction forces nothing. -/
theorem no_admissible_triad_forces (A B C : World → Prop)
    (hA : Admissible A) (hB : Admissible B) (hC : Admissible C) (h : A W1
    ∧ B W1 ∧ C W1) :
    ¬ Forces (fun Z => A Z ∧ B Z ∧ C Z) :=
  fun f => W2f_not_rh (f W2f W2f_closed
    (admissible_transfers A hA h.1, admissible_transfers B hB h.2.1,
    admissible_transfers C hC h.2.2))

/-- A list of admissible premises true on W1 forces nothing. -/
theorem no_admissible_coalition_forces (L : List (World → Prop))
    (hL : ∀ A ∈ L, Admissible A) (h1 : ∀ A ∈ L, A W1) :
    ¬ Forces (fun Z => ∀ A ∈ L, A Z) :=
  fun f => W2f_not_rh (f W2f W2f_closed (fun A hA => admissible_transfers
  A (hL A hA) (h1 A hA)))

/-- Least erasure is not admissible: it fails on W2f and is not a reading
of the record. -/
theorem spend_is_not_admissible : ¬ Admissible LeastErasure := by
  intro h
  rcases h with k | r
  · exact W2f_not_rh ((least_erasure_is_the_value W2f).mp (k W2f))
  · exact least_erasure_reads_past_the_record_free r

/-- Three predicates on a point: the freedom axis, always true; the seat
axis, the point is fixed
    by the fold; the collapse axis, the displacement equals its own
    negative. -/
def freedomAxis (_ : Pt) : Prop := True
def ramAxis (z : Pt) : Prop := fold z = z
def collapseAxis (z : Pt) : Prop := oddPart z = - oddPart z

/-- All three axes hold at a point exactly when it lies on the line. -/
theorem three_axis_lock (z : Pt) : (freedomAxis z ∧ ramAxis z ∧
collapseAxis z) ↔ onLine z := by
  constructor
  · intro (_, h, _)
    exact (fold_fixed_iff z).mp h
  · intro h
    refine (trivial, (fold_fixed_iff z).mpr h, ?_)
    have e : oddPart z = 0 := (onLine_iff_odd_zero z).mp h
    unfold collapseAxis
    omega

/-- The freedom axis holds at an off-line point. -/
theorem freedom_alone_open : freedomAxis (0, 14) ∧ ¬ onLine (0, 14) :=
  (trivial, by decide)

/-! ### XVII.b · The certified height -/

/-- The zeros certified on the line up to height T. -/
def LineBelow (T : Nat) : World := fun z => z.x = 1 ∧ z.t ≤ T
/-- The same zeros, with one off-line pair just above the certified
height. -/
def TwinAbove (T : Nat) : World := fun z => (z.x = 1 ∧ z.t ≤ T) ∨ (z =
(0, T+1) ∨ z = (2, T+1))

/-- For every T the world certified to T lies on the line, and a
fold-closed world agrees with it
    at every height up to T and carries an off-line pair above T. The
    certificate of Platt and
    Trudgian (2021) to height $3 \cdot 10^{12}$ is such a T. -/
theorem certified_height_never_forces (T : Nat) :
    RH (LineBelow T) ∧ FoldClosed (TwinAbove T) ∧ ¬ RH (TwinAbove T) ∧
    ∀ z, z.t ≤ T → (TwinAbove T z ↔ LineBelow T z) := by
  refine (fun z hz => hz.1, ?_, ?_, ?_)
  · intro z hz
    rcases hz with (h1, h2) | h | h
    · exact Or.inl (show 2 - z.x = 1 by omega, h2)
    · subst h
      exact Or.inr (Or.inr rfl)
    · subst h
      exact Or.inr (Or.inl rfl)
  · intro h
    have h0 : (0 : Int) = 1 := h (0, T+1) (Or.inr (Or.inl rfl))
    exact absurd h0 (by decide)
  · intro z hz
    constructor
    · intro h
      rcases h with h | h | h
      · exact h
      · subst h
        have : T + 1 ≤ T := hz
        omega
      · subst h
        have : T + 1 ≤ T := hz
        omega
    · intro h
      exact Or.inl h

/-- Below T the certified world and its twin have the same record. -/
theorem certified_worlds_share_the_record_below (T : Nat) (r : Pt) (hr :
r.t ≤ T) :
    recordOf (LineBelow T) r ↔ recordOf (TwinAbove T) r := by
  constructor
  · intro (s, hs, hsr)
    exact (s, Or.inl hs, hsr)
  · intro (s, hs, hsr)
    rcases hs with h | h | h
    · exact (s, h, hsr)
    · subst h
      have e : (T + 1 : Nat) = r.t := congrArg Pt.t hsr
      omega
    · subst h
      have e : (T + 1 : Nat) = r.t := congrArg Pt.t hsr
      omega

/-! ### XVII.c · The heat flow on polynomials: under $\partial_t H = -\partial_x^2 H$ the
constant term of $z^2 + bz + c$ moves to $c - 2t$ and the discriminant rises
by $8t$ -/

namespace HeatFlow

theorem sq_nonneg (x : Int) : 0 ≤ x * x := by
  cases Int.le_total 0 x with
  | inl h => exact Int.mul_nonneg h h
  | inr h =>
    have hn : 0 ≤ -x := by omega
    have := Int.mul_nonneg hn hn
    rw [Int.neg_mul_neg] at this
    exact this

theorem cube_diff (u k : Int) :
    u*u*u - (u-k)*(u-k)*(u-k) = 3*(u*u*k) - 3*(u*k*k) + k*k*k := by
  simp only [Int.sub_mul, Int.mul_sub, Int.mul_assoc]
  simp only [Int.mul_comm k u, Int.mul_left_comm k u]
  omega

/-- Discriminant of $z^2 + bz + c$. -/
def disc (b c : Int) : Int := b*b - 4*c
/-- The flow on the constant term: $\exp(-tD^2)(z^2 + bz + c) = z^2 + bz + (c - 2t)$. -/
def flowC (c t : Int) : Int := c - 2*t
/-- Real-rooted at time t: the flowed discriminant is nonnegative. -/
def RR (b c t : Int) : Prop := 0 ≤ disc b (flowC c t)
instance (b c t : Int) : Decidable (RR b c t) := inferInstanceAs
(Decidable (0 ≤ disc b (flowC c t)))

```

```
/-- disc(t) = disc(0) + 8t. -/
theorem disc_flow (b c t : Int) : disc b (flowC c t) = disc b c + 8*t :=
by
  unfold disc flowC; omega

/-- Real-rootedness is preserved forward in time. -/
theorem forward_preserves (b c s t : Int) (hst : s ≤ t) (h : RR b c s) :
RR b c t := by
  unfold RR at *; rw [disc_flow] at *; omega

/-- Every quadratic is real-rooted after some finite time. -/
theorem erasure_finite (b c : Int) : ∃ t, 0 ≤ t ∧ RR b c t :=
  ((disc b c).natAbs, by omega, by unfold RR; rw [disc_flow]; omega)

/-- The least erasure time, Λ of the quadratic: 0 if already real-rooted,
else ⌈−disc/8⌉. -/
def lam (b c : Int) : Int := if 0 ≤ disc b c then 0 else (7 - disc b c) /
8

theorem lam_nonneg (b c : Int) : 0 ≤ lam b c := by
  unfold lam; split <;> omega

/-- At time Λ the quadratic is real-rooted. -/
theorem real_at_lam (b c : Int) : RR b c (lam b c) := by
  unfold RR lam; rw [disc_flow]; split <;> omega

/-- If Λ > 0, the quadratic is not real-rooted at time Λ − 1. -/
theorem lam_least (b c : Int) (h : 0 < lam b c) : ¬ RR b c (lam b c - 1)
:= by
  unfold RR; rw [disc_flow]; unfold lam at *; split at * <;> omega

/-- Λ = 0 exactly when the quadratic is real-rooted at time 0. -/
theorem lam_zero_iff_real (b c : Int) : lam b c = 0 ↔ RR b c 0 := by
  unfold RR lam; rw [disc_flow]; split <;> omega

/-- For z² + c the squared imaginary part of the roots is max(c − 2t, 0):
de Bruijn's bound holds
    with equality. -/
def imSq (c t : Int) : Int := max (flowC c t) 0
theorem de_bruijn_tight (c t : Int) (ht : 0 ≤ t) : imSq c t = max (imSq c
0 - 2*t) 0 := by
  unfold imSq flowC; omega

/-- For z² + 2k the roots are ±√(2t − 2k): at t = k they meet at 0. -/
def rootSq (c t : Int) : Int := 2*t - c
theorem collision (k t : Int) :
    rootSq (2*k) k = 0 ∧ (t < k → rootSq (2*k) t < 0) ∧ (k < t → 0 <
    rootSq (2*k) t) ∧
    rootSq (2*k) (t+1) = rootSq (2*k) t + 2 := by
  unfold rootSq; omega

/-- Discriminant of z³ + pz + q. -/
def disc3 (p q : Int) : Int := -4*(p*p*p) - 27*(q*q)
/-- The flow on the linear coefficient: exp(−tD²)(z³ + pz + q) = z³ + (p −
6t)z + q. -/
def flowP (p t : Int) : Int := p - 6*t

/-- One time step raises the cubic discriminant by 72((u − 3)² + 3), u
the current linear
    coefficient. -/
theorem disc3_step (p q t : Int) :
    disc3 (flowP p (t+1)) q - disc3 (flowP p t) q = 72*((p - 6*t - 3)*(p -
    6*t - 3) + 3) := by
  have key : ∀ u k : Int, u*u*u - (u-k)*(u-k)*(u-k) = 3*(u*u*k) -
  3*(u*k*k) + k*k*k := by
    intro u k
    simp only [Int.sub_mul, Int.mul_sub, Int.mul_assoc]
    simp only [Int.mul_comm k u, Int.mul_left_comm k u]
    omega
  have h := key (p - 6*t) 6
  have e0 : flowP p (t+1) = (p - 6*t) - 6 := by unfold flowP; omega
  have e1 : (p - 6*t)*(p - 6*t)*6 = 6*((p - 6*t)*(p - 6*t)) :=
  Int.mul_comm _ _
  have e2 : (p - 6*t)*6*6 = 36*(p - 6*t) := by rw [Int.mul_assoc,
  Int.mul_comm]; rfl
  have e3 : (p - 6*t - 3)*(p - 6*t - 3) = (p - 6*t)*(p - 6*t) - 6*(p -
  6*t) + 9 := by
    generalize p - 6*t = u
    simp only [Int.sub_mul, Int.mul_sub]; rw [Int.mul_comm 3 u]; omega
  have e4 : (6:Int)*6*6 = 216 := by decide
  rw [e1, e2, e4] at h
  unfold disc3; rw [e0, e3]
  have e5 : flowP p t = p - 6*t := rfl
  rw [e5]
  omega

/-- The cubic discriminant strictly increases with time. -/
theorem disc3_strict (p q t : Int) : disc3 (flowP p t) q < disc3 (flowP p
(t+1)) q := by
  have h := disc3_step p q t
  have := sq_nonneg (p - 6*t - 3)
  omega

/-- Over any number of steps the cubic discriminant never falls. -/
theorem disc3_monotone_rec (p q : Int) : ∀ n : Nat, disc3 (flowP p 0) q ≤
disc3 (flowP p n) q := by
  intro n
  induction n with
  | zero => exact Int.le_refl _
  | succ n ih =>
    have st := disc3_strict p q (n : Int)
    have e : ((n + 1 : Nat) : Int) = (n : Int) + 1 := by omega
    rw [e]; omega

/-- disc3_monotone_rec, restated. -/
theorem disc3_monotone (p q : Int) (n : Nat) : disc3 (flowP p 0) q ≤
disc3 (flowP p n) q :=
  disc3_monotone_rec p q n

theorem cubic_forward_preserves (p q : Int) (h : 0 ≤ disc3 (flowP p 0) q)
(n : Nat) :
    0 ≤ disc3 (flowP p n) q := Int.le_trans h (disc3_monotone_rec p q n)

/-- The flow is injective on the constant term: the polynomial at time t
determines the polynomial
    at time 0. -/
theorem flow_injective (c c' t : Int) (h : flowC c t = flowC c' t) : c =
c' := by
  unfold flowC at h; omega

/-- At every positive time there are two quadratics real-rooted at t, one
real-rooted at 0 and one
    not. -/
theorem certificate_two_worlds (t : Int) (ht : 0 < t) :
    RR 0 0 t ∧ RR 0 t t ∧ RR 0 0 0 ∧ ¬ RR 0 t 0 := by
  unfold RR; simp only [disc_flow]; unfold disc; omega

/-- No function of real-rootedness at time t returns real-rootedness at
time 0 on every quadratic. -/
theorem certificate_decides_nothing (t : Int) (ht : 0 < t) :
    ¬ ∃ g : Bool → Bool, ∀ b c : Int, g (decide (RR b c t)) = decide (RR b
    c 0) := by
  intro (g, hg)
  have (h1, h2, h3, h4) := certificate_two_worlds t ht
  have a := hg 0 0
  have b := hg 0 t
  rw [decide_eq_true h1, decide_eq_true h3] at a
  rw [decide_eq_true h2, decide_eq_false h4] at b
  rw [a] at b
  exact Bool.noConfusion b

/-- For every t > 0 some quadratic is real-rooted from t on and not at 0,
with Λ > 0. -/
theorem certification_never_forces (t : Int) (ht : 0 < t) :
    ∃ c : Int, (∀ s, t ≤ s → RR 0 c s) ∧ ¬ RR 0 c 0 ∧ 0 < lam 0 c :=
  (t, fun s hs => forward_preserves 0 t t s hs (certificate_two_worlds t
  ht).2.1,
   (certificate_two_worlds t ht).2.2.2,
   by unfold lam disc; split <;> omega)

/-- A double root is real-rooted at 0 with Λ = 0, and not real-rooted at
any earlier time. -/
theorem zero_slack (a : Int) :
    RR (2*a) (a*a) 0 ∧ lam (2*a) (a*a) = 0 ∧ ∀ s, 0 < s → ¬ RR (2*a)
    (a*a) (-s) := by
  have hd : disc (2*a) (a*a) = 0 := by
    unfold disc; rw [Int.mul_assoc, Int.mul_left_comm a 2 a]; omega
  refine ⟨?_, ?_, ?_⟩
  · unfold RR; rw [disc_flow, hd]; omega
  · unfold lam; rw [hd]; simp
  · intro s hs; unfold RR; rw [disc_flow, hd]; omega

/-- A quadratic real-rooted at time 1 may or may not be real-rooted at
time 0: z² − 2 is, z² + 2 is
    not. -/
theorem backward_not_forced : RR 0 (-2) 1 ∧ RR 0 2 1 ∧ RR 0 (-2) 0 ∧ ¬ RR
0 2 0 := by
  unfold RR; simp only [disc_flow]; unfold disc; decide

/-- The largest Λ over a family of quadratics. -/
def famLam : List (Int × Int) → Int
  | [] => 0
  | (b, c) :: r => max (lam b c) (famLam r)

theorem famLam_nonneg_rec : ∀ F : List (Int × Int), 0 ≤ famLam F := by
  intro F
  induction F with
  | nil => exact Int.le_refl 0
  | cons m r ih =>
    obtain (b, c) := m
    unfold famLam; omega

theorem family_least_erasure_rec :
    ∀ F : List (Int × Int), famLam F = 0 ↔ ∀ m ∈ F, RR m.1 m.2 0 := by
  intro F
  induction F with
  | nil => simp [famLam]
  | cons m r ih =>
    obtain (b, c) := m
    have hn := famLam_nonneg_rec r
    have hl := lam_nonneg b c
    have hz := lam_zero_iff_real b c
    constructor
    · intro h m hm
      unfold famLam at h
      cases hm with
      | head => exact hz.mp (by omega)
      | tail _ hm' => exact ih.mp (by omega) m hm'
    · intro h
      have h1 : lam b c = 0 := hz.mpr (h (b, c) (List.Mem.head r))
      have h2 : famLam r = 0 := ih.mpr (fun m hm => h m (List.Mem.tail _
      hm))
      unfold famLam; omega

/-- The family's Λ is 0 exactly when every member is real-rooted at time
0. -/
```

```
theorem family_least_erasure (F : List (Int × Int)) : famLam F = 0 ↔ ∀ m ∈ F, RR m.1 m.2 0 :=
  family_least_erasure_rec F

end HeatFlow

/-- Real-rootedness at time 0 is exactly Λ ≤ 0, since 0 ≤ Λ. -/
theorem heat_bit_is_one_inequality (b c : Int) : HeatFlow.RR b c 0 ↔ HeatFlow.lam b c ≤ 0 := by
  have h1 := HeatFlow.lam_nonneg b c
  have h2 := HeatFlow.lam_zero_iff_real b c
  constructor
  · intro h
    have := h2.mpr h
    omega
  · intro h
    exact h2.mp (by omega)

/-- Over a family, every member is real-rooted at time 0 exactly when the family's Λ ≤ 0. -/
theorem family_bit_is_one_inequality (F : List (Int × Int)) :
    (∀ m ∈ F, HeatFlow.RR m.1 m.2 0) ↔ HeatFlow.famLam F ≤ 0 := by
  have h1 := HeatFlow.famLam_nonneg_rec F
  have h2 := HeatFlow.family_least_erasure F
  constructor
  · intro h
    have := h2.mpr h
    omega
  · intro h
    exact h2.mp (by omega)

/-- The de Bruijn–Newman structure and the quadratic model satisfy the same equivalences, the model
    with its lower bound proved. -/
theorem heat_model_reads_as_the_carrier {V : Type} [LE V] (D : DBN V) (b c : Int) :
    (RH D.zeros ↔ D.lam ≤ D.zero) ∧ (HeatFlow.RR b c 0 ↔ HeatFlow.lam b c ≤ 0) ∧
    (0 ≤ HeatFlow.lam b c) :=
  (rh_iff_the_sign D, heat_bit_is_one_inequality b c, HeatFlow.lam_nonneg b c)

/-! ### XVII.d · The Liouville arrow λ(n) = (−1)^Ω(n), computed -/

namespace Liouville

/-- Ω, the number of prime factors with multiplicity, by trial division. -/
def omegaF : Nat → Nat → Nat → Nat
  | 0, _, _ => 0
  | fuel + 1, n, d =>
    if n ≤ 1 then 0
    else if d * d > n then 1
    else if n % d = 0 then 1 + omegaF fuel (n / d) d
    else omegaF fuel n (d + 1)
def bigOmega (n : Nat) : Nat := omegaF (2 * n + 2) n 2
/-- The arrow λ(n) = (−1)^Ω(n). -/
def lam (n : Nat) : Int := if bigOmega n % 2 = 0 then 1 else -1

/-- λ(ab) = λ(a)λ(b) on 1..40 × 1..40. -/
theorem lam_mult : ((List.range 40).all fun a => (List.range 40).all fun b =>
      lam ((a+1)*(b+1)) == lam (a+1) * lam (b+1)) = true := by decide

/-- λ(p) = −1 for every prime p below 100. -/
theorem lam_flips_at_primes :
    ([2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97].all fun p => lam p == -1) = true :=
    by
  decide

def L : Nat → Int
  | 0 => 0
  | n + 1 => L n + lam (n + 1)

/-- L(x) = Σ_{n ≤ x} λ(n) ≤ 0 for 2 ≤ x ≤ 200. The inequality fails at x = 906,150,257 (Haselgrove
    1958; Tanaka 1980). -/
theorem polya_holds_to_200 : ((List.range 199).all fun k => decide (L (k + 2) ≤ 0)) = true := by decide

end Liouville

/-- λ is the sign arrow of the computed count Ω. -/
theorem liouville_is_the_sign_arrow (n : Nat) : Liouville.lam n = signArrow (Liouville.bigOmega n) := rfl

/-- A zero set with the faithfulness of λ: the hypothesis holds exactly when L(x) = O(x^{1/2+ε})
    for every ε > 0. Carried as a hypothesis (Landau 1899). -/
structure LiouvilleFace where
  zeros : World
  faithful : Prop
  faithful_iff : RH zeros ↔ faithful

theorem rh_from_faithful (F : LiouvilleFace) (h : F.faithful) : RH F.zeros := F.faithful_iff.mpr h
theorem faithful_is_the_bit (F : LiouvilleFace) : F.faithful ↔ RH F.zeros := F.faithful_iff.symm

/-- On one zero set, Weil positivity, Λ ≤ 0, the non-negativity of the Li stream and the
    faithfulness of λ are equivalent, each cited equivalence entering as a field. -/
theorem the_posit_in_every_coordinate {V : Type} [LE V] (P : PrimeAct) (D : DBN V) (Li : LiStream)
    (F : LiouvilleFace) (hD : D.zeros = P.E.zeros) (hL : Li.zeros = P.E.zeros)
    (hF : F.zeros = P.E.zeros) :
    (WeilPositive P.E ↔ D.lam ≤ D.zero) ∧ (WeilPositive P.E ↔ ∀ n, 1 ≤ n → Li.nonneg n) ∧
    (WeilPositive P.E ↔ F.faithful) := by
  have hW : WeilPositive P.E ↔ RH P.E.zeros := spend_is_the_line P.E
  refine ⟨?_, ?_, ?_⟩
  · rw [hW, ← hD]
    exact rh_iff_the_sign D
  · rw [hW, ← hL]
    exact (li_is_the_bit_stream Li).symm
  · rw [hW, ← hF]
    exact F.faithful_iff

/-! ### XVII.e · Five readings, one proposition -/

/-- Lossless registration: every zero is fixed by registration. -/
def Lossless (Z : World) : Prop := ∀ z, Z z → reg z = z
/-- Every zero has displacement 0. -/
def DepthZero (Z : World) : Prop := ∀ z, Z z → oddPart z = 0
/-- The Li mode of a zero, in the chart's own scale: |ρ − 1|² and |ρ|² doubled. -/
def N1 (z : Pt) : Int := (z.x - 2)*(z.x - 2) + 4*((z.t : Int)*(z.t : Int))
def N0 (z : Pt) : Int := z.x*z.x + 4*((z.t : Int)*(z.t : Int))
/-- Stability: no Li mode grows. -/
def Stable (Z : World) : Prop := ∀ z, Z z → N1 z ≤ N0 z

theorem modes (z : Pt) : N1 z - N0 z = 4 - 4 * z.x := by
  simp only [N1, N0, Int.sub_mul, Int.mul_sub]
  omega

/-- Registration fixes a zero exactly when the zero lies on the line. -/
theorem lossless_iff_rh (Z : World) : Lossless Z ↔ RH Z := by
  constructor
  · intro h z hz
    have e := congrArg Pt.x (h z hz)
    exact e.symm
  · intro h z hz
    exact reg_fixes_line z (h z hz)

/-- On a fold-closed world, stability of every Li mode is the line: a mode below the line grows,
    and its fold-partner stands above. -/
theorem stable_iff_rh (Z : World) (hZ : FoldClosed Z) : Stable Z ↔ RH Z := by
  constructor
  · intro h z hz
    have a := h z hz
    have b := h (fold z) (hZ z hz)
    have ea := modes z
    have eb := modes (fold z)
    have ex : (fold z).x = 2 - z.x := rfl
    rw [ex] at eb
    show z.x = 1
    omega
  · intro h z hz
    have e : z.x = 1 := h z hz
    have := modes z
    omega

/-- On every fold-closed world the line, lossless registration, displacement zero, no left zero,
    the stability of every Li mode and least erasure are equivalent. -/
theorem five_readings_agree (Z : World) (hZ : FoldClosed Z) :
    (RH Z ↔ Lossless Z) ∧ (RH Z ↔ DepthZero Z) ∧ (RH Z ↔ ¬ Left Z) ∧
    (RH Z ↔ Stable Z) ∧ (RH Z ↔ LeastErasure Z) :=
  ⟨(lossless_iff_rh Z).symm, rh_iff_even_eigenspace Z, rh_iff_no_left Z hZ,
   (stable_iff_rh Z hZ).symm, (least_erasure_is_the_value Z).symm⟩

/-- And no record-respecting reading returns any of the five, since each is the hypothesis. -/
theorem no_record_reading_returns_any_face (g : World → Prop) (hg : RespectsRecord g) :
    ¬ (∀ Z, FoldClosed Z → (g Z ↔ Lossless Z)) ∧ ¬ (∀ Z, FoldClosed Z → (g Z ↔ Stable Z)) := by
  constructor
  · intro h
    exact record_decides_nothing_free g hg (fun Z hZ => (h Z hZ).trans (lossless_iff_rh Z))
  · intro h
    exact record_decides_nothing_free g hg (fun Z hZ => (h Z hZ).trans (stable_iff_rh Z hZ))

/-! ### XVII.f · The supply has no reader index -/

/-- A proposition indexed by a type of readers, constant. -/
def Requirement (Reader : Type) (V : Prop) : Reader → Prop := fun _ => V

/-- Every reader faces the same proposition. -/
theorem same_for_every_reader {Reader : Type} (V : Prop) (a b : Reader) :
    Requirement Reader V a ↔ Requirement Reader V b := Iff.rfl

/-- A requirement that differs between two readers is not the fixed value. -/
theorem no_private_bit {Reader : Type} (V : Prop) (R : Reader → Prop) (hR : ∀ r, R r ↔ V)
    (a b : Reader) : R a ↔ R b := (hR a).trans (hR b).symm

/-- A reader-indexed requirement equal to Weil positivity for every reader is the same for all
```

```
    readers. -/
theorem spend_has_no_reader {Reader : Type} (P : PrimeAct) (R : Reader → Prop)
    (hR : ∀ r, R r ↔ WeilPositive P.E) (a b : Reader) : R a ↔ R b :=
  no_private_bit (WeilPositive P.E) R hR a b

/-! ### XVII.g · Section XVII, whole -/

/-- Section XVII, whole. -/
theorem the_harvest :
    (∀ (L : List (World → Prop)), (∀ A ∈ L, Admissible A) → (∀ A ∈ L, A W1) →
      ¬ Forces (fun Z => ∀ A ∈ L, A Z)) ∧
    ¬ Admissible LeastErasure ∧
    (∀ T : Nat, RH (LineBelow T) ∧ FoldClosed (TwinAbove T) ∧ ¬ RH (TwinAbove T)) ∧
    (∀ b c : Int, (HeatFlow.RR b c 0 ↔ HeatFlow.lam b c ≤ 0) ∧ 0 ≤ HeatFlow.lam b c) ∧
    (∀ t : Int, 0 < t → ¬ ∃ g : Bool → Bool, ∀ b c : Int,
      g (decide (HeatFlow.RR b c t)) = decide (HeatFlow.RR b c 0)) ∧
    (∀ n : Nat, Liouville.lam n = signArrow (Liouville.bigOmega n)) ∧
    (∀ Z : World, FoldClosed Z →
      (RH Z ↔ Lossless Z) ∧ (RH Z ↔ DepthZero Z) ∧ (RH Z ↔ ¬ Left Z) ∧
      (RH Z ↔ Stable Z) ∧ (RH Z ↔ LeastErasure Z)) ∧
    (∀ {Reader : Type} (P : PrimeAct) (R : Reader → Prop),
      (∀ r, R r ↔ WeilPositive P.E) → ∀ a b : Reader, R a ↔ R b) :=
  ⟨no_admissible_coalition_forces, spend_is_not_admissible,
   fun T => ((certified_height_never_forces T).1,
   (certified_height_never_forces T).2.1,
     (certified_height_never_forces T).2.2.1),
   fun b c => ⟨heat_bit_is_one_inequality b c, HeatFlow.lam_nonneg b c⟩,
   HeatFlow.certificate_decides_nothing, liouville_is_the_sign_arrow,
   five_readings_agree,
   fun P R hR a b => spend_has_no_reader P R hR a b⟩


/-! ## XVIII · A theory placed on three strata; the root; the two can'ts; the round trip; the ladder blocked -/

/-! ### XVIII.a · Placement: computation, provability, truth -/

/-- A theory: its sentences, what it proves, what finite computation settles, what holds in its
    intended model. -/
structure Theory where
  Sent  : Type
  Prov  : Sent → Prop
  Comp  : Sent → Prop
  True_ : Sent → Prop

/-- Σ₁-completeness, abstractly: what finite computation settles, the theory proves. -/
def RungOnLadder (T : Theory) : Prop := ∀ s, T.Comp s → T.Prov s
/-- Soundness: what the theory proves holds in its intended model. -/
def Sound (T : Theory) : Prop := ∀ s, T.Prov s → T.True_ s

/-- Under Σ₁-completeness and soundness: what computation settles the theory proves, what the
    theory proves holds, and what computation settles holds. -/
theorem placement (T : Theory) (hR : RungOnLadder T) (hS : Sound T) :
    (∀ s, T.Comp s → T.Prov s) ∧ (∀ s, T.Prov s → T.True_ s) ∧ (∀ s, T.Comp s → T.True_ s) :=
  ⟨hR, hS, fun s h => hS s (hR s h)⟩

/-- An unsound theory: it proves everything and its model refutes half of it. -/
def unsound : Theory := ⟨Bool, fun _ => True, fun _ => False, fun b => b = true⟩

/-- The unsound theory is Σ₁-complete and not sound. -/
theorem soundness_is_load_bearing : RungOnLadder unsound ∧ ¬ Sound unsound := by
  refine ⟨fun _ h => h.elim, fun h => ?_⟩
  have := h false trivial
  cases this

/-- A sound incomplete theory: true on both sentences, proving one. -/
def incomplete : Theory := ⟨Bool, fun b => b = true, fun _ => False, fun _ => True⟩

/-- A sound theory can leave a truth unproved. -/
theorem ground_exceeds_ladder :
    Sound incomplete ∧ incomplete.True_ false ∧ ¬ incomplete.Prov false :=
  ⟨fun _ _ => trivial, trivial, fun h => Bool.noConfusion h⟩

/-- Independent axioms are separate bits: every combination of two values is realized. -/
theorem independent_axioms_are_separate_bits :
    ∀ a b : Bool, ∃ w : Bool × Bool, w.1 = a ∧ w.2 = b :=
  fun a b => ⟨(a, b), rfl, rfl⟩

/-! ### XVIII.b · The root crosses every keyless sentence and no keyed one -/

/-- Over any type of worlds: a sentence is keyless when every world has it, keyed when some world
    lacks it. -/
def KeylessOn {W : Type} (P : W → Prop) : Prop := ∀ w, P w
def KeyedOn {W : Type} (P : W → Prop) : Prop := ∃ w, ¬ P w

/-- The root proposition, 0 < 1. -/
def RootAct : Prop := (0 : Int) < 1
theorem root_act : RootAct := show (0 : Int) < 1 by decide


/-- The root is keyless over every type of worlds. -/
theorem root_is_keyless {W : Type} : KeylessOn (fun _ : W => RootAct) :=
  fun _ => root_act

/-- The root implies every keyless sentence on every world. -/
theorem keyless_crosses {W : Type} (P : W → Prop) (hP : KeylessOn P) : ∀ w, RootAct → P w :=
  fun w _ => hP w

/-- The root does not imply a keyed sentence on every world. -/
theorem root_decides_no_keyed {W : Type} (P : W → Prop) (hK : KeyedOn P) :
    ¬ ∀ w, RootAct → P w :=
  fun h => hK.elim (fun w hw => hw (h w root_act))

/-- Choice over ZF as a keyed sentence: two worlds, one each way (Gödel 1938; Cohen 1963). -/
def choiceHolds : Bool → Prop := fun w => w = true
theorem choice_is_keyed : KeyedOn choiceHolds := ⟨false, fun h => Bool.noConfusion h⟩
theorem root_does_not_cross_choice :
    (∀ _w : Bool, RootAct) ∧ choiceHolds true ∧ ¬ choiceHolds false :=
  ⟨fun _ => root_act, rfl, fun h => Bool.noConfusion h⟩

/-- The hypothesis is keyed over the worlds of the chart. -/
theorem line_is_keyed_over_worlds : KeyedOn RH ∧ ∃ Z, FoldClosed Z ∧ RH Z :=
  ⟨⟨W2f, W2f_not_rh⟩, ⟨W1, W1_closed, W1_rh⟩⟩

/-- The root does not imply the hypothesis on every world. -/
theorem root_does_not_cross_the_line : ¬ ∀ Z : World, RootAct → RH Z :=
  root_decides_no_keyed RH line_is_keyed_over_worlds.1

/-! ### XVIII.c · The two can'ts of a foundation are not mirror images -/

/-- A hypothesis with a provability predicate, Σ₁-completeness on the denial (a false Π⁰₁
    hypothesis has a finite witness, which the theory proves: Davis, Matiyasevich and Robinson
    1976; Lagarias 2002) and soundness on the denial, both carried as hypotheses. -/
structure Setting where
  hyp       : Prop
  Prov      : Prop → Prop
  sigma1    : ¬ hyp → Prov (¬ hyp)
  sound_neg : Prov (¬ hyp) → ¬ hyp

/-- On the chart one off-line zero refutes the hypothesis. -/
theorem refutation_is_one_point (Z : World) (z : Pt) (hz : Z z) (h : ¬ onLine z) : ¬ RH Z :=
  fun r => h (r z hz)

/-- If the theory cannot refute the hypothesis, the hypothesis holds. -/
theorem cant_refute_seals (S : Setting) (h : ¬ S.Prov (¬ S.hyp)) : S.hyp :=
  Classical.byContradiction (fun n => h (S.sigma1 n))

theorem cant_refute_seals_dec (S : Setting) [Decidable S.hyp] (h : ¬ S.Prov (¬ S.hyp)) : S.hyp :=
  Decidable.byContradiction (fun n => h (S.sigma1 n))

/-- Under soundness on the denial, the hypothesis holds exactly when the foundation cannot refute it. -/
theorem rh_iff_cant_refute (S : Setting) [Decidable S.hyp] : S.hyp ↔ ¬ S.Prov (¬ S.hyp) :=
  ⟨fun r p => S.sound_neg p r, cant_refute_seals_dec S⟩

/-- A hypothesis the theory neither proves nor refutes holds. -/
theorem independence_forces_truth (S : Setting) (_hp : ¬ S.Prov S.hyp)
    (hn : ¬ S.Prov (¬ S.hyp)) :
    S.hyp :=
  cant_refute_seals S hn

/-- A setting in which the hypothesis holds and is not provable. -/
def independentTrue : Setting := ⟨True, fun _ => False, fun n => absurd trivial n, fun p => p.elim⟩

theorem cant_prove_does_not_seal_false :
    ¬ independentTrue.hyp ∧ ¬ independentTrue.Prov independentTrue.hyp :=
  ⟨trivial, id⟩

/-- Cannot-refute implies the hypothesis; cannot-prove is consistent with it. -/
theorem cant_asymmetry :
    (∀ S : Setting, ¬ S.Prov (¬ S.hyp) → S.hyp) ∧ (∃ S : Setting, S.hyp ∧ ¬ S.Prov S.hyp) :=
  ⟨cant_refute_seals, ⟨independentTrue, cant_prove_does_not_seal_false⟩⟩

/-! ### XVIII.d · The round trip is the identity -/

/-- A sentence conjoined with the root. -/
def viaRoot {W : Type} (P : W → Prop) : W → Prop := fun w => RootAct ∧ P w

/-- Conjoining the root changes no sentence's value. -/
theorem round_trip_identity {W : Type} (P : W → Prop) (w : W) : viaRoot P w ↔ P w :=
  ⟨fun h => h.2, fun h => ⟨root_act, h⟩⟩

/-- An absolute sentence, the same in every world, returns unchanged. -/
theorem absolute_returns_unchanged {W : Type} (P : W → Prop) (hAbs : ∀ w v, P w ↔ P v) (w : W) :
    ∀ v, viaRoot P v ↔ P w :=
  fun v => (round_trip_identity P v).trans (hAbs v w)
```

```
/-- The trip adds presence and nothing else: it returns identically for a
sentence and its denial. -/
theorem trip_adds_presence_only {W : Type} (P : W → Prop) (w : W) :
    RootAct ∧ (viaRoot P w ↔ P w) ∧ (viaRoot (fun v => ¬ P v) w ↔ ¬ P w)
    :=
  ⟨root_act, round_trip_identity P w, round_trip_identity (fun v => ¬ P
    v) w⟩

/-- For any true Q, (Q → RH Z) ↔ RH Z. -/
theorem massless_arrow (Q : Prop) (hq : Q) (Z : World) : (Q → RH Z) ↔ RH Z
:=
  ⟨fun f => f hq, fun h _ => h⟩

/-- Any true premise satisfies the three properties of the root. -/
theorem any_true_premise_serves (R : Prop) (hR : R) {W : Type} (P : W →
Prop) :
    ((∀ w, P w) → ∀ w, R → P w) ∧ ((∃ w, ¬ P w) → ¬ ∀ w, R → P w) ∧ (∀ w,
    (R ∧ P w) ↔ P w) :=
  ⟨fun hP w _ => hP w, fun h₁ h => h₁.elim (fun w hw => hw (h w hR)),
   fun _ => ⟨fun h => h.2, fun h => ⟨hR, h⟩⟩⟩

/-- Conjoined with the root, the hypothesis is itself on W1 and on W2f. -/
theorem the_line_round_trips :
    (∀ Z : World, viaRoot RH Z ↔ RH Z) ∧ viaRoot RH W1 ∧ ¬ viaRoot RH W2f
    :=
  ⟨fun Z => round_trip_identity RH Z, ⟨root_act, W1_rh⟩, fun h =>
   W2f_not_rh h.2⟩

/-! ### XVIII.e · The ladder blocked, in six clauses -/

/-- Six clauses: no reading of the record decides the hypothesis; no
keyless premise forces it; no
    admissible coalition holding on W1 forces it; the root alone does not
    give it; a setting carries
    a true hypothesis it does not prove; and the hypothesis forces the
    line and is entailed by every
    premise that forces it. -/
theorem ladder_blocked :
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) ∧
    (∀ A : World → Prop, Keyless A → ¬ Forces A) ∧
    (∀ L : List (World → Prop), (∀ A ∈ L, Admissible A) → (∀ A ∈ L, A W1)
    →
      ¬ Forces (fun Z => ∀ A ∈ L, A Z)) ∧
    (¬ ∀ Z : World, RootAct → RH Z) ∧
    (∃ S : Setting, S.hyp ∧ ¬ S.Prov S.hyp) ∧
    (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z) :=
  ⟨record_decides_nothing_free, keyless_forces_nothing_free,
   no_admissible_coalition_forces,
    root_does_not_cross_the_line, ⟨independentTrue,
    cant_prove_does_not_seal_false⟩,
    rh_is_the_weakest_forcing_premise⟩

/-- Section XVIII, whole. -/
theorem the_road :
    (∀ T : Theory, RungOnLadder T → Sound T → ∀ s, T.Comp s → T.True_ s) ∧
    (RungOnLadder unsound ∧ ¬ Sound unsound) ∧
    (Sound incomplete ∧ incomplete.True_ false ∧ ¬ incomplete.Prov false)
    ∧
    (∀ {W : Type} (P : W → Prop), KeylessOn P → ∀ w, RootAct → P w) ∧
    (∀ {W : Type} (P : W → Prop), KeyedOn P → ¬ ∀ w, RootAct → P w) ∧
    (KeyedOn choiceHolds ∧ KeyedOn RH) ∧
    (∀ S : Setting, ¬ S.Prov (¬ S.hyp) → S.hyp) ∧
    (∃ S : Setting, S.hyp ∧ ¬ S.Prov S.hyp) ∧
    (∀ {W : Type} (P : W → Prop) (w : W), viaRoot P w ↔ P w) ∧
    (∀ (Q : Prop), Q → ∀ Z : World, (Q → RH Z) ↔ RH Z) ∧
    (∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z → RH Z) :=
  ⟨fun T hR hS => ⟨placement T hR hS⟩.2.2, soundness_is_load_bearing,
   ground_exceeds_ladder,
    fun P hP => keyless_crosses P hP, fun P hK => root_decides_no_keyed P
    hK,
    ⟨choice_is_keyed, line_is_keyed_over_worlds.1⟩,
    cant_refute_seals, ⟨independentTrue, cant_prove_does_not_seal_false⟩,
    fun P w => round_trip_identity P w, massless_arrow,
    rh_is_the_weakest_forcing_premise.2⟩

/-! ## XIX · The vocabulary law, the even coalition, the triaxial cut,
the ladder's proof, the route ledger -/

/-! ### XIX.a · The vocabulary law -/

/-- On every world, a consequence of the hypothesis is a consequence of
least erasure and
    conversely, and a premise entailing one entails the other. -/
theorem vocabulary_law (Z : World) (P : Prop) :
    ((RH Z → P) ↔ (LeastErasure Z → P)) ∧ ((P → RH Z) ↔ (P → LeastErasure
    Z)) :=
  ⟨⟨fun h l => h ((least_erasure_is_the_value Z).mp l),
    fun h r => h ((least_erasure_is_the_value Z).mpr r)⟩,
   ⟨fun h p => (least_erasure_is_the_value Z).mpr (h p),
    fun h p => (least_erasure_is_the_value Z).mp (h p)⟩⟩

/-- A consequence of any one face is a consequence of the least-erasure
face. -/
theorem every_face_proves_every_face (H : Faces) (P : Prop) :
    ((H.line → P) ↔ (H.least → P)) ∧ ((H.lamZero → P) ↔ (H.least → P)) ∧
    ((H.faithful → P) ↔ (H.least → P)) ∧ ((H.weil → P) ↔ (H.least → P)) :=
  ⟨⟨fun h l => h (H.line_iff_least.mpr l), fun h r => h
   (H.line_iff_least.mp r)⟩,
   ⟨fun h l => h (H.lam_iff_least.mpr l), fun h r => h
   (H.lam_iff_least.mp r)⟩,
   ⟨fun h l => h (H.faith_iff_least.mpr l), fun h r => h
   (H.faith_iff_least.mp r)⟩,
   ⟨fun h l => h (H.weil_iff_least.mpr l), fun h r => h
   (H.weil_iff_least.mp r)⟩⟩

/-- On every fold-closed world a consequence of any of the four chart
readings is a consequence of
    least erasure. -/
theorem chart_faces_prove_each_other (Z : World) (hZ : FoldClosed Z) (P :
Prop) :
    ((Lossless Z → P) ↔ (LeastErasure Z → P)) ∧ ((DepthZero Z → P) ↔
    (LeastErasure Z → P)) ∧
    ((¬ Left Z → P) ↔ (LeastErasure Z → P)) ∧ ((Stable Z → P) ↔
    (LeastErasure Z → P)) :=
  ⟨⟨fun h l => h ((lossless_iff_rh Z).mpr ((least_erasure_is_the_value
   Z).mp l)),
    fun h r => h ((least_erasure_is_the_value Z).mpr ((lossless_iff_rh
    Z).mp r))⟩,
   ⟨fun h l => h ((rh_iff_even_eigenspace Z).mp
   ((least_erasure_is_the_value Z).mp l)),
    fun h r => h ((least_erasure_is_the_value Z).mpr
    ((rh_iff_even_eigenspace Z).mpr r))⟩,
   ⟨fun h l => h ((rh_iff_no_left Z hZ).mp ((least_erasure_is_the_value
   Z).mp l)),
    fun h r => h ((least_erasure_is_the_value Z).mpr ((rh_iff_no_left Z
    hZ).mpr r))⟩,
   ⟨fun h l => h ((stable_iff_rh Z hZ).mpr ((least_erasure_is_the_value
   Z).mp l)),
    fun h r => h ((least_erasure_is_the_value Z).mpr ((stable_iff_rh Z
    hZ).mp r))⟩⟩

/-! ### XIX.b · The even coalition: closed under every combination, blind
to the value -/

/-- The fold is an involution. -/
theorem fold_fold (z : Pt) : fold (fold z) = z := by
  cases z with
  | mk x t =>
    show Pt.mk (2 - (2 - x)) t = Pt.mk x t
    congr 1
    omega

/-- The fold law is keyless. -/
theorem fold_law_admissible : Admissible (fun _ : World => ∀ z, fold
(fold z) = z) :=
  Or.inl (fun _ => fold_fold)

/-- Two worlds of one record have one record. -/
theorem same_record_eq (Z Z' : World) (h : SameRecord Z Z') : recordOf Z =
recordOf Z' :=
  funext (fun r => propext (h r))

/-- A function of the record is a reading of the record. -/
theorem record_reading_admissible (h : World → Prop) : Admissible (fun Z
=> h (recordOf Z)) :=
  Or.inr (fun Z Z' hs => by
    show h (recordOf Z) ↔ h (recordOf Z')
    rw [same_record_eq Z Z' hs])

/-- A constant premise is admissible. -/
theorem admissible_const (P : Prop) : Admissible (fun _ : World => P) :=
  Or.inr (fun _ _ _ => Iff.rfl)

/-- The denial of a keyless premise respects the record: it is false
everywhere. -/
theorem keyless_denial_respects (A : World → Prop) (hA : Keyless A) :
    RespectsRecord (fun Z => ¬ A Z) :=
  fun Z Z' _ => ⟨fun n => absurd (hA Z) n, fun n => absurd (hA Z') n⟩

/-- Admissible premises are closed under negation. -/
theorem admissible_not (A : World → Prop) (hA : Admissible A) :
Admissible (fun Z => ¬ A Z) := by
  rcases hA with k | r
  · exact Or.inr (keyless_denial_respects A k)
  · exact Or.inr (fun Z Z' h => ⟨fun n a => n ((r Z Z' h).mpr a), fun n a
    => n ((r Z Z' h).mp a)⟩)

/-- Admissible premises are closed under conjunction. -/
theorem admissible_and (A B : World → Prop) (hA : Admissible A) (hB :
Admissible B) :
    Admissible (fun Z => A Z ∧ B Z) := by
  rcases hA with ka | ra <;> rcases hB with kb | rb
  · exact Or.inl (fun Z => ⟨ka Z, kb Z⟩)
  · exact Or.inr (fun Z Z' h =>
      ⟨fun ⟨_, b⟩ => ⟨ka Z', (rb Z Z' h).mp b⟩, fun ⟨_, b⟩ => ⟨ka Z, (rb Z
      Z' h).mpr b⟩⟩)
  · exact Or.inr (fun Z Z' h =>
      ⟨fun ⟨a, _⟩ => ⟨(ra Z Z' h).mp a, kb Z'⟩, fun ⟨a, _⟩ => ⟨(ra Z Z'
      h).mpr a, kb Z⟩⟩)
  · exact Or.inr (fun Z Z' h =>
      ⟨fun ⟨a, b⟩ => ⟨(ra Z Z' h).mp a, (rb Z Z' h).mp b⟩,
       fun ⟨a, b⟩ => ⟨(ra Z Z' h).mpr a, (rb Z Z' h).mpr b⟩⟩)

/-- Admissible premises are closed under disjunction. -/
theorem admissible_or (A B : World → Prop) (hA : Admissible A) (hB :
Admissible B) :
    Admissible (fun Z => A Z ∨ B Z) := by
  rcases hA with ka | ra <;> rcases hB with kb | rb
  · exact Or.inl (fun Z => Or.inl (ka Z))
  · exact Or.inl (fun Z => Or.inl (ka Z))
  · exact Or.inl (fun Z => Or.inr (kb Z))
  · exact Or.inr (fun Z Z' h =>
      ⟨fun o => o.elim (fun a => Or.inl ((ra Z Z' h).mp a)) (fun b =>
      Or.inr ((rb Z Z' h).mp b)⟩,
```

```
        fun o => o.elim (fun a => Or.inl ((ra Z Z' h).mpr a)) (fun b =>
        Or.inr ((rb Z Z' h).mpr b))))

/-- No admissible premise agrees with the hypothesis on every fold-closed
world. -/
theorem admissible_never_decides (A : World → Prop) (hA : Admissible A) :
    ¬ ∀ Z, FoldClosed Z → (A Z ↔ RH Z) := by
  intro h
  rcases hA with k | r
  · exact W2f_not_rh ((h W2f W2f_closed).mp (k W2f))
  · exact record_decides_nothing_free A r h

/-- A coalition: any Boolean combination of atomic premises, however
deep. -/
inductive Coalition (ι : Type) where
  | atom : ι → Coalition ι
  | top : Coalition ι
  | bot : Coalition ι
  | not : Coalition ι → Coalition ι
  | and : Coalition ι → Coalition ι → Coalition ι
  | or : Coalition ι → Coalition ι → Coalition ι

/-- The premise a coalition asserts, given its atoms. -/
def Coalition.eval {ι : Type} (at_ : ι → World → Prop) : Coalition ι →
World → Prop
  | .atom i => at_ i
  | .top => fun _ => True
  | .bot => fun _ => False
  | .not c => fun Z => ¬ Coalition.eval at_ c Z
  | .and c d => fun Z => Coalition.eval at_ c Z ∧ Coalition.eval at_ d Z
  | .or c d => fun Z => Coalition.eval at_ c Z ∨ Coalition.eval at_ d Z

/-- Every coalition of admissible atoms is admissible. -/
theorem coalition_admissible {ι : Type} (at_ : ι → World → Prop) (h : ∀
i, Admissible (at_ i)) :
    ∀ c : Coalition ι, Admissible (Coalition.eval at_ c) := by
  intro c
  induction c with
  | atom i => exact h i
  | top => exact admissible_const True
  | bot => exact admissible_const False
  | not c ih =>
    show Admissible (fun Z => ¬ Coalition.eval at_ c Z)
    exact admissible_not _ ih
  | and c d ihc ihd =>
    show Admissible (fun Z => Coalition.eval at_ c Z ∧ Coalition.eval at_
    d Z)
    exact admissible_and _ _ ihc ihd
  | or c d ihc ihd =>
    show Admissible (fun Z => Coalition.eval at_ c Z ∨ Coalition.eval at_
    d Z)
    exact admissible_or _ _ ihc ihd

/-- No coalition of admissible atoms agrees with the hypothesis on every
fold-closed world. -/
theorem no_coalition_decides {ι : Type} (at_ : ι → World → Prop) (h : ∀
i, Admissible (at_ i))
    (c : Coalition ι) : ¬ ∀ Z, FoldClosed Z → (Coalition.eval at_ c Z ↔
    RH Z) :=
  admissible_never_decides _ (coalition_admissible at_ h c)

/-! ### XIX.c · The triaxial cut, irreducible -/

/-- The fold law and every reading of the record are admissible;
admissible premises are closed
    under Boolean combination; no admissible premise decides the
    hypothesis; off the line one
    supplied side bit fixes any fold-flipped target by a unique
    calibration, both values realized;
    least erasure is not admissible; and nothing weaker than the
    hypothesis forces the line. -/
theorem triaxial_cut_irreducible :
    Admissible (fun _ : World => ∀ z, fold (fold z) = z) ∧
    (∀ h : World → Prop, Admissible (fun Z => h (recordOf Z))) ∧
    (∀ (ι : Type) (at_ : ι → World → Prop), (∀ i, Admissible (at_ i)) →
      ∀ c : Coalition ι, Admissible (Coalition.eval at_ c)) ∧
    (∀ A : World → Prop, Admissible A → ¬ ∀ Z, FoldClosed Z → (A Z ↔ RH
    Z)) ∧
    (∀ (d : Pt → Bool) (z : Pt), ¬ onLine z → d (fold z) = !d z →
      ∃ c : Bool, (d z = xor (side z) c ∧ d (fold z) = xor (side (fold
      z)) c) ∧
        ∀ c', (d z = xor (side z) c' ∧ d (fold z) = xor (side (fold z))
        c') → c' = c) ∧
    (∀ z : Pt, ¬ onLine z →
      (side (fold z) = !side z ∧ side z = xor (side z) false) ∧
      ((!side (fold z)) = !(!side z) ∧ (!side z) = xor (side z) true)) ∧
    ¬ Admissible LeastErasure ∧
    (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z) :=
  ⟨fold_law_admissible, record_reading_admissible,
   fun _ at_ h c => coalition_admissible at_ h c,
   admissible_never_decides,
   colocation_is_a_calibration, calibration_realized_both_ways,
   spend_is_not_admissible, rh_is_the_weakest_forcing_premise⟩

/-! ### XIX.d · The ladder's proof lands on least erasure; no foundation
is claimed closed -/

/-- A reading of a theory's sentences on the chart: each sentence names a
world, and the sentence
    holds in the intended model exactly when that world satisfies the
    hypothesis. -/
structure ReadsAsLine (T : Theory) where
  world : T.Sent → World
  reads : ∀ s, T.True_ s ↔ RH (world s)

/-- If a sound theory proves a sentence that reads as the hypothesis on a
world, that world has
    least erasure. -/
theorem ladder_proof_lands_on_least_erasure (T : Theory) (hS : Sound T)
(R : ReadsAsLine T)
    (s : T.Sent) (hp : T.Prov s) : LeastErasure (R.world s) :=
  (least_erasure_is_the_value (R.world s)).mpr ((R.reads s).mp (hS s hp))

/-- Where the theory also proves the sentence whenever it holds, its
proof is exactly least
    erasure on the world the sentence names. -/
theorem ladder_proof_iff_least_erasure (T : Theory) (hS : Sound T) (R :
ReadsAsLine T)
    (s : T.Sent) (hc : T.True_ s → T.Prov s) : T.Prov s ↔ LeastErasure
    (R.world s) :=
  ⟨ladder_proof_lands_on_least_erasure T hS R s,
   fun l => hc ((R.reads s).mpr ((least_erasure_is_the_value (R.world
   s)).mp l))⟩

/-- A sound theory with one sentence, proved, reading as the line world.
-/
def lineTheory : Theory := ⟨Unit, fun _ => True, fun _ => False, fun _ =>
True⟩
theorem lineTheory_sound : Sound lineTheory := fun _ _ => trivial
def lineTheoryReads : ReadsAsLine lineTheory :=
  ⟨fun _ => W1, fun _ => ⟨fun _ => W1_rh, fun _ => trivial⟩⟩

/-- A sound theory proving a sentence that reads as the hypothesis
exists, and its proof lands on
    least erasure. -/
theorem a_sound_theory_may_prove_the_line :
    Sound lineTheory ∧ lineTheory.Prov () ∧ LeastErasure
    (lineTheoryReads.world ()) :=
  ⟨lineTheory_sound, trivial,
   ladder_proof_lands_on_least_erasure lineTheory lineTheory_sound
   lineTheoryReads () trivial⟩

/-! ### XIX.e · The route ledger, computed -/

/-- Six routes to the hypothesis. -/
inductive Route where
  | ladderFromRecord | foundationAsSystem | placement | groundByAct |
  independence | realPartCertified
  deriving DecidableEq, Repr

/-- A route's status: blocked by a theorem of this file; closed by the
act; carried on a cited
    theorem; or open, with no closure claimed. -/
inductive RouteStatus where
  | blocked | closedByAct | cited | open
  deriving DecidableEq, Repr

/-- A status is a function of three recorded bits: a theorem of this file
blocks the route; the act
    closes it; a cited theorem carries it. -/
def routeStatus (blocked byAct cited : Bool) : RouteStatus :=
  if blocked then .blocked else if byAct then .closedByAct else if cited
  then .cited else .open

structure RouteEntry where
  route : Route
  blocked : Bool
  byAct : Bool
  cited : Bool
  printed : RouteStatus

/-- The ledger: the ladder from the record, blocked (unicorn_block); the
foundation as a system,
    open (a_sound_theory_may_prove_the_line); placement, on the two cited
    premises (placement);
    the ground closed by the act (rh_from_the_act); independence,
    blocked, since it would force
    truth (independence_forces_truth); the real part to the certified
    height, cited (Platt and
    Trudgian 2021, certified_height_never_forces). -/
def routeLedger : List RouteEntry :=
  [⟨.ladderFromRecord, true, false, false, .blocked⟩,
   ⟨.foundationAsSystem, false, false, false, .open⟩,
   ⟨.placement, false, false, true, .cited⟩,
   ⟨.groundByAct, false, true, false, .closedByAct⟩,
   ⟨.independence, true, false, false, .blocked⟩,
   ⟨.realPartCertified, false, false, true, .cited⟩]

/-- Every printed status is the computed one. -/
theorem route_ledger_is_computed :
    routeLedger.all (fun e => routeStatus e.blocked e.byAct e.cited ==
    e.printed) = true := by
  decide

/-- Six routes: two blocked, one closed by the act, two cited, one open.
-/
theorem route_ledger_counts :
    routeLedger.length = 6 ∧
    (routeLedger.filter (fun e => e.printed == .blocked)).length = 2 ∧
    (routeLedger.filter (fun e => e.printed == .closedByAct)).length = 1 ∧
    (routeLedger.filter (fun e => e.printed == .cited)).length = 2 ∧
    (routeLedger.filter (fun e => e.printed == .open)).length = 1 := by
  decide

/-- Nothing escapes the ledger: every status is one of the four. -/
```

```
theorem route_status_total : ∀ a b c : Bool,
    routeStatus a b c = .blocked ∨ routeStatus a b c = .closedByAct ∨
    routeStatus a b c = .cited ∨ routeStatus a b c = .open := by
  decide

/-- A route is blocked exactly when a theorem blocks it: no other bit
produces the verdict. -/
theorem blocked_iff_theorem : ∀ a b c : Bool, routeStatus a b c =
.blocked ↔ a = true := by
  decide

/-! ### XIX.f · The two-part reading -/

/-- (a) From the posit the hypothesis follows. (b) No reading of the
record supplies the posit. (c)
    The posit is the hypothesis. (d) Nothing weaker forces. -/
theorem reader_frame :
    (∀ A : ActualZeros, RH A.zeros) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z) :=
  ⟨rh_from_the_act, record_decides_nothing_free,
   least_erasure_is_the_value,
    rh_is_the_weakest_forcing_premise⟩

/-- Section XIX, whole. -/
theorem the_front :
    (∀ (Z : World) (P : Prop), ((RH Z → P) ↔ (LeastErasure Z → P)) ∧
      ((P → RH Z) ↔ (P → LeastErasure Z))) ∧
    (∀ (H : Faces) (P : Prop), ((H.line → P) ↔ (H.least → P)) ∧
    ((H.lamZero → P) ↔ (H.least → P)) ∧
      ((H.faithful → P) ↔ (H.least → P)) ∧ ((H.weil → P) ↔ (H.least →
      P))) ∧
    (∀ (ι : Type) (at_ : ι → World → Prop), (∀ i, Admissible (at_ i)) →
      ∀ c : Coalition ι, ¬ ∀ Z, FoldClosed Z → (Coalition.eval at_ c Z ↔
      RH Z)) ∧
    ¬ Admissible LeastErasure ∧
    (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
    RH Z) ∧
    (∀ (T : Theory), Sound T → ∀ (R : ReadsAsLine T) (s : T.Sent), T.Prov
    s →
      LeastErasure (R.world s)) ∧
    (Sound lineTheory ∧ lineTheory.Prov () ∧ LeastErasure
    (lineTheoryReads.world ())) ∧
    routeLedger.all (fun e => routeStatus e.blocked e.byAct e.cited ==
    e.printed) = true ∧
    ((∀ A : ActualZeros, RH A.zeros) ∧
     (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
     RH Z)) ∧
     (∀ Z : World, LeastErasure Z ↔ RH Z)) :=
  ⟨vocabulary_law, every_face_proves_every_face,
   fun _ at_ h c => no_coalition_decides at_ h c, spend_is_not_admissible,
   rh_is_the_weakest_forcing_premise,
   fun T hS R s hp => ladder_proof_lands_on_least_erasure T hS R s hp,
   a_sound_theory_may_prove_the_line, route_ledger_is_computed,
   ⟨rh_from_the_act, record_decides_nothing_free,
   least_erasure_is_the_value⟩⟩

/-! ## XX · Least erasure affirmed: the record, the ledger, the price,
the one form, the act -/

/-! ### XX.a · The record is carried by a least-erasure world, and by
only one -/

/-- Registration lands on the line. -/
theorem reg_on_line (z : Pt) : onLine (reg z) := rfl

/-- The record of every world satisfies the hypothesis. -/
theorem record_rh (Z : World) : RH (recordOf Z) := by
  intro r ⟨s, _, hsr⟩
  rw [← hsr]
  exact rfl

/-- The record of every world has least erasure. -/
theorem record_has_least_erasure (Z : World) : LeastErasure (recordOf Z)
:=
  ⟨least_erasure_is_the_value (recordOf Z)⟩.mpr (record_rh Z)

/-- The record of every world is fold-closed: it lies on the line, which
the fold fixes. -/
theorem record_closed (Z : World) : FoldClosed (recordOf Z) := by
  intro r ⟨s, hs, hsr⟩
  rw [← hsr]
  exact ⟨s, hs, ((fold_fixed_iff (reg s)).mpr rfl).symm⟩

/-- The record of Z shares the record of Z, has least erasure, and every
least-erasure world with
    that record equals it pointwise. -/
theorem record_carried_by_least_erasure (Z : World) :
    SameRecord Z (recordOf Z) ∧ LeastErasure (recordOf Z) ∧
    ∀ Z', SameRecord Z Z' → LeastErasure Z' → ∀ z, Z' z ↔ recordOf Z z :=
    by
  refine ⟨record_same Z, record_has_least_erasure Z, ?_⟩
  intro Z' hs hle z
  have h1 : SameRecord Z' (recordOf Z) := fun r => (hs r).symm.trans
  (record_same Z r)
  exact lossless_unique Z' (recordOf Z) h1 ((least_erasure_is_the_value
  Z').mp hle) (record_rh Z) z

/-! ### XX.b · Every reading of the record holds on a least-erasure world
-/

/-- An admissible premise that holds on a world holds on that world's
record, which has least
    erasure. -/
theorem admissible_holds_on_the_lossless_world (A : World → Prop) (hA :
Admissible A) (Z : World)
    (h : A Z) : A (recordOf Z) ∧ LeastErasure (recordOf Z) := by
  refine ⟨?_, record_has_least_erasure Z⟩
  rcases hA with k | r
  · exact k (recordOf Z)
  · exact (r Z (recordOf Z) (record_same Z)).mp h

/-- Every reading of the record that holds on a world holds on that
world's record. -/
theorem every_reading_holds_on_the_lossless_world (g : World → Prop) (hg :
RespectsRecord g)
    (Z : World) (h : g Z) : g (recordOf Z) ∧ LeastErasure (recordOf Z) :=
    by
  refine ⟨(hg Z (recordOf Z) (record_same Z)).mp h,
  record_has_least_erasure Z⟩

/-- Every coalition of admissible atoms that holds on a world holds on
that world's record. -/
theorem every_coalition_holds_on_the_lossless_world {ι : Type} (at_ : ι →
World → Prop)
    (h : ∀ i, Admissible (at_ i)) (c : Coalition ι) (Z : World) (hc :
    Coalition.eval at_ c Z) :
    Coalition.eval at_ c (recordOf Z) ∧ LeastErasure (recordOf Z) := by
  refine ⟨?_, record_has_least_erasure Z⟩
  rcases coalition_admissible at_ h c with k | r
  · exact k (recordOf Z)
  · exact (r Z (recordOf Z) (record_same Z)).mp hc

/-- An admissible premise that denies least erasure on every fold-closed
world holds on no world. -/
theorem record_never_testifies_against (A : World → Prop) (hA :
Admissible A)
    (hdeny : ∀ Z, FoldClosed Z → A Z → ¬ LeastErasure Z) : ∀ Z, ¬ A Z :=
    by
  intro Z h
  rcases hA with k | r
  · exact hdeny (recordOf Z) (record_closed Z) (k (recordOf Z))
  (record_has_least_erasure Z)
  · exact hdeny (recordOf Z) (record_closed Z) ((r Z (recordOf Z)
  (record_same Z)).mp h)
      (record_has_least_erasure Z)

/-! ### XX.c · The erasure ledger: one bit per off-line pair, zero
exactly at least erasure -/

/-- An off-line pair at offset k is two distinct points, exchanged by the
fold, with one registered
    point. -/
theorem pair_two_points_one_record (k : Int) (h : Nat) (hk : k ≠ 0) :
    ((1 - k, h) : Pt) ≠ (1 + k, h) ∧ reg (1 - k, h) = reg (1 + k, h) ∧
    fold (1 - k, h) = (1 + k, h) := by
  refine ⟨?_, rfl, ?_⟩
  · intro e
    have := congrArg Pt.x e
    change 1 - k = 1 + k at this
    omega
  · show Pt.mk (2 - (1 - k)) h = Pt.mk (1 + k) h
    congr 1
    omega

/-- A finite configuration: the heights of its on-line zeros, and its
off-line pairs, each an
    offset with a height. An offset of zero is no pair. -/
structure FinCfg where
  online : List Nat
  pairs : List (Int × Nat)

/-- The world a finite configuration names on the chart. -/
def FinCfg.world (C : FinCfg) : World := fun z =>
  (z.x = 1 ∧ z.t ∈ C.online) ∨
  ∃ p ∈ C.pairs, p.1 ≠ 0 ∧ (z = (1 - p.1, p.2) ∨ z = (1 + p.1, p.2))

/-- The off-line pairs, counted. -/
def offCount : List (Int × Nat) → Nat
  | [] => 0
  | p :: ps => (if p.1 = 0 then 0 else 1) + offCount ps

/-- Erased bits: the number of off-line pairs. -/
def FinCfg.erasedBits (C : FinCfg) : Nat := offCount C.pairs

theorem offCount_zero_iff (ps : List (Int × Nat)) : offCount ps = 0 ↔ ∀ p
∈ ps, p.1 = 0 := by
  induction ps with
  | nil => exact ⟨fun _ p hp => (nomatch hp), fun _ => rfl⟩
  | cons p ps ih =>
    show (if p.1 = 0 then 0 else 1) + offCount ps = 0 ↔ ∀ q ∈ p :: ps,
    q.1 = 0
    by_cases hp : p.1 = 0
    · rw [if_pos hp, Nat.zero_add, ih]
      constructor
      · intro h q hq
        cases hq with
        | head => exact hp
        | tail _ hq' => exact h q hq'
      · intro h q hq
        exact h q (List.Mem.tail p hq)
    · rw [if_neg hp]
      constructor
```

```
      · intro h
        omega
      · intro h
        exact absurd (h p (List.Mem.head ps)) hp

/-- Every finite configuration is fold-closed: the on-line points are fixed and each pair is
    exchanged. -/
theorem fincfg_closed (C : FinCfg) : FoldClosed C.world := by
  intro z hz
  rcases hz with (hx, ht) | (p, hp, hk, hz)
  · exact Or.inl (show 2 - z.x = 1 by omega, ht)
  · refine Or.inr (p, hp, hk, ?_)
    rcases hz with e | e
    · subst e
      exact Or.inr (by show Pt.mk (2 - (1 - p.1)) p.2 = Pt.mk (1 + p.1) p.2; congr 1; omega)
    · subst e
      exact Or.inl (by show Pt.mk (2 - (1 + p.1)) p.2 = Pt.mk (1 - p.1) p.2; congr 1; omega)

/-- Least erasure holds exactly when the erased bits are zero. -/
theorem least_erasure_iff_zero_erased (C : FinCfg) : LeastErasure C.world ↔ C.erasedBits = 0 := by
  rw [least_erasure_is_the_value]
  show RH C.world ↔ offCount C.pairs = 0
  rw [offCount_zero_iff]
  constructor
  · intro h p hp
    refine Decidable.byContradiction (fun hk => ?_)
    have hz : C.world (1 - p.1, p.2) := Or.inr (p, hp, hk, Or.inl rfl)
    have := h _ hz
    change 1 - p.1 = 1 at this
    omega
  · intro h z hz
    rcases hz with (hx, _) | (p, hp, hk, _)
    · exact hx
    · exact absurd (h p hp) hk

/-! ### XX.d · The price: one floor per erased bit, zero exactly at least erasure -/

/-- k_B T ln 2 at 300 K in units of 10⁻⁴⁵ J: k_B = 1380649 × 10⁻²⁹ J/K (SI 2019), ln 2 =
    6931471805599453 × 10⁻¹⁶ to sixteen digits. -/
def landauerFloor : Nat := 1380649 * 300 * 6931471805599453

theorem landauer_floor_exact : landauerFloor = 2870978885078723755499100 := by decide

/-- The price of a registration: one floor per erased bit if irreversible, 0 if reversible. -/
def FinCfg.price (C : FinCfg) (irreversible : Bool) : Nat :=
  if irreversible then C.erasedBits * landauerFloor else 0

/-- An irreversible registration has price 0 exactly when the configuration has least erasure. -/
theorem price_zero_iff_least_erasure (C : FinCfg) : C.price true = 0 ↔ LeastErasure C.world := by
  rw [least_erasure_iff_zero_erased]
  show C.erasedBits * landauerFloor = 0 ↔ C.erasedBits = 0
  constructor
  · intro h
    rcases Nat.mul_eq_zero.mp h with h1 | h1
    · exact h1
    · exact absurd h1 (by decide)
  · intro h
    rw [h, Nat.zero_mul]

/-- One more off-line pair. -/
def FinCfg.addPair (C : FinCfg) (k : Int) (h : Nat) : FinCfg :=
  (C.online, (k, h) :: C.pairs)

/-- Adding an off-line pair adds one floor; the price is the erased bits times the floor. -/
theorem denial_price_linear (C : FinCfg) (k : Int) (h : Nat) (hk : k ≠ 0) :
    (C.addPair k h).price true = C.price true + landauerFloor ∧
    C.price true = C.erasedBits * landauerFloor := by
  refine (?_, rfl)
  show ((if k = 0 then 0 else 1) + offCount C.pairs) * landauerFloor
      = offCount C.pairs * landauerFloor + landauerFloor
  rw [if_neg hk, Nat.add_mul, Nat.one_mul, Nat.add_comm]

/-- What is held reversibly commits no bit. -/
theorem reversible_commits_nothing (C : FinCfg) : C.price false = 0 := rfl

/-! ### XX.e · The one form of anything against least erasure -/

/-- Least erasure holds exactly when no point of the world lies off the line. No classical axiom. -/
theorem least_erasure_iff_no_point_off (Z : World) : LeastErasure Z ↔ ¬ OffLine Z := by
  rw [least_erasure_is_the_value]
  constructor
  · intro h (z, hz, hoff)
    exact hoff (h z hz)
  · intro h z hz
    exact Decidable.byContradiction (fun hoff => h (z, hz, hoff))

/-- Least erasure fails exactly when some point of the world lies off the line. Classical. -/
theorem rejection_is_a_witness (Z : World) : ¬ LeastErasure Z ↔ OffLine Z := by
  rw [least_erasure_iff_no_point_off]
  exact (fun h => Classical.byContradiction (fun hn => h hn), fun h hn => hn h)

/-- Two settings on one hypothesis agree on whether the hypothesis is refutable. -/
theorem falsifier_form_constant (S₁ S₂ : Setting) [Decidable S₁.hyp] [Decidable S₂.hyp]
    (h : S₁.hyp = S₂.hyp) : (¬ S₁.Prov (¬ S₁.hyp)) ↔ (¬ S₂.Prov (¬ S₂.hyp)) := by
  constructor
  · intro h1 p2
    exact S₂.sound_neg p2 (Eq.mp h (cant_refute_seals_dec S₁ h1))
  · intro h2 p1
    exact S₁.sound_neg p1 (Eq.mpr h (cant_refute_seals_dec S₂ h2))

/-! ### XX.f · The act -/

/-- Any inhabitant of any type yields the root proposition. -/
theorem act_reenacts_root {Act : Type} (_a : Act) : RootAct := root_act

/-- The root does not imply the denial of the hypothesis on every world. -/
theorem root_does_not_cross_the_denial : ¬ ∀ Z : World, RootAct → ¬ RH Z :=
  fun h => h W1 root_act W1_rh

/-- A self-grounding supply of least erasure with an act exists exactly when least erasure holds. -/
theorem assent_is_the_value (Z : World) :
    (∃ G : SelfGrounding (LeastErasure Z), Nonempty G.Act) ↔ LeastErasure Z :=
  supply_iff (LeastErasure Z)

/-! ### XX.g · Freedom given, the sign spent -/

/-- A prime is one fibre of exactly two, never on the diagonal; an off-line pair is two points
    exchanged by the fold with one record; a bit is two distinct values. -/
theorem one_shape_two_registers :
    (∀ p : Nat, isPrime p →
      (∀ a b, mul_orbit p a b ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1)) ∧ (1 : Nat) ≠ p) ∧
    (∀ p : Nat, isPrime p → ¬ ∃ a, mul_orbit p a a) ∧
    (∀ (k : Int) (h : Nat), k ≠ 0 → ((1 - k, h) : Pt) ≠ (1 + k, h) ∧
      reg (1 - k, h) = reg (1 + k, h) ∧ fold (1 - k, h) = (1 + k, h)) ∧
    ((∀ b : Bool, b = true ∨ b = false) ∧ (true : Bool) ≠ false) :=
  ⟨freedom_is_exactly_two, prime_off_seat, pair_two_points_one_record,
   aperture_one_bit_wide⟩

/-- The free basis holds and forces nothing; least erasure is Weil positivity; positivity is keyed;
    a self-grounding supply of positivity is the hypothesis. -/
theorem freedom_is_given_the_sign_is_spent :
    (∀ p q : Nat, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 = 0) ∧
    ¬ Forces (fun _ => ∀ p q, isPrime p → isPrime q → p ≠ q →
      ∃ f : Nat → Int, CompletelyAdditive f ∧ f p = 1 ∧ f q = 0 ∧ f 1 = 0) ∧
    (∀ E : ExplicitFormula, LeastErasure E.zeros ↔ WeilPositive E) ∧
    (WeilPositive positiveInstance ∧ ¬ WeilPositive negativeInstance) ∧
    (∀ E : ExplicitFormula, (∃ G : SelfGrounding (WeilPositive E), Nonempty G.Act) ↔ RH E.zeros) :=
  ⟨prime_freedom_independent, prime_freedom_forces_nothing, least_erasure_is_positivity,
   positivity_is_keyed, prime_witness_iff⟩

/-! ### XX.h · Least erasure, affirmed whole -/

/-- Section XX, whole. -/
theorem least_erasure_affirmed :
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ Z : World, SameRecord Z (recordOf Z) ∧ LeastErasure (recordOf Z) ∧
      ∀ Z', SameRecord Z Z' → LeastErasure Z' → ∀ z, Z' z ↔ recordOf Z z) ∧
    (∀ (A : World → Prop), Admissible A → ∀ Z, A Z → A (recordOf Z) ∧ LeastErasure (recordOf Z)) ∧
    (∀ (A : World → Prop), Admissible A →
      (∀ Z, FoldClosed Z → A Z → ¬ LeastErasure Z) → ∀ Z, ¬ A Z) ∧
    (∀ C : FinCfg, LeastErasure C.world ↔ C.erasedBits = 0) ∧
    (∀ C : FinCfg, C.price true = 0 ↔ LeastErasure C.world) ∧
    (∀ Z : World, LeastErasure Z ↔ ¬ OffLine Z) ∧
    (∀ (S₁ S₂ : Setting) [Decidable S₁.hyp] [Decidable S₂.hyp], S₁.hyp = S₂.hyp →
      ((¬ S₁.Prov (¬ S₁.hyp)) ↔ (¬ S₂.Prov (¬ S₂.hyp)))) ∧
    (∀ Z : World, (∃ G : SelfGrounding (LeastErasure Z), Nonempty G.Act) ↔ LeastErasure Z) ∧
    (∀ E : ExplicitFormula, LeastErasure E.zeros ↔ WeilPositive E) :=
  ⟨least_erasure_is_the_value, record_carried_by_least_erasure,
   admissible_holds_on_the_lossless_world, record_never_testifies_against,
   least_erasure_iff_zero_erased, price_zero_iff_least_erasure,
   least_erasure_iff_no_point_off,
   fun S₁ S₂ _ _ h => falsifier_form_constant S₁ S₂ h,
   assent_is_the_value,
   least_erasure_is_positivity⟩


/-! ## XXI · Every assignment realized; the arrow for every n; the closure, whole

The primes admit every assignment of integers: for each `a` on the primes, `n ↦ Σ_{p ≤ n} v_p(n) a(p)`
```

```
is completely additive and takes the value `a(p)` at every prime, and by `determined_by_primes` it is the
only such function. The count `Ω(n) = Σ_p v_p(n)` and the arrow `λ(n) = (−1)^{Ω(n)}` are defined for every
`n`, and `λ` is completely multiplicative with `λ(p) = −1`. The Liouville face is pinned to the constructed
walk. On the chart, least erasure is the value, the fixed-point condition and leastness in the fibre; the
closure is bound in one theorem; and the same laws hold on any carrier with an involution and a registration,
the chart one instance. -/

/-- The finite sum `sumBelow N g = g 0 + ⋯ + g (N − 1)`. -/
def sumBelow : Nat → (Nat → Int) → Int
  | 0, _ => 0
  | N + 1, g => sumBelow N g + g N

theorem sumBelow_add (g h : Nat → Int) (N : Nat) :
    sumBelow N (fun i => g i + h i) = sumBelow N g + sumBelow N h := by
  induction N with
  | zero => show (0 : Int) = 0 + 0; omega
  | succ N ih =>
    show sumBelow N (fun i => g i + h i) + (g N + h N) = (sumBelow N g + g N) + (sumBelow N h + h N)
    rw [ih]; omega

theorem sumBelow_congr (g h : Nat → Int) (N : Nat) (hgh : ∀ i, i < N → g i = h i) :
    sumBelow N g = sumBelow N h := by
  induction N with
  | zero => rfl
  | succ N ih =>
    show sumBelow N g + g N = sumBelow N h + h N
    rw [ih (fun i hi => hgh i (by omega)), hgh N (by omega)]

theorem sumBelow_stable (g : Nat → Int) (M k : Nat) (hz : ∀ i, M ≤ i → i < M + k → g i = 0) :
    sumBelow (M + k) g = sumBelow M g := by
  induction k with
  | zero => rfl
  | succ k ih =>
    show sumBelow (M + k) g + g (M + k) = sumBelow M g
    rw [ih (fun i h1 h2 => hz i h1 (by omega)), hz (M + k) (by omega) (by omega)]
    omega

theorem sumBelow_zero (g : Nat → Int) (N : Nat) (hz : ∀ i, i < N → g i = 0) : sumBelow N g = 0 := by
  induction N with
  | zero => rfl
  | succ N ih =>
    show sumBelow N g + g N = 0
    rw [ih (fun i hi => hz i (by omega)), hz N (by omega)]
    omega

theorem sumBelow_single (g : Nat → Int) (j N : Nat) (hj : j < N) (hz : ∀ i, i < N → i ≠ j → g i = 0) :
    sumBelow N g = g j := by
  induction N with
  | zero => omega
  | succ N ih =>
    show sumBelow N g + g N = g j
    by_cases hjN : j = N
    · rw [hjN, sumBelow_zero g N (fun i hi => hz i (by omega) (by omega))]
      omega
    · rw [ih (by omega) (fun i hi hij => hz i (by omega) hij), hz N (by omega) (fun h => hjN h.symm)]
      omega

/-- Primality, decided: two at least, and its own least divisor. -/
def primeTest (p : Nat) : Bool := decide (2 ≤ p ∧ leastDivisor p = p)

theorem primeTest_iff (p : Nat) : primeTest p = true ↔ isPrime p := by
  constructor
  · intro h
    obtain ⟨h2, hl⟩ : 2 ≤ p ∧ leastDivisor p = p := of_decide_eq_true h
    have hp := leastDivisor_prime p h2
    rw [hl] at hp
    exact hp
  · intro hp
    apply decide_eq_true
    refine ⟨hp.1, ?_⟩
    have hs := leastDivisor_spec p hp.1
    rcases hp.2 (leastDivisor p) hs.1.2 with h1 | h1
    · have h2 := hs.1.1
      omega
    · exact h1

theorem pExp_zero_above (p n : Nat) (hn : 0 < n) (hp : n < p) : pExp p n = 0 :=
  pExp_eq_zero_of_not_dvd p n (fun hd => absurd (Nat.le_of_dvd hn hd) (by omega))

/-- The term of the extension at `p`: the exponent of `p` in `n` times the value assigned to `p`, at primes only. -/
def extendTerm (a : Nat → Int) (n p : Nat) : Int := if primeTest p = true then (pExp p n : Int) * a p else 0

/-- The extension of an assignment on the primes: `n ↦ Σ_{p ≤ n} v_p(n) a(p)`. -/
def extendAssignment (a : Nat → Int) (n : Nat) : Int := sumBelow (n + 1) (extendTerm a n)

theorem extendTerm_zero_above (a : Nat → Int) (n i : Nat) (hn : 0 < n) (hi : n < i) : extendTerm a n i = 0 := by
  unfold extendTerm
  rw [pExp_zero_above i n hn hi]
  by_cases hb : primeTest i = true
  · rw [if_pos hb]; simp
  · rw [if_neg hb]

theorem extendAssignment_bound (a : Nat → Int) (n B : Nat) (hn : 0 < n) (hB : n < B) :
    extendAssignment a n = sumBelow B (extendTerm a n) := by
  obtain ⟨k, rfl⟩ : ∃ k, B = n + 1 + k := ⟨B - (n + 1), by omega⟩
  unfold extendAssignment
  rw [sumBelow_stable (extendTerm a n) (n + 1) k (fun i h1 _ =>
    extendTerm_zero_above a n i hn (by omega))]

theorem extendTerm_mul (a : Nat → Int) (m n p : Nat) (hm : 0 < m) (hn : 0 < n) :
    extendTerm a (m * n) p = extendTerm a m p + extendTerm a n p := by
  unfold extendTerm
  by_cases hb : primeTest p = true
  · rw [if_pos hb, if_pos hb, if_pos hb, pExp_mul p m n ((primeTest_iff p).mp hb) hm hn]
    have e : ((pExp p m + pExp p n : Nat) : Int) = (pExp p m : Int) + (pExp p n : Int) := by omega
    rw [e, Int.add_mul]
  · rw [if_neg hb, if_neg hb, if_neg hb]
    omega

theorem extendAssignment_mul (a : Nat → Int) (m n : Nat) (hm : 0 < m) (hn : 0 < n) :
    extendAssignment a (m * n) = extendAssignment a m + extendAssignment a n := by
  have hmle : m ≤ m * n := by
    first
    | exact Nat.le_mul_of_pos_right m hn
    | exact Nat.le_mul_of_pos_right hn
    | (have h := Nat.mul_le_mul_left m hn; rwa [Nat.mul_one] at h)
  have hnle : n ≤ m * n := by
    first
    | exact Nat.le_mul_of_pos_left n hm
    | exact Nat.le_mul_of_pos_left hm
    | (have h := Nat.mul_le_mul_right n hm; rwa [Nat.one_mul] at h)
  rw [extendAssignment_bound a (m * n) (m * n + 1) (by omega) (by omega),
      extendAssignment_bound a m (m * n + 1) hm (by omega),
      extendAssignment_bound a n (m * n + 1) hn (by omega),
      sumBelow_congr (extendTerm a (m * n)) (fun i => extendTerm a m i + extendTerm a n i) (m * n + 1)
        (fun i _ => extendTerm_mul a m n i hm hn)]
  exact sumBelow_add (extendTerm a m) (extendTerm a n) (m * n + 1)

theorem extendTerm_other (a : Nat → Int) (q i : Nat) (hq : isPrime q) (hi : i ≠ q) : extendTerm a q i = 0 := by
  unfold extendTerm
  by_cases hb : primeTest i = true
  · rw [if_pos hb, pExp_other i q ((primeTest_iff i).mp hb) hq hi]; simp
  · rw [if_neg hb]

theorem extendAssignment_prime (a : Nat → Int) (q : Nat) (hq : isPrime q) : extendAssignment a q = a q := by
  unfold extendAssignment
  rw [sumBelow_single (extendTerm a q) q (q + 1) (by omega) (fun i _ hiq => extendTerm_other a q i hq hiq)]
  unfold extendTerm
  rw [if_pos ((primeTest_iff q).mpr hq), pExp_self q hq]
  simp

theorem extendAssignment_additive (a : Nat → Int) : CompletelyAdditive (extendAssignment a) :=
  fun m n hm hn => extendAssignment_mul a m n hm hn

/-- **Theorem (every assignment is realized).** For every assignment `a` of integers to the primes, the extension
    is completely additive, takes the value `a(p)` at every prime, and is the only completely additive function
    that does: the primes are a free basis, existence and uniqueness both proved. -/
theorem primes_admit_every_assignment (a : Nat → Int) :
    CompletelyAdditive (extendAssignment a) ∧ (∀ p, isPrime p →
    extendAssignment a p = a p) ∧
    ∀ g : Nat → Int, CompletelyAdditive g → (∀ p, isPrime p → g p = a p) →
      ∀ n, 0 < n → g n = extendAssignment a n :=
  ⟨extendAssignment_additive a, extendAssignment_prime a,
   fun g hg hga => determined_by_primes g (extendAssignment a) hg
    (extendAssignment_additive a)
     (fun p hp => (hga p hp).trans (extendAssignment_prime a p hp).symm)⟩

/-- `Ω(n) = Σ_p v_p(n)`, the prime factors of `n` counted with multiplicity, for every `n`. -/
def omegaAll (n : Nat) : Int := extendAssignment (fun _ => 1) n

/-- `λ(n) = (−1)^{Ω(n)}`, for every `n`. -/
def liouvilleAll (n : Nat) : Int := if omegaAll n % 2 = 0 then 1 else -1

theorem omegaAll_mul (m n : Nat) (hm : 0 < m) (hn : 0 < n) : omegaAll (m * n) = omegaAll m + omegaAll n :=
  extendAssignment_mul (fun _ => 1) m n hm hn

theorem omegaAll_prime (p : Nat) (hp : isPrime p) : omegaAll p = 1 :=
  extendAssignment_prime (fun _ => 1) p hp

/-- **The arrow, for every n.** `λ` is completely multiplicative, on all positive integers. -/
```

```
theorem liouvilleAll_mul (m n : Nat) (hm : 0 < m) (hn : 0 < n) :
    liouvilleAll (m * n) = liouvilleAll m * liouvilleAll n := by
  unfold liouvilleAll
  rw [omegaAll_mul m n hm hn]
  generalize omegaAll m = x
  generalize omegaAll n = y
  by_cases hx : x % 2 = 0 <;> by_cases hy : y % 2 = 0
  · rw [if_pos (show (x + y) % 2 = 0 by omega), if_pos hx, if_pos hy];
    decide
  · rw [if_neg (show ¬ (x + y) % 2 = 0 by omega), if_pos hx, if_neg hy];
    decide
  · rw [if_neg (show ¬ (x + y) % 2 = 0 by omega), if_neg hx, if_pos hy];
    decide
  · rw [if_pos (show (x + y) % 2 = 0 by omega), if_neg hx, if_neg hy];
    decide

theorem liouvilleAll_prime (p : Nat) (hp : isPrime p) : liouvilleAll p =
-1 := by
  unfold liouvilleAll
  rw [omegaAll_prime p hp]
  decide

theorem liouvilleAll_one : liouvilleAll 1 = 1 := by decide

set_option maxRecDepth 100000 in
/-- The arrow for every n agrees with the computed arrow of Section XVII
on 1..30. -/
theorem liouvilleAll_agrees_to_30 : ∀ n, n < 30 → Liouville.lam (n + 1) =
liouvilleAll (n + 1) := by decide

/-- The Liouville walk `L(x) = Σ_{1 ≤ n ≤ x} λ(n)`, for every `x`. -/
def liouvilleWalk : Nat → Int
  | 0 => 0
  | n + 1 => liouvilleWalk n + liouvilleAll (n + 1)

/-- The Liouville sentence in integers: for all `k, m ≥ 1` there is `C`
with `|L(x)|^{2m} ≤ C^{2m} x^{m+2k}` for every
    `x`, that is `L(x) = O(x^{1/2 + k/m})` at every exponent above one
    half. -/
def FaithfulLambda : Prop :=
  ∀ k m : Nat, 0 < k → 0 < m → ∃ C : Nat, ∀ x : Nat,
    (liouvilleWalk x).natAbs ^ (2 * m) ≤ C ^ (2 * m) * x ^ (m + 2 * k)

/-- The Liouville face pinned to the arithmetic: its sentence is
`FaithfulLambda`, a statement about the constructed
    walk. The link to the zeros is the cited equivalence of
    `LiouvilleFace`, carried as the one field. -/
structure LiouvilleFacePinned where
  zeros : World
  landau : RH zeros ↔ FaithfulLambda

theorem pinned_face_reads (F : LiouvilleFacePinned) : RH F.zeros ↔
FaithfulLambda := F.landau

/-- The pinned face is a Liouville face, with its sentence fixed. -/
def LiouvilleFacePinned.toFace (F : LiouvilleFacePinned) : LiouvilleFace
:=
  { zeros := F.zeros, faithful := FaithfulLambda, faithful_iff :=
    F.landau }

theorem pinned_face_is_a_face (F : LiouvilleFacePinned) :
F.toFace.faithful = FaithfulLambda := rfl

/-- **The fixed-point form.** A configuration satisfies the hypothesis
exactly when it is its own record. -/
theorem rh_iff_own_record (Z : World) : RH Z ↔ ∀ z, Z z ↔ recordOf Z z :=
by
  constructor
  · intro hZ z
    constructor
    · intro hz
      exact ⟨z, hz, reg_fixes_line z (hZ z hz)⟩
    · intro ⟨s, hs, hsr⟩
      subst hsr
      rw [reg_fixes_line s (hZ s hs)]
      exact hs
  · intro h z hz
    obtain ⟨s, _, hsr⟩ := (h z).mp hz
    subst hsr
    exact rfl

/-- The erased part of a configuration: its zeros off the line. -/
def erasedPart (Z : World) : World := fun z => Z z ∧ ¬ onLine z

/-- **The variational form.** Least erasure is leastness: a configuration
has least erasure exactly when its erased
    part lies inside the erased part of every configuration with the same
    record. -/
theorem least_erasure_is_least (Z : World) :
    LeastErasure Z ↔ ∀ Z', SameRecord Z Z' → ∀ z, erasedPart Z z →
    erasedPart Z' z := by
  rw [least_erasure_is_the_value]
  constructor
  · intro hZ _ _ z hz
    exact absurd (hZ z hz.1) hz.2
  · intro h z hz
    rcases Decidable.em (onLine z) with hon | hoff
    · exact hon
    · obtain ⟨⟨s, _, hsr⟩, hoff'⟩ := h (recordOf Z) (record_same Z) z
    ⟨hz, hoff⟩
      exact (hoff' (by rw [← hsr]; exact rfl)).elim

/-- The record is the least member of its fibre, and the only one: it has
least erasure, and every configuration of
    the fibre with least erasure has exactly its zeros. -/
theorem record_is_the_least (Z : World) :
    LeastErasure (recordOf Z) ∧ ∀ Z', SameRecord Z Z' → LeastErasure Z' →
    ∀ z, Z' z ↔ recordOf Z z := by
  have hrh : RH (recordOf Z) := by
    intro r hr
    obtain ⟨s, _, hsr⟩ := hr
    subst hsr
    exact rfl
  refine ⟨(least_erasure_is_the_value _).mpr hrh, fun Z' h hle z => ?_⟩
  exact lossless_unique Z' (recordOf Z) (fun r => (h r).symm.trans
  (record_same Z r))
    ((least_erasure_is_the_value Z').mp hle) hrh z

/-- **The closure, whole.** On the chart: the seat is the fixed set of
the fold; registration lands on it and fixes
    it; over one record both worlds exist, and no record-respecting
    reading decides the value on fold-closed
    configurations; least erasure is the value, is the fixed-point
    condition, and is leastness in the fibre; the
    lossless member of a fibre is unique; nothing escapes the cut; and
    the act prints the hypothesis at the actual
    zero set. -/
theorem the_closure :
    (∀ z : Pt, fold z = z ↔ onLine z) ∧
    (∀ z : Pt, onLine (reg z) ∧ (onLine z → reg z = z)) ∧
    (SameRecord W1 W2 ∧ RH W1 ∧ ¬ RH W2) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) ∧
    (∀ Z : World, (LeastErasure Z ↔ RH Z) ∧ (RH Z ↔ ∀ z, Z z ↔ recordOf Z
    z) ∧
      (LeastErasure Z ↔ ∀ Z', SameRecord Z Z' → ∀ z, erasedPart Z z →
      erasedPart Z' z)) ∧
    (∀ Z Z' : World, SameRecord Z Z' → RH Z → RH Z' → ∀ z, Z z ↔ Z' z) ∧
    (∀ z : Pt, (onLine z ∧ fold z = z ∧ reg z = z) ∨
      (¬ onLine z ∧ fold z ≠ z ∧ reg (fold z) = reg z ∧ ∀ w, reg w ≠ z)) ∧
    (∀ A : ActualZeros, RH A.zeros) :=
  ⟨fold_fixed_iff, fun z => ⟨(show (reg z).x = 1 from rfl),
  reg_fixes_line z⟩, ⟨same_record_12, W1_rh, W2_not_rh⟩,
   record_decides_nothing,
   fun Z => ⟨least_erasure_is_the_value Z, rh_iff_own_record Z,
   least_erasure_is_least Z⟩,
   lossless_unique, nothing_escapes_one_cut, rh_from_the_act⟩

namespace Carrier

/-- A fold on any carrier: an involution. -/
structure Fold (X : Type) where
  σ : X → X
  invol : ∀ x, σ (σ x) = x

/-- A registration for a fold: it lands on the fixed set, fixes it, and
forgets the side. -/
structure Registration {X : Type} (F : Fold X) where
  r : X → X
  lands : ∀ x, F.σ (r x) = r x
  fixes : ∀ x, F.σ x = x → r x = x
  forgets : ∀ x, r (F.σ x) = r x

def RHc {X : Type} (F : Fold X) (Z : X → Prop) : Prop := ∀ x, Z x → F.σ x
= x
def recordC {X : Type} {F : Fold X} (R : Registration F) (Z : X → Prop) :
X → Prop :=
  fun y => ∃ s, Z s ∧ R.r s = y
def SameRecordC {X : Type} {F : Fold X} (R : Registration F) (Z Z' : X →
Prop) : Prop :=
  ∀ y, recordC R Z y ↔ recordC R Z' y
def OffC {X : Type} (F : Fold X) (Z : X → Prop) : Prop := ∃ x, Z x ∧ F.σ x
≠ x
def LeastErasureC {X : Type} {F : Fold X} (R : Registration F) (Z : X →
Prop) : Prop :=
  ∀ Z', SameRecordC R Z Z' → OffC F Z → OffC F Z'

theorem record_same_C {X : Type} {F : Fold X} (R : Registration F) (Z : X
→ Prop) :
    SameRecordC R Z (recordC R Z) := by
  intro y
  constructor
  · intro (s, hs, hsy)
    exact ⟨R.r s, (s, hs, rfl), by rw [R.fixes (R.r s) (R.lands s)];
    exact hsy⟩
  · intro (s', (s, hs, hss'), hs'y)
    subst hss'
    exact (s, hs, by rw [← hs'y, R.fixes (R.r s) (R.lands s)])

theorem least_erasure_is_the_value_C {X : Type} {F : Fold X} (R :
Registration F)
    (em : ∀ x, F.σ x = x ∨ F.σ x ≠ x) (Z : X → Prop) : LeastErasureC R Z ↔
    RHc F Z := by
  constructor
  · intro hle x hx
    rcases em x with h | h
    · exact h
    · obtain ⟨y, (s, _, hsy), hy⟩ := hle (recordC R Z) (record_same_C R
    Z) (x, hx, h)
      exact (hy (by rw [← hsy]; exact R.lands s)).elim
  · intro hZ _ _ (x, hx, h)
    exact absurd (hZ x hx) h

theorem rh_iff_own_record_C {X : Type} {F : Fold X} (R : Registration F)
(Z : X → Prop) :
    RHc F Z ↔ ∀ x, Z x ↔ recordC R Z x := by
  constructor
```

```
  · intro hZ x
    constructor
    · intro hx
      exact (x, hx, R.fixes x (hZ x hx))
    · intro (s, hs, hsx)
      subst hsx
      rw [R.fixes s (hZ s hs)]
      exact hs
  · intro h x hx
    obtain (s, _, hsx) := (h x).mp hx
    subst hsx
    exact R.lands s

theorem one_record_C {X : Type} {F : Fold X} (R : Registration F) (x : X)
:
    SameRecordC R (fun s => s = R.r x) (fun s => s = x ∨ s = F.σ x) := by
  intro y
  constructor
  · intro (s, hs, hy)
    refine (x, Or.inl rfl, ?_)
    rw [← hy, hs, R.fixes (R.r x) (R.lands x)]
  · intro (s, hs, hy)
    refine (R.r x, rfl, ?_)
    rw [R.fixes (R.r x) (R.lands x), ← hy]
    rcases hs with h | h
    · rw [h]
    · rw [h, R.forgets]

theorem worlds_differ_C {X : Type} {F : Fold X} (R : Registration F) (x :
X) (h : F.σ x ≠ x) :
    RHc F (fun s => s = R.r x) ∧ ¬ RHc F (fun s => s = x ∨ s = F.σ x) :=
  (fun _ hs => hs ▸ R.lands x, fun hall => h (hall x (Or.inl rfl)))

theorem no_reading_decides_C {X : Type} {F : Fold X} (R : Registration F)
(x : X) (h : F.σ x ≠ x)
    (g : (X → Prop) → Prop) (hg : ∀ Z Z', SameRecordC R Z Z' → (g Z ↔ g
    Z')) :
    ¬ ((g (fun s => s = R.r x) ↔ RHc F (fun s => s = R.r x)) ∧
       (g (fun s => s = x ∨ s = F.σ x) ↔ RHc F (fun s => s = x ∨ s = F.σ
       x))) := by
  intro (h1, h2)
  exact (worlds_differ_C R x h).2 (h2.mp ((hg _ _ (one_record_C R x)).mp
  (h1.mpr (worlds_differ_C R x h).1)))

/-- On any carrier the lossless member of a record is unique: two
configurations of one record that both
    satisfy the hypothesis carry the same points. -/
theorem lossless_unique_C {X : Type} {F : Fold X} (R : Registration F) (Z
Z' : X → Prop)
    (hs : SameRecordC R Z Z') (h : RHc F Z) (h' : RHc F Z') : ∀ x, Z x ↔
    Z' x :=
  fun x => ((rh_iff_own_record_C R Z).mp h x).trans ((hs x).trans
  ((rh_iff_own_record_C R Z').mp h' x).symm)

/-- The chart's fold, as a carrier fold. -/
def chartFold : Fold Pt := (fold, fold_fold)

/-- The chart's registration, as a carrier registration. -/
def chartReg : Registration chartFold :=
  (reg, fun z => (fold_fixed_iff (reg z)).mpr rfl, fun z h =>
  reg_fixes_line z ((fold_fixed_iff z).mp h),
   fun _ => rfl)

/-- **The chart is one instance.** Its fixed set is the line, and the
carrier's hypothesis is the chart's. -/
theorem chart_is_an_instance :
    (∀ z, chartFold.σ z = z ↔ onLine z) ∧ ∀ Z : World, RHc chartFold Z ↔
    RH Z :=
  (fold_fixed_iff, fun _ => (fun h z hz => (fold_fixed_iff z).mp (h z
  hz), fun h z hz => (fold_fixed_iff z).mpr (h z hz)))

/-- **The closure on any carrier.** On every carrier with an involution
and a registration on which fixedness is
    decided: least erasure is the value; the value is lossless
    registration; the lossless member of a record is
    unique; and every point off the fixed set gives two worlds of one
    record that differ in the value. -/
theorem the_closure_on_any_carrier {X : Type} {F : Fold X} (R :
Registration F) (em : ∀ x, F.σ x = x ∨ F.σ x ≠ x) :
    (∀ Z, LeastErasureC R Z ↔ RHc F Z) ∧ (∀ Z, RHc F Z ↔ ∀ x, Z x ↔
    recordC R Z x) ∧
    (∀ Z Z', SameRecordC R Z Z' → RHc F Z → RHc F Z' → ∀ x, Z x ↔ Z' x) ∧
    (∀ x, F.σ x ≠ x → SameRecordC R (fun s => s = R.r x) (fun s => s = x ∨
    s = F.σ x) ∧
      RHc F (fun s => s = R.r x) ∧ ¬ RHc F (fun s => s = x ∨ s = F.σ x))
      :=
  (least_erasure_is_the_value_C R em, rh_iff_own_record_C R,
  lossless_unique_C R,
   fun x h => (one_record_C R x, worlds_differ_C R x h))

end Carrier

/-! ## XXII · The socket: every route to the value lands in the one field

An unconditional closure would be a closed term of type `LeastErasure Z`
at the actual zero set, placed in the field
`supply` of `ActualZeros`. This section proves that the field is the only
socket and that nothing reaches the value by
another way. A term of the hypothesis at a fold-closed configuration
exists exactly when the field can be filled there,
and the closure theorem of Section XXI takes any term in the field with
no other change. A premise on configurations
either forces the line or holds at a fold-closed configuration where the
value fails; a keyless premise forces nothing;
a forcing premise that respects the record fails at every nonempty
fold-closed configuration; a forcing premise either
is the value on every fold-closed configuration or fails at one that
satisfies the hypothesis; and a forcing premise
that holds at a configuration fills the field there. A sentence carried
to the zeros by a cited equivalence fills the
field exactly when it holds, whatever the sentence. The named sentences
depend on their own configurations: Weil
positivity differs between two explicit formulas with every field, no
prefix of a Li stream forces, and the Liouville
sentence, read as a property of an assignment at the primes, fails at the
assignment that sends every prime to zero.
The one refutation is one point. -/

/-- The field can be filled at a fold-closed configuration exactly when
the hypothesis holds there, and exactly when
    least erasure does. -/
theorem socket_is_the_value (Z : World) (hZ : FoldClosed Z) :
    ((∃ S : ActualZeros, S.zeros = Z) ↔ RH Z) ∧ ((∃ S : ActualZeros,
    S.zeros = Z) ↔ LeastErasure Z) := by
  have h1 : (∃ S : ActualZeros, S.zeros = Z) ↔ RH Z :=
    (fun (S, hS) => hS ▸ rh_from_the_act S, fun h => ((Z, hZ,
    (least_erasure_is_the_value Z).mpr h), rfl))
  exact (h1, h1.trans (least_erasure_is_the_value Z).symm)

/-- The closure theorem takes a term in the field and changes nothing
else: its last conjunct, applied to the
    configuration with the term, gives the hypothesis. -/
theorem closure_takes_a_term (Z : World) (hZ : FoldClosed Z) (s :
LeastErasure Z) : RH Z :=
  the_closure.2.2.2.2.2.2.2 (Z, hZ, s)

/-- At W1 the field holds a closed term, and the closure gives the
hypothesis there with nothing assumed. -/
theorem closure_at_a_term : RH W1 :=
  closure_takes_a_term W1 W1_closed ((least_erasure_is_the_value W1).mpr
  W1_rh)

/-- Every premise either forces the line or holds at a fold-closed
configuration where the value fails. Classical. -/
theorem forces_or_has_a_twin (A : World → Prop) : Forces A ∨ ∃ Z,
FoldClosed Z ∧ A Z ∧ ¬ RH Z :=
  Classical.byCases (p := ∃ Z, FoldClosed Z ∧ A Z ∧ ¬ RH Z) Or.inr
    (fun h => Or.inl (fun Z hZ ha => Classical.byContradiction (fun hn =>
    h (Z, hZ, ha, hn))))

/-- A configuration with one more pair, at real parts 0 and 1 and at the
height `t`. -/
def withPair (Z : World) (t : Nat) : World := fun w => Z w ∨ w = (0, t) ∨
w = (2, t)

theorem withPair_closed (Z : World) (hZ : FoldClosed Z) (t : Nat) :
FoldClosed (withPair Z t) := by
  intro w hw
  rcases hw with h | h | h
  · exact Or.inl (hZ w h)
  · subst h
    exact Or.inr (Or.inr rfl)
  · subst h
    exact Or.inr (Or.inl rfl)

/-- The pair added at the height of a zero leaves the record unchanged:
both new points have the registration of that zero. -/
theorem withPair_same_record (Z : World) (z : Pt) (hz : Z z) : SameRecord
Z (withPair Z z.t) := by
  intro r
  constructor
  · intro (s, hs, hsr)
    exact (s, Or.inl hs, hsr)
  · intro (s, hs, hsr)
    rcases hs with h | h | h
    · exact (s, h, hsr)
    · subst h
      exact (z, hz, hsr)
    · subst h
      exact (z, hz, hsr)

theorem withPair_not_rh (Z : World) (t : Nat) : ¬ RH (withPair Z t) := by
  intro h
  have e : (0 : Int) = 1 := h (0, t) (Or.inr (Or.inl rfl))
  omega

/-- A premise that respects the record and forces the line fails at every
nonempty fold-closed configuration: the
    configuration and its copy with one more pair share a record, and the
    copy fails the hypothesis. -/
theorem record_forcing_fails_everywhere (A : World → Prop) (hr :
RespectsRecord A) (hf : Forces A) (Z : World)
    (hZ : FoldClosed Z) (hne : ∃ z, Z z) : ¬ A Z := by
  intro ha
  obtain (z, hz) := hne
  exact withPair_not_rh Z z.t (hf (withPair Z z.t) (withPair_closed Z hZ
  z.t)
    ((hr Z (withPair Z z.t) (withPair_same_record Z z hz)).mp ha))

/-- A premise that forces the line either is the value on every
fold-closed configuration or fails at a fold-closed
    configuration that satisfies the hypothesis. Classical. -/
theorem forcing_is_the_value_or_stronger (A : World → Prop) (hf : Forces
A) :
    (∀ Z, FoldClosed Z → (A Z ↔ RH Z)) ∨ ∃ Z, FoldClosed Z ∧ RH Z ∧ ¬ A Z
    :=
  Classical.byCases (p := ∃ Z, FoldClosed Z ∧ RH Z ∧ ¬ A Z) Or.inr
```

```
    (fun h => Or.inl (fun Z hZ => (hf Z hZ, fun hrh =>
    Classical.byContradiction (fun hna => h (Z, hZ, hrh, hna)))))

/-- A premise that forces the line and holds at a fold-closed
configuration fills the field there. -/
theorem forcing_fills_the_socket (A : World → Prop) (hf : Forces A) (Z :
World) (hZ : FoldClosed Z) (ha : A Z) :
    ∃ S : ActualZeros, S.zeros = Z :=
  (socket_is_the_value Z hZ).1.mpr (hf Z hZ ha)

/-- A sentence carried to a configuration by a cited equivalence with the
hypothesis. -/
structure SocketFace where
  zeros : World
  sentence : Prop
  cited : RH zeros ↔ sentence

/-- Whatever the sentence, it holds exactly when the field can be filled
at its configuration. -/
theorem face_is_the_socket (F : SocketFace) (hF : FoldClosed F.zeros) :
    F.sentence ↔ ∃ S : ActualZeros, S.zeros = F.zeros :=
  F.cited.symm.trans (socket_is_the_value F.zeros hF).1.symm

def SocketFace.ofWeil (E : ExplicitFormula) : SocketFace := ⟨E.zeros,
WeilPositive E, (spend_is_the_line E).symm⟩
def SocketFace.ofLi (L : LiStream) : SocketFace := ⟨L.zeros, ∀ n, 1 ≤ n →
L.nonneg n, L.li⟩
def SocketFace.ofSign {V : Type} [LE V] (D : DBN V) : SocketFace :=
⟨D.zeros, D.lam ≤ D.zero, rh_iff_the_sign D⟩
def SocketFace.ofLiouville (F : LiouvilleFacePinned) : SocketFace :=
⟨F.zeros, FaithfulLambda, F.landau⟩
def SocketFace.ofPosit (Z : World) : SocketFace := ⟨Z, LeastErasure Z,
(least_erasure_is_the_value Z).symm⟩

/-- Weil positivity, the Li stream, the sign of Λ, the Liouville sentence
and least erasure itself each fill the field
    exactly when they hold. -/
theorem named_faces_are_the_socket :
    (∀ E : ExplicitFormula, FoldClosed E.zeros → (WeilPositive E ↔ ∃ S :
    ActualZeros, S.zeros = E.zeros)) ∧
    (∀ L : LiStream, FoldClosed L.zeros → ((∀ n, 1 ≤ n → L.nonneg n) ↔ ∃ S
    : ActualZeros, S.zeros = L.zeros)) ∧
    (∀ (V : Type) [LE V] (D : DBN V), FoldClosed D.zeros → (D.lam ≤
    D.zero ↔ ∃ S : ActualZeros, S.zeros = D.zeros)) ∧
    (∀ F : LiouvilleFacePinned, FoldClosed F.zeros → (FaithfulLambda ↔ ∃ S
    : ActualZeros, S.zeros = F.zeros)) ∧
    (∀ Z : World, FoldClosed Z → (LeastErasure Z ↔ ∃ S : ActualZeros,
    S.zeros = Z)) :=
  ⟨fun E hE => face_is_the_socket (SocketFace.ofWeil E) hE,
   fun L hL => face_is_the_socket (SocketFace.ofLi L) hL,
   fun _ _ D hD => face_is_the_socket (SocketFace.ofSign D) hD,
   fun F hF => face_is_the_socket (SocketFace.ofLiouville F) hF,
   fun Z hZ => face_is_the_socket (SocketFace.ofPosit Z) hZ⟩

/-- The sign arrow of an assignment at the primes: `(−1)^{f(n)}` for the
completely additive `f` with the value `a(p)`
    at every prime. The assignment that sends every prime to one gives
    `λ`. -/
def arrowOf (a : Nat → Int) (n : Nat) : Int := if extendAssignment a n % 2
= 0 then 1 else -1

/-- The walk of an assignment: the sum of its arrow over `1 ≤ n ≤ x`. -/
def walkOf (a : Nat → Int) : Nat → Int
  | 0 => 0
  | n + 1 => walkOf a n + arrowOf a (n + 1)

/-- The Liouville sentence for an assignment at the primes. -/
def FaithfulOf (a : Nat → Int) : Prop :=
  ∀ k m : Nat, 0 < k → 0 < m → ∃ C : Nat, ∀ x : Nat,
    (walkOf a x).natAbs ^ (2 * m) ≤ C ^ (2 * m) * x ^ (m + 2 * k)

theorem walk_is_the_walk_of_one (x : Nat) : liouvilleWalk x = walkOf (fun
_ => 1) x := by
  induction x with
  | zero => rfl
  | succ n ih =>
    show liouvilleWalk n + liouvilleAll (n + 1) = walkOf (fun _ => 1) n +
    arrowOf (fun _ => 1) (n + 1)
    rw [ih]
    rfl

/-- The Liouville sentence is the sentence of the assignment that sends
every prime to one. -/
theorem faithful_is_the_sentence_of_one : FaithfulLambda ↔ FaithfulOf
(fun _ => 1) := by
  unfold FaithfulLambda FaithfulOf
  simp only [walk_is_the_walk_of_one]

/-- The assignment that sends every prime to zero extends to zero
everywhere. -/
theorem blind_assignment_vanishes (n : Nat) : extendAssignment (fun _ =>
0) n = 0 := by
  unfold extendAssignment
  exact sumBelow_zero _ _ (fun i _ => by unfold extendTerm; split <;>
  simp)

/-- At that assignment the arrow is one everywhere and the walk is the
identity. -/
theorem blind_walk (x : Nat) : walkOf (fun _ => 0) x = (x : Int) := by
  induction x with
  | zero => rfl
  | succ n ih =>
    show walkOf (fun _ => 0) n + arrowOf (fun _ => 0) (n + 1) = ((n + 1 :
    Nat) : Int)
    rw [ih]
    unfold arrowOf
    rw [blind_assignment_vanishes, if_pos (by decide)]
    omega

/-- The Liouville sentence fails at the assignment that sends every prime
to zero: at `k = 1, m = 3` the walk `x` would
    need `x^6 ≤ C^6 x^5`, which fails at `x = C^6 + 1`. -/
theorem blind_is_unfaithful : ¬ FaithfulOf (fun _ => 0) := by
  intro h
  obtain ⟨C, hC⟩ := h 1 3 (by decide) (by decide)
  have hx := hC (C ^ 6 + 1)
  rw [blind_walk] at hx
  change (C ^ 6 + 1) ^ 6 ≤ C ^ 6 * (C ^ 6 + 1) ^ 5 at hx
  have hpos : 0 < (C ^ 6 + 1) ^ 5 := Nat.pow_pos (Nat.succ_pos _)
  have e : (C ^ 6 + 1) ^ 6 = (C ^ 6 + 1) ^ 5 * (C ^ 6 + 1) :=
  Nat.pow_succ (C ^ 6 + 1) 5
  rw [e] at hx
  generalize (C ^ 6 + 1) ^ 5 = y at hx hpos
  generalize C ^ 6 = c at hx
  rw [Nat.mul_succ, Nat.mul_comm c y] at hx
  omega

/-- The Liouville sentence reads the primes: it is the sentence of one
assignment, it fails at another, and so no proof
    of it holds for every assignment. -/
theorem faithful_reads_the_primes :
    (FaithfulLambda ↔ FaithfulOf (fun _ => 1)) ∧ ¬ FaithfulOf (fun _ =>
    0) ∧ ¬ ∀ a : Nat → Int, FaithfulOf a :=
  ⟨faithful_is_the_sentence_of_one, blind_is_unfaithful, fun h =>
  blind_is_unfaithful (h _)⟩

/-- **The socket, whole.** The field of the act is the one socket: it can
be filled at a fold-closed configuration
    exactly when the hypothesis holds, and the closure takes any term in
    it with no other change. Every premise on
    configurations forces the line or holds at a fold-closed
    configuration where the value fails; a keyless premise
    forces nothing; a premise that respects the record and forces the
    line fails at every nonempty fold-closed
    configuration; a forcing premise is the value or fails at a
    configuration that satisfies the hypothesis, and fills
    the field wherever it holds. Every sentence carried to the zeros by a
    cited equivalence fills the field exactly when
    it holds. The named sentences depend on their configurations, and the
    one refutation is one point. -/
theorem the_socket :
    (∀ Z : World, FoldClosed Z → ((∃ S : ActualZeros, S.zeros = Z) ↔ RH
    Z)) ∧
    (∀ Z : World, FoldClosed Z → LeastErasure Z → RH Z) ∧
    (∀ A : World → Prop, Forces A ∨ ∃ Z, FoldClosed Z ∧ A Z ∧ ¬ RH Z) ∧
    (∀ A : World → Prop, Keyless A → ¬ Forces A) ∧
    (∀ A : World → Prop, RespectsRecord A → Forces A → ∀ Z, FoldClosed Z →
    (∃ z, Z z) → ¬ A Z) ∧
    (∀ A : World → Prop, Forces A → (∀ Z, FoldClosed Z → (A Z ↔ RH Z)) ∨ ∃
    Z, FoldClosed Z ∧ RH Z ∧ ¬ A Z) ∧
    (∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z → ∃ S :
    ActualZeros, S.zeros = Z) ∧
    (∀ F : SocketFace, FoldClosed F.zeros → (F.sentence ↔ ∃ S :
    ActualZeros, S.zeros = F.zeros)) ∧
    (WeilPositive positiveInstance ∧ ¬ WeilPositive negativeInstance) ∧
    (∀ N : Nat, ∃ L : LiStream, (∀ n, n < N → L.nonneg n) ∧ ¬ RH L.zeros)
    ∧
    ((FaithfulLambda ↔ FaithfulOf (fun _ => 1)) ∧ ¬ FaithfulOf (fun _ =>
    0)) ∧
    (∀ Z : World, ¬ LeastErasure Z ↔ OffLine Z) :=
  ⟨fun Z hZ => (socket_is_the_value Z hZ).1, closure_takes_a_term,
  forces_or_has_a_twin, keyless_forces_nothing,
   record_forcing_fails_everywhere, forcing_is_the_value_or_stronger,
   forcing_fills_the_socket, face_is_the_socket,
   positivity_is_keyed, li_prefix_never_forces,
   ⟨faithful_is_the_sentence_of_one, blind_is_unfaithful⟩,
   rejection_is_a_witness⟩

/-! ### XXII.b · At resolution two the open strip holds a pair off the
line

At resolution one the chart's open strip `0 < x < 2` holds only the line,
so a premise that places every zero inside
the open strip forces the line there: a fact of the coarse chart, not of
the zeros. On the chart at resolution two, with
`x = 4 Re s`, the line `x = 2`, the fold `x ↦ 4 − x` and registration
`(x, t) ↦ (2, t)`, the open strip `0 < x < 4`
holds the pair `x = 1, 3`, real parts one quarter and three quarters.
There the open-strip premise forces nothing, no
reading of the record decides the value, and the refusals of the socket
hold for premises that range over strip
configurations only. -/

def fold₂ (z : Pt) : Pt := ⟨4 - z.x, z.t⟩
def reg₂ (z : Pt) : Pt := ⟨2, z.t⟩
def onLine₂ (z : Pt) : Prop := z.x = 2
def InStrip₂ (z : Pt) : Prop := 0 < z.x ∧ z.x < 4
/-- A strip configuration at resolution two: fold-closed, and every zero
inside the open strip. -/
def StripWorld₂ (Z : World) : Prop := (∀ z, Z z → Z (fold₂ z)) ∧ ∀ z, Z z
→ InStrip₂ z
def RH₂ (Z : World) : Prop := ∀ z, Z z → onLine₂ z
def SameRecord₂ (Z Z' : World) : Prop := ∀ r, (∃ s, Z s ∧ reg₂ s = r) ↔
(∃ s, Z' s ∧ reg₂ s = r)
/-- A premise respects the record when it reads alike on two strip
configurations of one record. -/
def RespectsRecord₂ (A : World → Prop) : Prop :=
```

```
  ∀ Z Z', StripWorld₂ Z → StripWorld₂ Z' → SameRecord₂ Z Z' → (A Z ↔ A Z')
/-- A premise forces the line when every strip configuration it holds on
satisfies the hypothesis. -/
def Forces₂ (A : World → Prop) : Prop := ∀ Z, StripWorld₂ Z → A Z → RH₂ Z

/-- The seat at resolution two: the fixed set of the fold is the line. -/
theorem fold₂_fixed_iff (z : Pt) : fold₂ z = z ↔ onLine₂ z := by
  cases z with
  | mk x t =>
    show Pt.mk (4 - x) t = Pt.mk x t ↔ x = 2
    constructor
    · intro h
      have hx := congrArg Pt.x h
      change 4 - x = x at hx
      omega
    · intro h
      subst h
      rfl

/-- At resolution one the open strip is the line: the premise that places
every zero inside it forces the line. -/
theorem open_strip_forces_at_resolution_one : Forces (fun Z => ∀ z, Z z →
0 < z.x ∧ z.x < 2) := by
  intro Z _ h z hz
  have := h z hz
  show z.x = 1
  omega

/-- The pair at real parts one quarter and three quarters, at the height
`t`. -/
def innerPair₂ (t : Nat) : World := fun w => w = (1, t) ∨ w = (3, t)

theorem innerPair₂_strip (t : Nat) : StripWorld₂ (innerPair₂ t) := by
  refine (fun w hw => ?_, fun w hw => ?_)
  · rcases hw with h | h
    · subst h
      exact Or.inr rfl
    · subst h
      exact Or.inl rfl
  · rcases hw with h | h
    · subst h
      exact (show (0 : Int) < 1 by decide, show (1 : Int) < 4 by decide)
    · subst h
      exact (show (0 : Int) < 3 by decide, show (3 : Int) < 4 by decide)

theorem innerPair₂_not_rh (t : Nat) : ¬ RH₂ (innerPair₂ t) := by
  intro h
  have e : (1 : Int) = 2 := h (1, t) (Or.inl rfl)
  omega

/-- At resolution two the open-strip premise forces nothing: the inner
pair satisfies it and fails the hypothesis. -/
theorem open_strip_forces_nothing_at_resolution_two : ¬ Forces₂ (fun Z =>
∀ z, Z z → InStrip₂ z) :=
  fun h => innerPair₂_not_rh 0 (h (innerPair₂ 0) (innerPair₂_strip 0)
  (innerPair₂_strip 0).2)

/-- A premise true on every configuration forces nothing on strip
configurations. -/
theorem keyless_forces_nothing₂ (A : World → Prop) (hA : ∀ Z, A Z) : ¬
Forces₂ A :=
  fun h => innerPair₂_not_rh 0 (h _ (innerPair₂_strip 0) (hA _))

/-- A strip configuration with the inner pair added at the height `t`. -/
def withPair₂ (Z : World) (t : Nat) : World := fun w => Z w ∨ w = (1, t) ∨
w = (3, t)

theorem withPair₂_strip (Z : World) (hZ : StripWorld₂ Z) (t : Nat) :
StripWorld₂ (withPair₂ Z t) := by
  refine (fun w hw => ?_, fun w hw => ?_)
  · rcases hw with h | h | h
    · exact Or.inl (hZ.1 w h)
    · subst h
      exact Or.inr (Or.inr rfl)
    · subst h
      exact Or.inr (Or.inl rfl)
  · rcases hw with h | h | h
    · exact hZ.2 w h
    · subst h
      exact (show (0 : Int) < 1 by decide, show (1 : Int) < 4 by decide)
    · subst h
      exact (show (0 : Int) < 3 by decide, show (3 : Int) < 4 by decide)

theorem withPair₂_same_record (Z : World) (z : Pt) (hz : Z z) :
SameRecord₂ Z (withPair₂ Z z.t) := by
  intro r
  constructor
  · intro (s, hs, hsr)
    exact (s, Or.inl hs, hsr)
  · intro (s, hs, hsr)
    rcases hs with h | h | h
    · exact (s, h, hsr)
    · subst h
      exact (z, hz, hsr)
    · subst h
      exact (z, hz, hsr)

theorem withPair₂_not_rh (Z : World) (t : Nat) : ¬ RH₂ (withPair₂ Z t) :=
by
  intro h
  have e : (1 : Int) = 2 := h (1, t) (Or.inr (Or.inl rfl))
  omega

/-- A premise that respects the record and forces the line on strip
configurations fails at every nonempty strip
    configuration: its copy with the inner pair at the height of one of
    its zeros shares its record and fails the
    hypothesis. -/
theorem record_forcing_fails_in_the_strip (A : World → Prop) (hr :
RespectsRecord₂ A) (hf : Forces₂ A) (Z : World)
    (hZ : StripWorld₂ Z) (hne : ∃ z, Z z) : ¬ A Z := by
  intro ha
  obtain (z, hz) := hne
  have hs := withPair₂_strip Z hZ z.t
  exact withPair₂_not_rh Z z.t (hf _ hs ((hr Z _ hZ hs
  (withPair₂_same_record Z z hz)).mp ha))

/-- No reading that respects the record decides the hypothesis on strip
configurations. -/
theorem no_reading_decides_in_the_strip (g : World → Prop) (hg :
RespectsRecord₂ g) :
    ¬ ∀ Z, StripWorld₂ Z → (g Z ↔ RH₂ Z) := by
  intro h
  have hL : StripWorld₂ (fun w => w = (2, 14)) :=
    (fun w hw => by rw [hw]; rfl, fun w hw => by rw [hw]; exact (by
    decide, by decide))
  have hLrh : RH₂ (fun w => w = (2, 14)) := fun w hw => by rw [hw]; rfl
  have hs := withPair₂_strip _ hL 14
  have hsame : SameRecord₂ (fun w => w = (2, 14)) (withPair₂ (fun w => w =
  (2, 14)) 14) :=
    withPair₂_same_record _ (2, 14) rfl
  exact withPair₂_not_rh _ 14 ((h _ hs).mp ((hg _ _ hL hs hsame).mp ((h _
  hL).mpr hLrh)))

/-- Every premise either forces the line on strip configurations or holds
at a strip configuration where the value
    fails. Classical. -/
theorem forces_or_has_a_twin₂ (A : World → Prop) : Forces₂ A ∨ ∃ Z,
StripWorld₂ Z ∧ A Z ∧ ¬ RH₂ Z :=
  Classical.byCases (p := ∃ Z, StripWorld₂ Z ∧ A Z ∧ ¬ RH₂ Z) Or.inr
    (fun h => Or.inl (fun Z hZ ha => Classical.byContradiction (fun hn =>
    h (Z, hZ, ha, hn))))

/-- **The socket in the strip.** At resolution one the open strip is the
line; at resolution two the fold fixes the
    line, the open strip holds a pair off the line at every height, the
    open-strip premise and every keyless premise
    force nothing, no reading of the record decides, a record-respecting
    forcing premise fails at every nonempty strip
    configuration, and every premise forces or holds where the value
    fails. -/
theorem the_socket_in_the_strip :
    Forces (fun Z => ∀ z, Z z → 0 < z.x ∧ z.x < 2) ∧
    (∀ z : Pt, fold₂ z = z ↔ onLine₂ z) ∧
    (∀ t : Nat, StripWorld₂ (innerPair₂ t) ∧ ¬ RH₂ (innerPair₂ t)) ∧
    ¬ Forces₂ (fun Z => ∀ z, Z z → InStrip₂ z) ∧
    (∀ A : World → Prop, (∀ Z, A Z) → ¬ Forces₂ A) ∧
    (∀ g : World → Prop, RespectsRecord₂ g → ¬ ∀ Z, StripWorld₂ Z → (g Z ↔
    RH₂ Z)) ∧
    (∀ A : World → Prop, RespectsRecord₂ A → Forces₂ A → ∀ Z, StripWorld₂
    Z → (∃ z, Z z) → ¬ A Z) ∧
    (∀ A : World → Prop, Forces₂ A ∨ ∃ Z, StripWorld₂ Z ∧ A Z ∧ ¬ RH₂ Z)
    :=
  ⟨open_strip_forces_at_resolution_one, fold₂_fixed_iff, fun t =>
  ⟨innerPair₂_strip t, innerPair₂_not_rh t⟩,
   open_strip_forces_nothing_at_resolution_two, keyless_forces_nothing₂,
   no_reading_decides_in_the_strip,
   record_forcing_fails_in_the_strip, forces_or_has_a_twin₂⟩

/-! ## XXII.c · Two denials, and the witness the act forces

A denial has two parts: the act of making it, and the sentence it
asserts. Two denials are read side by side, over
every self-grounding root. The denial of the root asserts that the root
fails; the act of making it instances the
root, so its sentence is refuted by its own act and only the root
remains. The denial of the hypothesis asserts that
a zero lies off the line; the act of making it instances the root, which
the twin carries too, and its sentence denies
the self-grounding assertion of the line at the zero set, which the twin
does not carry: in time the twin is timeless
and unseeded. The assertion stands at the actual zero set by the act of
section XI, and there the denial is refuted.
On the act every computation that reads the zero set reports points on
the line, and a computation that reports a
point off the line reads no act. -/

/-- A seeded timeless witness of the line, for a world in time: one locus
at every instant, the same truth value of
    the hypothesis at every instant, and the hypothesis at instant 0, the
    seed. -/
structure SeededWitness (W : TWorld) : Prop where
  same_locus : SameLocus W
  timeless   : TimelessT W
  seed       : RH (W 0)

/-- A world in time carries a seeded timeless witness exactly when the
hypothesis holds at every instant. -/
theorem seeded_witness_iff_bound (W : TWorld) : SeededWitness W ↔ BoundT W
:=
  ⟨fun w => ⟨template_binds_iff_seed W w.timeless).mpr w.seed,
   fun h => ⟨fold_face_keyless W, fun t s => ⟨fun _ => h s, fun _ => h
   t⟩, h 0⟩⟩

/-- The constant twin, timeless and fold-closed at every instant, carries
no seeded witness. -/
```

```
theorem twin_has_no_seeded_witness : ¬ SeededWitness WTwin :=
  fun w => W2f_not_rh w.seed

/-- The constant on-line world carries one. -/
theorem line_world_has_a_seeded_witness : SeededWitness WOn :=
  (seeded_witness_iff_bound WOn).mpr closure_face_on_line

/-- Every denial, whatever its sentence, is an act, and every act
instances the root. -/
theorem every_denial_is_an_act {root C : Prop} (G : SelfGrounding root)
(a : G.Act) (_c : C) : root :=
  G.instances a

/-- A denial of a self-grounding root is refuted by its own act: its
sentence denies what its act instances. -/
theorem root_denial_refutes_itself {root : Prop} (G : SelfGrounding root)
(a : G.Act) (n : ¬ root) : False :=
  n (G.instances a)

/-- A denial of the hypothesis at a world denies the self-grounding
assertion of the line there. -/
theorem line_denial_denies_the_assertion (Z : World) (h : ¬ RH Z) :
    ¬ ∃ G : SelfGrounding (RH Z), Nonempty G.Act :=
  fun e => h ((supply_iff (RH Z)).mp e)

/-- In time, a denial of the hypothesis at instant 0 denies every seeded
witness. -/
theorem line_denial_denies_every_seed (W : TWorld) (h : ¬ RH (W 0)) : ¬
SeededWitness W :=
  fun w => h w.seed

/-- The twin carries the root, fold-closure, the record of W1 and the
timeless form; it carries neither a
    self-grounding assertion of the line nor a seeded witness. -/
theorem twin_carries_the_root_and_no_seed {root : Prop} (G :
SelfGrounding root) :
    root ∧ FoldClosed W2f ∧ SameRecord W1 W2f ∧ TimelessT WTwin ∧ ¬ RH
    W2f ∧
    (¬ ∃ G' : SelfGrounding (RH W2f), Nonempty G'.Act) ∧ ¬ SeededWitness
    WTwin :=
  ⟨G.instances G.anAct, W2f_closed, same_record_f, fun _ _ => Iff.rfl,
  W2f_not_rh, line_not_self_grounding,
   twin_has_no_seeded_witness⟩

/-- The root alone forces nothing: it holds on every world, the twin
written out included. -/
theorem the_root_alone_forces_nothing {root : Prop} (G : SelfGrounding
root) : ¬ Forces (fun _ => root) :=
  keyless_forces_nothing_free (fun _ => root) (fun _ => G.instances
  G.anAct)

/-- At every instance of the actual zero set the self-grounding assertion
of the line exists. -/
theorem the_act_asserts_the_line (A : ActualZeros) : ∃ G : SelfGrounding
(RH A.zeros), Nonempty G.Act :=
  (supply_iff (RH A.zeros)).mpr (rh_from_the_act A)

/-- The constant family of the actual zero set carries a seeded witness.
-/
theorem the_act_seeds_its_constant_world (A : ActualZeros) :
SeededWitness (fun _ => A.zeros) :=
  (seeded_witness_iff_bound _).mpr (fun _ => rh_from_the_act A)

/-- On the act the denial of the hypothesis is refuted. -/
theorem line_denial_refuted_on_the_act (A : ActualZeros) (h : ¬ RH
A.zeros) : False :=
  h (rh_from_the_act A)

/-- TWO DENIALS, ONE FORM. For every self-grounding root: every denial is
an act and instances the root; the denial
    of the root is refuted by its own act; the denial of the hypothesis
    denies the self-grounding assertion of the line
    and, in time, the seed; the twin carries the root and neither of
    these, so the root alone forces nothing, and a
    seeded witness exists exactly where the line holds at every instant;
    on the act the assertion exists, the seed
    stands, and the denial is refuted. -/
theorem two_denials {root : Prop} (G : SelfGrounding root) :
    (∀ C : Prop, G.Act → C → root) ∧
    (G.Act → ¬ root → False) ∧
    (∀ Z : World, ¬ RH Z → ¬ ∃ G' : SelfGrounding (RH Z), Nonempty
    G'.Act) ∧
    (∀ W : TWorld, SeededWitness W ↔ BoundT W) ∧
    (root ∧ FoldClosed W2f ∧ SameRecord W1 W2f ∧ TimelessT WTwin ∧ ¬ RH
    W2f ∧
      (¬ ∃ G' : SelfGrounding (RH W2f), Nonempty G'.Act) ∧ ¬
      SeededWitness WTwin) ∧
    SeededWitness WOn ∧
    ¬ Forces (fun _ => root) ∧
    (∀ A : ActualZeros, (∃ G' : SelfGrounding (RH A.zeros), Nonempty
    G'.Act) ∧
      SeededWitness (fun _ => A.zeros) ∧ (¬ RH A.zeros → False)) :=
  ⟨fun _ a c => every_denial_is_an_act G a c,
   fun a n => root_denial_refutes_itself G a n,
   line_denial_denies_the_assertion,
   seeded_witness_iff_bound,
   twin_carries_the_root_and_no_seed G,
   line_world_has_a_seeded_witness,
   the_root_alone_forces_nothing G,
   fun A => ⟨the_act_asserts_the_line A, the_act_seeds_its_constant_world
   A, line_denial_refuted_on_the_act A⟩⟩

/-- At the root proposition 0 < 1, with the unit act. -/
theorem two_denials_at_the_root :
    (∀ C : Prop, rootSelfGrounding.Act → C → RootAct) ∧
    (rootSelfGrounding.Act → ¬ RootAct → False) ∧
    (∀ Z : World, ¬ RH Z → ¬ ∃ G' : SelfGrounding (RH Z), Nonempty
    G'.Act) ∧
    (∀ W : TWorld, SeededWitness W ↔ BoundT W) ∧
    (RootAct ∧ FoldClosed W2f ∧ SameRecord W1 W2f ∧ TimelessT WTwin ∧ ¬
    RH W2f ∧
      (¬ ∃ G' : SelfGrounding (RH W2f), Nonempty G'.Act) ∧ ¬
      SeededWitness WTwin) ∧
    SeededWitness WOn ∧
    ¬ Forces (fun _ => RootAct) ∧
    (∀ A : ActualZeros, (∃ G' : SelfGrounding (RH A.zeros), Nonempty
    G'.Act) ∧
      SeededWitness (fun _ => A.zeros) ∧ (¬ RH A.zeros → False)) :=
  two_denials rootSelfGrounding

/-- A computation that ends: the finite list of points it reports as
zeros. -/
structure Computation where
  reports : List Pt

/-- A computation reads an act when every point it reports is a zero of
the act's zero set. -/
def Computation.Reads (c : Computation) (A : ActualZeros) : Prop := ∀ z, z
∈ c.reports → A.zeros z

/-- On the act nothing is left open: the hypothesis, least erasure, no
point off the line, the zero set its own
    record, and every consequence of the hypothesis at the zero set. -/
theorem nothing_left_open_on_the_act (A : ActualZeros) :
    RH A.zeros ∧ LeastErasure A.zeros ∧ ¬ OffLine A.zeros ∧ (∀ z, A.zeros
    z ↔ recordOf A.zeros z) ∧
    (∀ Q : World → Prop, (∀ Z, RH Z → Q Z) → Q A.zeros) :=
  ⟨rh_from_the_act A, A.supply, fun ⟨z, hz, h⟩ => h (rh_from_the_act A z
  hz),
   ⟨rh_iff_own_record A.zeros).mp (rh_from_the_act A), fun _ hQ => hQ
   A.zeros (rh_from_the_act A)⟩⟩

/-- Every computation that reads an act reports only points on the line.
-/
theorem computation_reading_the_act_lands_on_the_line (A : ActualZeros)
(c : Computation) (h : c.Reads A) :
    ∀ z, z ∈ c.reports → onLine z :=
  fun z hz => rh_from_the_act A z (h z hz)

/-- A computation that reports a point off the line reads no act. -/
theorem off_line_report_reads_no_act (c : Computation) (z : Pt) (hz : z ∈
c.reports) (h : ¬ onLine z) :
    ¬ ∃ A : ActualZeros, c.Reads A :=
  fun ⟨A, hA⟩ => h (rh_from_the_act A z (hA z hz))

/-- The computation that reports the zero of the twin at real part 0. -/
def twinReport : Computation := ⟨[⟨0, 14⟩]⟩

/-- A computation that reads no act is not forced: the twin's report
stands off the line and reads no act. -/
theorem a_computation_reading_no_act_is_not_forced :
    ¬ (∀ c : Computation, ∀ z, z ∈ c.reports → onLine z) ∧ ¬ ∃ A :
    ActualZeros, twinReport.Reads A :=
  ⟨fun h => absurd (h twinReport ⟨0, 14⟩ (List.Mem.head _)) (by decide : ¬
  ((0 : Int) = 1)),
   off_line_report_reads_no_act twinReport ⟨0, 14⟩ (List.Mem.head _) (by
   decide : ¬ ((0 : Int) = 1))⟩

/-- THE WITNESS FORCED BY THE ACT. On every act nothing is left open;
every computation that reads the act reports
    only points on the line; a computation that reports a point off the
    line reads no act; and the forcing reaches
    exactly the computations that read an act. -/
theorem the_witness_forced_by_the_act :
    (∀ A : ActualZeros, RH A.zeros ∧ LeastErasure A.zeros ∧ ¬ OffLine
    A.zeros ∧
      (∀ z, A.zeros z ↔ recordOf A.zeros z) ∧ (∀ Q : World → Prop, (∀ Z,
      RH Z → Q Z) → Q A.zeros)) ∧
    (∀ (A : ActualZeros) (c : Computation), c.Reads A → ∀ z, z ∈
    c.reports → onLine z) ∧
    (∀ (c : Computation) (z : Pt), z ∈ c.reports → ¬ onLine z → ¬ ∃ A :
    ActualZeros, c.Reads A) ∧
    ¬ (∀ c : Computation, ∀ z, z ∈ c.reports → onLine z) :=
  ⟨nothing_left_open_on_the_act,
  computation_reading_the_act_lands_on_the_line,
  off_line_report_reads_no_act,
   a_computation_reading_no_act_is_not_forced.1⟩

/-- Section XXII.c, whole, over every self-grounding root. -/
theorem the_denials_and_the_witness {root : Prop} (G : SelfGrounding
root) :
    ((∀ C : Prop, G.Act → C → root) ∧
     (G.Act → ¬ root → False) ∧
     (∀ Z : World, ¬ RH Z → ¬ ∃ G' : SelfGrounding (RH Z), Nonempty
     G'.Act) ∧
     (∀ W : TWorld, SeededWitness W ↔ BoundT W) ∧
     (root ∧ FoldClosed W2f ∧ SameRecord W1 W2f ∧ TimelessT WTwin ∧ ¬ RH
     W2f ∧
       (¬ ∃ G' : SelfGrounding (RH W2f), Nonempty G'.Act) ∧ ¬
       SeededWitness WTwin) ∧
     SeededWitness WOn ∧
     ¬ Forces (fun _ => root) ∧
     (∀ A : ActualZeros, (∃ G' : SelfGrounding (RH A.zeros), Nonempty
     G'.Act) ∧
       SeededWitness (fun _ => A.zeros) ∧ (¬ RH A.zeros → False))) ∧
    ((∀ A : ActualZeros, RH A.zeros ∧ LeastErasure A.zeros ∧ ¬ OffLine
    A.zeros ∧
```

```
      (∀ z, A.zeros z ↔ recordOf A.zeros z) ∧ (∀ Q : World → Prop, (∀ Z,
       RH Z → Q Z) → Q A.zeros)) ∧
      (∀ (A : ActualZeros) (c : Computation), c.Reads A → ∀ z, z ∈
      c.reports → onLine z) ∧
      (∀ (c : Computation) (z : Pt), z ∈ c.reports → ¬ onLine z → ¬ ∃ A :
      ActualZeros, c.Reads A) ∧
      ¬ (∀ c : Computation, ∀ z, z ∈ c.reports → onLine z)) :=
  (two_denials G, the_witness_forced_by_the_act)

/-! ## XXIII · Double security: the two channels of the line

The formal gate: nothing passes for the hypothesis but least erasure, and
the act passes it with an empty cone. The
actuation gate: nothing passes against it but a point off the line; the
root, which every act re-enacts, crosses
neither way; a registration costs nothing exactly at least erasure, one
floor per erased bit. Between the gates the
bit stays keyed. Every conjunct is a theorem above; this section adds
their conjunction and no content. -/

/-- DOUBLE SECURITY. The formal gate, the actuation gate, and the keyed
bit between them. -/
theorem double_security :
    (∀ A : ActualZeros, RH A.zeros) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ Z : World, ¬ LeastErasure Z ↔ OffLine Z) ∧
    (∀ Act : Type, Act → RootAct) ∧
    (¬ ∀ Z : World, RootAct → RH Z) ∧
    (¬ ∀ Z : World, RootAct → ¬ RH Z) ∧
    (¬ ∃ G : SelfGrounding (RH W2f), Nonempty G.Act) ∧
    (∀ C : FinCfg, C.price true = 0 ↔ LeastErasure C.world) ∧
    landauerFloor = 2870978885078723755499100 ∧
    ¬ RespectsRecord LeastErasure ∧
    (∃ Z : World, FoldClosed Z ∧ ¬ LeastErasure Z) :=
  ⟨rh_from_the_act, least_erasure_is_the_value, rejection_is_a_witness,
   fun _ a => act_reenacts_root a, root_does_not_cross_the_line,
   root_does_not_cross_the_denial,
   line_not_self_grounding, price_zero_iff_least_erasure,
   landauer_floor_exact,
   least_erasure_reads_past_the_record,
   ⟨W2, W2_closed, fun h => W2_not_rh ((least_erasure_is_the_value W2).mp
   h)⟩⟩

/-! ## XXIV · the root, undeniable in act, and the lock -/

/-- THE ROOT IS UNDENIABLE IN ACT: a denial of a self-grounding root is
an act, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial :
G.Act) : R :=
  G.instances denial

/-- The root is held by the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

/-- No outside proof adds anything to a self-grounding root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q :
Prop) : (Q → R) ↔ R :=
  ⟨fun _ => G.instances G.anAct, fun r _ => r⟩

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it holds by its act; the
two fold-closed worlds share one
record, the hypothesis holds on the first and fails on the second; so
read uniformly on the worlds the
root forces no value. -/
theorem undeniable_root_forces_no_value {R : Prop} (G : SelfGrounding R) :
    R ∧ (FoldClosed W1 ∧ RH W1) ∧ (FoldClosed W2 ∧ ¬ RH W2) ∧ SameRecord
      W1 W2 ∧ ¬ Forces (fun _ => R) :=
  ⟨G.instances G.anAct, ⟨W1_closed, W1_rh⟩, ⟨W2_closed, W2_not_rh⟩,
   same_record_12,
    keyless_forces_nothing (fun _ => R) (fun _ => G.instances G.anAct)⟩

/-- The root's form: every existent of a domain actuates, its actuation
positive. -/
def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

/-- The indicator of a decided proposition: one where it holds, zero
where it fails. -/
def indicator {p : Prop} : Decidable p → Int
  | isTrue _  => 1
  | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds. -/
theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h ↔ p
:=
  match h with
  | isTrue hp  => ⟨fun _ => hp, fun _ => show (0 : Int) < 1 by decide⟩
  | isFalse hn => ⟨fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by decide),
    fun hp => absurd hp hn⟩

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent, whose
actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) := fun _ => show
(0 : Int) < 1 by decide

/-- THE ROOT READ AT THE ZEROS: to exist is to actuate, `Root` itself,
with the existents the points of
the world and the actuation of a point one where it stands on the line
and zero where it does not. -/
def RootAtZeros (Z : World) : Prop :=
  Root { z : Pt // Z z } (fun z => indicator (inferInstanceAs (Decidable
  (onLine z.1))))

/-- THE ROOT READ AT THE ZEROS IS THE HYPOTHESIS. -/
theorem root_at_zeros_is_the_hypothesis (Z : World) : RootAtZeros Z ↔ RH Z
:=
  ⟨fun h z hz => (indicator_pos _).mp (h ⟨z, hz⟩), fun h z =>
  (indicator_pos _).mpr (h z.1 z.2)⟩

/-- THE ROOT READ AT THE ZEROS IS LEAST ERASURE. -/
theorem root_at_zeros_is_least_erasure (Z : World) : RootAtZeros Z ↔
LeastErasure Z :=
  ⟨fun h => (least_erasure_is_the_value Z).mpr
  ((root_at_zeros_is_the_hypothesis Z).mp h),
   fun h => (root_at_zeros_is_the_hypothesis Z).mpr
   ((least_erasure_is_the_value Z).mp h)⟩

/-- THE ROOT READ AT THE ZEROS IS KEYED: it holds on the fold-closed
world on the line and fails on its
fold-closed twin of one record. -/
theorem root_at_zeros_is_keyed :
    (FoldClosed W1 ∧ RootAtZeros W1) ∧ (FoldClosed W2 ∧ ¬ RootAtZeros W2)
    :=
  ⟨(W1_closed, (root_at_zeros_is_the_hypothesis W1).mpr W1_rh),
   (W2_closed, fun h => W2_not_rh ((root_at_zeros_is_the_hypothesis
   W2).mp h))⟩

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read uniformly
on the worlds it forces
nothing; read at the zeros it is least erasure, keyed, which decides
where the root alone does not. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
    R ∧ ¬ Forces (fun _ => R) ∧ (∀ Z : World, RootAtZeros Z ↔
    LeastErasure Z) ∧
    ((FoldClosed W1 ∧ RootAtZeros W1) ∧ (FoldClosed W2 ∧ ¬ RootAtZeros
    W2)) :=
  ⟨G.instances G.anAct, keyless_forces_nothing (fun _ => R) (fun _ =>
  G.instances G.anAct),
   root_at_zeros_is_least_erasure,
   root_at_zeros_is_keyed⟩

/-- LEAST ERASURE, UNCONDITIONAL: on every world, with no hypothesis and
no premise, least erasure of
the world is the hypothesis of the line on it. -/
theorem least_erasure_unconditional : ∀ Z : World, LeastErasure Z ↔ RH Z
:=
  least_erasure_is_the_value

/-- THE LOCK, one theorem: the constructed root holds with no hypothesis;
the root holds by the act and
holds on both worlds of one record; read uniformly it forces nothing;
read at the zeros it is least
erasure, keyed; least erasure is the hypothesis on every world,
unconditionally; the closing theorem gives
the hypothesis on the actual zeros by one act; and no record-respecting
reading decides the value on the
fold-closed worlds. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
    Root Unit (fun _ => 1) ∧
    R ∧
    (RH W1 ∧ FoldClosed W2 ∧ ¬ RH W2) ∧
    ¬ Forces (fun _ => R) ∧
    (∀ Z : World, RootAtZeros Z ↔ LeastErasure Z) ∧
    ((FoldClosed W1 ∧ RootAtZeros W1) ∧ (FoldClosed W2 ∧ ¬ RootAtZeros
    W2)) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ A : ActualZeros, RH A.zeros) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) :=
  ⟨constructed_root_holds, G.instances G.anAct, ⟨W1_rh, W2_closed,
  W2_not_rh⟩, ⟨root_read_on_row_is_keyed G⟩.2.1,
   root_at_zeros_is_least_erasure, root_at_zeros_is_keyed,
   least_erasure_unconditional, rh_from_the_act,
   record_decides_nothing⟩

/-! ## XXV · Admissible forcing is vacuous; the closure at every
resolution; the readings of the root joined

An admissible premise that forces the line holds at no nonempty
fold-closed configuration, and the constant false
premise is admissible and forces, so the class is not empty. At
resolution `n` the line is `x = n`, the fold is
`x ↦ 2n − x` and registration is `(x, t) ↦ (n, t)`; the chart is
resolution one and Section XXII.b is resolution two.
The closure of Section XXI holds there with least erasure as the one
field, and from `n = 2` on the open strip
`0 < x < 2n` holds the pair `x = n − 1, n + 1` off the line, so the strip
does not supply the bit; at resolution one
the pair at offset `k ≥ 2` lies outside the closed strip. The readings of
the root are joined in one theorem, and the
lock is stated at the constructed root. On a finite configuration a
search through its pairs, which terminates, finds a
point off the line exactly when least erasure fails. The posit is an act;
no reading of the record returns any of the
five chart readings; the primes are a free basis; least erasure is the
self-verification of the hypothesis; the
certified world is fold-closed; every route of the ledger carries the
theorem its status names; and the tenth falsifier,
a point off the line, is exactly the failure of the line and is barred on
the zero set the act supplies. -/

/-! ### XXV.a · Admissible forcing is vacuous -/

/-- ADMISSIBLE FORCING IS VACUOUS: an admissible premise that forces the
line fails at every nonempty fold-closed
```

```
configuration. -/
theorem admissible_forcing_is_vacuous (A : World → Prop) (hA : Admissible
A) (hf : Forces A) (Z : World)
    (hZ : FoldClosed Z) (hne : ∃ z, Z z) : ¬ A Z :=
  Or.elim hA (fun k _ => keyless_forces_nothing_free A k hf)
    (fun r ha => match hne with
      | ⟨z, hz⟩ =>
        absurd (hf (withPair Z z.t) (withPair_closed Z hZ z.t)
            ((r Z (withPair Z z.t) (withPair_same_record Z z hz)).mp ha)
            ⟨0, z.t⟩ (Or.inr (Or.inl rfl)))
          (by decide : ¬ ((0 : Int) = 1)))

/-- COALITION FORCING IS VACUOUS: a coalition of admissible atoms that
forces the line fails at every nonempty
fold-closed configuration. -/
theorem coalition_forcing_is_vacuous {ι : Type} (at_ : ι → World → Prop)
(h : ∀ i, Admissible (at_ i))
    (c : Coalition ι) (hf : Forces (Coalition.eval at_ c)) (Z : World)
    (hZ : FoldClosed Z) (hne : ∃ z, Z z) :
    ¬ Coalition.eval at_ c Z :=
  admissible_forcing_is_vacuous _ (coalition_admissible at_ h c) hf Z hZ
  hne

/-- AN ADMISSIBLE PREMISE FORCES: the constant false premise is
admissible and forces the line, holding nowhere. -/
theorem an_admissible_premise_forces : Admissible (fun _ => False) ∧
Forces (fun _ => False) :=
  ⟨admissible_const False, fun _ _ h => h.elim⟩

/-! ### XXV.b · The closure at resolution n -/

/-- TWICE IS A SUM: twice an integer is the integer added to itself. -/
theorem int_two_mul (a : Int) : 2 * a = a + a := by
  rw [show (2 : Int) = 1 + 1 from rfl, Int.add_mul, Int.one_mul]

/-- TWICE LESS ONCE: twice an integer less the integer is the integer. -/
theorem int_two_mul_sub (n : Int) : 2 * n - n = n :=
  (congrArg (· - n) (int_two_mul n)).trans (Int.add_sub_cancel n n)

/-- The fold at resolution `n`: the line is `x = n`. -/
def foldAt (n : Int) (z : Pt) : Pt := ⟨2 * n - z.x, z.t⟩

/-- Registration at resolution `n`: it lands on the line `x = n` and
keeps the height. -/
def regAt (n : Int) (z : Pt) : Pt := ⟨n, z.t⟩

/-- THE FOLD AT RESOLUTION N IS AN INVOLUTION: folding twice returns the
point. -/
theorem foldAt_invol (n : Int) (z : Pt) : foldAt n (foldAt n z) = z :=
  congrArg (fun y => Pt.mk y z.t) (Int.sub_sub_self (2 * n) z.x)

/-- THE SEAT AT RESOLUTION N: the fold fixes a point exactly when the
point lies on the line `x = n`. -/
theorem foldAt_fixed_iff (n : Int) (z : Pt) : foldAt n z = z ↔ z.x = n :=
  ⟨fun h => (Int.eq_of_mul_eq_mul_left (by decide : (2 : Int) ≠ 0)
      (((Int.sub_add_cancel (2 * n) z.x).symm.trans (congrArg (· + z.x)
      (congrArg Pt.x h))).trans
        (int_two_mul z.x).symm)).symm,
   fun h => congrArg (fun y => Pt.mk y z.t)
      (((congrArg (fun y => 2 * n - y) h).trans (int_two_mul_sub
      n)).trans h.symm)⟩

/-- THE CHART AND THE STRIP ARE TWO RESOLUTIONS: the fold at resolution
one is the chart's fold, and at resolution two
it is the fold of Section XXII.b. -/
theorem foldAt_specializes : (∀ z, foldAt 1 z = fold z) ∧ (∀ z, foldAt 2 z
= fold₂ z) :=
  ⟨fun _ => rfl, fun _ => rfl⟩

/-- THE FOLD KEEPS THE OPEN STRIP: at resolution `n` a point with `0 < x <
2n` folds to a point with the same bounds. -/
theorem foldAt_strip (n : Int) (z : Pt) (h : 0 < z.x ∧ z.x < 2 * n) :
    0 < (foldAt n z).x ∧ (foldAt n z).x < 2 * n :=
  ⟨Int.sub_pos_of_lt h.2, Int.sub_lt_self (2 * n) h.1⟩

/-- THE PAIR ABOUT THE LINE: at resolution `n` the fold exchanges the
points at `x = n − 1` and `x = n + 1`. -/
theorem foldAt_pair (n : Int) (t : Nat) :
    foldAt n ⟨n - 1, t⟩ = ⟨n + 1, t⟩ ∧ foldAt n ⟨n + 1, t⟩ = ⟨n - 1, t⟩ :=
  have e : foldAt n ⟨n - 1, t⟩ = ⟨n + 1, t⟩ := congrArg (fun y => Pt.mk y
  t)
    (((congrArg (· - (n - 1)) (int_two_mul n)).trans (Int.add_sub_assoc n
    n (n - 1))).trans
      (congrArg (n + ·) (Int.sub_sub_self n 1)))
  ⟨e, (congrArg (foldAt n) e.symm).trans (foldAt_invol n _)⟩

/-- The fold at resolution `n`, as a carrier fold. -/
def chartAt (n : Int) : Carrier.Fold Pt := ⟨foldAt n, foldAt_invol n⟩

/-- Registration at resolution `n`, as a carrier registration: it lands
on the line, fixes it, and forgets the side. -/
def regChartAt (n : Int) : Carrier.Registration (chartAt n) :=
  ⟨regAt n, fun z => (foldAt_fixed_iff n (regAt n z)).mpr rfl,
   fun z h => congrArg (fun y => Pt.mk y z.t) ((foldAt_fixed_iff n z).mp
   h).symm, fun _ => rfl⟩

/-- A zero set at resolution `n` with its fold-closure, its open strip,
and the one posit, least erasure. -/
structure ActualZerosAt (n : Int) where
  /-- The zero set, as a world on the chart. -/
  zeros : World
  /-- The zero set is closed under the fold at resolution `n`. -/
  closed : ∀ z, zeros z → zeros (foldAt n z)
  /-- Every zero lies inside the open strip `0 < x < 2n`. -/
  strip : ∀ z, zeros z → 0 < z.x ∧ z.x < 2 * n
  /-- The posit: least erasure of the zero set under registration at
  resolution `n`. -/
  supply : Carrier.LeastErasureC (regChartAt n) zeros

/-- THE CLOSURE AT RESOLUTION N: from the posit, every zero is fixed by
the fold at resolution `n`. -/
theorem rh_from_the_act_at (n : Int) (A : ActualZerosAt n) : Carrier.RHc
(chartAt n) A.zeros :=
  (Carrier.least_erasure_is_the_value_C (regChartAt n)
    (fun x => match decEq ((chartAt n).σ x) x with
      | isTrue h => Or.inl h
      | isFalse h => Or.inr h) A.zeros).mp A.supply

/-- THE STRIP DOES NOT SUPPLY THE BIT: from resolution two on, the pair
at `x = n − 1` and `x = n + 1` at height zero
is fold-closed, lies inside the open strip, and is not fixed by the fold.
-/
theorem strip_does_not_supply_at (n : Int) (hn : 2 ≤ n) :
    ∃ Z : World, (∀ z, Z z → Z (foldAt n z)) ∧ (∀ z, Z z → 0 < z.x ∧ z.x <
    2 * n) ∧
      ¬ Carrier.RHc (chartAt n) Z :=
  have h0 : (0 : Int) ≤ n := Int.le_trans (by decide : (0 : Int) ≤ 2) hn
  have hlo : n - 1 < n := Int.sub_lt_self n (by decide : (0 : Int) < 1)
  have hle : n ≤ 2 * n :=
    Eq.subst (motive := fun y => n ≤ y) (int_two_mul n).symm
    (Int.le_add_of_nonneg_right h0)
  have hs : 0 < ((n - 1, 0) : Pt).x ∧ ((n - 1, 0) : Pt).x < 2 * n :=
    ⟨Int.sub_pos_of_lt (Int.lt_of_lt_of_le (by decide : (1 : Int) < 2)
    hn), Int.lt_of_lt_of_le hlo hle⟩
  have hs' : 0 < ((n + 1, 0) : Pt).x ∧ ((n + 1, 0) : Pt).x < 2 * n :=
    ⟨Eq.subst (motive := fun w : Pt => 0 < w.x ∧ w.x < 2 * n) (foldAt_pair
    n 0).1 (foldAt_strip n _ hs)⟩
  ⟨fun w => w = ⟨n - 1, 0⟩ ∨ w = ⟨n + 1, 0⟩,
   fun w hw => Or.elim hw
     (fun e => Or.inr ((congrArg (foldAt n) e).trans (foldAt_pair n 0).1))
     (fun e => Or.inl ((congrArg (foldAt n) e).trans (foldAt_pair n
     0).2)),
   fun w hw => Or.elim hw
     (fun e => Eq.subst (motive := fun v : Pt => 0 < v.x ∧ v.x < 2 * n)
     e.symm hs)
     (fun e => Eq.subst (motive := fun v : Pt => 0 < v.x ∧ v.x < 2 * n)
     e.symm hs'),
   fun h => Int.lt_irrefl n (Eq.subst (motive := fun y => y < n)
     ((foldAt_fixed_iff n (n - 1, 0)).mp (h (n - 1, 0) (Or.inl rfl)))
     hlo)⟩

/-- THE PAIR OUTSIDE THE CLOSED STRIP: at resolution one an offset of at
least two places the pair outside
`0 ≤ x ≤ 2`. -/
theorem pair_outside_the_closed_strip (k : Int) (hk : 2 ≤ k) : 1 - k < 0 ∧
2 < 1 + k :=
  ⟨(Int.sub_neg_of_lt (Int.lt_of_lt_of_le (by decide : (1 : Int) < 2) hk),
    Int.lt_of_lt_of_le (by decide : (2 : Int) < 1 + 2)
    (Int.add_le_add_left hk 1))⟩

/-! ### XXV.c · The readings of the root, joined -/

/-- THE READINGS OF THE ROOT, JOINED: the constructed root is the
proposition `0 < 1`; it has a self-grounding supply
with an act; read uniformly on the worlds it forces nothing; read at the
zeros it is the hypothesis; and it does not
hold of every domain, since a domain whose actuation is zero refutes it.
-/
theorem root_readings_joined :
    (Root Unit (fun _ => 1) ↔ RootAct) ∧ Nonempty (SelfGrounding (Root
    Unit (fun _ => 1))) ∧
    ¬ Forces (fun _ => Root Unit (fun _ => 1)) ∧ (∀ Z : World,
    RootAtZeros Z ↔ RH Z) ∧
    ¬ ∀ (U : Type) (ΔE : U → Int), Root U ΔE :=
  ⟨⟨fun h => h (), fun h _ => h⟩, ⟨Unit, (), fun _ =>
  constructed_root_holds⟩⟩,
   keyless_forces_nothing_free _ (fun _ => constructed_root_holds),
   root_at_zeros_is_the_hypothesis,
   fun h => absurd (h Unit (fun _ => 0) ()) (by decide : ¬ ((0 : Int) <
   0)))

/-- THE LOCK ON THE CONSTRUCTED ROOT: the lock, with the root taken to be
the constructed root and its supply the unit
act. -/
theorem the_lock_on_the_constructed_root :
    Root Unit (fun _ => 1) ∧
    Root Unit (fun _ => 1) ∧
    (RH W1 ∧ FoldClosed W2 ∧ ¬ RH W2) ∧
    ¬ Forces (fun _ => Root Unit (fun _ => 1)) ∧
    (∀ Z : World, RootAtZeros Z ↔ LeastErasure Z) ∧
    ((FoldClosed W1 ∧ RootAtZeros W1) ∧ (FoldClosed W2 ∧ ¬ RootAtZeros
    W2)) ∧
    (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
    (∀ A : ActualZeros, RH A.zeros) ∧
    (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
    RH Z)) :=
  the_lock ⟨Unit, (), fun _ => constructed_root_holds⟩

/-! ### XXV.d · The rejection, computed -/

/-- The first off-line pair of a list of pairs, found by a search from
the front. -/
def firstOff : List (Int × Nat) → Option (Int × Nat)
  | [] => none
  | p :: ps => if p.1 = 0 then firstOff ps else some p
```

```
/-- THE SEARCH IS TOTAL: it returns nothing and every pair has offset zero, or it returns a pair of the list with a
nonzero offset. -/
theorem firstOff_spec (ps : List (Int × Nat)) :
    (firstOff ps = none ∧ ∀ p ∈ ps, p.1 = 0) ∨ ∃ p, firstOff ps = some p ∧ p ∈ ps ∧ p.1 ≠ 0 :=
  match ps with
  | [] => Or.inl (rfl, fun _ h => nomatch h)
  | p :: qs =>
    if e : p.1 = 0 then
      have hc : firstOff (p :: qs) = firstOff qs := if_pos e
      match firstOff_spec qs with
      | Or.inl (hn, hz) => Or.inl (hc.trans hn, fun q hq => match q, hq with
          | _, List.Mem.head _ => e
          | _, List.Mem.tail _ hq' => hz _ hq')
      | Or.inr (q, hq, hm, hne) => Or.inr (q, hc.trans hq, List.Mem.tail p hm, hne)
    else
      Or.inr (p, if_neg e, List.Mem.head qs, e)

/-- AN OFFSET IS READ OFF ITS POINT: the point at `x = 1 − a` lies on the line only when `a = 0`. -/
theorem offset_off_line (a : Int) (t : Nat) (h : a ≠ 0) : ¬ onLine (1 - a, t) :=
  fun e => h (((Int.sub_sub_self 1 a).symm.trans (congrArg (fun y => 1 - y) e)).trans (by decide))

/-- THE REJECTION, COMPUTED: if least erasure fails on a finite configuration, the search returns a pair `(k, t)` of
the configuration with `k ≠ 0` whose point `(1 − k, t)` lies in the world and off the line. -/
theorem rejection_is_computed (C : FinCfg) (h : ¬ LeastErasure C.world) :
    ∃ p, firstOff C.pairs = some p ∧ p ∈ C.pairs ∧ p.1 ≠ 0 ∧ C.world (1 - p.1, p.2) ∧
      ¬ onLine (1 - p.1, p.2) :=
  match firstOff_spec C.pairs with
  | Or.inl (_, hz) => absurd ((least_erasure_is_the_value C.world).mpr (fun _ hz' =>
      Or.elim hz' (fun hx => hx.1) (fun (p, hp, hk, _) => absurd (hz p hp) hk))) h
  | Or.inr (p, hf, hm, hk) => (p, hf, hm, hk, Or.inr (p, hm, hk, Or.inl rfl), offset_off_line p.1 p.2 hk)

/-- THE SEARCH DECIDES LEAST ERASURE: a finite configuration has least erasure exactly when the search returns
nothing. -/
theorem least_erasure_iff_search_empty (C : FinCfg) : LeastErasure C.world ↔ firstOff C.pairs = none :=
  (fun hle => match firstOff_spec C.pairs with
    | Or.inl (hn, _) => hn
    | Or.inr (p, _, hm, hk) =>
      absurd ((least_erasure_is_the_value C.world).mp hle _ (Or.inr (p, hm, hk, Or.inl rfl)))
        (offset_off_line p.1 p.2 hk)),
   fun hn => match firstOff_spec C.pairs with
    | Or.inl (_, hz) => (least_erasure_is_the_value C.world).mpr (fun _ hz' =>
      Or.elim hz' (fun hx => hx.1) (fun (p, hp, hk, _) => absurd (hz p hp) hk))
    | Or.inr (_, hs, _, _) => Option.noConfusion (hn.symm.trans hs))

/-! ### XXV.e · The posit, the readings, the basis, the recursion, the certified world, the ledger -/

/-- THE POSIT IS AN ACT: at the actual zeros the posit has a self-grounding supply with an act. -/
theorem the_posit_is_a_deed (A : ActualZeros) : ∃ G : SelfGrounding (LeastErasure A.zeros), Nonempty G.Act :=
  ((Unit, (), fun _ => A.supply), (()))

/-- A READING KEYED ON THE TWINS IS NOT RETURNED: a reading true on W1 and false on W2f agrees with no reading of the
record on every fold-closed world. -/
theorem keyed_reading_not_returned (P : World → Prop) (h1 : P W1) (h2 : ¬ P W2f) (g : World → Prop)
    (hg : RespectsRecord g) : ¬ ∀ Z, FoldClosed Z → (g Z ↔ P Z) :=
  fun h => h2 ((h W2f W2f_closed).mp ((hg W1 W2f same_record_f).mp ((h W1 W1_closed).mpr h1)))

/-- NO READING OF THE RECORD RETURNS ANY READING: lossless registration, displacement zero, no left zero, stability
and least erasure each hold on W1 and fail on W2f, so no reading of the record agrees with any of them. -/
theorem no_record_reading_returns_any_reading (g : World → Prop) (hg : RespectsRecord g) :
    ¬ (∀ Z, FoldClosed Z → (g Z ↔ Lossless Z)) ∧ ¬ (∀ Z, FoldClosed Z → (g Z ↔ DepthZero Z)) ∧
    ¬ (∀ Z, FoldClosed Z → (g Z ↔ ¬ Left Z)) ∧ ¬ (∀ Z, FoldClosed Z → (g Z ↔ Stable Z)) ∧
    ¬ (∀ Z, FoldClosed Z → (g Z ↔ LeastErasure Z)) :=
  (keyed_reading_not_returned Lossless
    (fun z hz => Eq.subst (motive := fun w => reg w = w) (Eq.symm hz) rfl)
    (fun h => absurd (congrArg Pt.x (h (0, 14) (Or.inl rfl))) (by decide : ¬ ((1 : Int) = 0))) g hg,
   keyed_reading_not_returned DepthZero
    (fun z hz => Eq.subst (motive := fun w => oddPart w = 0) (Eq.symm hz) rfl)
    (fun h => absurd (h (0, 14) (Or.inl rfl)) (by decide : ¬ (oddPart (0, 14) = 0))) g hg,
   keyed_reading_not_returned (fun Z => ¬ Left Z)
    (fun (z, hz, hx) => Eq.subst (motive := fun w : Pt => ¬ w.x < 1) (Eq.symm hz)
      (by decide : ¬ (((1, 14) : Pt).x < 1)) hx)
    (fun h => h ((0, 14), Or.inl rfl, by decide)) g hg,
   keyed_reading_not_returned Stable
    (fun z hz => Eq.subst (motive := fun w => N1 w ≤ N0 w) (Eq.symm hz) (by decide))
    (fun h => absurd (h (0, 14) (Or.inl rfl)) (by decide : ¬ (N1 (0, 14) ≤ N0 (0, 14)))) g hg,
   keyed_reading_not_returned LeastErasure ((least_erasure_is_the_value W1).mpr W1_rh)
    (fun h => W2f_not_rh ((least_erasure_is_the_value W2f).mp h)) g hg)

/-- THE PRIMES ARE A FREE BASIS: every assignment of integers to the primes extends to exactly one completely
additive function. -/
theorem primes_are_a_free_basis (a : Nat → Int) :
    ∃ f, CompletelyAdditive f ∧ (∀ p, isPrime p → f p = a p) ∧
      ∀ g, CompletelyAdditive g → (∀ p, isPrime p → g p = a p) → ∀ n, 0 < n → g n = f n :=
  (extendAssignment a, primes_admit_every_assignment a)

/-- LEAST ERASURE IS THE RECURSION: on every world least erasure holds exactly when the hypothesis is implied by its
own denial; the line is decided point by point, so no classical step enters. -/
theorem least_erasure_iff_recursion (Z : World) : LeastErasure Z ↔ SelfVerifyingP (RH Z) :=
  (fun h _ => (least_erasure_is_the_value Z).mp h,
   fun h => (least_erasure_is_the_value Z).mpr (fun z hz =>
     match Int.decEq z.x 1 with
     | isTrue e => e
     | isFalse ne => h (fun r => ne (r z hz)) z hz))

/-- THE CERTIFIED WORLD IS FOLD-CLOSED: the world certified to height `T` is closed under the fold, and a fold-closed
world agrees with it to `T` and carries an off-line pair above. -/
theorem certified_height_never_forces_closed (T : Nat) :
    FoldClosed (LineBelow T) ∧
    (RH (LineBelow T) ∧ FoldClosed (TwinAbove T) ∧ ¬ RH (TwinAbove T) ∧
      ∀ z, z.t ≤ T → (TwinAbove T z ↔ LineBelow T z)) :=
  (fun z hz => ((congrArg (fun y => 2 - y) hz.1).trans (by decide), hz.2),
   fun _ hz => hz.1,
   fun z hz => match hz with
    | Or.inl hl => Or.inl ((congrArg (fun y => 2 - y) hl.1).trans (by decide), hl.2)
    | Or.inr (Or.inl e) => Or.inr (Or.inr ((congrArg fold e).trans rfl))
    | Or.inr (Or.inr e) => Or.inr (Or.inl ((congrArg fold e).trans rfl)),
   fun h => absurd (h (0, T + 1) (Or.inr (Or.inl rfl))) (by decide : ¬ ((0 : Int) = 1)),
   fun z hz => (fun h => match h with
      | Or.inl hl => hl
      | Or.inr (Or.inl e) =>
        absurd (Eq.subst (motive := fun w : Pt => w.t ≤ T) e hz) (Nat.not_succ_le_self T)
      | Or.inr (Or.inr e) =>
        absurd (Eq.subst (motive := fun w : Pt => w.t ≤ T) e hz) (Nat.not_succ_le_self T),
    fun h => Or.inl h))

/-- The statement each route of the ledger carries: the theorem its status names. -/
def routeClaim : Route → Prop
  | .ladderFromRecord => ∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔ RH Z)
  | .foundationAsSystem => Sound lineTheory ∧ lineTheory.Prov () ∧ LeastErasure (lineTheoryReads.world ())
  | .placement => ∀ T : Theory, RungOnLadder T → Sound T →
      (∀ s, T.Comp s → T.Prov s) ∧ (∀ s, T.Prov s → T.True_ s) ∧ (∀ s, T.Comp s → T.True_ s)
  | .groundByAct => ∀ A : ActualZeros, RH A.zeros
  | .independence => ∀ S : Setting, ¬ S.Prov S.hyp → ¬ S.Prov (¬ S.hyp) → S.hyp
  | .realPartCertified => ∀ T : Nat, RH (LineBelow T) ∧ FoldClosed (TwinAbove T) ∧ ¬ RH (TwinAbove T) ∧
      ∀ z, z.t ≤ T → (TwinAbove T z ↔ LineBelow T z)

/-- EVERY ROUTE CARRIES ITS THEOREM: each of the six routes is backed by the theorem its ledger entry names. -/
theorem every_route_carries_its_theorem : ∀ r : Route, routeClaim r :=
  fun r =>
  match r with
  | .ladderFromRecord => unicorn_block
  | .foundationAsSystem => a_sound_theory_may_prove_the_line
  | .placement => placement
  | .groundByAct => rh_from_the_act
  | .independence => independence_forces_truth
  | .realPartCertified => certified_height_never_forces

/-! ### XXV.f · The tenth falsifier, barred by the act -/

/-- THE TENTH IS THE REFUTER: on every configuration a computed point off the line is exactly the failure of the line. -/
theorem the_tenth_is_the_refuter (Z : World) : OffLine Z ↔ ¬ RH Z :=
  (fun (z, hz, hn) h => hn (h z hz), not_rh_has_offline_witness Z)

/-- THE TENTH IS BARRED BY THE ACT: on the zero set the act supplies, no point lies off the line; the one refuter of the
value does not exist there. -/
theorem the_tenth_is_barred_by_the_act (A : ActualZeros) : ¬ OffLine A.zeros :=
  fun (z, hz, hn) => hn (rh_from_the_act A z hz)

/-- THE KINETIC BAR: a reading supplied on the actual zero set gives the line there, and with it the refuter's absence;
```

```
the formal side leaves the search open, and the field the act supplies
closes it. -/
theorem the_kinetic_bar (A : ActualZeros) : RH A.zeros ∧ ¬ OffLine
A.zeros :=
  (rh_from_the_act A, the_tenth_is_barred_by_the_act A)


/-! ## XXVI · The closure, bound whole

One theorem binds the closure, for every self-grounding root: the
two-part reading of section XIX.f, the lock of
section XXIV, the socket of section XXII, the irreducible cut of section
XIX.c, the two denials and the witness of
section XXII.c, and the bar of section XXV.f. Every conjunct is a theorem
above; this section adds their conjunction
and no content. -/

/-- THE CLOSURE, BOUND IN ONE THEOREM. For every self-grounding root:
from the posit the hypothesis follows, no reading
of the record supplies it, the posit is the hypothesis, and nothing
weaker forces the line; the lock; the field of the
act can be filled at a fold-closed configuration exactly where the
hypothesis holds; no admissible premise decides and
least erasure is not admissible; the two denials; the witness forced by
the act; and on the zero set the act supplies
the hypothesis holds and no point lies off the line. -/
theorem the_capstone {root : Prop} (G : SelfGrounding root) :
    ((∀ A : ActualZeros, RH A.zeros) ∧
     (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
     RH Z)) ∧
     (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
     (Forces RH ∧ ∀ A : World → Prop, Forces A → ∀ Z, FoldClosed Z → A Z →
     RH Z)) ∧
    (Root Unit (fun _ => 1) ∧
     root ∧
     (RH W1 ∧ FoldClosed W2 ∧ ¬ RH W2) ∧
     ¬ Forces (fun _ => root) ∧
     (∀ Z : World, RootAtZeros Z ↔ LeastErasure Z) ∧
     ((FoldClosed W1 ∧ RootAtZeros W1) ∧ (FoldClosed W2 ∧ ¬ RootAtZeros
     W2)) ∧
     (∀ Z : World, LeastErasure Z ↔ RH Z) ∧
     (∀ A : ActualZeros, RH A.zeros) ∧
     (∀ g : World → Prop, RespectsRecord g → ¬ ∀ Z, FoldClosed Z → (g Z ↔
     RH Z))) ∧
    (∀ Z : World, FoldClosed Z → ((∃ S : ActualZeros, S.zeros = Z) ↔ RH
    Z)) ∧
    ((∀ A : World → Prop, Admissible A → ¬ ∀ Z, FoldClosed Z → (A Z ↔ RH
    Z)) ∧ ¬ Admissible LeastErasure) ∧
    ((∀ C : Prop, G.Act → C → root) ∧
     (G.Act → ¬ root → False) ∧
     (∀ Z : World, ¬ RH Z → ¬ ∃ G' : SelfGrounding (RH Z), Nonempty
     G'.Act) ∧
     (∀ W : TWorld, SeededWitness W ↔ BoundT W) ∧
     (root ∧ FoldClosed W2f ∧ SameRecord W1 W2f ∧ TimelessT WTwin ∧ ¬ RH
     W2f ∧
       (¬ ∃ G' : SelfGrounding (RH W2f), Nonempty G'.Act) ∧ ¬
       SeededWitness WTwin) ∧
     SeededWitness WOn ∧
     ¬ Forces (fun _ => root) ∧
     (∀ A : ActualZeros, (∃ G' : SelfGrounding (RH A.zeros), Nonempty
     G'.Act) ∧
       SeededWitness (fun _ => A.zeros) ∧ (¬ RH A.zeros → False))) ∧
    ((∀ A : ActualZeros, RH A.zeros ∧ LeastErasure A.zeros ∧ ¬ OffLine
    A.zeros ∧
      (∀ z, A.zeros z ↔ recordOf A.zeros z) ∧ (∀ Q : World → Prop, (∀ Z,
      RH Z → Q Z) → Q A.zeros)) ∧
     (∀ (A : ActualZeros) (c : Computation), c.Reads A → ∀ z, z ∈
     c.reports → onLine z) ∧
     (∀ (c : Computation) (z : Pt), z ∈ c.reports → ¬ onLine z → ¬ ∃ A :
     ActualZeros, c.Reads A) ∧
     ¬ (∀ c : Computation, ∀ z, z ∈ c.reports → onLine z)) ∧
    (∀ A : ActualZeros, RH A.zeros ∧ ¬ OffLine A.zeros) :=
  (reader_frame, the_lock G, fun Z hZ => (socket_is_the_value Z hZ).1,
   (triaxial_cut_irreducible.2.2.2.1,
    triaxial_cut_irreducible.2.2.2.2.2.2.1),
   two_denials G, the_witness_forced_by_the_act, the_kinetic_bar)

end PrimeFreedom

/-! ## Cones, printed: sections I to XVI -/

#print axioms PrimeFreedom.arrow_exists
#print axioms PrimeFreedom.aperture_one_bit_wide
#print axioms PrimeFreedom.prime_fibre
#print axioms PrimeFreedom.prime_off_seat
#print axioms PrimeFreedom.freedom_is_exactly_two
#print axioms PrimeFreedom.seat_is_zero
#print axioms PrimeFreedom.exists_prime_dvd
#print axioms PrimeFreedom.leastDivisor_prime
#print axioms PrimeFreedom.determined_by_primes
#print axioms PrimeFreedom.fold_preserves_offline
#print axioms PrimeFreedom.offline_zero_quadruple
#print axioms PrimeFreedom.not_rh_has_offline_witness
#print axioms PrimeFreedom.denial_posits_the_orbit
#print axioms PrimeFreedom.record_blind_at_every_scale
#print axioms PrimeFreedom.euclid_lemma
#print axioms PrimeFreedom.pExp_mul
#print axioms PrimeFreedom.padicMeasure_additive
#print axioms PrimeFreedom.prime_freedom_independent
#print axioms PrimeFreedom.primes_base_of_freedom
#print axioms PrimeFreedom.arrow_multiplicative
#print axioms PrimeFreedom.arrow_at_prime
#print axioms PrimeFreedom.finite_never_forces
#print axioms PrimeFreedom.limit_not_forced
#print axioms PrimeFreedom.nothing_escapes_one_cut
#print axioms PrimeFreedom.sides_together
#print axioms PrimeFreedom.rh_iff_no_left
#print axioms PrimeFreedom.record_decides_nothing
#print axioms PrimeFreedom.unicorn_block
#print axioms PrimeFreedom.keyless_forces_nothing
#print axioms PrimeFreedom.least_erasure_is_the_value
#print axioms PrimeFreedom.supply_iff
#print axioms PrimeFreedom.faces_are_one
#print axioms PrimeFreedom.rh_from_the_act_generic
#print axioms PrimeFreedom.rh_from_the_act
#print axioms PrimeFreedom.rh_ground_closure_complete

#print axioms PrimeFreedom.primes_exist
#print axioms PrimeFreedom.primes_exist_forces_nothing
#print axioms PrimeFreedom.prime_freedom_forces_nothing
#print axioms PrimeFreedom.arrow_forces_nothing
#print axioms PrimeFreedom.free_basis_coexists_with_offline_world
#print axioms PrimeFreedom.fold_off_line_stays_off
#print axioms PrimeFreedom.positivity_on_zero_side
#print axioms PrimeFreedom.least_erasure_is_positivity
#print axioms PrimeFreedom.prime_witness_iff
#print axioms PrimeFreedom.rh_from_prime_witness
#print axioms PrimeFreedom.positivity_is_keyed
#print axioms PrimeFreedom.freedom_does_not_pick_the_sign
#print axioms PrimeFreedom.prime_witness_ledger
#print axioms PrimeFreedom.posit_is_the_conclusion
#print axioms PrimeFreedom.rh_from_prime_act
#print axioms PrimeFreedom.acts_are_one
#print axioms PrimeFreedom.record_decides_nothing_free
#print axioms PrimeFreedom.unicorn_block_free
#print axioms PrimeFreedom.keyless_forces_nothing_free
#print axioms PrimeFreedom.arrow_forces_nothing_free
#print axioms PrimeFreedom.least_erasure_reads_past_the_record_free
#print axioms PrimeFreedom.positivity_is_keyed_free

/-! ## Cones, pinned: sections VII to XIII (a compile in which any pinned
cone changes fails) -/
/-- info: 'PrimeFreedom.least_erasure_is_the_value' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_is_the_value
/-- info: 'PrimeFreedom.least_erasure_is_positivity' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_is_positivity
/-- info: 'PrimeFreedom.spend_is_the_line' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.spend_is_the_line
/-- info: 'PrimeFreedom.record_decides_nothing_free' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_decides_nothing_free
/-- info: 'PrimeFreedom.keyless_forces_nothing_free' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.keyless_forces_nothing_free
/-- info: 'PrimeFreedom.rh_from_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.rh_from_the_act
/-- info: 'PrimeFreedom.rh_from_prime_act' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.rh_from_prime_act
/-- info: 'PrimeFreedom.acts_are_one' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.acts_are_one
/-- info: 'PrimeFreedom.primes_base_of_freedom' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.primes_base_of_freedom
/-- info: 'PrimeFreedom.spend_is_not_the_free_basis' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.spend_is_not_the_free_basis
/-- info: 'PrimeFreedom.rh_on_the_spent_bit' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.rh_on_the_spent_bit
/-- info: 'PrimeFreedom.prime_arc_sealed' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.prime_arc_sealed
/-- info: 'PrimeFreedom.rh_is_the_weakest_forcing_premise' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.rh_is_the_weakest_forcing_premise
/-- info: 'PrimeFreedom.li_prefix_never_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.li_prefix_never_forces
/-- info: 'PrimeFreedom.rh_from_the_sign' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.rh_from_the_sign
/-- info: 'PrimeFreedom.rh_iff_the_sign' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.rh_iff_the_sign
/-- info: 'PrimeFreedom.spends_are_one' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.spends_are_one
/-- info: 'PrimeFreedom.line_not_self_grounding' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.line_not_self_grounding
/-- info: 'PrimeFreedom.sign_realized_both_ways' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.sign_realized_both_ways
/-- info: 'PrimeFreedom.the_vestigial_posit' depends on axioms: [propext]
-/
#guard_msgs in #print axioms PrimeFreedom.the_vestigial_posit
/-- info: 'PrimeFreedom.exactly_one_face_is_keyed' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.exactly_one_face_is_keyed
/-- info: 'PrimeFreedom.arrow_is_not_the_line' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.arrow_is_not_the_line
/-- info: 'PrimeFreedom.template_binds_iff_seed' does not depend on any
axioms -/
```

```
#guard_msgs in #print axioms PrimeFreedom.template_binds_iff_seed
/-- info: 'PrimeFreedom.the_seed_is_the_spend' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_seed_is_the_spend
/-- info: 'PrimeFreedom.the_template_plugged_in' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_template_plugged_in
/-! ## Cones, pinned: section XVI -/
/-- info: 'PrimeFreedom.fold_negates_odd' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.fold_negates_odd
/-- info: 'PrimeFreedom.rh_iff_even_eigenspace' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.rh_iff_even_eigenspace
/-- info: 'PrimeFreedom.record_blind_to_odd' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_blind_to_odd
/-- info: 'PrimeFreedom.side_odd_off_line' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.side_odd_off_line
/-- info: 'PrimeFreedom.record_never_reads_the_side' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.record_never_reads_the_side
/-- info: 'PrimeFreedom.colocation_is_a_calibration' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.colocation_is_a_calibration
/-- info: 'PrimeFreedom.calibration_realized_both_ways' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.calibration_realized_both_ways
/-- info: 'PrimeFreedom.recursion_fails_on_the_twin' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.recursion_fails_on_the_twin
/-- info: 'PrimeFreedom.least_erasure_is_the_recursion' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_is_the_recursion
/-- info: 'PrimeFreedom.the_mirror_at_minus_one' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.the_mirror_at_minus_one

/-! ## Cones, printed: section XVII -/
#print axioms PrimeFreedom.admissible_transfers
#print axioms PrimeFreedom.no_admissible_triad_forces
#print axioms PrimeFreedom.no_admissible_coalition_forces
#print axioms PrimeFreedom.spend_is_not_admissible
#print axioms PrimeFreedom.three_axis_lock
#print axioms PrimeFreedom.freedom_alone_open
#print axioms PrimeFreedom.certified_height_never_forces
#print axioms PrimeFreedom.certified_worlds_share_the_record_below
#print axioms PrimeFreedom.HeatFlow.disc_flow
#print axioms PrimeFreedom.HeatFlow.forward_preserves
#print axioms PrimeFreedom.HeatFlow.erasure_finite
#print axioms PrimeFreedom.HeatFlow.lam_nonneg
#print axioms PrimeFreedom.HeatFlow.real_at_lam
#print axioms PrimeFreedom.HeatFlow.lam_least
#print axioms PrimeFreedom.HeatFlow.lam_zero_iff_real
#print axioms PrimeFreedom.HeatFlow.de_bruijn_tight
#print axioms PrimeFreedom.HeatFlow.collision
#print axioms PrimeFreedom.HeatFlow.disc3_step
#print axioms PrimeFreedom.HeatFlow.disc3_strict
#print axioms PrimeFreedom.HeatFlow.disc3_monotone
#print axioms PrimeFreedom.HeatFlow.cubic_forward_preserves
#print axioms PrimeFreedom.HeatFlow.flow_injective
#print axioms PrimeFreedom.HeatFlow.certificate_two_worlds
#print axioms PrimeFreedom.HeatFlow.certificate_decides_nothing
#print axioms PrimeFreedom.HeatFlow.certification_never_forces
#print axioms PrimeFreedom.HeatFlow.zero_slack
#print axioms PrimeFreedom.HeatFlow.backward_not_forced
#print axioms PrimeFreedom.HeatFlow.family_least_erasure
#print axioms PrimeFreedom.heat_bit_is_one_inequality
#print axioms PrimeFreedom.family_bit_is_one_inequality
#print axioms PrimeFreedom.heat_model_reads_as_the_carrier
#print axioms PrimeFreedom.Liouville.lam_mult
#print axioms PrimeFreedom.Liouville.lam_flips_at_primes
#print axioms PrimeFreedom.Liouville.polya_holds_to_200
#print axioms PrimeFreedom.liouville_is_the_sign_arrow
#print axioms PrimeFreedom.rh_from_faithful
#print axioms PrimeFreedom.faithful_is_the_bit
#print axioms PrimeFreedom.the_posit_in_every_coordinate
#print axioms PrimeFreedom.modes
#print axioms PrimeFreedom.lossless_iff_rh
#print axioms PrimeFreedom.stable_iff_rh
#print axioms PrimeFreedom.five_readings_agree
#print axioms PrimeFreedom.no_record_reading_returns_any_face
#print axioms PrimeFreedom.same_for_every_reader
#print axioms PrimeFreedom.no_private_bit
#print axioms PrimeFreedom.spend_has_no_reader
#print axioms PrimeFreedom.the_harvest

/-! ## Cones, pinned: section XVII -/
/-- info: 'PrimeFreedom.no_admissible_coalition_forces' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.no_admissible_coalition_forces
/-- info: 'PrimeFreedom.spend_is_not_admissible' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.spend_is_not_admissible
/-- info: 'PrimeFreedom.certified_height_never_forces' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.certified_height_never_forces
/-- info: 'PrimeFreedom.HeatFlow.lam_nonneg' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.HeatFlow.lam_nonneg
/-- info: 'PrimeFreedom.HeatFlow.lam_zero_iff_real' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.HeatFlow.lam_zero_iff_real
/-- info: 'PrimeFreedom.HeatFlow.certificate_decides_nothing' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.HeatFlow.certificate_decides_nothing
/-- info: 'PrimeFreedom.HeatFlow.zero_slack' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.HeatFlow.zero_slack
/-- info: 'PrimeFreedom.heat_bit_is_one_inequality' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.heat_bit_is_one_inequality
/-- info: 'PrimeFreedom.liouville_is_the_sign_arrow' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.liouville_is_the_sign_arrow
/-- info: 'PrimeFreedom.Liouville.polya_holds_to_200' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.Liouville.polya_holds_to_200
/-- info: 'PrimeFreedom.the_posit_in_every_coordinate' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.the_posit_in_every_coordinate
/-- info: 'PrimeFreedom.five_readings_agree' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.five_readings_agree
/-- info: 'PrimeFreedom.spend_has_no_reader' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.spend_has_no_reader
/-- info: 'PrimeFreedom.the_harvest' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_harvest

/-! ## Cones, printed: section XVIII -/

#print axioms PrimeFreedom.placement
#print axioms PrimeFreedom.soundness_is_load_bearing
#print axioms PrimeFreedom.ground_exceeds_ladder
#print axioms PrimeFreedom.independent_axioms_are_separate_bits
#print axioms PrimeFreedom.root_is_keyless
#print axioms PrimeFreedom.keyless_crosses
#print axioms PrimeFreedom.root_decides_no_keyed
#print axioms PrimeFreedom.choice_is_keyed
#print axioms PrimeFreedom.root_does_not_cross_choice
#print axioms PrimeFreedom.line_is_keyed_over_worlds
#print axioms PrimeFreedom.root_does_not_cross_the_line
#print axioms PrimeFreedom.refutation_is_one_point
#print axioms PrimeFreedom.cant_refute_seals
#print axioms PrimeFreedom.cant_refute_seals_dec
#print axioms PrimeFreedom.rh_iff_cant_refute
#print axioms PrimeFreedom.independence_forces_truth
#print axioms PrimeFreedom.cant_prove_does_not_seal_false
#print axioms PrimeFreedom.cant_asymmetry
#print axioms PrimeFreedom.round_trip_identity
#print axioms PrimeFreedom.absolute_returns_unchanged
#print axioms PrimeFreedom.trip_adds_presence_only
#print axioms PrimeFreedom.massless_arrow
#print axioms PrimeFreedom.any_true_premise_serves
#print axioms PrimeFreedom.the_line_round_trips
#print axioms PrimeFreedom.ladder_blocked
#print axioms PrimeFreedom.the_road

/-! ## Cones, pinned: section XVIII -/
/-- info: 'PrimeFreedom.placement' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.placement
/-- info: 'PrimeFreedom.soundness_is_load_bearing' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.soundness_is_load_bearing
/-- info: 'PrimeFreedom.ground_exceeds_ladder' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.ground_exceeds_ladder
/-- info: 'PrimeFreedom.root_decides_no_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_decides_no_keyed
/-- info: 'PrimeFreedom.root_does_not_cross_the_line' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_does_not_cross_the_line
/-- info: 'PrimeFreedom.cant_refute_seals' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.cant_refute_seals
/-- info: 'PrimeFreedom.rh_iff_cant_refute' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.rh_iff_cant_refute
/-- info: 'PrimeFreedom.independence_forces_truth' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.independence_forces_truth
/-- info: 'PrimeFreedom.cant_prove_does_not_seal_false' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.cant_prove_does_not_seal_false
/-- info: 'PrimeFreedom.round_trip_identity' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.round_trip_identity
/-- info: 'PrimeFreedom.massless_arrow' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.massless_arrow
/-- info: 'PrimeFreedom.ladder_blocked' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.ladder_blocked
/-- info: 'PrimeFreedom.the_road' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_road

/-! ## Cones, printed: section XIX -/

#print axioms PrimeFreedom.vocabulary_law
#print axioms PrimeFreedom.every_face_proves_every_face
#print axioms PrimeFreedom.chart_faces_prove_each_other
#print axioms PrimeFreedom.fold_fold
#print axioms PrimeFreedom.fold_law_admissible
#print axioms PrimeFreedom.same_record_eq
#print axioms PrimeFreedom.record_reading_admissible
```

```
#print axioms PrimeFreedom.admissible_const
#print axioms PrimeFreedom.keyless_denial_respects
#print axioms PrimeFreedom.admissible_not
#print axioms PrimeFreedom.admissible_and
#print axioms PrimeFreedom.admissible_or
#print axioms PrimeFreedom.admissible_never_decides
#print axioms PrimeFreedom.coalition_admissible
#print axioms PrimeFreedom.no_coalition_decides
#print axioms PrimeFreedom.triaxial_cut_irreducible
#print axioms PrimeFreedom.ladder_proof_lands_on_least_erasure
#print axioms PrimeFreedom.ladder_proof_iff_least_erasure
#print axioms PrimeFreedom.lineTheory_sound
#print axioms PrimeFreedom.a_sound_theory_may_prove_the_line
#print axioms PrimeFreedom.route_ledger_is_computed
#print axioms PrimeFreedom.route_ledger_counts
#print axioms PrimeFreedom.route_status_total
#print axioms PrimeFreedom.blocked_iff_theorem
#print axioms PrimeFreedom.reader_frame
#print axioms PrimeFreedom.the_front

/-! ## Cones, pinned: section XIX -/
/-- info: 'PrimeFreedom.vocabulary_law' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.vocabulary_law
/-- info: 'PrimeFreedom.every_face_proves_every_face' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.every_face_proves_every_face
/-- info: 'PrimeFreedom.chart_faces_prove_each_other' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.chart_faces_prove_each_other
/-- info: 'PrimeFreedom.record_reading_admissible' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.record_reading_admissible
/-- info: 'PrimeFreedom.admissible_not' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.admissible_not
/-- info: 'PrimeFreedom.admissible_and' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.admissible_and
/-- info: 'PrimeFreedom.admissible_or' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.admissible_or
/-- info: 'PrimeFreedom.admissible_never_decides' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.admissible_never_decides
/-- info: 'PrimeFreedom.coalition_admissible' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.coalition_admissible
/-- info: 'PrimeFreedom.no_coalition_decides' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.no_coalition_decides
/-- info: 'PrimeFreedom.triaxial_cut_irreducible' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.triaxial_cut_irreducible
/-- info: 'PrimeFreedom.ladder_proof_lands_on_least_erasure' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.ladder_proof_lands_on_least_erasure
/-- info: 'PrimeFreedom.ladder_proof_iff_least_erasure' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.ladder_proof_iff_least_erasure
/-- info: 'PrimeFreedom.a_sound_theory_may_prove_the_line' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.a_sound_theory_may_prove_the_line
/-- info: 'PrimeFreedom.route_ledger_is_computed' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.route_ledger_is_computed
/-- info: 'PrimeFreedom.blocked_iff_theorem' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.blocked_iff_theorem
/-- info: 'PrimeFreedom.reader_frame' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.reader_frame
/-- info: 'PrimeFreedom.the_front' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_front

/-! ## Cones, pinned: section XX -/
/-- info: 'PrimeFreedom.reg_on_line' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.reg_on_line
/-- info: 'PrimeFreedom.record_rh' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_rh
/-- info: 'PrimeFreedom.record_has_least_erasure' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_has_least_erasure
/-- info: 'PrimeFreedom.record_closed' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.record_closed
/-- info: 'PrimeFreedom.record_carried_by_least_erasure' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_carried_by_least_erasure
/-- info: 'PrimeFreedom.admissible_holds_on_the_lossless_world' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.admissible_holds_on_the_lossless_world
/-- info: 'PrimeFreedom.every_reading_holds_on_the_lossless_world' does
not depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.every_reading_holds_on_the_lossless_world
/-- info: 'PrimeFreedom.every_coalition_holds_on_the_lossless_world' does
not depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.every_coalition_holds_on_the_lossless_world
/-- info: 'PrimeFreedom.record_never_testifies_against' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.record_never_testifies_against
/-- info: 'PrimeFreedom.pair_two_points_one_record' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.pair_two_points_one_record
/-- info: 'PrimeFreedom.offCount_zero_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.offCount_zero_iff
/-- info: 'PrimeFreedom.fincfg_closed' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.fincfg_closed
/-- info: 'PrimeFreedom.least_erasure_iff_zero_erased' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_iff_zero_erased
/-- info: 'PrimeFreedom.landauer_floor_exact' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.landauer_floor_exact
/-- info: 'PrimeFreedom.price_zero_iff_least_erasure' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.price_zero_iff_least_erasure
/-- info: 'PrimeFreedom.denial_price_linear' depends on axioms: [propext]
-/
#guard_msgs in #print axioms PrimeFreedom.denial_price_linear
/-- info: 'PrimeFreedom.reversible_commits_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.reversible_commits_nothing
/-- info: 'PrimeFreedom.least_erasure_iff_no_point_off' depends on
axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_iff_no_point_off
/-- info: 'PrimeFreedom.rejection_is_a_witness' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.rejection_is_a_witness
/-- info: 'PrimeFreedom.falsifier_form_constant' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.falsifier_form_constant
/-- info: 'PrimeFreedom.act_reenacts_root' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.act_reenacts_root
/-- info: 'PrimeFreedom.root_does_not_cross_the_denial' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_does_not_cross_the_denial
/-- info: 'PrimeFreedom.assent_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.assent_is_the_value
/-- info: 'PrimeFreedom.one_shape_two_registers' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.one_shape_two_registers
/-- info: 'PrimeFreedom.freedom_is_given_the_sign_is_spent' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
PrimeFreedom.freedom_is_given_the_sign_is_spent
/-- info: 'PrimeFreedom.least_erasure_affirmed' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_affirmed

/-! ## Cones, printed: every theorem of sections I to XX whose cone is
not printed above -/

#print axioms PrimeFreedom.orientation_two_valued
#print axioms PrimeFreedom.orientations_distinct
#print axioms PrimeFreedom.seat_is_one
#print axioms PrimeFreedom.findLeastFrom_correct
#print axioms PrimeFreedom.leastDivisor_spec
#print axioms PrimeFreedom.dvd_remainder_mul
#print axioms PrimeFreedom.leastMulDvd_spec
#print axioms PrimeFreedom.findGt_spec
#print axioms PrimeFreedom.findGt_max
#print axioms PrimeFreedom.pExp_dvd
#print axioms PrimeFreedom.two_pow_ge
#print axioms PrimeFreedom.pow_dvd_bound
#print axioms PrimeFreedom.pExp_le_of_dvd
#print axioms PrimeFreedom.mul_mul_mul_comm_nat
#print axioms PrimeFreedom.pExp_self
#print axioms PrimeFreedom.pExp_eq_zero_of_not_dvd
#print axioms PrimeFreedom.pExp_one
#print axioms PrimeFreedom.pExp_other
#print axioms PrimeFreedom.signArrow_add
#print axioms PrimeFreedom.fold_fixed_iff
#print axioms PrimeFreedom.reg_fixes_line
#print axioms PrimeFreedom.unicorn_never_registered
#print axioms PrimeFreedom.W1_closed
#print axioms PrimeFreedom.W1_rh
#print axioms PrimeFreedom.pair_closed
#print axioms PrimeFreedom.pair_not_rh
#print axioms PrimeFreedom.pair_same_record
#print axioms PrimeFreedom.W2_closed
#print axioms PrimeFreedom.W2_not_rh
#print axioms PrimeFreedom.same_record_12
#print axioms PrimeFreedom.fibre_is_infinite
#print axioms PrimeFreedom.lossless_unique
#print axioms PrimeFreedom.sentence_separates_record_does_not
#print axioms PrimeFreedom.record_is_lossless
#print axioms PrimeFreedom.record_same
#print axioms PrimeFreedom.least_erasure_reads_past_the_record
#print axioms PrimeFreedom.posit_is_the_value
#print axioms PrimeFreedom.fold_onLine_iff
#print axioms PrimeFreedom.fold_ne_of_offline
#print axioms PrimeFreedom.primes_exist_keyless
#print axioms PrimeFreedom.prime_freedom_keyless
#print axioms PrimeFreedom.arrow_keyless
#print axioms PrimeFreedom.W2f_closed
#print axioms PrimeFreedom.W2f_not_rh
#print axioms PrimeFreedom.same_record_f
#print axioms PrimeFreedom.weaker_never_forces
#print axioms PrimeFreedom.weaker_at_the_twin_never_forces
#print axioms PrimeFreedom.rh_from_li
#print axioms PrimeFreedom.li_is_the_bit_stream
```

```
#print axioms PrimeFreedom.root_has_an_act
#print axioms PrimeFreedom.fold_face_keyless
#print axioms PrimeFreedom.fold_face_on_twin
#print axioms PrimeFreedom.fold_face_on_line
#print axioms PrimeFreedom.record_face_keyless
#print axioms PrimeFreedom.arrow_face_keyless
#print axioms PrimeFreedom.closure_face_on_line
#print axioms PrimeFreedom.closure_face_fails_on_twin
#print axioms PrimeFreedom.fold_keeps_even
#print axioms PrimeFreedom.residence_orientation_reversing
#print axioms PrimeFreedom.reg_kills_odd
#print axioms PrimeFreedom.reg_keeps_even
#print axioms PrimeFreedom.reg_idempotent
#print axioms PrimeFreedom.onLine_iff_odd_zero
#print axioms PrimeFreedom.odd_pair_opposite
#print axioms PrimeFreedom.wall_on_the_chart
#print axioms PrimeFreedom.value_is_recursion
#print axioms PrimeFreedom.recursion_is_value
#print axioms PrimeFreedom.recursion_on_the_line
#print axioms PrimeFreedom.root_recursion
#print axioms PrimeFreedom.HeatFlow.sq_nonneg
#print axioms PrimeFreedom.HeatFlow.cube_diff
#print axioms PrimeFreedom.HeatFlow.disc3_monotone_rec
#print axioms PrimeFreedom.HeatFlow.famLam_nonneg_rec
#print axioms PrimeFreedom.HeatFlow.family_least_erasure_rec
#print axioms PrimeFreedom.root_act

/-! ## Cones, pinned: section XXI -/

/-- info: 'PrimeFreedom.sumBelow_add' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.sumBelow_add
/-- info: 'PrimeFreedom.sumBelow_congr' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.sumBelow_congr
/-- info: 'PrimeFreedom.sumBelow_stable' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.sumBelow_stable
/-- info: 'PrimeFreedom.sumBelow_zero' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.sumBelow_zero
/-- info: 'PrimeFreedom.sumBelow_single' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.sumBelow_single
/-- info: 'PrimeFreedom.primeTest_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.primeTest_iff
/-- info: 'PrimeFreedom.pExp_zero_above' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.pExp_zero_above
/-- info: 'PrimeFreedom.extendTerm_zero_above' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendTerm_zero_above
/-- info: 'PrimeFreedom.extendAssignment_bound' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendAssignment_bound
/-- info: 'PrimeFreedom.extendTerm_mul' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendTerm_mul
/-- info: 'PrimeFreedom.extendAssignment_mul' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendAssignment_mul
/-- info: 'PrimeFreedom.extendTerm_other' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendTerm_other
/-- info: 'PrimeFreedom.extendAssignment_prime' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendAssignment_prime
/-- info: 'PrimeFreedom.extendAssignment_additive' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.extendAssignment_additive
/-- info: 'PrimeFreedom.primes_admit_every_assignment' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.primes_admit_every_assignment
/-- info: 'PrimeFreedom.omegaAll_mul' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.omegaAll_mul
/-- info: 'PrimeFreedom.omegaAll_prime' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.omegaAll_prime
/-- info: 'PrimeFreedom.liouvilleAll_mul' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.liouvilleAll_mul
/-- info: 'PrimeFreedom.liouvilleAll_prime' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.liouvilleAll_prime
/-- info: 'PrimeFreedom.liouvilleAll_one' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.liouvilleAll_one
/-- info: 'PrimeFreedom.liouvilleAll_agrees_to_30' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.liouvilleAll_agrees_to_30
/-- info: 'PrimeFreedom.pinned_face_reads' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.pinned_face_reads
/-- info: 'PrimeFreedom.pinned_face_is_a_face' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.pinned_face_is_a_face
/-- info: 'PrimeFreedom.rh_iff_own_record' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.rh_iff_own_record
/-- info: 'PrimeFreedom.least_erasure_is_least' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_is_least
/-- info: 'PrimeFreedom.record_is_the_least' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.record_is_the_least
/-- info: 'PrimeFreedom.the_closure' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_closure
/-- info: 'PrimeFreedom.Carrier.record_same_C' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.record_same_C
/-- info: 'PrimeFreedom.Carrier.least_erasure_is_the_value_C' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.Carrier.least_erasure_is_the_value_C
/-- info: 'PrimeFreedom.Carrier.rh_iff_own_record_C' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.rh_iff_own_record_C
/-- info: 'PrimeFreedom.Carrier.one_record_C' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.one_record_C
/-- info: 'PrimeFreedom.Carrier.worlds_differ_C' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.worlds_differ_C
/-- info: 'PrimeFreedom.Carrier.no_reading_decides_C' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.no_reading_decides_C
/-- info: 'PrimeFreedom.Carrier.chart_is_an_instance' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.chart_is_an_instance
/-- info: 'PrimeFreedom.Carrier.the_closure_on_any_carrier' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.Carrier.the_closure_on_any_carrier

/-- info: 'PrimeFreedom.Carrier.lossless_unique_C' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.Carrier.lossless_unique_C

/-! ## Cones, pinned: section XXII -/
/-- info: 'PrimeFreedom.socket_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.socket_is_the_value
/-- info: 'PrimeFreedom.closure_takes_a_term' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.closure_takes_a_term
/-- info: 'PrimeFreedom.closure_at_a_term' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.closure_at_a_term
/-- info: 'PrimeFreedom.forces_or_has_a_twin' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.forces_or_has_a_twin
/-- info: 'PrimeFreedom.withPair_closed' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.withPair_closed
/-- info: 'PrimeFreedom.withPair_same_record' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.withPair_same_record
/-- info: 'PrimeFreedom.withPair_not_rh' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.withPair_not_rh
/-- info: 'PrimeFreedom.record_forcing_fails_everywhere' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.record_forcing_fails_everywhere
/-- info: 'PrimeFreedom.forcing_is_the_value_or_stronger' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.forcing_is_the_value_or_stronger
/-- info: 'PrimeFreedom.forcing_fills_the_socket' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.forcing_fills_the_socket
/-- info: 'PrimeFreedom.face_is_the_socket' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.face_is_the_socket
/-- info: 'PrimeFreedom.named_faces_are_the_socket' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.named_faces_are_the_socket
/-- info: 'PrimeFreedom.walk_is_the_walk_of_one' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.walk_is_the_walk_of_one
/-- info: 'PrimeFreedom.faithful_is_the_sentence_of_one' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.faithful_is_the_sentence_of_one
/-- info: 'PrimeFreedom.blind_assignment_vanishes' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.blind_assignment_vanishes
/-- info: 'PrimeFreedom.blind_walk' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.blind_walk
/-- info: 'PrimeFreedom.blind_is_unfaithful' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.blind_is_unfaithful
/-- info: 'PrimeFreedom.faithful_reads_the_primes' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.faithful_reads_the_primes
/-- info: 'PrimeFreedom.the_socket' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_socket

/-! ## Cones, pinned: section XXII.b -/
/-- info: 'PrimeFreedom.fold₂_fixed_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.fold₂_fixed_iff
/-- info: 'PrimeFreedom.open_strip_forces_at_resolution_one' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
PrimeFreedom.open_strip_forces_at_resolution_one
/-- info: 'PrimeFreedom.innerPair₂_strip' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.innerPair₂_strip
```

```
/-- info: 'PrimeFreedom.innerPair₂_not_rh' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.innerPair₂_not_rh
/-- info: 'PrimeFreedom.open_strip_forces_nothing_at_resolution_two'
depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
PrimeFreedom.open_strip_forces_nothing_at_resolution_two
/-- info: 'PrimeFreedom.keyless_forces_nothing₂' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.keyless_forces_nothing₂
/-- info: 'PrimeFreedom.withPair₂_strip' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.withPair₂_strip
/-- info: 'PrimeFreedom.withPair₂_same_record' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.withPair₂_same_record
/-- info: 'PrimeFreedom.withPair₂_not_rh' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.withPair₂_not_rh
/-- info: 'PrimeFreedom.record_forcing_fails_in_the_strip' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
PrimeFreedom.record_forcing_fails_in_the_strip
/-- info: 'PrimeFreedom.no_reading_decides_in_the_strip' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.no_reading_decides_in_the_strip
/-- info: 'PrimeFreedom.forces_or_has_a_twin₂' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.forces_or_has_a_twin₂
/-- info: 'PrimeFreedom.the_socket_in_the_strip' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_socket_in_the_strip

/-! ## Cones, pinned: section XXIII -/

/-- info: 'PrimeFreedom.double_security' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.double_security

/-! ## Cones, pinned: section XXIV -/

/-- info: 'PrimeFreedom.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.denial_reenacts_root

/-- info: 'PrimeFreedom.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.external_proof_adds_nothing

/-- info: 'PrimeFreedom.seated_undeniable' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.seated_undeniable

/-- info: 'PrimeFreedom.undeniable_root_forces_no_value' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.undeniable_root_forces_no_value

/-- info: 'PrimeFreedom.root_read_on_row_is_keyed' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.root_read_on_row_is_keyed

/-- info: 'PrimeFreedom.least_erasure_unconditional' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_unconditional

/-- info: 'PrimeFreedom.the_lock' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_lock

/-- info: 'PrimeFreedom.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.indicator_pos

/-- info: 'PrimeFreedom.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.constructed_root_holds

/-- info: 'PrimeFreedom.root_at_zeros_is_the_hypothesis' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_at_zeros_is_the_hypothesis

/-- info: 'PrimeFreedom.root_at_zeros_is_least_erasure' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_at_zeros_is_least_erasure

/-- info: 'PrimeFreedom.root_at_zeros_is_keyed' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.root_at_zeros_is_keyed

/-! ## Cones, pinned: section XXV -/

/-- info: 'PrimeFreedom.admissible_forcing_is_vacuous' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.admissible_forcing_is_vacuous

/-- info: 'PrimeFreedom.coalition_forcing_is_vacuous' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.coalition_forcing_is_vacuous

/-- info: 'PrimeFreedom.an_admissible_premise_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.an_admissible_premise_forces

/-- info: 'PrimeFreedom.int_two_mul' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.int_two_mul

/-- info: 'PrimeFreedom.int_two_mul_sub' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.int_two_mul_sub

/-- info: 'PrimeFreedom.foldAt_invol' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.foldAt_invol

/-- info: 'PrimeFreedom.foldAt_fixed_iff' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.foldAt_fixed_iff

/-- info: 'PrimeFreedom.foldAt_specializes' does not depend on any axioms
-/
#guard_msgs in #print axioms PrimeFreedom.foldAt_specializes

/-- info: 'PrimeFreedom.foldAt_strip' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.foldAt_strip

/-- info: 'PrimeFreedom.foldAt_pair' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.foldAt_pair

/-- info: 'PrimeFreedom.rh_from_the_act_at' depends on axioms: [propext]
-/
#guard_msgs in #print axioms PrimeFreedom.rh_from_the_act_at

/-- info: 'PrimeFreedom.strip_does_not_supply_at' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.strip_does_not_supply_at

/-- info: 'PrimeFreedom.pair_outside_the_closed_strip' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.pair_outside_the_closed_strip

/-- info: 'PrimeFreedom.root_readings_joined' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_readings_joined

/-- info: 'PrimeFreedom.the_lock_on_the_constructed_root' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_lock_on_the_constructed_root

/-- info: 'PrimeFreedom.firstOff_spec' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.firstOff_spec

/-- info: 'PrimeFreedom.offset_off_line' depends on axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.offset_off_line

/-- info: 'PrimeFreedom.rejection_is_computed' depends on axioms:
[propext] -/
#guard_msgs in #print axioms PrimeFreedom.rejection_is_computed

/-- info: 'PrimeFreedom.least_erasure_iff_search_empty' depends on
axioms: [propext] -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_iff_search_empty

/-- info: 'PrimeFreedom.the_posit_is_a_deed' does not depend on any
axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_posit_is_a_deed

/-- info: 'PrimeFreedom.keyed_reading_not_returned' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.keyed_reading_not_returned

/-- info: 'PrimeFreedom.no_record_reading_returns_any_reading' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.no_record_reading_returns_any_reading

/-- info: 'PrimeFreedom.primes_are_a_free_basis' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.primes_are_a_free_basis

/-- info: 'PrimeFreedom.least_erasure_iff_recursion' does not depend on
any axioms -/
#guard_msgs in #print axioms PrimeFreedom.least_erasure_iff_recursion

/-- info: 'PrimeFreedom.certified_height_never_forces_closed' does not
depend on any axioms -/
#guard_msgs in #print axioms
PrimeFreedom.certified_height_never_forces_closed

/-- info: 'PrimeFreedom.every_route_carries_its_theorem' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.every_route_carries_its_theorem

/-- info: 'PrimeFreedom.the_tenth_is_the_refuter' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_tenth_is_the_refuter

/-- info: 'PrimeFreedom.the_tenth_is_barred_by_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_tenth_is_barred_by_the_act

/-- info: 'PrimeFreedom.the_kinetic_bar' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_kinetic_bar

/-! ## Cones, pinned: section XXII.c -/

/-- info: 'PrimeFreedom.seeded_witness_iff_bound' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.seeded_witness_iff_bound

/-- info: 'PrimeFreedom.twin_has_no_seeded_witness' does not depend on
any axioms -/
```

```
#guard_msgs in #print axioms PrimeFreedom.twin_has_no_seeded_witness

/-- info: 'PrimeFreedom.line_world_has_a_seeded_witness' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.line_world_has_a_seeded_witness

/-- info: 'PrimeFreedom.every_denial_is_an_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.every_denial_is_an_act

/-- info: 'PrimeFreedom.root_denial_refutes_itself' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.root_denial_refutes_itself

/-- info: 'PrimeFreedom.line_denial_denies_the_assertion' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.line_denial_denies_the_assertion

/-- info: 'PrimeFreedom.line_denial_denies_every_seed' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.line_denial_denies_every_seed

/-- info: 'PrimeFreedom.twin_carries_the_root_and_no_seed' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.twin_carries_the_root_and_no_seed

/-- info: 'PrimeFreedom.the_root_alone_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_root_alone_forces_nothing

/-- info: 'PrimeFreedom.the_act_asserts_the_line' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_act_asserts_the_line

/-- info: 'PrimeFreedom.the_act_seeds_its_constant_world' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_act_seeds_its_constant_world

/-- info: 'PrimeFreedom.line_denial_refuted_on_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.line_denial_refuted_on_the_act

/-- info: 'PrimeFreedom.two_denials' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.two_denials

/-- info: 'PrimeFreedom.two_denials_at_the_root' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.two_denials_at_the_root

/-- info: 'PrimeFreedom.nothing_left_open_on_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.nothing_left_open_on_the_act

/-- info: 'PrimeFreedom.computation_reading_the_act_lands_on_the_line' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.computation_reading_the_act_lands_on_the_line

/-- info: 'PrimeFreedom.off_line_report_reads_no_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.off_line_report_reads_no_act

/-- info: 'PrimeFreedom.a_computation_reading_no_act_is_not_forced' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.a_computation_reading_no_act_is_not_forced

/-- info: 'PrimeFreedom.the_witness_forced_by_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PrimeFreedom.the_witness_forced_by_the_act

/-- info: 'PrimeFreedom.the_denials_and_the_witness' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_denials_and_the_witness

/-! ## Cones, pinned: section XXVI -/

/-- info: 'PrimeFreedom.the_capstone' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PrimeFreedom.the_capstone

end PartII

/-! # PART III · Annihilation_Closure.lean, verbatim, with the second edition laws appended -/
section PartIII
/-
  Annihilation_Closure.lean · The Master Annihilation Closure.
  Core Lean 4.19.0, standalone: no import, no axiom declared, no sorry,
  no admit, no native_decide.

  BOOKS ONE AND TWO are `lean/Remembered_Offered.lean` carried
  byte-identical below, its own header included:
  the two denials, the price, the monism-fixed toy of the bound, and the
  harvest of 7 October 2026 with its
  capstones `the_chain` and `the_harvest`.

  BOOK THREE is the annihilation closure, harvested on 7 October 2026:
  I    four names, one proposition: least erasure, the value, Monism and
       the co-location of Being with Monism;
  II   Monism, the sole witness: every faithful witness is Monism, no
       reading of the record witnesses, and
       a witness is never universal;
  III  co-location is not universal: Being stands on the off-line pair
  configuration and Monism does not;
  IV   the timeless zone: one proposition, the locus resting both
  instruments, zero time remembering, every
       later time forgetting, closure at zero time exactly at Λ = 0, the
       upstream unforced;
  V    timeless is determinacy, not selection;
  VI   the check: the off-line pair configuration satisfies every
  principle of the closure and lacks the value,
       its twin has it;
       no principle true on every closed configuration decides the value;
  VII  the capstones: `the_timeless_zone_closed`, the timeless zone with
  no act anywhere in it, and
       `the_annihilation_closure`, the zone together with the act's
       conjunct of the time world.

  BOOK FOUR is the Unicorn, contained. `lean/RH_Unicorn_Block.lean` is
  carried byte-identical below, its own header
  included: the cut at a height, the formal block, the aperture one bit
  wide, existence supplying nothing, no bypass,
  the supply side silent, the certificate, and the three bits as one
  term. Book Four binds it to the chart:
  I    the cut on the chart, at every height;
  II   no bypass on the chart;
  III  the Unicorn blocked on the chart, above every height;
  IV   RA supplies nothing to the Unicorn part;
  V    on the act's zero set, every arrival above every height stands on
  the line;
  VI   the capstone, `the_unicorn_contained`.

  Stated whole, locked 7 October 2026: the timeless zone is closed, LE =
  RH = Monism, one proposition, with zero
  gap, proved; RH is the statement that ζ's zero set lies in the timeless
  zone. Book Three proves the first and
  assumes nothing; its sixth section proves that nothing every closed
  configuration shares can single out ζ. In
  the time world the statement enters only where Book One's `ActualZeros`
  is taken as input. The cones are printed
  at the foot and pinned.
-/
/-
  Remembered_Offered.lean · The Remembered and the Offered: One Self, One
  Place at a Time.
  Core Lean 4.19.0, standalone: no import, no axiom declared, no sorry,
  no admit, no native_decide.

  BOOK ONE is carried unchanged: Two_Denials.lean whole (the deed and the
  self-grounding root from Armed_Seat.lean,
  the chart, the record, the act and the deed's law from
  Force_Witness.lean, the monism witness from Codex.lean,
  the two denials at the two layers of one root); the finite
  configuration and its price from Inescapable_Capstone.lean;
  the toy of the bound from Codex.lean, monism fixed, namespace RALi.

  BOOK TWO is the harvest of 7 October 2026, in binding order:
  I    the Tongue always has a world; monism, not the Tongue, tells the
  worlds apart;
  II   one self, one place at a time: the floor and least erasure are two
  objects;
  III  the twins: one extension, two instruments, neither the voucher of
  the other;
  IV   the remembered and the offered: memory is lossless, a world is its
  own memory exactly when the value holds;
  V    forgetting and remembering: one sequence, a one-bit gap, always
  connected;
  Vb   the forcing by heat: return is free, resistance pays, the free
  path is one;
  VI   THE CHAIN: the root, forgetting, the locus in the middle,
  remembering, least erasure;
  VIb  zero time remembers, every later time forgets;
  VII  two doors and no third, the act shuts the second;
  VIII a band forces nothing;
  IX   the harvest, bound.

  Stated whole: everything is proved except the offer at the actual
  zeros, which enters only as the field `supply`
  of `ActualZeros`, the act, at premise grade. The cones are printed at
  the foot and pinned.
-/
set_option autoImplicit false
namespace RememberedOffered

/-! # BOOK ONE · carried unchanged

## Two_Denials.lean, whole -/

/-! ## PART ONE · copied unchanged

### from Armed_Seat.lean · the deed and the root -/

def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- A denial of the root is an act, and re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R :=
```

```
  G.instances denial

/-- A self-grounding root is self-verifying: the deed of denying it hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) : SelfVerifying R :=
  fun _ => G.instances G.anAct

/-- SEATED, UNDENIABLE: a self-grounding root holds, with no classical detour, by the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- The Root Axiom at the constructed one-point domain: every act is a deed, and a deed actuates. -/
def RA : Prop := (0 : Int) < 1
def raSelfGrounding : SelfGrounding RA := (Unit, (), fun _ => show (0 : Int) < 1 by decide)

theorem root_undeniable : RA := raSelfGrounding.instances ()

/-! ### from Force_Witness.lean · the chart, the record, the act, the deed's law -/

/-- A point is (d, t): d its offset from the line, t its height. The line is d = 0. -/
abbrev Point := Int × Int

/-- The fold: s ↦ 1 − s̄ on the offset chart, (d, t) ↦ (−d, t). -/
def fold (p : Point) : Point := (-p.1, p.2)

/-- The line: offset zero. -/
def onLine (p : Point) : Prop := p.1 = 0

/-- The registration: it keeps the height and forgets the side. -/
def reg (p : Point) : Point := (0, p.2)

/-- Integer negation is an involution, by the integer's constructors. -/
theorem neg_neg_free : ∀ d : Int, - -d = d := fun
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

/-- The fold is an involution. -/
theorem fold_involutive (p : Point) : fold (fold p) = p := by
  obtain (d, t) := p
  show (- -d, t) = (d, t)
  rw [neg_neg_free d]

/-- The registration lands on the line. -/
theorem reg_lands (p : Point) : onLine (reg p) := rfl

/-- The registration erases nothing at a point exactly when the point is on the line. -/
theorem reg_erases_nothing_iff (p : Point) : reg p = p ↔ onLine p := by
  obtain (d, t) := p
  exact (fun h => (congrArg Prod.fst h).symm, fun h => by cases h; rfl)

/-- A configuration: a set of points. Closed: the fold carries it to itself. -/
abbrev Config := Point → Prop

def Closed (S : Config) : Prop := ∀ p, S p → S (fold p)

/-- The value: every point of the configuration is on the line. -/
def Value (S : Config) : Prop := ∀ p, S p → onLine p

/-- Least erasure: the registration leaves every point of the configuration unchanged. -/
def LeastErasure (S : Config) : Prop := ∀ p, S p → reg p = p

/-- The record of a configuration: what the registration leaves. -/
def Rec (S : Config) (q : Point) : Prop := ∃ p, S p ∧ reg p = q

def SameRecord (S S' : Config) : Prop := ∀ q, Rec S q ↔ Rec S' q

/-- The registered world of a point and its pair world. -/
def registered (p : Point) : Config := fun s => s = reg p

def pairWorld (p : Point) : Config := fun s => s = p ∨ s = fold p

theorem pair_world_closed (p : Point) : Closed (pairWorld p) := fun _ hs =>
  match hs with
  | Or.inl e => Or.inr (congrArg fold e)
  | Or.inr e => Or.inl (e ▸ fold_involutive p)

/-- The registered world and the pair world leave one record. -/
theorem one_record (p : Point) : SameRecord (registered p) (pairWorld p) := fun q =>
  (fun (_, hs, hq) => (p, Or.inl rfl, by rw [← hq, hs]; rfl),
   fun (_, hs, hq) => (reg p, rfl, by
      rw [← hq]
      exact match hs with
        | Or.inl e => by rw [e]; rfl
        | Or.inr e => by rw [e]; rfl))

theorem registered_has_value (p : Point) : Value (registered p) := fun s hs => by
  rw [hs]; exact reg_lands p

theorem pair_world_lacks_value (p : Point) (h : ¬ onLine p) : ¬ Value (pairWorld p) :=
  fun hv => h (hv p (Or.inl rfl))

/-- THE ACT. The bit is supplied once, in the open, as a field of a type: a closed configuration with least
    erasure at it. The field is the value supplied; the type makes the supply visible to every reader. -/
structure ActualZeros where
  zeros : Config
  closed : Closed zeros
  supply : LeastErasure zeros

/-- FROM THE ACT, THE VALUE. -/
theorem value_from_the_act (Z : ActualZeros) : Value Z.zeros :=
  fun p hp => (reg_erases_nothing_iff p).mp (Z.supply p hp)

/-- AND OF NOTHING ELSE: a fact that holds whatever the configuration, the deed included, forces no value. -/
theorem the_deed_forces_nothing (D : Prop) (hD : D) : ¬ ∀ S : Config, Closed S → D → Value S := by
  intro hall
  exact pair_world_lacks_value (1, 0) (fun e => by cases e) (hall _ (pair_world_closed (1, 0)) hD)


/-! ### from Codex.lean, namespace RALi, Part VII · the monism witness, the root read twice -/

structure Q4 where
  r : Int
  i : Int
  j : Int
  k : Int
  deriving DecidableEq, Repr

/-- The geometric route's involution and the formal route's involution, written apart. -/
def sigmaGeo (q : Q4) : Q4 := (q.r, -q.i, -q.j, -q.k)
def sigmaForm (q : Q4) : Q4 := (q.r, -q.i, -q.j, -q.k)

/-- THE MONISM WITNESS. One involution, one principle, one seed. -/
structure MonismWitness (P : Nat → Prop) : Prop where
  one_involution : ∀ q, sigmaGeo q = sigmaForm q
  uniform        : ∀ n m, P n ↔ P m
  seed           : P 0

/-- 24. The seat field is constructed, not posited: σ = σ′ holds by rfl. -/
theorem one_involution_constructed : ∀ q, sigmaGeo q = sigmaForm q := fun _ => rfl

/-- 25. The timeless reading and the timed reading coincide under one principle:
the claim over all time is the claim at one instant. Deleting time loses nothing. -/
theorem timeless_equals_timed (P : Nat → Prop) (u : ∀ n m, P n ↔ P m) :
    (∀ n, P n) ↔ P 0 :=
  (fun h => h 0, fun h n => (u 0 n).mp h)

/-- 26. The witness closes the record, past and future. -/
theorem monism_closes (P : Nat → Prop) (w : MonismWitness P) : ∀ n, P n :=
  (timeless_equals_timed P w.uniform).mpr w.seed

/-- 27. The witness is exactly the claim: a monism witness for P exists iff P holds at
every index. Its value field carries the whole of the claim, as theorem 17 required. -/
theorem monism_witness_is_the_claim (P : Nat → Prop) :
    MonismWitness P ↔ ∀ n, P n :=
  (monism_closes P, fun h => (one_involution_constructed,
    fun n m => (fun _ => h m, fun _ => h n), h 0))

/-! ## PART TWO · two denials, one shape, at the two layers of one root

A no has two parts: the act of uttering it and the content it asserts.
The Tongue's no asserts ¬ R of the floor R.
The no to RH asserts that the zeros lack the value: that they form the false world. -/

/-- The point off the line from which the pair world, the false world, is built. -/
def offPoint : Point := (1, 0)

theorem off_point_is_off : ¬ onLine offPoint := fun h =>
  Nat.noConfusion (Int.ofNat.inj (show Int.ofNat 1 = Int.ofNat 0 from h))

/-- THE ROOT READ TWICE on a configuration: the monism witness of its line property, timeless. -/
abbrev RootReadTwice (S : Config) : Prop := MonismWitness (fun _ => Value S)

/-- THE SAME FIRST STEP: whatever a no asserts, uttering it is an act, and the act re-enacts the floor. -/
theorem every_no_reenacts_the_root {R C : Prop} (G : SelfGrounding R) (deed : G.Act) (_content : C) : R :=
  denial_reenacts_root G deed

/-- THE TONGUE'S NO CONTRADICTS THE FLOOR, BY ITS OWN ACT: its content denies the floor, and its own act
    re-enacts it. -/
theorem the_tongue_no_refutes_itself {R : Prop} (G : SelfGrounding R) (deed : G.Act) (content : ¬ R) : False :=
  content (denial_reenacts_root G deed)
```

```
/-- ONLY THE YES REMAINS AT THE FLOOR: the return is forced by the deed, with no key and no classical detour. -/
theorem only_the_yes_remains_at_the_root {R : Prop} (G : SelfGrounding R) : SelfVerifying R ∧ R :=
  (denial_instantiates G, seated_undeniable G)

/-- THE ROOT READ TWICE IS THE VALUE: on every configuration the monism witness stands exactly where every zero
    is on the line. -/
theorem the_root_read_twice_is_the_value (S : Config) : RootReadTwice S ↔ Value S :=
  (fun w => w.seed, fun h => (one_involution_constructed, fun _ _ => Iff.rfl, h))

/-- THE NO TO RH CONTRADICTS THE ROOT READ TWICE: a configuration lacking the value carries no monism. -/
theorem the_line_no_contradicts_the_root_read_twice (S : Config) (content : ¬ Value S) : ¬ RootReadTwice S :=
  fun w => content w.seed

/-- THE TRUE WORLD CARRIES MONISM: the registered world of the off-line point. -/
theorem the_true_world_carries_monism : RootReadTwice (registered offPoint) :=
  (one_involution_constructed, fun _ _ => Iff.rfl, registered_has_value offPoint)

/-- THE FALSE WORLD CARRIES NO MONISM: the pair world of the off-line point. -/
theorem the_false_world_carries_no_monism : ¬ RootReadTwice (pairWorld offPoint) :=
  fun w => pair_world_lacks_value offPoint off_point_is_off w.seed

/-- THE FALSE WORLD, AT THE TWO LAYERS: the floor stands there, as in every world; it is closed and leaves the
    record of the true world; it lacks the value; and it carries no
    monism. The floor is there and Being is not. -/
theorem the_false_world_carries_the_floor_and_no_monism {R : Prop} (G : SelfGrounding R) :
    R ∧ Closed (pairWorld offPoint) ∧ SameRecord (registered offPoint) (pairWorld offPoint) ∧
    ¬ Value (pairWorld offPoint) ∧ ¬ RootReadTwice (pairWorld offPoint) :=
  (seated_undeniable G, pair_world_closed offPoint, one_record offPoint,
   pair_world_lacks_value offPoint off_point_is_off,
   the_false_world_carries_no_monism)

/-- THE FLOOR ALONE DECIDES NO VALUE: it stands in both worlds of one record. The value is decided at the second
    layer, by the root read twice. -/
theorem the_floor_alone_decides_no_value {R : Prop} (G : SelfGrounding R) :
    ¬ ∀ S : Config, Closed S → R → Value S :=
  the_deed_forces_nothing R (seated_undeniable G)

/-- THE RETURN IS THE ACT: the act carries the root read twice at the actual zeros. -/
theorem the_act_is_the_return_to_the_root_read_twice (Z : ActualZeros) : RootReadTwice Z.zeros :=
  (the_root_read_twice_is_the_value Z.zeros).mpr (value_from_the_act Z)

/-- AN ACT EXISTS EXACTLY WHERE THE ROOT READ TWICE STANDS on a closed configuration. -/
theorem an_act_exactly_where_the_root_read_twice_stands (S : Config) :
    (∃ Z : ActualZeros, Z.zeros = S) ↔ (Closed S ∧ RootReadTwice S) := by
  constructor
  · intro (Z, h)
    subst h
    exact (Z.closed, the_act_is_the_return_to_the_root_read_twice Z)
  · intro (hc, hm)
    exact ((S, hc, fun p hp => (reg_erases_nothing_iff p).mpr
      (((the_root_read_twice_is_the_value S).mp hm) p hp)), rfl)

/-- ON THE ACT THE NO TO RH IS REFUTED, with zero contradiction: the act's own field. -/
theorem on_the_act_the_line_no_is_refuted (Z : ActualZeros) (content : ¬ Value Z.zeros) : False :=
  content (value_from_the_act Z)

/-- TWO DENIALS, ONE SHAPE, TWO LAYERS. Every no re-enacts the floor. The Tongue's no contradicts the floor and
    its own act returns it: only the yes remains. The no to RH contradicts the root read twice: on every
    configuration a lack of the value is a lack of monism, and the false world carries the floor and no monism,
    beside the true world of one record that carries both. So the floor alone decides no value, and the root read
    twice is the value. The return to the root read twice at the actual zeros is the act: there the monism witness
    stands, the value holds, and the no is refuted. -/
theorem two_denials {R : Prop} (G : SelfGrounding R) :
    (∀ C : Prop, G.Act → C → R) ∧
    (G.Act → ¬ R → False) ∧
    (SelfVerifying R ∧ R) ∧
    (∀ S : Config, ¬ Value S → ¬ RootReadTwice S) ∧
    (R ∧ Closed (pairWorld offPoint) ∧ SameRecord (registered offPoint) (pairWorld offPoint) ∧
      RootReadTwice (registered offPoint) ∧ ¬ RootReadTwice (pairWorld offPoint)) ∧
    (¬ ∀ S : Config, Closed S → R → Value S) ∧
    (∀ S : Config, RootReadTwice S ↔ Value S) ∧
    (∀ Z : ActualZeros, RootReadTwice Z.zeros ∧ Value Z.zeros ∧ (¬ Value Z.zeros → False)) :=
  (fun _ deed c => every_no_reenacts_the_root G deed c,
   fun deed c => the_tongue_no_refutes_itself G deed c,
   only_the_yes_remains_at_the_root G,
   the_line_no_contradicts_the_root_read_twice,
   (seated_undeniable G, pair_world_closed offPoint, one_record offPoint,
    the_true_world_carries_monism, the_false_world_carries_no_monism),
   the_floor_alone_decides_no_value G,
   the_root_read_twice_is_the_value,
   fun Z => (the_act_is_the_return_to_the_root_read_twice Z,
   value_from_the_act Z,
    on_the_act_the_line_no_is_refuted Z))

/-- AT THE ROOT AXIOM ITSELF. -/
theorem two_denials_at_RA :
    (∀ C : Prop, raSelfGrounding.Act → C → RA) ∧
    (raSelfGrounding.Act → ¬ RA → False) ∧
    (SelfVerifying RA ∧ RA) ∧
    (∀ S : Config, ¬ Value S → ¬ RootReadTwice S) ∧
    (RA ∧ Closed (pairWorld offPoint) ∧ SameRecord (registered offPoint) (pairWorld offPoint) ∧
      RootReadTwice (registered offPoint) ∧ ¬ RootReadTwice (pairWorld offPoint)) ∧
    (¬ ∀ S : Config, Closed S → RA → Value S) ∧
    (∀ S : Config, RootReadTwice S ↔ Value S) ∧
    (∀ Z : ActualZeros, RootReadTwice Z.zeros ∧ Value Z.zeros ∧ (¬ Value Z.zeros → False)) :=
  two_denials raSelfGrounding

/-! ## From Inescapable_Capstone.lean · the finite configuration and its price -/

/-- The price of exported freedom in native units: temperature is energy per bit, so b bits cost b·T. -/
def price (T bits : Nat) : Nat := bits * T

/-- A finite configuration: the heights of its zeros on the line, and its off-line pairs, each an offset that is not
    zero and a height. -/
structure FinCfg where
  line : List Int
  pairs : List (Int × Int)
  offset_ne : ∀ q ∈ pairs, q.1 ≠ 0
def FinCfg.pts (F : FinCfg) : Config := fun p => (p.1 = 0 ∧ p.2 ∈ F.line) ∨ ∃ q ∈ F.pairs, p = q ∨ p = fold q
/-- The erased bits of a finite configuration: one per off-line pair, two points registered to one. -/
def erased (F : FinCfg) : Nat := F.pairs.length

theorem fincfg_closed (F : FinCfg) : Closed F.pts := fun p hp =>
  match hp with
  | Or.inl (hl, ht) => Or.inl (by show -p.1 = 0; rw [hl]; rfl, ht)
  | Or.inr (q, hq, Or.inl e) => Or.inr (q, hq, Or.inr (congrArg fold e))
  | Or.inr (q, hq, Or.inr e) => Or.inr (q, hq, Or.inl (by rw [e]; exact fold_involutive q))

/-- LEAST ERASURE IS ZERO ERASED BITS: "least" is a count. -/
theorem least_erasure_iff_zero_erased (F : FinCfg) : LeastErasure F.pts ↔ erased F = 0 := by
  constructor
  · intro h
    unfold erased
    cases hp : F.pairs with
    | nil => rfl
    | cons q r =>
      have hq : q ∈ F.pairs := by rw [hp]; exact List.Mem.head r
      exact absurd ((reg_erases_nothing_iff q).mp (h q (Or.inr (q, hq, Or.inl rfl)))) (F.offset_ne q hq)
  · intro h p hp
    have hnil : F.pairs = [] := List.eq_nil_of_length_eq_zero h
    rcases hp with (hl, _) | (q, hq, _)
    · exact (reg_erases_nothing_iff p).mpr hl
    · rw [hnil] at hq; cases hq

/-- THE HEAT OF REGISTRATION IS ZERO EXACTLY AT THE VALUE: registering a finite configuration costs one T per
    off-line pair, and at any positive T it costs nothing exactly when every point stands on the line. -/
theorem price_zero_iff_value (F : FinCfg) (T : Nat) (hT : 0 < T) : price T (erased F) = 0 ↔ Value F.pts := by
  constructor
  · intro h p hp
    have h0 : erased F = 0 := by
      cases he : erased F with
      | zero => rfl
      | succ k => rw [he] at h; exact absurd h (Nat.pos_iff_ne_zero.mp (Nat.mul_pos (Nat.succ_pos k) hT))
    exact (reg_erases_nothing_iff p).mp ((least_erasure_iff_zero_erased F).mpr h0 p hp)
  · intro hv
    have h0 := (least_erasure_iff_zero_erased F).mp (fun p hp => (reg_erases_nothing_iff p).mpr (hv p hp))
    show erased F * T = 0
    rw [h0, Nat.zero_mul]

/-- Each further off-line pair adds one T: the total is the erased bits times the floor. -/
theorem price_counts_pairs (F : FinCfg) (T : Nat) : price T (erased F) = F.pairs.length * T := rfl

/-! ## From Codex.lean, monism fixed, namespace RALi · the toy of the bound, a toy of the bound, not of the flow -/

/-- Squared strip width after t time steps: max(d2 − 2t, 0). -/
def flow (d2 t : Nat) : Nat := d2 - 2 * t
```

```
def realAt (d2 t : Nat) : Prop := flow d2 t = 0

/-- 38. THE MONOTONICITY FORMULA. The width never grows along ζ's time. -/
theorem flow_monotone (d2 t : Nat) : flow d2 (t + 1) ≤ flow d2 t := by
  unfold flow; omega

/-- 39. Its transport, proved: once every zero is real, every zero stays real. This is
de Bruijn's theorem in the toy, the ζ-analogue of Perelman's monotonicity, at theorem grade. -/
theorem reality_transported (d2 t : Nat) (h : realAt d2 t) : realAt d2 (t + 1) := by
  unfold realAt flow at *; omega


def Lam (d2 : Nat) : Nat := (d2 + 1) / 2

theorem real_at_Lam (d2 : Nat) : realAt d2 (Lam d2) := by unfold realAt flow Lam; omega

/-- 40. In the toy, the strip is closed at time zero iff Λ = 0. For ζ, RH ↔ Λ = 0 is the cited
result of Newman with Rodgers and Tao and enters the cone as `hLam`. -/
theorem rh_iff_lambda_zero (d2 : Nat) : realAt d2 0 ↔ Lam d2 = 0 := by
  unfold realAt flow Lam
  exact (fun h => by omega, fun h => by omega)


/-- 42. THE RECORD FORGETS. After one step the record of the width-zero state and the
width-one state coincide: the forward flow is even in the RH bit, so no readout of the
flowed record decides RH. This is the fTOE wall, T1, executed on ζ's own time. -/
theorem flowed_record_forgets (T : Nat) (hT : 1 ≤ T) :
    ¬ ∃ g : Nat → Bool, ∀ d2, g (flow d2 T) = decide (d2 = 0) := by
  rintro (g, hg)
  have a := hg 0; have b := hg 1
  have e : flow 1 T = flow 0 T := by unfold flow; omega
  rw [e, a] at b
  exact absurd b (by decide)

/-- 43. The upstream direction is not a transport. Going backward from a real record,
both answers are admissible: the preimage of "real at T" holds the RH state and a non-RH
state. The forward arrow proves; the backward arrow must be supplied. -/
theorem upstream_is_not_forced (T : Nat) (hT : 1 ≤ T) :
    realAt 0 T ∧ realAt 1 T ∧ realAt 0 0 ∧ ¬ realAt 1 0 := by
  unfold realAt flow
  exact (by omega, by omega, rfl, fun h => by omega)

/-! # BOOK TWO · the harvest, in binding order -/

/-! ## I · The Tongue always has a world -/

/-- THE TONGUE ALWAYS HAS A WORLD: the floor stands on every configuration, the false world included, so the Tongue
    can speak in every world; only the content of its no, ¬R beside an act, has none. -/
theorem the_tongue_always_has_a_world {R : Prop} (G : SelfGrounding R) :
    (∀ _ : Config, R) ∧
    (R ∧ Closed (pairWorld offPoint) ∧ ¬ Value (pairWorld offPoint)) ∧
    ¬ ∃ _ : G.Act, ¬ R :=
  (fun _ => seated_undeniable G,
   (seated_undeniable G, pair_world_closed offPoint,
    pair_world_lacks_value offPoint off_point_is_off),
   fun (deed, c) => c (G.instances deed))

/-- MONISM BEFORE THE TONGUE: the floor, and with it the Tongue, stands in both worlds of one record; what tells
    them apart is the root read twice, present in the true world and absent in the false. -/
theorem monism_before_the_tongue {R : Prop} (G : SelfGrounding R) :
    SameRecord (registered offPoint) (pairWorld offPoint) ∧
    (R ∧ RootReadTwice (registered offPoint)) ∧
    (R ∧ ¬ RootReadTwice (pairWorld offPoint)) :=
  (one_record offPoint,
   (seated_undeniable G, the_true_world_carries_monism),
   (seated_undeniable G, the_false_world_carries_no_monism))

/-- The Tongue's no has no world: no act stands beside ¬R. -/
theorem tongue_no_has_no_world {R : Prop} (G : SelfGrounding R) : ¬ ∃ _ : G.Act, ¬ R :=
  fun (deed, c) => c (G.instances deed)

/-- The no to RH has a world: an act, a closed configuration, the floor holding, and the value failing, together. -/
theorem line_no_has_a_world {R : Prop} (G : SelfGrounding R) :
    ∃ S : Config, ∃ _ : G.Act, Closed S ∧ R ∧ ¬ Value S :=
  (pairWorld offPoint, G.anAct, pair_world_closed offPoint,
   seated_undeniable G,
   pair_world_lacks_value offPoint off_point_is_off)

/-- A deed does not make a closed configuration an act: the floor's acts supply no ActualZeros over every closed
    configuration. -/
theorem the_deed_is_not_the_supply {R : Prop} (G : SelfGrounding R) :
    ¬ ∀ S : Config, Closed S → G.Act → ∃ Z : ActualZeros, Z.zeros = S :=
  by
  intro h
  obtain (Z, hZ) := h (pairWorld offPoint) (pair_world_closed offPoint) G.anAct
  have v := value_from_the_act Z
  rw [hZ] at v
  exact pair_world_lacks_value offPoint off_point_is_off v

theorem at_RA :
    (¬ ∃ _ : raSelfGrounding.Act, ¬ RA) ∧
    (∃ S : Config, ∃ _ : raSelfGrounding.Act, Closed S ∧ RA ∧ ¬ Value S) ∧
    (¬ ∀ S : Config, Closed S → raSelfGrounding.Act → ∃ Z : ActualZeros,
     Z.zeros = S) :=
  (tongue_no_has_no_world raSelfGrounding, line_no_has_a_world raSelfGrounding,
   the_deed_is_not_the_supply raSelfGrounding)

/-! ## II · One self, one place at a time -/

/-- The floor and least erasure are two objects: identified on every closed configuration, they would agree on the
    pair world, where the floor holds and least erasure fails. -/
theorem floor_is_not_least_erasure {R : Prop} (G : SelfGrounding R) :
    ¬ ∀ S : Config, Closed S → (R ↔ LeastErasure S) := by
  intro h
  have le : LeastErasure (pairWorld offPoint) :=
    (h (pairWorld offPoint) (pair_world_closed offPoint)).mp
    (seated_undeniable G)
  exact pair_world_lacks_value offPoint off_point_is_off
    (fun p hp => (reg_erases_nothing_iff p).mp (le p hp))

/-- The voucher stays with the floor: it holds on both worlds of one record, the false one included. -/
theorem the_voucher_stays_with_the_floor {R : Prop} (G : SelfGrounding R) :
    R ∧ SameRecord (registered offPoint) (pairWorld offPoint) ∧
    LeastErasure (registered offPoint) ∧ ¬ LeastErasure (pairWorld offPoint) :=
  (seated_undeniable G, one_record offPoint,
   fun p hp => (reg_erases_nothing_iff p).mpr (registered_has_value offPoint p hp),
   fun le => pair_world_lacks_value offPoint off_point_is_off
     (fun p hp => (reg_erases_nothing_iff p).mp (le p hp)))

/-- Merging them at one world is the supply again: on any configuration, identifying the floor with least erasure
    there is least erasure there. -/
theorem merging_at_a_world_is_the_supply {R : Prop} (G : SelfGrounding R) (S : Config) :
    (R ↔ LeastErasure S) ↔ LeastErasure S :=
  (fun h => h.mp (seated_undeniable G), fun le => (fun _ => le, fun _ => seated_undeniable G))

/-! ## III · The twins -/

/-- Two instruments: registration and the fold are different maps. -/
theorem two_instruments : reg offPoint ≠ fold offPoint := by
  show ((0 : Int), (0 : Int)) ≠ (-(1 : Int), (0 : Int))
  decide

/-- One truth value on every world: each twin holds exactly where the other does. -/
theorem twins_on_every_world (S : Config) : LeastErasure S ↔ Value S :=
  (fun le p hp => (reg_erases_nothing_iff p).mp (le p hp),
   fun v p hp => (reg_erases_nothing_iff p).mpr (v p hp))

/-- Extensionally one proposition, read through two different maps. -/
theorem twins_are_one_extension : LeastErasure = Value :=
  funext fun S => propext (twins_on_every_world S)

/-- Neither twin vouches for the other: whatever supplies one supplies the other, so a premise that reaches one
    reaches both, and the floor reaches neither. -/
theorem no_twin_is_the_others_voucher {R : Prop} (G : SelfGrounding R) :
    (∀ S : Config, (LeastErasure S → Value S) ∧ (Value S → LeastErasure S)) ∧
    ¬ ∀ S : Config, Closed S → R → Value S :=
  (fun S => ((twins_on_every_world S).mp, (twins_on_every_world S).mpr),
   the_floor_alone_decides_no_value G)

/-! ## IV · The remembered and the offered -/

/-- No instrument both remembers and keeps the side: a map that lands every point on the line and keeps the
    height sends the off-line point and its fold partner to one point. -/
theorem no_instrument_remembers_and_keeps_the_side (g : Point → Point)
    (lands : ∀ p, onLine (g p)) (keeps : ∀ p, (g p).2 = p.2) :
    g offPoint = g (fold offPoint) ∧ offPoint ≠ fold offPoint := by
  have e1 : g offPoint = (0, 0) := Prod.ext (lands offPoint) (keeps offPoint)
  have e2 : g (fold offPoint) = (0, 0) := Prod.ext (lands (fold offPoint)) (keeps (fold offPoint))
  exact (e1.trans e2.symm, by decide)

/-- Memory is always lossless: the record of every configuration has the value. -/
theorem the_remembered_is_lossless (S : Config) : Value (Rec S) :=
  fun _ (p, _, hq) => hq ▸ reg_lands p

/-- The offered is the remembered exactly when the value holds: a configuration equals its own record, point for
    point, if and only if every point is on the line. -/
theorem offered_is_remembered_iff_value (S : Config) : (∀ q, Rec S q ↔ S q) ↔ Value S := by
  constructor
  · intro h p hp
    exact the_remembered_is_lossless S p ((h p).mpr hp)
```

```
  · intro v q
    constructor
    · intro ⟨p, hp, hq⟩
      have e : reg p = p := (reg_erases_nothing_iff p).mpr (v p hp)
      rw [← hq, e]; exact hp
    · intro hq
      exact ⟨q, hq, (reg_erases_nothing_iff q).mpr (v q hq)⟩

/-- At the act, the offered is the remembered. -/
theorem at_the_act_offered_is_remembered (Z : ActualZeros) : ∀ q, Rec
Z.zeros q ↔ Z.zeros q :=
  (offered_is_remembered_iff_value Z.zeros).mpr (value_from_the_act Z)

/-- Off the act, they part: the false world is not its own record. -/
theorem the_false_world_is_not_its_record : ¬ ∀ q, Rec (pairWorld
offPoint) q ↔ pairWorld offPoint q :=
  fun h => pair_world_lacks_value offPoint off_point_is_off
  ((offered_is_remembered_iff_value _).mp h)

/-! ## V · Forgetting and remembering: the one-bit gap, connected -/

/-- Remembering is stable: the memory of a memory is the memory. -/
theorem remembering_is_stable (S : Config) (q : Point) : Rec (Rec S) q ↔
Rec S q := by
  constructor
  · intro ⟨p, ⟨r, hr, hp⟩, hq⟩
    refine ⟨r, hr, ?_⟩
    rw [← hq, ← hp]
    exact ((reg_erases_nothing_iff (reg r)).mpr (reg_lands r)).symm
  · intro hq
    exact ⟨q, hq, (reg_erases_nothing_iff q).mpr
    (the_remembered_is_lossless S q hq)⟩

/-- THE ONE-BIT GAP, CONNECTED. The forgotten and the remembered always
share one record; the remembered is
    lossless and stable; and the only gap between a world and its memory
    is whether the world is its own memory,
    which is the value: one bit, never more, and never disconnected. -/
theorem one_bit_gap_connected (S : Config) :
    SameRecord S (Rec S) ∧ Value (Rec S) ∧ (∀ q, Rec (Rec S) q ↔ Rec S q)
    ∧
    ((∀ q, Rec S q ↔ S q) ↔ Value S) :=
  ⟨fun q => (remembering_is_stable S q).symm, the_remembered_is_lossless
  S, remembering_is_stable S,
   offered_is_remembered_iff_value S⟩

/-- Both sides of the gap occur on one record: the true world at home in
its memory, the false world not. -/
theorem the_gap_is_live :
    (∀ q, Rec (registered offPoint) q ↔ registered offPoint q) ∧
    ¬ (∀ q, Rec (pairWorld offPoint) q ↔ pairWorld offPoint q) ∧
    SameRecord (registered offPoint) (pairWorld offPoint) :=
  ⟨(offered_is_remembered_iff_value _).mpr (registered_has_value
  offPoint),
   the_false_world_is_not_its_record, one_record offPoint⟩

/-! ## Vb · The forcing by heat -/

/-- One pair off the line at offset one: a finite configuration that
resists. -/
def resisting : FinCfg := ⟨[], [((1 : Int), (0 : Int))], fun q hq => by
  cases hq with
  | head => decide
  | tail _ h => cases h⟩

/-- RETURN IS FREE, RESISTANCE PAYS: at any positive temperature, a
configuration with the value registers at zero
    heat, and one without it pays at least one unit of heat. -/
theorem return_is_free_resistance_pays (F : FinCfg) (T : Nat) (hT : 0 <
T) :
    (Value F.pts → price T (erased F) = 0) ∧ (¬ Value F.pts → T ≤ price T
    (erased F)) := by
  refine ⟨(price_zero_iff_value F T hT).mpr, fun hn => ?_⟩
  have hne : erased F ≠ 0 := fun h0 =>
    hn ((price_zero_iff_value F T hT).mp (by unfold price; rw [h0]; exact
    Nat.zero_mul T))
  unfold price
  have h1 : 1 ≤ erased F := Nat.pos_of_ne_zero hne
  calc T = 1 * T := (Nat.one_mul T).symm
    _ ≤ erased F * T := Nat.mul_le_mul_right T h1

/-- RESISTANCE IS POSSIBLE: it is no contradiction. A closed finite
configuration without the value exists, and it
    pays. -/
theorem resistance_is_possible (T : Nat) (hT : 0 < T) :
    Closed resisting.pts ∧ ¬ Value resisting.pts ∧ T ≤ price T (erased
    resisting) := by
  have hnv : ¬ Value resisting.pts := fun hv =>
    have h : ((1 : Int), (0 : Int)).1 = 0 :=
      hv ((1 : Int), (0 : Int)) (Or.inr ((⟨(1 : Int), (0 : Int)⟩),
      List.Mem.head _, Or.inl rfl))
    (by decide : ¬ ((1 : Int) = 0)) h
  exact ⟨fincfg_closed resisting, hnv, (return_is_free_resistance_pays
  resisting T hT).2 hnv⟩

/-- THE FREE PATH IS ONE: two worlds of one record that both have the
value are one world, point for point. -/
theorem the_free_path_is_unique (S S' : Config) (hs : SameRecord S S')
(hv : Value S) (hv' : Value S') :
    ∀ q, S q ↔ S' q := fun q =>
  ⟨fun h => ((offered_is_remembered_iff_value S').mpr hv' q).mp
      ((hs q).mp (((offered_is_remembered_iff_value S).mpr hv q).mpr h)),
   fun h => ((offered_is_remembered_iff_value S).mpr hv q).mp
      ((hs q).mpr (((offered_is_remembered_iff_value S').mpr hv' q).mpr
      h))⟩

/-! ## VI · THE CHAIN -/

/-- The locus is where the fold and the registration both rest. -/
theorem the_locus_rests_both (p : Point) (h : onLine p) : fold p = p ∧
reg p = p := by
  obtain ⟨d, t⟩ := p
  have hd : d = 0 := h
  subst hd
  exact ⟨rfl, rfl⟩

/-- THE CHAIN. The root holds; forgetting sends every point onto the
locus and loses the side; the locus is where
    the fold and the registration both rest; remembering reads the locus,
    so memory is lossless and stable; a world
    has least erasure exactly when it already stands on the locus,
    exactly when it is its own memory; on the act
    the zeros stand there; and the guard: the root alone places no world
    there, and both sides of the gap stand on
    one record. -/
theorem the_chain {R : Prop} (G : SelfGrounding R) (S : Config) :
    -- the root
    R ∧
    -- forgetting: onto the locus, the side lost
    (∀ p, onLine (reg p)) ∧ reg offPoint = reg (fold offPoint) ∧ offPoint
    ≠ fold offPoint ∧
    -- the middle: the locus rests both maps
    (∀ p, onLine p → fold p = p ∧ reg p = p) ∧
    -- remembering: lossless and stable
    Value (Rec S) ∧ (∀ q, Rec (Rec S) q ↔ Rec S q) ∧ SameRecord S (Rec S)
    ∧
    -- least erasure: already on the locus, the world its own memory
    (LeastErasure S ↔ Value S) ∧ ((∀ q, Rec S q ↔ S q) ↔ Value S) ∧
    -- the act
    (∀ Z : ActualZeros, Value Z.zeros ∧ ∀ q, Rec Z.zeros q ↔ Z.zeros q) ∧
    -- the guard
    (¬ ∀ T : Config, Closed T → R → Value T) ∧
    ((∀ q, Rec (registered offPoint) q ↔ registered offPoint q) ∧
      ¬ (∀ q, Rec (pairWorld offPoint) q ↔ pairWorld offPoint q) ∧
      SameRecord (registered offPoint) (pairWorld offPoint)) :=
  ⟨seated_undeniable G,
   reg_lands, rfl, by decide,
   the_locus_rests_both,
   the_remembered_is_lossless S, remembering_is_stable S,
   (one_bit_gap_connected S).1,
   twins_on_every_world_local S, offered_is_remembered_iff_value S,
   fun Z => ⟨value_from_the_act Z, at_the_act_offered_is_remembered Z⟩,
   the_floor_alone_decides_no_value G,
   the_gap_is_live⟩
where
  twins_on_every_world_local (S : Config) : LeastErasure S ↔ Value S :=
    ⟨fun le p hp => (reg_erases_nothing_iff p).mp (le p hp),
     fun v p hp => (reg_erases_nothing_iff p).mpr (v p hp)⟩

/-! ## VIb · Zero time remembers -/

/-- ZERO TIME REMEMBERS, EVERY LATER TIME FORGETS: at t = 0 the record
separates the world on the line (d2 = 0)
    from the world off it (d2 = 1); at every t ≥ 1 the two records
    coincide. -/
theorem zero_time_remembers :
    (realAt 0 0 ∧ ¬ realAt 1 0) ∧ ∀ T, 1 ≤ T → (realAt 0 T ∧ realAt 1 T)
    := by
  refine ⟨⟨rfl, fun h => ?_⟩, fun T hT => ⟨?_, ?_⟩⟩
  · unfold realAt flow at h; omega
  · unfold realAt flow; omega
  · unfold realAt flow; omega

/-- LEAST TIME IS LEAST FORGETTING: the width never grows, and once
forgotten the separation never returns. -/
theorem forgetting_is_monotone (d2 t : Nat) : flow d2 (t + 1) ≤ flow d2 t
∧ (realAt d2 t → realAt d2 (t + 1)) := by
  unfold realAt flow; constructor <;> omega

/-! ## VII · Two doors, no third -/

/-- Two doors and no third: every configuration has the value or holds a
member off the line. -/
theorem two_doors (S : Config) : Value S ∨ ∃ p, S p ∧ ¬ onLine p := by
  by_cases h : ∃ p, S p ∧ ¬ onLine p
  · exact Or.inr h
  · exact Or.inl (fun p hp => Classical.byContradiction (fun hn => h ⟨p,
  hp, hn⟩))

/-- Never both: the doors exclude each other. -/
theorem doors_exclusive (S : Config) : ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine
p) :=
  fun ⟨v, p, hp, hn⟩ => hn (v p hp)

/-- Both doors open on closed configurations of one record: the true
world through the first, the false world
    through the second. -/
theorem both_doors_open :
    (Closed (registered offPoint) ∧ Value (registered offPoint)) ∧
    (Closed (pairWorld offPoint) ∧ ∃ p, pairWorld offPoint p ∧ ¬ onLine
    p) ∧
    SameRecord (registered offPoint) (pairWorld offPoint) :=
  ⟨⟨fun s hs => by rw [hs]; rfl, registered_has_value offPoint⟩,
   ⟨pair_world_closed offPoint, ⟨offPoint, Or.inl rfl, off_point_is_off⟩⟩,
   one_record offPoint⟩
```

```
/-- On the act, one door: the second is shut. -/
theorem on_the_act_one_door (Z : ActualZeros) : ¬ ∃ p, Z.zeros p ∧ ¬ onLine p :=
  fun (p, hp, hn) => hn (value_from_the_act Z p hp)

/-! ## VIII · A band forces nothing -/

/-- The band of width k: every member within offset k of the line. On the chart of the eighth grain,
    d = 8·Re s − 4, a zero-free half-plane Re s > 7/8 with the fold reads as the band of width 3. -/
def Band (k : Nat) (S : Config) : Prop := ∀ p, S p → p.1.natAbs ≤ k

/-- The band is weaker than the value: every world with the value satisfies every band. -/
theorem value_gives_band (k : Nat) (S : Config) (h : Value S) : Band k S := by
  intro p hp
  have d0 : p.1 = 0 := h p hp
  rw [d0]
  exact Nat.zero_le k

/-- Both worlds of one record satisfy the band of width 3. -/
theorem band_on_both_worlds : Band 3 (registered offPoint) ∧ Band 3 (pairWorld offPoint) := by
  constructor
  · intro p hp
    rw [hp]
    show ((0 : Int)).natAbs ≤ 3
    decide
  · intro p hp
    cases hp with
    | inl e => rw [e]; show ((1 : Int)).natAbs ≤ 3; decide
    | inr e => rw [e]; show ((-(1 : Int))).natAbs ≤ 3; decide

/-- A BAND FORCES NOTHING: the pair world at offset one holds a member off the line and satisfies the band. -/
theorem band_does_not_force : ¬ ∀ S : Config, Band 3 S → Value S :=
  fun h => pair_world_lacks_value offPoint off_point_is_off (h _ band_on_both_worlds.2)

/-- Only the band of width zero is the value. -/
theorem band_zero_is_value (S : Config) : Band 0 S ↔ Value S := by
  constructor
  · intro h p hp
    exact Int.natAbs_eq_zero.mp (Nat.le_zero.mp (h p hp))
  · exact value_gives_band 0 S

/-! ## IX · The harvest, bound -/

/-- THE HARVEST, BOUND. The Tongue always has a world, and only the content of its no has none; one self, one
    place at a time; the twins hold on every world; the forgotten and the remembered share one record, the remembered
    lossless and stable, the gap one bit; return is free and resistance pays; forgetting lands on the locus, where
    both maps rest; zero time remembers and every later time forgets; the doors exclude each other and the act shuts
    the second; a band forces nothing; and the guard: the root alone places no world on the line. -/
theorem the_harvest {R : Prop} (G : SelfGrounding R) :
    ((∀ _ : Config, R) ∧ (R ∧ Closed (pairWorld offPoint) ∧ ¬ Value (pairWorld offPoint)) ∧ ¬ ∃ _ : G.Act, ¬ R) ∧
    (¬ ∀ S : Config, Closed S → (R ↔ LeastErasure S)) ∧
    (∀ S : Config, LeastErasure S ↔ Value S) ∧
    (∀ S : Config, SameRecord S (Rec S) ∧ Value (Rec S) ∧ (∀ q, Rec (Rec S) q ↔ Rec S q) ∧
      ((∀ q, Rec S q ↔ S q) ↔ Value S)) ∧
    (∀ (F : FinCfg) (T : Nat), 0 < T →
      (Value F.pts → price T (erased F) = 0) ∧ (¬ Value F.pts → T ≤ price T (erased F))) ∧
    ((∀ p, onLine (reg p)) ∧ (∀ p, onLine p → fold p = p ∧ reg p = p)) ∧
    ((realAt 0 0 ∧ ¬ realAt 1 0) ∧ ∀ T, 1 ≤ T → (realAt 0 T ∧ realAt 1 T)) ∧
    ((∀ S : Config, ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine p)) ∧ (∀ Z : ActualZeros, ¬ ∃ p, Z.zeros p ∧ ¬ onLine p)) ∧
    (¬ ∀ S : Config, Band 3 S → Value S) ∧
    (¬ ∀ S : Config, Closed S → R → Value S) :=
  ⟨the_tongue_always_has_a_world G,
   floor_is_not_least_erasure G,
   twins_on_every_world,
   one_bit_gap_connected,
   return_is_free_resistance_pays,
   ⟨reg_lands, the_locus_rests_both⟩,
   zero_time_remembers,
   ⟨doors_exclusive, on_the_act_one_door⟩,
   band_does_not_force,
   the_floor_alone_decides_no_value G⟩

end RememberedOffered

/-! ## The cones, every one pinned: a compile in which any cone changes fails. -/

/-- info: 'RememberedOffered.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.denial_reenacts_root
/-- info: 'RememberedOffered.denial_instantiates' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.denial_instantiates
/-- info: 'RememberedOffered.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.seated_undeniable
/-- info: 'RememberedOffered.root_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.root_undeniable
/-- info: 'RememberedOffered.neg_neg_free' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.neg_neg_free
/-- info: 'RememberedOffered.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.fold_involutive
/-- info: 'RememberedOffered.reg_lands' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.reg_lands
/-- info: 'RememberedOffered.reg_erases_nothing_iff' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.reg_erases_nothing_iff
/-- info: 'RememberedOffered.pair_world_closed' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.pair_world_closed
/-- info: 'RememberedOffered.one_record' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.one_record
/-- info: 'RememberedOffered.registered_has_value' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.registered_has_value
/-- info: 'RememberedOffered.pair_world_lacks_value' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.pair_world_lacks_value
/-- info: 'RememberedOffered.value_from_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.value_from_the_act
/-- info: 'RememberedOffered.the_deed_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_deed_forces_nothing
/-- info: 'RememberedOffered.one_involution_constructed' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.one_involution_constructed
/-- info: 'RememberedOffered.timeless_equals_timed' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.timeless_equals_timed
/-- info: 'RememberedOffered.monism_closes' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.monism_closes
/-- info: 'RememberedOffered.monism_witness_is_the_claim' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.monism_witness_is_the_claim
/-- info: 'RememberedOffered.off_point_is_off' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.off_point_is_off
/-- info: 'RememberedOffered.every_no_reenacts_the_root' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.every_no_reenacts_the_root
/-- info: 'RememberedOffered.the_tongue_no_refutes_itself' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_tongue_no_refutes_itself
/-- info: 'RememberedOffered.only_the_yes_remains_at_the_root' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.only_the_yes_remains_at_the_root
/-- info: 'RememberedOffered.the_root_read_twice_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_root_read_twice_is_the_value
/-- info: 'RememberedOffered.the_line_no_contradicts_the_root_read_twice' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_line_no_contradicts_the_root_read_twice
/-- info: 'RememberedOffered.the_true_world_carries_monism' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_true_world_carries_monism
/-- info: 'RememberedOffered.the_false_world_carries_no_monism' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_false_world_carries_no_monism
/-- info: 'RememberedOffered.the_false_world_carries_the_floor_and_no_monism' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_false_world_carries_the_floor_and_no_monism
/-- info: 'RememberedOffered.the_floor_alone_decides_no_value' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_floor_alone_decides_no_value
/-- info: 'RememberedOffered.the_act_is_the_return_to_the_root_read_twice' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_act_is_the_return_to_the_root_read_twice
/-- info: 'RememberedOffered.an_act_exactly_where_the_root_read_twice_stands' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.an_act_exactly_where_the_root_read_twice_stands
/-- info: 'RememberedOffered.on_the_act_the_line_no_is_refuted' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.on_the_act_the_line_no_is_refuted
/-- info: 'RememberedOffered.two_denials' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.two_denials
/-- info: 'RememberedOffered.two_denials_at_RA' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.two_denials_at_RA
/-- info: 'RememberedOffered.fincfg_closed' does not depend on any axioms -/
```

```
#guard_msgs in #print axioms RememberedOffered.fincfg_closed
/-- info: 'RememberedOffered.least_erasure_iff_zero_erased' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.least_erasure_iff_zero_erased
/-- info: 'RememberedOffered.price_zero_iff_value' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.price_zero_iff_value
/-- info: 'RememberedOffered.price_counts_pairs' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.price_counts_pairs
/-- info: 'RememberedOffered.flow_monotone' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.flow_monotone
/-- info: 'RememberedOffered.reality_transported' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.reality_transported
/-- info: 'RememberedOffered.real_at_Lam' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.real_at_Lam
/-- info: 'RememberedOffered.rh_iff_lambda_zero' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.rh_iff_lambda_zero
/-- info: 'RememberedOffered.flowed_record_forgets' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.flowed_record_forgets
/-- info: 'RememberedOffered.upstream_is_not_forced' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.upstream_is_not_forced
/-- info: 'RememberedOffered.the_tongue_always_has_a_world' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.the_tongue_always_has_a_world
/-- info: 'RememberedOffered.monism_before_the_tongue' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.monism_before_the_tongue
/-- info: 'RememberedOffered.tongue_no_has_no_world' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.tongue_no_has_no_world
/-- info: 'RememberedOffered.line_no_has_a_world' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.line_no_has_a_world
/-- info: 'RememberedOffered.the_deed_is_not_the_supply' does not depend
on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_deed_is_not_the_supply
/-- info: 'RememberedOffered.at_RA' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.at_RA
/-- info: 'RememberedOffered.floor_is_not_least_erasure' does not depend
on any axioms -/
#guard_msgs in #print axioms RememberedOffered.floor_is_not_least_erasure
/-- info: 'RememberedOffered.the_voucher_stays_with_the_floor' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.the_voucher_stays_with_the_floor
/-- info: 'RememberedOffered.merging_at_a_world_is_the_supply' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.merging_at_a_world_is_the_supply
/-- info: 'RememberedOffered.two_instruments' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.two_instruments
/-- info: 'RememberedOffered.twins_on_every_world' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.twins_on_every_world
/-- info: 'RememberedOffered.twins_are_one_extension' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.twins_are_one_extension
/-- info: 'RememberedOffered.no_twin_is_the_others_voucher' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.no_twin_is_the_others_voucher
/-- info: 'RememberedOffered.no_instrument_remembers_and_keeps_the_side'
does not depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.no_instrument_remembers_and_keeps_the_side
/-- info: 'RememberedOffered.the_remembered_is_lossless' does not depend
on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_remembered_is_lossless
/-- info: 'RememberedOffered.offered_is_remembered_iff_value' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.offered_is_remembered_iff_value
/-- info: 'RememberedOffered.at_the_act_offered_is_remembered' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.at_the_act_offered_is_remembered
/-- info: 'RememberedOffered.the_false_world_is_not_its_record' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.the_false_world_is_not_its_record
/-- info: 'RememberedOffered.remembering_is_stable' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.remembering_is_stable
/-- info: 'RememberedOffered.one_bit_gap_connected' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.one_bit_gap_connected
/-- info: 'RememberedOffered.the_gap_is_live' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.the_gap_is_live
/-- info: 'RememberedOffered.return_is_free_resistance_pays' does not
depend on any axioms -/
#guard_msgs in #print axioms
RememberedOffered.return_is_free_resistance_pays
/-- info: 'RememberedOffered.resistance_is_possible' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.resistance_is_possible
/-- info: 'RememberedOffered.the_free_path_is_unique' does not depend on
any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_free_path_is_unique
/-- info: 'RememberedOffered.the_locus_rests_both' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.the_locus_rests_both
/-- info: 'RememberedOffered.the_chain' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_chain
/-- info: 'RememberedOffered.zero_time_remembers' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.zero_time_remembers
/-- info: 'RememberedOffered.forgetting_is_monotone' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.forgetting_is_monotone
/-- info: 'RememberedOffered.two_doors' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.two_doors
/-- info: 'RememberedOffered.doors_exclusive' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.doors_exclusive
/-- info: 'RememberedOffered.both_doors_open' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.both_doors_open
/-- info: 'RememberedOffered.on_the_act_one_door' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.on_the_act_one_door
/-- info: 'RememberedOffered.value_gives_band' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.value_gives_band
/-- info: 'RememberedOffered.band_on_both_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.band_on_both_worlds
/-- info: 'RememberedOffered.band_does_not_force' does not depend on any
axioms -/
#guard_msgs in #print axioms RememberedOffered.band_does_not_force
/-- info: 'RememberedOffered.band_zero_is_value' depends on axioms:
[propext] -/
#guard_msgs in #print axioms RememberedOffered.band_zero_is_value
/-- info: 'RememberedOffered.the_harvest' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_harvest

#print axioms RememberedOffered.the_chain
#print axioms RememberedOffered.the_harvest

namespace RememberedOffered

/-! # BOOK THREE · the annihilation closure -/

/-! ## I · Four names, one proposition -/

/-- CO-LOCATION IS THE VALUE: Being and Monism stand together on a
configuration exactly where every point is on
the line. -/
theorem colocation_is_the_value {R : Prop} (G : SelfGrounding R) (S :
Config) :
    (R ∧ RootReadTwice S) ↔ Value S :=
  (fun ⟨_, m⟩ => (the_root_read_twice_is_the_value S).mp m,
   fun v => (seated_undeniable G, (the_root_read_twice_is_the_value
   S).mpr v))

/-- FOUR NAMES, ONE PROPOSITION: on every configuration least erasure,
the value, Monism and the co-location of
Being with Monism are one proposition. -/
theorem four_names_one_proposition {R : Prop} (G : SelfGrounding R) (S :
Config) :
    (LeastErasure S ↔ Value S) ∧ (RootReadTwice S ↔ Value S) ∧ ((R ∧
    RootReadTwice S) ↔ Value S) :=
  ⟨twins_on_every_world S, the_root_read_twice_is_the_value S,
  colocation_is_the_value G S⟩

/-! ## II · Monism, the sole witness -/

/-- MONISM IS THE SOLE WITNESS. Every faithful witness of the value is
the root read twice, as one proposition; no
reading of the record witnesses, since the true configuration and the
off-line pair configuration leave one
record; the floor witnesses nothing, standing on both; and Monism stands
on the true configuration and not on the
pair, on that one record. -/
theorem monism_is_the_sole_witness {R : Prop} (G : SelfGrounding R) :
    (∀ W : Config → Prop, (∀ S, W S ↔ Value S) → W = RootReadTwice) ∧
    (¬ ∃ h : Config → Prop, ∀ S, h (Rec S) ↔ Value S) ∧
    (¬ ∀ S : Config, Closed S → R → Value S) ∧
    (SameRecord (registered offPoint) (pairWorld offPoint) ∧
      RootReadTwice (registered offPoint) ∧ ¬ RootReadTwice (pairWorld
      offPoint)) :=
  ⟨fun _ hW => funext fun S => propext ((hW S).trans
  (the_root_read_twice_is_the_value S).symm),
   fun ⟨_, hh⟩ =>
     have e : Rec (registered offPoint) = Rec (pairWorld offPoint) :=
       funext fun q => propext (one_record offPoint q)
     pair_world_lacks_value offPoint off_point_is_off
       ((hh _).mp (e ▸ (hh _).mpr (registered_has_value offPoint))),
   the_floor_alone_decides_no_value G,
   ⟨one_record offPoint, the_true_world_carries_monism,
   the_false_world_carries_no_monism⟩⟩

/-- A WITNESS IS NEVER UNIVERSAL: whatever holds on every configuration
is not a witness of the value, since the
```

```
off-line pair configuration would carry it. Universality and witnessing exclude each other. -/
theorem a_witness_is_never_universal (W : Config → Prop) (hu : ∀ S, W S) : ¬ ∀ S, W S ↔ Value S :=
  fun hw => pair_world_lacks_value offPoint off_point_is_off ((hw (pairWorld offPoint)).mp (hu _))

/-- So Monism is not universal over configurations: the root read twice fails on the off-line pair configuration. -/
theorem monism_is_not_universal : ¬ ∀ S : Config, RootReadTwice S :=
  fun hu => the_false_world_carries_no_monism (hu _)

/-! ## III · Co-location is not universal -/

/-- Co-location over every configuration is refuted: Being stands on the off-line pair configuration and Monism
does not. -/
theorem colocation_is_not_universal {R : Prop} (_G : SelfGrounding R) :
    ¬ ∀ S : Config, R ∧ RootReadTwice S :=
  fun h => the_false_world_carries_no_monism (h (pairWorld offPoint)).2

/-! ## IV · The timeless zone -/

/-- THE TIMELESS ZONE. Least erasure and the value are one proposition; at the locus the two instruments coincide;
at zero time the record tells the true configuration from the pair, at every later time it does not; the closure at zero
time is Λ = 0; and the passage back to zero time is not forced by the forward arrow. -/
theorem the_timeless_zone :
    LeastErasure = Value ∧
    (∀ p, onLine p → fold p = p ∧ reg p = p) ∧
    ((realAt 0 0 ∧ ¬ realAt 1 0) ∧ ∀ T, 1 ≤ T → (realAt 0 T ∧ realAt 1 T)) ∧
    (∀ d2, realAt d2 0 ↔ Lam d2 = 0) ∧
    (∀ T, 1 ≤ T → realAt 0 T ∧ realAt 1 T ∧ realAt 0 0 ∧ ¬ realAt 1 0) :=
  (twins_are_one_extension, the_locus_rests_both, zero_time_remembers, rh_iff_lambda_zero,
   upstream_is_not_forced)

/-! ## V · Timeless is determinacy, not selection -/

/-- TIMELESS IS DETERMINACY, NOT SELECTION. At zero time the record's freedom is gone: it separates the true world
from the pair. Every configuration stands behind at most one door. And both doors stay open on closed
configurations: the off-line pair configuration is closed, lacks the value, and has the floor standing on it.
Removing the freedom fixes each configuration as what it is; it does not single out ζ. -/
theorem timeless_is_determinacy_not_selection {R : Prop} (G : SelfGrounding R) :
    (realAt 0 0 ∧ ¬ realAt 1 0) ∧
    (∀ S : Config, ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine p)) ∧
    (Closed (registered offPoint) ∧ Value (registered offPoint)) ∧
    (Closed (pairWorld offPoint) ∧ R ∧ ¬ Value (pairWorld offPoint)) ∧
    ¬ (∀ S : Config, Closed S → Value S) :=
  (zero_time_remembers.1, doors_exclusive, both_doors_open.1,
   (pair_world_closed offPoint, seated_undeniable G,
    pair_world_lacks_value offPoint off_point_is_off),
   fun h => pair_world_lacks_value offPoint off_point_is_off (h _
    (pair_world_closed offPoint)))

/-! ## VI · The check -/

/-- THE CHECK. The off-line pair configuration satisfies every principle of the timeless closure and lacks the value: it is
closed; the floor stands on it; least erasure, the value and Monism are one proposition on it; corrected monism
holds, it carries no witness; it leaves the true world's record; at zero time the width-one world is determinately
not real, with Λ = 1. Its twin, the true world, satisfies the same principles and has the value. -/
theorem the_check {R : Prop} (G : SelfGrounding R) :
    (∃ Z : Config,
      Closed Z ∧ R ∧ (LeastErasure Z ↔ Value Z) ∧ (RootReadTwice Z ↔ Value Z) ∧
      ¬ RootReadTwice Z ∧ SameRecord (registered offPoint) Z ∧ ¬ Value Z) ∧
    (∃ T : Config,
      Closed T ∧ R ∧ (LeastErasure T ↔ Value T) ∧ (RootReadTwice T ↔ Value T) ∧
      RootReadTwice T ∧ SameRecord T (pairWorld offPoint) ∧ Value T) ∧
    (¬ realAt 1 0 ∧ Lam 1 ≠ 0 ∧ realAt 0 0 ∧ Lam 0 = 0) :=
  ((pairWorld offPoint, pair_world_closed offPoint, seated_undeniable G,
    twins_on_every_world _, the_root_read_twice_is_the_value _,
    the_false_world_carries_no_monism, one_record offPoint,
    pair_world_lacks_value offPoint off_point_is_off),
   (registered offPoint, both_doors_open.1.1, seated_undeniable G,
    twins_on_every_world _, the_root_read_twice_is_the_value _,
    the_true_world_carries_monism, one_record offPoint,
    registered_has_value offPoint),
   (zero_time_remembers.1.2, by decide, zero_time_remembers.1.1, by decide))

/-- NO PRINCIPLE TRUE ON EVERY CLOSED CONFIGURATION DECIDES THE VALUE: whatever holds on all of them holds on the
off-line pair configuration, which lacks the value. -/
theorem no_closed_principle_decides (D : Config → Prop) (hD : ∀ S, Closed S → D S) :
    ¬ ∀ S, Closed S → D S → Value S :=
  fun h => pair_world_lacks_value offPoint off_point_is_off
    (h _ (pair_world_closed offPoint) (hD _ (pair_world_closed offPoint)))

/-! ## VII · The capstone -/

/-- THE TIMELESS ZONE, CLOSED, with no act anywhere in it. Four names are one proposition on every configuration;
Monism is the sole witness; no witness is universal; neither Monism nor co-location is universal; LE = RH; zero time
separates the true configuration from the pair, and closure at zero time is Λ = 0; every configuration stands behind
exactly one door; and nothing true on every closed configuration decides the value. -/
theorem the_timeless_zone_closed {R : Prop} (G : SelfGrounding R) :
    (∀ S : Config, (LeastErasure S ↔ Value S) ∧ (RootReadTwice S ↔ Value S) ∧ ((R ∧ RootReadTwice S) ↔ Value S)) ∧
    (∀ W : Config → Prop, (∀ S, W S ↔ Value S) → W = RootReadTwice) ∧
    (∀ W : Config → Prop, (∀ S, W S) → ¬ ∀ S, W S ↔ Value S) ∧
    (¬ ∀ S : Config, RootReadTwice S) ∧
    (¬ ∀ S : Config, R ∧ RootReadTwice S) ∧
    (LeastErasure = Value ∧ (realAt 0 0 ∧ ¬ realAt 1 0) ∧ (∀ d2, realAt d2 0 ↔ Lam d2 = 0)) ∧
    (∀ S : Config, (Value S ∨ ∃ p, S p ∧ ¬ onLine p) ∧ ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine p)) ∧
    (∀ D : Config → Prop, (∀ S, Closed S → D S) → ¬ ∀ S, Closed S → D S → Value S) :=
  (four_names_one_proposition G,
   (monism_is_the_sole_witness G).1,
   a_witness_is_never_universal,
   monism_is_not_universal,
   colocation_is_not_universal G,
   (twins_are_one_extension, zero_time_remembers.1, rh_iff_lambda_zero),
   fun S => (two_doors S, doors_exclusive S),
   no_closed_principle_decides)

/-- THE ANNIHILATION CLOSURE. Four names are one proposition on every configuration; Monism is the sole witness and
no witness is universal; co-location is not universal; the timeless zone holds; timelessness is determinacy and
not selection; the check stands; no principle shared by every closed configuration decides the value; and on any
term of the act the value holds at its zeros, while the run leaves the value free. -/
theorem the_annihilation_closure {R : Prop} (G : SelfGrounding R) :
    (∀ S : Config, (LeastErasure S ↔ Value S) ∧ (RootReadTwice S ↔ Value S) ∧ ((R ∧ RootReadTwice S) ↔ Value S)) ∧
    (∀ W : Config → Prop, (∀ S, W S ↔ Value S) → W = RootReadTwice) ∧
    (¬ ∀ S : Config, RootReadTwice S) ∧
    (¬ ∀ S : Config, R ∧ RootReadTwice S) ∧
    (LeastErasure = Value ∧ (realAt 0 0 ∧ ¬ realAt 1 0) ∧ (∀ d2, realAt d2 0 ↔ Lam d2 = 0)) ∧
    (∃ Z : Config, Closed Z ∧ R ∧ ¬ RootReadTwice Z ∧ SameRecord (registered offPoint) Z ∧ ¬ Value Z) ∧
    (∀ D : Config → Prop, (∀ S, Closed S → D S) → ¬ ∀ S, Closed S → D S → Value S) ∧
    (∀ Z : ActualZeros, Value Z.zeros) :=
  (four_names_one_proposition G,
   (monism_is_the_sole_witness G).1,
   monism_is_not_universal,
   colocation_is_not_universal G,
   (twins_are_one_extension, zero_time_remembers.1, rh_iff_lambda_zero),
   (pairWorld offPoint, pair_world_closed offPoint, seated_undeniable G,
    the_false_world_carries_no_monism, one_record offPoint,
    pair_world_lacks_value offPoint off_point_is_off),
   no_closed_principle_decides,
   value_from_the_act)

end RememberedOffered

/-
RH_Unicorn_Block.lean · the Unicorn part of the Riemann Hypothesis, closed to derivation, its
aperture one bit wide, its supply side silent. Core Lean 4.19.0, no library, no sorry, no user-declared axiom. 2026-09-25.

Part III divided the hypothesis at a height T into a Real part and a Unicorn part and proved
the Real part by certificate. Parts I and II proved the wall, the crossing, the bound, and the
cure theorem, and did not say what they say of the Unicorn part. This file says it: closed to
derivation, the aperture one bit wide, the register silent beyond it. ΔM = 0.
-/
namespace RHUnicorn

variable {α β : Type}

/-- The line property, the Real part at T, the Unicorn part at T, as in Part III. -/
def RH (Z onL : α → Prop) : Prop := ∀ z, Z z → onL z
def RealPartAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z ≤ T → onL z
def UnicornAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z > T → onL z

/-- THE FORMAL BLOCK OF THE UNICORN PART. Let τ act on the zeros, let ρ be a record even at a
    seat z above the height, and let d decide the line property, odd at z. Then no reading g of
    the record agrees with d even on the region above the height alone. The Unicorn part is
    closed to every even register; the closure is a theorem and does not expire. -/
theorem unicorn_block (height : α → Nat) (T : Nat) (τ : α → α) (ρ : α → β) (d : α → Bool)
```

```
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hp : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    ¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w := by
  intro (g, hg)
  have h1 := hg z habove
  have h2 := hg (τ z) habove'
  rw [hp, hd, h1] at h2
  revert h2
  cases d z <;> intro h2 <;> exact Bool.noConfusion h2

/-- THE APERTURE, ONE BIT WIDE. At a seat where d is odd, one odd bit s
at the seat would decide d
    on the seat and its partner through a calibration that exists and is
    unique. The width of
    the aperture, a theorem; it says nothing of whether anything passes
    through it. -/
theorem aperture_one_bit_wide (τ : α → α) (s d : α → Bool) (z : α)
    (hs : s (τ z) = !s z) (hd : d (τ z) = !d z) :
    ∃ c : Bool, (d z = xor (s z) c ∧ d (τ z) = xor (s (τ z)) c) ∧
      ∀ c' : Bool, (d z = xor (s z) c' ∧ d (τ z) = xor (s (τ z)) c') → c'
      = c :=
  (xor (d z) (s z),
    (by cases s z <;> cases d z <;> rfl,
     by rw [hs, hd]; cases s z <;> cases d z <;> rfl),
    by
      intro c' hc
      obtain (h1, -) := hc
      generalize hsz : s z = sv
      generalize hdz : d z = dv
      rw [hsz, hdz] at h1
      cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd
      h1 (by decide))

/-- EXISTENCE SUPPLIES NOTHING. Conditioning the Unicorn part on an
inhabited premise, the root
    axiom for one, leaves it exactly where it was; and no class of frames
    on which the premise
    is to decide it decides more than already held on the class. Theorems
    E and I of Part I,
    read at the Unicorn part. -/
theorem unicorn_rule_exact (Z onL : α → Prop) (height : α → Nat) (T :
Nat) (A : Prop) (ha : A) :
    (A ↔ UnicornAt Z onL height T) ↔ UnicornAt Z onL height T :=
  (fun h => h ha, fun hu _ => hu)

theorem unicorn_no_cure {Frame : Type} (A : Prop) (ha : A) (C U : Frame →
Prop) :
    (∀ X, C X → A → U X) ↔ (∀ X, C X → U X) :=
  (fun h X hc => h X hc ha, fun h X hc _ => h X hc)

/-- NO BYPASS. Given the Real part, the fused hypothesis is exactly the
Unicorn part, so a proof
    of the hypothesis contains a proof of the Unicorn part and the block
    is not avoided by
    aiming at the whole. -/
theorem unicorn_no_bypass (Z onL : α → Prop) (height : α → Nat) (T : Nat)
    (hr : RealPartAt Z onL height T) : RH Z onL ↔ UnicornAt Z onL height T
    := by
  constructor
  · intro h z hz _; exact h z hz
  · intro hu z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle

/-- THE SUPPLY SIDE IS SILENCE, AT THE REGISTER. Over one even record,
both orientations of d at
    the seat are equally refused to every reading above the height: the
    register cannot say
    which way the seat lies, so it cannot say what a supply would bring,
    nor that one exists,
    nor that none does. -/
theorem supply_side_silent (height : α → Nat) (T : Nat) (τ : α → α) (ρ : α
→ β) (d : α → Bool)
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hp : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w) ∧
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = !d w) ∧
    (!d z) ≠ d z :=
  (unicorn_block height T τ ρ d z habove habove' hp hd,
   unicorn_block height T τ ρ (fun w => !d w) z habove habove' hp (by
     show (!d (τ z)) = !(!d z)
     rw [hd]),
   by cases d z <;> decide)

/-- The division at the height, on natural heights: the hypothesis is
exactly its Real part and
    its Unicorn part. -/
theorem rh_iff_real_and_unicorn (Z onL : α → Prop) (height : α → Nat) (T :
Nat) :
    RH Z onL ↔ RealPartAt Z onL height T ∧ UnicornAt Z onL height T := by
  constructor
  · intro h; exact ⟨fun z hz _ => h z hz, fun z hz _ => h z hz⟩
  · intro ⟨hr, hu⟩ z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle

/-- A certificate at the height: a complete list of the zeros up to T,
each checked on the line. -/
structure Certificate (Z onL : α → Prop) (height : α → Nat) (T : Nat)
where
  items    : List α
  complete : ∀ z, Z z → height z ≤ T → z ∈ items
  checked  : ∀ z, z ∈ items → onL z

theorem real_part_of_certificate (Z onL : α → Prop) (height : α → Nat) (T
: Nat)
    (c : Certificate Z onL height T) : RealPartAt Z onL height T :=
  fun z hz hle => c.checked z (c.complete z hz hle)

/-- THE THREE BITS AS ONE TERM · THE REGISTER'S PROOF OF THE HYPOTHESIS,
COMPLETE. Under a
    certificate at T and the seat data above T, the hypothesis divides
    exactly (bit one), its
    Real part at T holds (bit two), every reading of the even register is
    refused above T (bit
    three, refused), and one supplied bit calibrates the seat uniquely
    (the door). The hypothesis
    is not thereby proved; nothing derivable about it is left underived.
    -/
theorem rh_register_proof_complete (Z onL : α → Prop) (height : α → Nat)
(T : Nat)
    (c : Certificate Z onL height T) (τ : α → α) (ρ : α → β) (d s : α →
    Bool) (z : α)
    (habove : height z > T) (habove' : height (τ z) > T) (hp : ρ (τ z) = ρ
    z)
    (hd : d (τ z) = !d z) (hs : s (τ z) = !s z) :
    (RH Z onL ↔ RealPartAt Z onL height T ∧ UnicornAt Z onL height T) ∧
    RealPartAt Z onL height T ∧
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w) ∧
    ∃ k : Bool, (d z = xor (s z) k ∧ d (τ z) = xor (s (τ z)) k) ∧
      ∀ k' : Bool, (d z = xor (s z) k' ∧ d (τ z) = xor (s (τ z)) k') → k'
      = k :=
  ⟨rh_iff_real_and_unicorn Z onL height T, real_part_of_certificate Z onL
  height T c,
   unicorn_block height T τ ρ d z habove habove' hp hd,
   aperture_one_bit_wide τ s d z hs hd⟩

end RHUnicorn

#print axioms RHUnicorn.unicorn_block
#print axioms RHUnicorn.aperture_one_bit_wide
#print axioms RHUnicorn.unicorn_rule_exact
#print axioms RHUnicorn.unicorn_no_cure
#print axioms RHUnicorn.unicorn_no_bypass
#print axioms RHUnicorn.supply_side_silent
#print axioms RHUnicorn.rh_iff_real_and_unicorn
#print axioms RHUnicorn.real_part_of_certificate
#print axioms RHUnicorn.rh_register_proof_complete

namespace RememberedOffered

/-! # BOOK FOUR · the Unicorn, contained -/

/-- The height of a point of the chart, as a natural number. -/
def ht (p : Point) : Nat := p.2.toNat

/-- I · THE CUT ON THE CHART: at every height, the value is exactly its
Real part and its Unicorn part. -/
theorem the_cut_on_the_chart (S : Config) (T : Nat) :
    Value S ↔ RHUnicorn.RealPartAt S onLine ht T ∧ RHUnicorn.UnicornAt S
    onLine ht T :=
  RHUnicorn.rh_iff_real_and_unicorn S onLine ht T

/-- II · NO BYPASS ON THE CHART: given the Real part at a height, the
value is exactly the Unicorn part there. -/
theorem no_bypass_on_the_chart (S : Config) (T : Nat) (hr :
RHUnicorn.RealPartAt S onLine ht T) :
    Value S ↔ RHUnicorn.UnicornAt S onLine ht T :=
  RHUnicorn.unicorn_no_bypass S onLine ht T hr

/-- III · THE UNICORN BLOCKED ON THE CHART: above every height, no
reading of the record decides the side of a
point, since the fold pair above the height leaves one record and
opposite sides. -/
theorem unicorn_blocked_on_the_chart (T : Nat) :
    ¬ ∃ g : Point → Bool, ∀ w : Point, ht w > T → g (reg w) = decide (w.1
    > 0) :=
  RHUnicorn.unicorn_block ht T fold reg (fun w => decide (w.1 > 0)) ((1 :
  Int), Int.ofNat (T + 1))
    (show T + 1 > T by omega) (show T + 1 > T by omega) rfl rfl

/-- IV · RA SUPPLIES NOTHING TO THE UNICORN PART: conditioning it on the
root leaves it exactly where it was. -/
theorem ra_supplies_nothing_to_the_unicorn (S : Config) (T : Nat) :
    (RA → RHUnicorn.UnicornAt S onLine ht T) ↔ RHUnicorn.UnicornAt S
    onLine ht T :=
  RHUnicorn.unicorn_rule_exact S onLine ht T RA (seated_undeniable
  raSelfGrounding)

/-- V · ON THE ACT'S ZERO SET THE UNICORN IS CLOSED: every arrival above
every height, in the act's zero set, stands
on the line. -/
theorem the_unicorn_closed_on_the_act (Z : ActualZeros) (T : Nat) :
RHUnicorn.UnicornAt Z.zeros onLine ht T :=
  fun p hp _ => value_from_the_act Z p hp

/-- On the act's zero set, no arrival stands off the line. -/
theorem no_arrival_off_the_line_on_the_act (Z : ActualZeros) : ¬ ∃ p,
Z.zeros p ∧ ¬ onLine p :=
  fun (p, hp, hoff) => hoff (value_from_the_act Z p hp)

/-- VI · THE UNICORN, CONTAINED. At every height the value divides into
its Real and Unicorn parts; given the Real
part the value is exactly the Unicorn part; above every height no reading
of the record decides the side; RA
```

```
supplies nothing to the Unicorn part; and on the act's zero set every arrival above every height stands on the line,
with no arrival off it. -/
theorem the_unicorn_contained :
    (∀ (S : Config) (T : Nat),
      Value S ↔ RHUnicorn.RealPartAt S onLine ht T ∧ RHUnicorn.UnicornAt S onLine ht T) ∧
    (∀ (S : Config) (T : Nat),
      RHUnicorn.RealPartAt S onLine ht T → (Value S ↔ RHUnicorn.UnicornAt S onLine ht T)) ∧
    (∀ T : Nat, ¬ ∃ g : Point → Bool, ∀ w : Point, ht w > T → g (reg w) = decide (w.1 > 0)) ∧
    (∀ (S : Config) (T : Nat),
      (RA → RHUnicorn.UnicornAt S onLine ht T) ↔ RHUnicorn.UnicornAt S onLine ht T) ∧
    (∀ (Z : ActualZeros) (T : Nat), RHUnicorn.UnicornAt Z.zeros onLine ht T) ∧
    (∀ Z : ActualZeros, ¬ ∃ p, Z.zeros p ∧ ¬ onLine p) :=
  ⟨the_cut_on_the_chart, no_bypass_on_the_chart,
   unicorn_blocked_on_the_chart,
    ra_supplies_nothing_to_the_unicorn, the_unicorn_closed_on_the_act,
    no_arrival_off_the_line_on_the_act⟩

end RememberedOffered

/-! ## Receipts of Book Three, pinned -/
/-- info: 'RememberedOffered.colocation_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.colocation_is_the_value
/-- info: 'RememberedOffered.four_names_one_proposition' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.four_names_one_proposition
/-- info: 'RememberedOffered.monism_is_the_sole_witness' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.monism_is_the_sole_witness
/-- info: 'RememberedOffered.a_witness_is_never_universal' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.a_witness_is_never_universal
/-- info: 'RememberedOffered.monism_is_not_universal' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.monism_is_not_universal
/-- info: 'RememberedOffered.colocation_is_not_universal' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.colocation_is_not_universal
/-- info: 'RememberedOffered.the_timeless_zone' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_timeless_zone
/-- info: 'RememberedOffered.timeless_is_determinacy_not_selection' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.timeless_is_determinacy_not_selection
/-- info: 'RememberedOffered.the_check' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_check
/-- info: 'RememberedOffered.no_closed_principle_decides' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.no_closed_principle_decides
/-- info: 'RememberedOffered.the_timeless_zone_closed' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_timeless_zone_closed
/-- info: 'RememberedOffered.the_annihilation_closure' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_annihilation_closure

#print axioms RememberedOffered.the_annihilation_closure

/-! ## Receipts of Book Four, pinned -/
/-- info: 'RHUnicorn.unicorn_block' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.unicorn_block
/-- info: 'RHUnicorn.aperture_one_bit_wide' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.aperture_one_bit_wide
/-- info: 'RHUnicorn.unicorn_rule_exact' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.unicorn_rule_exact
/-- info: 'RHUnicorn.unicorn_no_cure' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.unicorn_no_cure
/-- info: 'RHUnicorn.unicorn_no_bypass' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.unicorn_no_bypass
/-- info: 'RHUnicorn.supply_side_silent' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.supply_side_silent
/-- info: 'RHUnicorn.rh_iff_real_and_unicorn' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.rh_iff_real_and_unicorn
/-- info: 'RHUnicorn.real_part_of_certificate' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.real_part_of_certificate
/-- info: 'RHUnicorn.rh_register_proof_complete' does not depend on any axioms -/
#guard_msgs in #print axioms RHUnicorn.rh_register_proof_complete
/-- info: 'RememberedOffered.the_cut_on_the_chart' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_cut_on_the_chart
/-- info: 'RememberedOffered.no_bypass_on_the_chart' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.no_bypass_on_the_chart
/-- info: 'RememberedOffered.ra_supplies_nothing_to_the_unicorn' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.ra_supplies_nothing_to_the_unicorn
/-- info: 'RememberedOffered.the_unicorn_closed_on_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_unicorn_closed_on_the_act
/-- info: 'RememberedOffered.no_arrival_off_the_line_on_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.no_arrival_off_the_line_on_the_act
/-- info: 'RememberedOffered.unicorn_blocked_on_the_chart' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.unicorn_blocked_on_the_chart
/-- info: 'RememberedOffered.the_unicorn_contained' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_unicorn_contained

#print axioms RememberedOffered.the_unicorn_contained

/-! # THE ONE · appended to Annihilation_Closure.lean (sha256 cb474a41…), nothing above changed.
Every name and every reading the closure carries becomes one object, by equality and not by twin biconditionals;
the act's zero set becomes one notion with the closed configuration that has the value; the floor keeps its own
place; and the one sentence the closure leaves is printed as the closure's own last conjunct. -/
namespace RememberedOffered
theorem the_one {R : Prop} (G : SelfGrounding R) :
    (LeastErasure = Value ∧ RootReadTwice = Value ∧ (fun S => R ∧ RootReadTwice S) = Value) ∧
    (∀ S : Config, (∃ Z : ActualZeros, Z.zeros = S) ↔ (Closed S ∧ Value S)) ∧
    (¬ ∀ S : Config, Closed S → (R ↔ LeastErasure S)) ∧
    (∀ S : Config, (R ↔ LeastErasure S) ↔ LeastErasure S) ∧
    (∀ Z : ActualZeros, Value Z.zeros) ∧
    (∀ D : Config → Prop, (∀ S, Closed S → D S) → ¬ ∀ S, Closed S → D S → Value S) :=
  ⟨(twins_are_one_extension,
    funext fun S => propext (the_root_read_twice_is_the_value S),
    funext fun S => propext (colocation_is_the_value G S)),
   fun S => (an_act_exactly_where_the_root_read_twice_stands S).trans
     (fun ⟨c, m⟩ => ⟨c, (the_root_read_twice_is_the_value S).mp m⟩,
      fun ⟨c, v⟩ => ⟨c, (the_root_read_twice_is_the_value S).mpr v⟩),
   floor_is_not_least_erasure G,
   merging_at_a_world_is_the_supply G,
   value_from_the_act,
   no_closed_principle_decides⟩
end RememberedOffered

/-! # THE LAST ACT · appended to The_One.lean, nothing above changed.
A deed is a term of `G.Act`, the act type of a self-grounding root: every decentralized, impersonal, physical
computation, atomic and irreducible, is a deed. The supply is a term of `ActualZeros`. Tested here: every deed being
the supply at one closed zero set S, ζ's in the reading, and at every closed zero set. -/
namespace RememberedOffered
theorem the_last_act {R : Prop} (G : SelfGrounding R) (S : Config) (hS : Closed S) :
    -- every deed being the supply at S is exactly the value at S
    ((G.Act → ∃ Z : ActualZeros, Z.zeros = S) ↔ Value S) ∧
    -- every deed being the supply at every closed zero set, as a law, is refuted
    (¬ ∀ S' : Config, Closed S' → G.Act → ∃ Z : ActualZeros, Z.zeros = S') ∧
    -- the deed stands in the world without the value too
    (R ∧ Closed (pairWorld offPoint) ∧ ¬ Value (pairWorld offPoint)) ∧
    -- existence supplies nothing to the part above any height
    (∀ T : Nat, (RA → RHUnicorn.UnicornAt S onLine ht T) ↔ RHUnicorn.UnicornAt S onLine ht T) :=
  ⟨(fun h => by obtain ⟨Z, hZ⟩ := h G.anAct; rw [← hZ]; exact value_from_the_act Z,
    fun v _ => ⟨S, hS, fun p hp => (reg_erases_nothing_iff p).mpr (v p hp)⟩, rfl⟩),
   the_deed_is_not_the_supply G,
   ⟨seated_undeniable G, pair_world_closed offPoint,
    pair_world_lacks_value offPoint off_point_is_off⟩,
   fun T => ra_supplies_nothing_to_the_unicorn S T⟩
end RememberedOffered

/-! # LEAST ERASURE AND LEAST ESCAPE · appended to The_Last_Act.lean, nothing above changed.
The escape of a finite configuration is its number of off-line pairs, the bits registration erases. The pair that
denies RH, a zero at offset one with its mirror, is `resisting`. -/
namespace RememberedOffered
/-- The configuration with no zeros at all: zero escape. -/
def calm : FinCfg := ⟨[], [], fun _ hq => by cases hq⟩

theorem least_erasure_and_least_escape :
    -- I · the escaping pair escapes by exactly one bit and pays exactly one floor
    (erased resisting = 1 ∧ ∀ T : Nat, price T (erased resisting) = T) ∧
    -- II · least escape is zero escape, is least erasure, is the value: the minimiser stands on the line
    (∀ F : FinCfg, (∀ F' : FinCfg, erased F ≤ erased F') ↔ Value F.pts) ∧
    -- III · the minimum is attained
    (∃ F : FinCfg, erased F = 0) ∧
    -- IV · the escaping pair is closed and is not a minimiser
    (Closed resisting.pts ∧ ¬ ∀ F' : FinCfg, erased resisting ≤ erased F') ∧
    -- V · nothing true of every closed configuration makes a configuration a minimiser
    (∀ D : Config → Prop, (∀ S, Closed S → D S) →
      ¬ ∀ F : FinCfg, D F.pts → ∀ F' : FinCfg, erased F ≤ erased F') := by
  have least_iff : ∀ F : FinCfg, (∀ F' : FinCfg, erased F ≤ erased F') ↔ Value F.pts := fun F =>
```

```
    (fun h => (twins_on_every_world F.pts).mp
        ((least_erasure_iff_zero_erased F).mpr (Nat.le_zero.mp (h calm))),
     fun v F' => by
        have h0 : erased F = 0 := (least_erasure_iff_zero_erased F).mp
        ((twins_on_every_world F.pts).mpr v)
        rw [h0]; exact Nat.zero_le _)
  have not_value : ¬ Value resisting.pts := (resistance_is_possible 1 (by
    decide)).2.1
  refine ⟨rfl, fun T => Nat.one_mul T), least_iff, ⟨calm, rfl⟩,
    (fincfg_closed resisting, fun h => not_value ((least_iff
    resisting).mp h)), ?_⟩
  intro D hD h
  exact not_value ((least_iff resisting).mp (h resisting (hD _
  (fincfg_closed resisting))))
end RememberedOffered

/-! # THE LOOP RA–RAM–ZEROS · appended to Least_Escape.lean, nothing
above changed.
RA is the floor R of a self-grounding root. RAM is the root read twice,
`RootReadTwice`, the corrected monism witness
with one involution, uniformity and its seed. The loop runs R → 
RootReadTwice S → Value S → R at a zero set S. -/
namespace RememberedOffered
theorem the_loop {R : Prop} (G : SelfGrounding R) (S : Config) :
    -- the corrected monism witness is the claim, seed included
    (RootReadTwice S ↔ Value S) ∧
    -- the loop's forward leg from RA reaches the zeros exactly when the
    value already holds
    ((R → RootReadTwice S) ↔ Value S) ∧
    -- the return leg to RA is free: it holds whatever the zeros do
    (Value S → R) ∧ (¬ Value S → R) ∧
    -- the whole loop closes exactly at the value
    ((R ↔ RootReadTwice S) ↔ Value S) ∧
    -- the loop is sign-blind: it closes for the value and for its denial
    alike
    ((SelfVerifying (Value S) ↔ Value S) ∧ (SelfVerifying (¬ Value S) ↔ ¬
    Value S)) ∧
    -- on one record the loop closes on the true world and stays open on
    the pair: it cannot select
    (SameRecord (registered offPoint) (pairWorld offPoint) ∧ Closed
    (pairWorld offPoint) ∧
      (R ↔ RootReadTwice (registered offPoint)) ∧ ¬ (R ↔ RootReadTwice
      (pairWorld offPoint))) := by
  have r : R := seated_undeniable G
  have m := the_root_read_twice_is_the_value S
  have sv : ∀ P : Prop, SelfVerifying P ↔ P := fun P =>
    (fun h => Classical.byContradiction (fun n => n (h n)), fun p _ => p)
  refine ⟨m, (fun h => m.mp (h r), fun v _ => m.mpr v), fun _ => r, fun _
  => r,
    (fun h => m.mp (h.mp r), fun v => (fun _ => m.mpr v, fun _ => r)),
    ⟨sv _, sv _⟩,
    one_record offPoint, pair_world_closed offPoint,
    (fun _ => the_true_world_carries_monism, fun _ => r),
    fun h => the_false_world_carries_no_monism (h.mp r)⟩
end RememberedOffered

/-! # ROW FOUR FROM THE ZERO SIDE · appended to The_Loop.lean, nothing
above changed.
A computation that ends is the finite list of points it reports as zeros;
it reads a zero set when every point it
reports belongs to it. Row 4 of the RH paper, applied to the hypothetical
off-line pair. -/
namespace RememberedOffered
theorem row_four_from_the_zero_side (S : Config) :
    -- on any act, every computation reading it reports only zeros on the
    line
    (∀ (Z : ActualZeros) (L : List Point), (∀ p ∈ L, Z.zeros p) → ∀ p ∈
    L, onLine p) ∧
    -- so a report holding an off-line zero reads no act: on the act, the
    hypothetical is a phantom
    (∀ L : List Point, (∃ p ∈ L, ¬ onLine p) → ¬ ∃ Z : ActualZeros, ∀ p ∈
    L, Z.zeros p) ∧
    -- yet that report reads a closed configuration, the pair world,
    which carries the floor
    ((∀ p ∈ [offPoint, fold offPoint], pairWorld offPoint p) ∧ Closed
    (pairWorld offPoint)) ∧
    -- at a closed zero set: every report of it lands on the line exactly
    when it has the value
    (Closed S → ((∀ L : List Point, (∀ p ∈ L, S p) → ∀ p ∈ L, onLine p) ↔
    Value S)) := by
  refine ⟨fun Z L hL p hp => value_from_the_act Z p (hL p hp),
    fun L ⟨p, hp, hoff⟩ ⟨Z, hZ⟩ => hoff (value_from_the_act Z p (hZ p
    hp)), ⟨?_, pair_world_closed offPoint⟩, ?_⟩
  · intro p hp
    cases hp with
    | head => exact Or.inl rfl
    | tail _ h =>
      cases h with
      | head => exact Or.inr rfl
      | tail _ h' => cases h'
  · intro _
    constructor
    · intro h p hp
      exact h ⟨p⟩ (fun q hq => by
        cases hq with
        | head => exact hp
        | tail _ h' => cases h') p (List.Mem.head _)
    · intro v L hL p hp
      exact v p (hL p hp)
end RememberedOffered

/-! # BIDIRECTIONAL HALTED UNIDUCTION · appended to Row_Four.lean,
nothing above changed.
ZERO SIDE: a computation reports finitely many zeros; it reads a zero set
when every report belongs to it (Row 4).
PRIME SIDE: through the explicit formula each zero contributes a mode
x^(Re ρ) to ψ(x) − x, so the prime error is
bounded by x^(1/2 + ε) exactly when no zero's offset exceeds the line
(von Koch 1901; Landau, Ingham) [cited]. On the
offset chart: no point stands to the head's side of the line. -/
namespace RememberedOffered
def ZeroReports (S : Config) : Prop := ∀ L : List Point, (∀ p ∈ L, S p) → 
∀ p ∈ L, onLine p
def PrimeBound (S : Config) : Prop := ∀ p, S p → p.1 ≤ 0

theorem prime_side_halts (S : Config) (hS : Closed S) : PrimeBound S ↔ 
Value S :=
  ⟨fun hb p hp => by
     have h1 : p.1 ≤ 0 := hb p hp
     have h2 : -p.1 ≤ 0 := hb (fold p) (hS p hp)
     show p.1 = 0; omega,
   fun v p hp => by
     have h : p.1 = 0 := v p hp
     show p.1 ≤ 0; omega⟩

theorem bidirectional_halted_uniduction (S : Config) (hS : Closed S) :
    -- the zero side halts at the value of the whole zero set
    (ZeroReports S ↔ Value S) ∧
    -- the prime side halts at the value of the whole zero set
    (PrimeBound S ↔ Value S) ∧
    -- so both directions halt at one proposition
    (ZeroReports S ↔ PrimeBound S) ∧
    -- and neither sweep passes it: on one record, both readings hold on
    the true world and fail on the pair world
    (SameRecord (registered offPoint) (pairWorld offPoint) ∧
      (ZeroReports (registered offPoint) ∧ PrimeBound (registered
      offPoint)) ∧
      (¬ ZeroReports (pairWorld offPoint) ∧ ¬ PrimeBound (pairWorld
      offPoint))) ∧
    -- no principle true of every closed configuration supplies either
    side
    (¬ (∀ S' : Config, Closed S' → ZeroReports S') ∧ ¬ (∀ S' : Config,
    Closed S' → PrimeBound S')) ∧
    -- on the act both sides halt at yes
    (∀ Z : ActualZeros, ZeroReports Z.zeros ∧ PrimeBound Z.zeros) := by
  have zr : ∀ T : Config, Closed T → (ZeroReports T ↔ Value T) := fun T
  hT => (row_four_from_the_zero_side T).2.2.2 hT
  have pv : ¬ Value (pairWorld offPoint) := pair_world_lacks_value
  offPoint off_point_is_off
  have rc : Closed (registered offPoint) := fun s hs => by
    rw [hs]; show fold (reg offPoint) = reg offPoint; rfl
  refine ⟨zr S hS, prime_side_halts S hS, ⟨zr S hS⟩.trans
  (prime_side_halts S hS).symm,
    ⟨one_record offPoint,
     ⟨(zr _ rc).mpr (registered_has_value offPoint), ⟨prime_side_halts _
     rc⟩.mpr (registered_has_value offPoint)⟩,
     ⟨fun h => pv ((zr _ (pair_world_closed offPoint)).mp h),
      fun h => pv ((prime_side_halts _ (pair_world_closed offPoint)).mp
      h)⟩⟩,
    ⟨fun h => pv ((zr _ (pair_world_closed offPoint)).mp (h _
    (pair_world_closed offPoint))),
     fun h => pv ((prime_side_halts _ (pair_world_closed offPoint)).mp (h
     _ (pair_world_closed offPoint)))⟩,
    fun Z => ⟨(zr _ Z.closed).mpr (value_from_the_act Z),
    (prime_side_halts _ Z.closed).mpr (value_from_the_act Z)⟩⟩
end RememberedOffered

/-! # THE ATOMIC PROOF · appended to Halted_Uniduction.lean, nothing
above changed.
One statement: the reduction complete, the bit irreducible by theorem,
its form forced by theorem, the bit atomic,
and the closure from it at theorem grade. The bit stands in the statement
as the act's field, visible to every
reader. -/
namespace RememberedOffered
theorem the_atomic_proof {R : Prop} (G : SelfGrounding R) (S : Config)
(hS : Closed S) :
    -- I · REDUCTION, complete: every reading of the hypothesis the seed
    names is the one proposition
    ((ZeroReports S ↔ Value S) ∧ (PrimeBound S ↔ Value S) ∧ (LeastErasure
    S ↔ Value S) ∧
      (RootReadTwice S ↔ Value S)) ∧
    -- II · IRREDUCIBLE, by theorem: nothing shared by every closed
    configuration forces it, the deed included
    ((∀ D : Config → Prop, (∀ S', Closed S' → D S') → ¬ ∀ S', Closed S' →
    D S' → Value S') ∧
      ¬ (∀ S' : Config, Closed S' → R → Value S')) ∧
    -- III · FORCED IN FORM, by theorem: a premise that forces the value
    and follows from it is the value
    (∀ P : Config → Prop, (∀ S', P S' → Value S') → (∀ S', Value S' → P
    S') → ∀ S', P S' ↔ Value S') ∧
    -- IV · THE BIT, atomic: the act stands at this zero set exactly
    where the value holds
    ((∃ Z : ActualZeros, Z.zeros = S) ↔ Value S) ∧
    -- V · CLOSURE from the bit, theorem grade: on the act, the value,
    the zero reports and the prime bound
    (∀ Z : ActualZeros, Value Z.zeros ∧ ZeroReports Z.zeros ∧ PrimeBound
    Z.zeros) :=
  ⟨⟨(row_four_from_the_zero_side S).2.2.2 hS, prime_side_halts S hS,
  twins_on_every_world S,
    the_root_read_twice_is_the_value S⟩,
   ⟨no_closed_principle_decides, the_floor_alone_decides_no_value G⟩,
   fun _ f w S' => ⟨f S', w S'⟩,
   ⟨(the_one G).2.1 S).trans (fun ⟨_, v⟩ => v, fun v => ⟨hS, v⟩),
   fun Z => ⟨value_from_the_act Z, ⟨bidirectional_halted_uniduction S
   hS).2.2.2.2.2 Z⟩⟩
```

```
end RememberedOffered

/-! # THE ATOMIC NECESSITY · Row 9 · appended to the atomic proof, nothing above changed.
The hypothesis is one dot: one proposition, read the same from every side and in every vocabulary. A dot has no
other dot beside it to be read against, and no room inside it for a second kind of proof. -/
namespace RememberedOffered
theorem the_atomic_necessity (S : Config) (hS : Closed S) :
    -- I · every proof of the value at S, from any premise whatever, is a supply of the act's field at S
    (∀ Q : Prop, (Q → Value S) → Q → ∃ Z : ActualZeros, Z.zeros = S) ∧
    -- II · the dot: any two proofs of the value are one proof
    (∀ h₁ h₂ : Value S, h₁ = h₂) ∧
    -- III · the supply at S is unique: two supplies with the same zero set are one supply
    (∀ Z₁ Z₂ : ActualZeros, Z₁.zeros = S → Z₂.zeros = S → Z₁ = Z₂) ∧
    -- IV · every vocabulary and both sides name the same dot
    (LeastErasure = Value ∧ RootReadTwice = Value ∧ (ZeroReports S ↔ Value S) ∧ (PrimeBound S ↔ Value S)) ∧
    -- V · the supply exists at S exactly where the dot holds
    ((∃ Z : ActualZeros, Z.zeros = S) ↔ Value S) ∧
    -- VI · no room for a proof from what every closed configuration shares
    ¬ (∀ S' : Config, Closed S' → Value S') := by
  refine (fun Q hQ q => ((S, hS, (twins_on_every_world S).mpr (hQ q)), rfl),
    fun _ _ => rfl, ?_,
    (twins_are_one_extension, funext fun T => propext
    (the_root_read_twice_is_the_value T),
     (bidirectional_halted_uniduction S hS).1,
     (bidirectional_halted_uniduction S hS).2.1),
    (fun (Z, hZ) => hZ ▸ value_from_the_act Z,
     fun v => ((S, hS, (twins_on_every_world S).mpr v), rfl)),
    fun h => pair_world_lacks_value offPoint off_point_is_off (h _
    (pair_world_closed offPoint)))
  intro Z₁ Z₂ h₁ h₂
  obtain (z₁, c₁, s₁) := Z₁
  obtain (z₂, c₂, s₂) := Z₂
  change z₁ = S at h₁
  change z₂ = S at h₂
  subst h₁
  subst h₂
  rfl
end RememberedOffered

/-! # BOTH VALUES ARE ATOMIC · appended, nothing above changed. The bit has two values and each has the atomic form:
the value's proof is a supply of the one field; the denial's proof is one point off the line. -/
namespace RememberedOffered
theorem both_values_atomic (S : Config) (hS : Closed S) :
    ((∃ Z : ActualZeros, Z.zeros = S) ↔ Value S) ∧ (¬ Value S ↔ ∃ p, S p ∧ ¬ onLine p) ∧
    ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine p) :=
  ((the_atomic_necessity S hS).2.2.2.2.1,
   (fun hn => (two_doors S).resolve_left hn, fun (p, hp, hoff) hv => hoff (hv p hp)),
   doors_exclusive S)
end RememberedOffered

/-! # THE FORM OF A FIELD, EVERY PREMISE SORTED, THE HEIGHT IN ABSOLUTE VALUE · appended, nothing above changed.
Parts I to III of the atomic necessity hold of every proposition carried as the field of a type, the denial included:
they are the form of a field. What makes that form the form of every proof of the hypothesis is part IV: every
reading of the hypothesis from either side and in every vocabulary is this one proposition. Every premise on
configurations is sorted three ways against the value. And the part above a height is read with the height in
absolute value, the form of the cited verification, which covers both half-planes. -/
namespace RememberedOffered
/-- A proposition on configurations, carried as the field of a type at a closed configuration. -/
structure Field (V : Config → Prop) where
  zeros : Config
  closed : Closed zeros
  supply : V zeros

/-- THE FORM OF A FIELD: for every proposition carried as a field, every proof of it at a closed configuration is a
    supply of the field there, any two proofs are one, and the supply is unique. -/
theorem the_form_of_a_field (V : Config → Prop) (S : Config) (hS : Closed S) :
    (∀ Q : Prop, (Q → V S) → Q → ∃ B : Field V, B.zeros = S) ∧
    (∀ h₁ h₂ : V S, h₁ = h₂) ∧
    (∀ B₁ B₂ : Field V, B₁.zeros = S → B₂.zeros = S → B₁ = B₂) := by
  refine (fun Q hQ q => ((S, hS, hQ q), rfl), fun _ _ => rfl, ?_)
  intro B₁ B₂ h₁ h₂
  obtain (z₁, c₁, s₁) := B₁
  obtain (z₂, c₂, s₂) := B₂
  change z₁ = S at h₁
  change z₂ = S at h₂
  subst h₁
  subst h₂
  rfl

/-- THE FORM IS THE HYPOTHESIS'S: the act's type is the field of the value, supply for supply; the form of a field
    holds at the value and at its denial alike; and every reading of the hypothesis the seed names is the value, so
    the form of the field of the value is the form of every proof of the hypothesis. -/
theorem the_form_is_the_hypothesis (S : Config) (hS : Closed S) :
    ((∃ Z : ActualZeros, Z.zeros = S) ↔ ∃ B : Field Value, B.zeros = S) ∧
    ((∀ Q : Prop, (Q → Value S) → Q → ∃ B : Field Value, B.zeros = S) ∧
      (∀ Q : Prop, (Q → ¬ Value S) → Q → ∃ B : Field (fun T => ¬ Value T), B.zeros = S)) ∧
    (LeastErasure = Value ∧ RootReadTwice = Value ∧ (ZeroReports S ↔ Value S) ∧ (PrimeBound S ↔ Value S)) :=
  ((fun (Z, hZ) => ((Z.zeros, Z.closed, value_from_the_act Z), hZ),
    fun (B, hB) => ((B.zeros, B.closed, (twins_on_every_world B.zeros).mpr B.supply), hB)),
   ((the_form_of_a_field Value S hS).1, (the_form_of_a_field (fun T => ¬ Value T) S hS).1),
   (the_atomic_necessity S hS).2.2.2.1)

/-- EVERY PREMISE SORTED: a premise on configurations either fails to force the value at some closed configuration,
    or forces it and is the value on every closed configuration, or forces it and is strictly stronger, failing at
    some closed configuration with the value. -/
theorem premise_trichotomy (P : Config → Prop) :
    (∃ S, Closed S ∧ P S ∧ ¬ Value S) ∨
    ((∀ S, Closed S → P S → Value S) ∧ ∀ S, Closed S → (P S ↔ Value S)) ∨
    ((∀ S, Closed S → P S → Value S) ∧ ∃ S, Closed S ∧ Value S ∧ ¬ P S)
    := by
  rcases Classical.em (∀ S, Closed S → P S → Value S) with f | f
  · rcases Classical.em (∀ S, Closed S → Value S → P S) with e | e
    · exact Or.inr (Or.inl (f, fun S hS => (f S hS, e S hS)))
    · refine Or.inr (Or.inr (f, Classical.byContradiction fun h => e fun S hS v => ?_))
      exact Classical.byContradiction fun np => h (S, hS, v, np)
  · refine Or.inl (Classical.byContradiction fun h => f fun S hS p => ?_)
    exact Classical.byContradiction fun nv => h (S, hS, p, nv)

/-- The height of a point of the chart in absolute value: the form of the cited verification, both half-planes. -/
def htAbs (p : Point) : Nat := p.2.natAbs

/-- THE UNICORN, CONTAINED, WITH THE HEIGHT IN ABSOLUTE VALUE: at every height the value is its verified part, every
    zero with |t| ≤ T, and its part above; given the verified part the value is exactly the part above; above every
    height no reading of the record decides the side; the root supplies nothing to the part above; on the act's zero
    set every arrival above every height stands on the line. -/
theorem the_unicorn_contained_abs :
    (∀ (S : Config) (T : Nat),
      Value S ↔ RHUnicorn.RealPartAt S onLine htAbs T ∧
      RHUnicorn.UnicornAt S onLine htAbs T) ∧
    (∀ (S : Config) (T : Nat),
      RHUnicorn.RealPartAt S onLine htAbs T → (Value S ↔
      RHUnicorn.UnicornAt S onLine htAbs T)) ∧
    (∀ T : Nat, ¬ ∃ g : Point → Bool, ∀ w : Point, htAbs w > T → g (reg w) = decide (w.1 > 0)) ∧
    (∀ (S : Config) (T : Nat),
      (RA → RHUnicorn.UnicornAt S onLine htAbs T) ↔ RHUnicorn.UnicornAt S onLine htAbs T) ∧
    (∀ (Z : ActualZeros) (T : Nat), RHUnicorn.UnicornAt Z.zeros onLine htAbs T) ∧
    (∀ Z : ActualZeros, ¬ ∃ p, Z.zeros p ∧ ¬ onLine p) :=
  (fun S T => RHUnicorn.rh_iff_real_and_unicorn S onLine htAbs T,
   fun S T hr => RHUnicorn.unicorn_no_bypass S onLine htAbs T hr,
   fun T => RHUnicorn.unicorn_block htAbs T fold reg (fun w => decide (w.1 > 0)) ((1 : Int), Int.ofNat (T + 1))
     (show T + 1 > T by omega) (show T + 1 > T by omega) rfl rfl,
   fun S T => RHUnicorn.unicorn_rule_exact S onLine htAbs T RA
   (seated_undeniable raSelfGrounding),
   fun Z _ p hp _ => value_from_the_act Z p hp,
   no_arrival_off_the_line_on_the_act)
end RememberedOffered

/-! # ROWS 10 AND 11 · appended, nothing above changed. Row 10: the unconditional route from shared premises and the
record is blocked. Row 11: accepting the hypothesis, by any proof, an unconditional one included, forces acceptance of
the atomic proof provided in this paper; the only other door is one point off the line. -/
namespace RememberedOffered

/-- ROW 10 · THE UNCONDITIONAL ROUTE FROM SHARED PREMISES AND THE RECORD IS BLOCKED. The value is no law of
    reflection-closed configurations; nothing every closed configuration shares forces it, the root included; no
    reading of the record decides it; and, by a law of logic true of every proposition, every premise that does force
    it is either the value itself or strictly stronger than the value. -/
theorem the_unconditional_route_is_blocked {R : Prop} (G : SelfGrounding R) :
    (¬ ∀ S : Config, Closed S → Value S) ∧
    (∀ D : Config → Prop, (∀ S, Closed S → D S) → ¬ ∀ S, Closed S → D S → Value S) ∧
    (¬ ∀ S : Config, Closed S → R → Value S) ∧
    (∀ g : Config → Prop, (∀ S S', SameRecord S S' → (g S ↔ g S')) →
      ¬ ∀ S, Closed S → (g S ↔ Value S)) ∧
    (∀ P : Config → Prop, (∀ S, Closed S → P S → Value S) →
      (∀ S, Closed S → (P S ↔ Value S)) ∨ ∃ S, Closed S ∧ Value S ∧ ¬ P S) := by
  have pc := pair_world_closed offPoint
  have pv := pair_world_lacks_value offPoint off_point_is_off
  have rc : Closed (registered offPoint) := fun s hs => by
```

```
    rw [hs]; show fold (reg offPoint) = reg offPoint; rfl
  refine (fun h => pv (h _ pc), no_closed_principle_decides,
    the_floor_alone_decides_no_value G, ?_, ?_)
  · intro g hg h
    exact pv ((h _ pc).mp ((hg _ _ (one_record offPoint)).mp ((h _
      rc).mpr (registered_has_value offPoint))))
  · intro P f
    rcases premise_trichotomy P with (S, hS, p, nv) | (_, e) | (_, s)
    · exact absurd (f S hS p) nv
    · exact Or.inl e
    · exact Or.inr s

/-- ROW 11 · ACCEPTING THE HYPOTHESIS FORCES ACCEPTANCE OF THE ATOMIC
PROOF. At every reflection-closed zero set:
    every proof of the value, from any premise, an unconditional one
    included, is a supply of the act's field, by the
    definition of the act's type, which carries the value itself as its
    field; the atomic proof this paper provides is
    that form made explicit. Any two proofs are one and the supply is
    unique; the only other door is one point off the
    line; there is no third value; and the structure, the reduction and
    the forced form, holds with no premise. -/
theorem accepting_rh_forces_the_atomic_proof (S : Config) (hS : Closed S)
:
    (∀ Q : Prop, (Q → Value S) → Q → ∃ Z : ActualZeros, Z.zeros = S) ∧
    (Value S → ∃ Z : ActualZeros, Z.zeros = S) ∧
    (∀ h₁ h₂ : Value S, h₁ = h₂) ∧
    (∀ Z₁ Z₂ : ActualZeros, Z₁.zeros = S → Z₂.zeros = S → Z₁ = Z₂) ∧
    (¬ Value S ↔ ∃ p, S p ∧ ¬ onLine p) ∧
    (Value S ∨ ∃ p, S p ∧ ¬ onLine p) ∧ ¬ (Value S ∧ ∃ p, S p ∧ ¬ onLine
    p) ∧
    ((ZeroReports S ↔ Value S) ∧ (PrimeBound S ↔ Value S) ∧ (LeastErasure
    S ↔ Value S) ∧
      (RootReadTwice S ↔ Value S)) ∧
    (∀ P : Config → Prop, (∀ S', P S' → Value S') → (∀ S', Value S' → P
    S') → ∀ S', P S' ↔ Value S') := by
  have n := the_atomic_necessity S hS
  have b := both_values_atomic S hS
  refine ⟨n.1, fun v => n.2.2.2.2.1.mpr v, n.2.1, n.2.2.1, b.2.1,
  two_doors S, b.2.2, ?_, ?_⟩
  · exact (the_atomic_proof raSelfGrounding S hS).1
  · exact (the_atomic_proof raSelfGrounding S hS).2.2.1

end RememberedOffered

end PartIII

/-! # PART IV · the arrow and the zero side locus -/
section PartIV
/-
  ARROW HINGE · the non-trivial-zero locus read directly off the arrow.
  Core Lean 4.19.0, no import, no axiom declared, no sorry.
  The arrow of the completed zeta runs between its only two poles, s = 0
  and s = 1. Chart: x = 4·Re s, height t,
  so the poles sit at x = 0 and x = 4 and the open strip is 0 < x < 4.
  Grain is free among the grains that put the
  midpoint on the chart, the even ones; every such grain gives the same
  hinge, Re s = 1/2, and an odd grain holds no
  chart point at the hinge (`hinge_iff_even`). Every chart of this seed
  has an even grain.
  I   the arrow alone: reversal swaps its poles, undoes itself, and fixes
  at most one point, the hinge, and exactly
      one when the poles have an even sum in the chart.
  II  the free bit: flipping the arrow's orientation leaves the reversal
  and the hinge unchanged.
  III the plane: the fold keeps height and moves offset; its fixed set is
  the line Re s = 1/2.
  IV  the lean: off the line a point carries the arrow's sign and the
  fold flips it; the registration merges it.
  V   the bar: a coherent off-line world sits inside the strip; no
  premise true of every closed world forces the value.
  VI  the act: a supplied least-erasure world carries the value.
  VII the zero side's own locus: the centre of every nonempty finite
  reflection-closed family is the line.
  VIII the line on every extension: the arrow fixes one value of the real
  part; on every map of the plane that acts on
      the real part by the reversal, the points whose real part it fixes
      are the line, and the centre balances on it;
      a finite family closed under the fold, listed in any order, has its
      centre on the line.
      The height-keeping extension is the fold, s ↦ 1 − s̄, which needs
      the conjugate symmetry [cited]; the functional
      equation alone, s ↦ 1 − s, fixes one point and reads the same line.
-/
set_option autoImplicit false
namespace Hinge

/-! ## I · the arrow alone -/
structure Arrow where
  tail : Int
  head : Int

def rev (A : Arrow) (x : Int) : Int := A.tail + A.head - x
def flip (A : Arrow) : Arrow := (A.head, A.tail)
def off (A : Arrow) (x : Int) : Int := 2 * x - (A.tail + A.head)

theorem rev_swaps (A : Arrow) : rev A A.tail = A.head ∧ rev A A.head =
A.tail := by
  unfold rev; constructor <;> omega
theorem rev_involutive (A : Arrow) (x : Int) : rev A (rev A x) = x := by
unfold rev; omega
/-- THE HINGE: a point is fixed by the arrow's reversal exactly when its
offset from the midpoint is zero. -/
theorem hinge (A : Arrow) (x : Int) : rev A x = x ↔ off A x = 0 := by
unfold rev off; omega
theorem hinge_unique (A : Arrow) (x y : Int) (hx : rev A x = x) (hy : rev
A y = y) : x = y := by
  unfold rev at hx hy; omega
/-- THE HINGE EXISTS on every arrow whose poles have an even sum in the
chart: then the reversal fixes exactly the
    midpoint. With `hinge_unique`, exactly one point. -/
theorem hinge_exists (A : Arrow) (h : (A.tail + A.head) % 2 = 0) : ∃ x,
rev A x = x :=
  ((A.tail + A.head) / 2, by unfold rev; omega)
/-- THE HINGE ON THE CHART: the reversal has a fixed point on the integer
chart exactly when the poles have an even
    sum; at an odd grain the midpoint falls between two chart points and
    the chart holds no hinge. -/
theorem hinge_iff_even (A : Arrow) : (∃ x, rev A x = x) ↔ (A.tail +
A.head) % 2 = 0 := by
  constructor
  · intro (x, hx)
    unfold rev at hx
    omega
  · exact hinge_exists A

/-! ## II · the free bit -/
/-- SIGN-BLIND: the reversal and the offset magnitude ignore which way
the arrow points. -/
theorem hinge_sign_blind (A : Arrow) (x : Int) : rev (flip A) x = rev A x
∧ off (flip A) x = off A x := by
  show A.head + A.tail - x = A.tail + A.head - x ∧ 2 * x - (A.head +
  A.tail) = 2 * x - (A.tail + A.head)
  constructor <;> omega
/-- The reversal flips every offset. -/
theorem rev_flips_offset (A : Arrow) (x : Int) : off A (rev A x) = - off A
x := by unfold off rev; omega

/-- Zeta's arrow: s = 0 to s = 1, at x = 4·Re s. -/
def zeta : Arrow := (0, 4)
theorem zeta_hinge (x : Int) : rev zeta x = x ↔ x = 2 := by
  show 0 + 4 - x = x ↔ x = 2; omega
/-- THE REVERSAL IS FORCED: the only distance-keeping map of the axis
that swaps the poles is the arrow's reversal.
    Distance-keeping is stated sign-blind, every difference kept up to
    its sign, so the Number alone fixes the map. -/
theorem the_only_rigid_reversal (f : Int → Int)
    (iso : ∀ x y, f x - f y = x - y ∨ f x - f y = y - x) (h0 : f 0 = 4)
    (h4 : f 4 = 0) :
    ∀ x, f x = rev zeta x := by
  intro x
  have a := iso x 0
  have b := iso x 4
  rw [h0] at a
  rw [h4] at b
  show f x = 0 + 4 - x
  omega

/-! ## III · the plane -/
abbrev Pt := Int × Int
def fold (p : Pt) : Pt := (rev zeta p.1, p.2)
def onLine (p : Pt) : Prop := p.1 = 2
def reg (p : Pt) : Pt := (2, p.2)
def inStrip (p : Pt) : Prop := 0 < p.1 ∧ p.1 < 4

theorem fold_keeps_height (p : Pt) : (fold p).2 = p.2 := rfl
/-- THE LOCUS: the fold's fixed set is the line x = 2, which is Re s =
1/2. -/
theorem locus (p : Pt) : fold p = p ↔ onLine p := by
  obtain (x, t) := p
  constructor
  · intro e
    have h1 : 0 + 4 - x = x := congrArg Prod.fst e
    show x = 2; omega
  · intro e
    have h1 : x = 2 := e
    subst h1; rfl

/-! ## IV · the lean -/
/-- OFF THE LINE A POINT LEANS: its offset is nonzero and the fold flips
its sign. -/
theorem off_line_leans (p : Pt) (h : ¬ onLine p) : off zeta p.1 ≠ 0 ∧ off
zeta (fold p).1 = - off zeta p.1 := by
  constructor
  · intro e
    have e1 : 2 * p.1 - (0 + 4) = 0 := e
    exact h (show p.1 = 2 by omega)
  · exact rev_flips_offset zeta p.1
theorem reg_keeps_on_line (p : Pt) (h : onLine p) : reg p = p := by
  obtain (x, t) := p
  have h1 : x = 2 := h
  subst h1; rfl
/-- OFF THE LINE THE REGISTRATION MERGES: two distinct points, one
record. -/
theorem reg_merges (p : Pt) (h : ¬ onLine p) : fold p ≠ p ∧ reg (fold p) =
reg p :=
  ⟨fun e => h ((locus p).mp e), rfl⟩

/-! ## V · the bar -/
def Closed (Z : Pt → Prop) : Prop := ∀ p, Z p → Z (fold p)
def Value (Z : Pt → Prop) : Prop := ∀ p, Z p → onLine p
def LeastErasure (Z : Pt → Prop) : Prop := ∀ p, Z p → reg p = p

theorem least_erasure_iff_value (Z : Pt → Prop) : LeastErasure Z ↔ Value Z
:= by
  constructor
```

```
  · intro h p hp
    have e : (1 : Int) * 2 = p.1 := by
      have e0 : (reg p).1 = p.1 := congrArg Prod.fst (h p hp)
      have e2 : (2 : Int) = p.1 := e0
      omega
    show p.1 = 2; omega
  · intro h p hp; exact reg_keeps_on_line p (h p hp)

/-- The off-line pair at Re s = 1/4 and 3/4, height 0. -/
def pairW : Pt → Prop := fun p => p = (1, 0) ∨ p = (3, 0)
theorem pair_closed : Closed pairW := by
  intro p hp
  rcases hp with e | e
  · subst e; exact Or.inr (by decide)
  · subst e; exact Or.inl (by decide)
theorem pair_in_strip : ∀ p, pairW p → inStrip p := by
  intro p hp
  rcases hp with e | e
  · subst e; exact (by decide, by decide)
  · subst e; exact (by decide, by decide)
theorem pair_lacks_value : ¬ Value pairW := by
  intro h
  have h1 : (1 : Int) = 2 := h (1, 0) (Or.inl rfl)
  exact absurd h1 (by decide)
/-- THE ARROW FORCES NO ZERO: no premise true of every closed world
forces the value. -/
theorem keyless_forces_nothing (Q : (Pt → Prop) → Prop) (hQ : ∀ Z, Closed
Z → Q Z) :
    ¬ ∀ Z, Closed Z → Q Z → Value Z :=
  fun hall => pair_lacks_value (hall pairW pair_closed (hQ pairW
  pair_closed))

/-! ## VI · the act -/
structure ActualZeros where
  zeros : Pt → Prop
  closed : Closed zeros
  supply : LeastErasure zeros
theorem value_from_the_act (Z : ActualZeros) : Value Z.zeros :=
(least_erasure_iff_value _).mp Z.supply

/-! ## VII · the zero side's own locus -/
/-- The real parts of a configuration, summed. -/
def sumX : List Pt → Int
  | [] => 0
  | p :: L => p.1 + sumX L
/-- THE ZERO SIDE'S LOCUS IS THE LINE: list any members with their
mirrors and the real parts balance on the line,
    four per member pair in this chart, so the centre of every
    reflection-closed configuration sits at Re s = 1/2. -/
theorem centre_is_the_line (L : List Pt) : sumX (L.map fold) + sumX L = 4
* (L.length : Int) := by
  induction L with
  | nil => rfl
  | cons p L ih =>
    have hf : (fold p).1 = 0 + 4 - p.1 := rfl
    simp only [List.map_cons, sumX, List.length_cons]
    push_cast
    omega
/-- THE SAME OBJECT, MEMBERS OFF IT: the pair at Re s = 1/4 and 3/4 is
reflection-closed, its centre is the line,
    it sits in the open strip, and neither member is on the line. -/
theorem centred_yet_off :
    Closed pairW ∧ sumX [(1, 0), (3, 0)] = 2 * 2 ∧ (∀ p, pairW p →
    inStrip p ∧ ¬ onLine p) := by
  refine (pair_closed, by decide, ?_)
  intro p hp
  refine (pair_in_strip p hp, ?_)
  rcases hp with e | e
  · subst e; intro h; have h1 : (1 : Int) = 2 := h; exact absurd h1 (by
  decide)
  · subst e; intro h; have h1 : (3 : Int) = 2 := h; exact absurd h1 (by
  decide)

/-! ## VIII · the line and the centre on every extension -/
/-- THE LINE OFF THE ARROW: on every map of the plane that acts on the
real part by the arrow's reversal, the points
    whose real part it fixes are exactly the line x = 2, Re s = 1/2. The
    arrow fixes one value of the real part; the
    line is the set of points with that real part, whatever the map does
    to height. -/
theorem the_line_off_the_arrow (F : Pt → Pt) (hF : ∀ p, (F p).1 = rev
zeta p.1) (p : Pt) :
    (F p).1 = p.1 ↔ onLine p := by
  rw [hF p]
  exact zeta_hinge p.1
/-- THE FOLD IS THE HEIGHT-KEEPING EXTENSION: the one map of the plane
that acts on the real part by the reversal and
    keeps height is the fold. Keeping height is the conjugate symmetry
    composed with the functional equation,
    s ↦ 1 − s̄ [Riemann 1859, cited]. -/
theorem fold_is_the_height_keeping_extension (F : Pt → Pt) (hF : ∀ p, (F
p).1 = rev zeta p.1)
    (hh : ∀ p, (F p).2 = p.2) (p : Pt) : F p = fold p :=
  Prod.ext (hF p) (hh p)
/-- The functional equation alone, s ↦ 1 − s, on the chart: the reversal
on the real part, the height reversed. -/
def pointRefl (p : Pt) : Pt := (rev zeta p.1, -p.2)
/-- THE FUNCTIONAL EQUATION ALONE FIXES ONE POINT AND READS THE SAME
LINE: s ↦ 1 − s fixes only s = 1/2, and the
    points whose real part it fixes are the line. -/
theorem the_point_reflection_reads_the_same_line :
    (∀ p : Pt, pointRefl p = p ↔ p = (2, 0)) ∧ (∀ p : Pt, (pointRefl p).1
    = p.1 ↔ onLine p) := by
  refine (fun p => ?_, the_line_off_the_arrow pointRefl (fun _ => rfl))
  obtain (x, t) := p
  constructor
  · intro e
    have h1 : 0 + 4 - x = x := congrArg Prod.fst e
    have h2 : -t = t := congrArg Prod.snd e
    have hx : x = 2 := by omega
    have ht : t = 0 := by omega
    subst hx
    subst ht
    rfl
  · intro e
    have hx : x = 2 := congrArg Prod.fst e
    have ht : t = 0 := congrArg Prod.snd e
    subst hx
    subst ht
    decide
/-- THE CENTRE ON EVERY EXTENSION: under every map that acts on the real
part by the reversal, the real parts of any
    finite family and of its image balance on the line; the centre does
    not depend on what the map does to height. -/
theorem centre_on_every_extension (F : Pt → Pt) (hF : ∀ p, (F p).1 = rev
zeta p.1) (L : List Pt) :
    sumX (L.map F) + sumX L = 4 * (L.length : Int) := by
  induction L with
  | nil => rfl
  | cons p L ih =>
    have hF : (F p).1 = 0 + 4 - p.1 := hF p
    simp only [List.map_cons, sumX, List.length_cons]
    push_cast
    omega

/-- The sum of real parts does not depend on the order in which a family
is listed. -/
theorem sumX_perm {L M : List Pt} (h : L.Perm M) : sumX L = sumX M := by
  induction h with
  | nil => rfl
  | cons x _ ih => simp only [sumX]; omega
  | swap x y l => simp only [sumX]; omega
  | trans _ _ ih1 ih2 => omega
/-- THE CENTRE OF A CLOSED FAMILY: a finite family whose mirror image is
the family itself, listed in any order, has
    its real parts summing to twice its size, so a nonempty one has its
    centre at x = 2, Re s = 1/2. -/
theorem centre_of_a_closed_family (L : List Pt) (h : (L.map fold).Perm L)
: sumX L = 2 * (L.length : Int) := by
  have e := centre_is_the_line L
  have p := sumX_perm h
  omega

/-! ## the hinge, whole -/
theorem arrow_gives_line_not_value :
    (∀ A : Arrow, rev A A.tail = A.head ∧ rev A A.head = A.tail) ∧
    (∀ (A : Arrow) (x : Int), rev A (rev A x) = x) ∧
    (∀ (A : Arrow) (x y : Int), rev A x = x → rev A y = y → x = y) ∧
    (∀ (A : Arrow) (x : Int), rev (flip A) x = rev A x ∧ off (flip A) x =
    off A x) ∧
    (∀ x : Int, rev zeta x = x ↔ x = 2) ∧
    (∀ p : Pt, fold p = p ↔ onLine p) ∧
    (∀ p : Pt, ¬ onLine p → off zeta p.1 ≠ 0 ∧ off zeta (fold p).1 = -
    off zeta p.1) ∧
    (∀ p : Pt, ¬ onLine p → fold p ≠ p ∧ reg (fold p) = reg p) ∧
    (∀ Z : Pt → Prop, LeastErasure Z ↔ Value Z) ∧
    (Closed pairW ∧ (∀ p, pairW p → inStrip p) ∧ ¬ Value pairW) ∧
    (∀ Q : (Pt → Prop) → Prop, (∀ Z, Closed Z → Q Z) → ¬ ∀ Z, Closed Z → Q
    Z → Value Z) ∧
    (∀ Z : ActualZeros, Value Z.zeros) :=
  (rev_swaps, rev_involutive, hinge_unique, hinge_sign_blind, zeta_hinge,
  locus, off_line_leans, reg_merges,
   least_erasure_iff_value, (pair_closed, pair_in_strip,
   pair_lacks_value), keyless_forces_nothing,
   value_from_the_act)
end Hinge


end PartIV

/-! # PART IV-b · Force_Witness.lean, verbatim from the signed master
v5.1.0, with the plug appended -/
section PartIVb
/-
  Force_Witness.lean · PhysOS Proof PSP-FORCE-WITNESS-01 · the
  force-witness closure.
  Core Lean 4.19.0. No import, no sorry, no admit, no native_decide, no
  axiom declared.

  Least erasure is the final ninth-gate bit. A run of the substrate is
  the atomic witness that the closure was
  computed: a run whose flag is true proves that a run exists. The value
  at the actual zero set is the field of
  the act: keyed, given, free.

  PART ONE is copied from Inescapable_Capstone.lean, every definition and
  every law unchanged, with the lemmas
  they stand on: the chart, the configurations, the record, the act, the
  ninth gate, the deed.
  PART TWO is new: the atomic step, the ninth gate packed, the deed of
  the substrate, the free bit, the closure
  and its guard.

  THE FREE BIT. Every run leaves the value to the act.
  the_deed_forces_nothing is the seated law, and
  the_run_does_not_force_the_value states it at every run. It is the
  fourth conjunct of the closure.
```

```
-/

namespace ForceWitness

/-! ## PART ONE · The seated carriers and laws, copied unchanged -/

/-- A point is (d, t): d its offset from the line, t its height. The line is d = 0. -/
abbrev Point := Int × Int

/-- The fold: s ↦ 1 − s̄ on the offset chart, (d, t) ↦ (−d, t). -/
def fold (p : Point) : Point := (-p.1, p.2)

/-- The line: offset zero. -/
def onLine (p : Point) : Prop := p.1 = 0

/-- The registration: it keeps the height and forgets the side. -/
def reg (p : Point) : Point := (0, p.2)

/-- Integer negation is an involution, by the integer's constructors. -/
theorem neg_neg_free : ∀ d : Int, - -d = d := fun
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

/-- An integer equal to its own negation is zero, by the integer's constructors. -/
theorem neg_self_zero : ∀ d : Int, -d = d → d = 0 := fun
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Int.noConfusion h
  | Int.negSucc _, h => Int.noConfusion h

/-- The fold is an involution. -/
theorem fold_involutive (p : Point) : fold (fold p) = p := by
  obtain (d, t) := p
  show (- -d, t) = (d, t)
  rw [neg_neg_free d]

/-- THE CUT IS THE LINE: the fold fixes a point exactly when it is on the line. -/
theorem the_cut_is_the_line (p : Point) : fold p = p ↔ onLine p := by
  obtain (d, t) := p
  exact (fun h => neg_self_zero d (congrArg Prod.fst h), fun h => by
    cases h; rfl)

/-- The registration lands on the line. -/
theorem reg_lands (p : Point) : onLine (reg p) := rfl

/-- The registration erases nothing at a point exactly when the point is on the line. -/
theorem reg_erases_nothing_iff (p : Point) : reg p = p ↔ onLine p := by
  obtain (d, t) := p
  exact (fun h => (congrArg Prod.fst h).symm, fun h => by cases h; rfl)

/-- A configuration: a set of points. Closed: the fold carries it to itself. -/
abbrev Config := Point → Prop

def Closed (S : Config) : Prop := ∀ p, S p → S (fold p)

/-- The value: every point of the configuration is on the line. -/
def Value (S : Config) : Prop := ∀ p, S p → onLine p

/-- Least erasure: the registration leaves every point of the configuration unchanged. -/
def LeastErasure (S : Config) : Prop := ∀ p, S p → reg p = p

/-- The record of a configuration: what the registration leaves. -/
def Rec (S : Config) (q : Point) : Prop := ∃ p, S p ∧ reg p = q

def SameRecord (S S' : Config) : Prop := ∀ q, Rec S q ↔ Rec S' q

/-- The registered world of a point and its pair world. -/
def registered (p : Point) : Config := fun s => s = reg p

def pairWorld (p : Point) : Config := fun s => s = p ∨ s = fold p

theorem pair_world_closed (p : Point) : Closed (pairWorld p) := fun _ hs =>
  match hs with
  | Or.inl e => Or.inr (congrArg fold e)
  | Or.inr e => Or.inl (e ▸ fold_involutive p)

/-- The registered world and the pair world leave one record. -/
theorem one_record (p : Point) : SameRecord (registered p) (pairWorld p) := fun q =>
  (fun (_, hs, hq) => (p, Or.inl rfl, by rw [← hq, hs]; rfl),
   fun (_, hs, hq) => (reg p, rfl, by
      rw [← hq]
      exact match hs with
        | Or.inl e => by rw [e]; rfl
        | Or.inr e => by rw [e]; rfl))

theorem registered_has_value (p : Point) : Value (registered p) := fun s hs => by
  rw [hs]; exact reg_lands p

theorem pair_world_lacks_value (p : Point) (h : ¬ onLine p) : ¬ Value (pairWorld p) :=
  fun hv => h (hv p (Or.inl rfl))

/-- THE RECORD DECIDES NOTHING: for every point off the line, no reading that respects the record returns the
    value on both its registered world and its pair world. -/
theorem record_decides_nothing (p : Point) (h : ¬ onLine p) (g : Config → Prop)
    (hg : ∀ S S', SameRecord S S' → (g S ↔ g S')) :
    ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔ Value (pairWorld p))) :=
  fun (h1, h2) => pair_world_lacks_value p h
    (h2.mp ((hg _ _ (one_record p)).mp (h1.mpr (registered_has_value p))))

/-- THE ACT. The bit is supplied once, in the open, as a field of a type: a closed configuration with least
    erasure at it. The field is the value supplied; the type makes the
    supply visible to every reader. -/
structure ActualZeros where
  zeros : Config
  closed : Closed zeros
  supply : LeastErasure zeros

/-- FROM THE ACT, THE VALUE. -/
theorem value_from_the_act (Z : ActualZeros) : Value Z.zeros :=
  fun p hp => (reg_erases_nothing_iff p).mp (Z.supply p hp)

def e (n i : Nat) : List Int := (List.range n).map (fun j => if j == i then 1 else 0)

def neg3 (s : Bool × Bool × Bool) : Bool × Bool × Bool := (!s.1, !s.2.1, !s.2.2)

def par3 (s : Bool × Bool × Bool) : Nat :=
  (if s.1 then 1 else 0) + (if s.2.1 then 1 else 0) + (if s.2.2 then 1 else 0)

/-- The orientation of a sign pattern, the determinant of its diagonal: −1 to the number of flips. -/
def orient (s : Bool × Bool × Bool) : Int := if par3 s % 2 == 0 then 1 else -1

/-- The ordered factor pairs of n, by first factor; the swap sends a to n / a. -/
def pairs (n : Nat) : List Nat := (List.range (n + 1)).filter (fun a => 0 < a && n % a == 0)

/-- The price of exported freedom in native units: temperature is energy per bit, so b bits cost b·T. -/
def price (T bits : Nat) : Nat := bits * T

/-- A finite configuration: the heights of its zeros on the line, and its off-line pairs, each an offset that is not
    zero and a height. -/
structure FinCfg where
  line : List Int
  pairs : List (Int × Int)
  offset_ne : ∀ q ∈ pairs, q.1 ≠ 0

/-- The erased bits of a finite configuration: one per off-line pair, two points registered to one. -/
def erased (F : FinCfg) : Nat := F.pairs.length

/-- The executed frame: a formal content and the bit of whether it ran. Reading the content is formal. -/
abbrev Exec (Q : Type) := Bool × Q

def ran {Q : Type} (s : Exec Q) : Bool := s.1

/-- The nine: the eight sign patterns and one more. -/
abbrev Gate := Option (Bool × Bool × Bool)

/-- The fold on the nine: total negation on the eight; the ninth is its own partner. -/
def gfold : Gate → Gate
  | none => none
  | some s => some (neg3 s)

/-- THE SEAT IS THE NINTH: the fold of the nine fixes a gate exactly when the gate is the ninth. -/
theorem seat_is_the_ninth (g : Gate) : gfold g = g ↔ g = none := by
  cases g with
  | none => exact (fun _ => rfl, fun _ => rfl)
  | some s =>
    obtain (a, b, c) := s
    constructor
    · intro h; cases a <;> cases b <;> cases c <;> cases h
    · intro h; cases h

/-- The orientation of a pattern is never zero. -/
theorem orient_ne_zero (s : Bool × Bool × Bool) : orient s ≠ 0 := by
  obtain (a, b, c) := s; cases a <;> cases b <;> cases c <;> decide

/-- Total negation flips the orientation. -/
theorem orient_flips (s : Bool × Bool × Bool) : orient (neg3 s) = -orient s := by
  obtain (a, b, c) := s; cases a <;> cases b <;> cases c <;> rfl

/-- The carrier of the nine onto the chart: a pattern lands on the wall or the edge by its orientation, the ninth on
    the line. -/
def toChart (t : Int) : Gate → Point
  | none => (0, t)
  | some s => (orient s, t)

/-- The carrier is equivariant: the fold of the nine is carried onto the fold of the chart. -/
theorem gate_carrier_equivariant (t : Int) (g : Gate) : toChart t (gfold g) = fold (toChart t g) := by
```

```
  cases g with
  | none => rfl
  | some s => obtain (a, b, c) := s; cases a <;> cases b <;> cases c <;>
    rfl

/-- And under every carrier that separates partners, only the ninth does.
-/
theorem faithful_carrier_lands_only_the_ninth (c : Gate → Point) (eqv : ∀
g, c (gfold g) = fold (c g))
    (sep : ∀ g, c (gfold g) = c g → gfold g = g) (g : Gate) : onLine (c
    g) ↔ g = none := by
  constructor
  · intro h
    exact (seat_is_the_ninth g).mp (sep g ((eqv g).trans
    ((the_cut_is_the_line (c g)).mpr h)))
  · intro h
    subst h
    have e : fold (c none) = c none := (eqv none).symm
    exact (the_cut_is_the_line (c none)).mp e

/-- The orientation carrier separates partners. -/
theorem orientation_carrier_separates (t : Int) (g : Gate) (h : toChart t
(gfold g) = toChart t g) :
    gfold g = g := by
  cases g with
  | none => rfl
  | some s =>
    exact absurd (neg_self_zero (orient s) ((orient_flips s).symm.trans
    (congrArg Prod.fst h))) (orient_ne_zero s)

/-- A gate owes a bit when some target is odd at it: no even reading
returns that target, and one supplied odd
    witness fixes it by one calibration bit. -/
def OwesBit (g : Gate) : Prop := ∃ d : Gate → Bool, d (gfold g) = !d g

/-- At the seat no bit is owed: nothing is odd at a fixed point. -/
theorem seat_owes_no_bit : ¬ OwesBit none := by
  intro (d, h)
  have h' : d none = !d none := h
  cases hd : d none <;> rw [hd] at h' <;> cases h'

/-- Off the seat a bit is owed, at every one of the eight. -/
theorem off_seat_owes_a_bit (s : Bool × Bool × Bool) : OwesBit (some s)
:= by
  refine (fun g => decide (g = some s), ?_)
  obtain (a, b, c) := s
  cases a <;> cases b <;> cases c <;> decide

/-- The registered orbit of a gate, as a finite configuration: the ninth
is one point on the line; each of the eight
    is one off-line pair. -/
def gateCfg (t : Int) : Gate → FinCfg
  | none => ([t], [], fun _ h => nomatch h)
  | some s => ([], [(orient s, t)], fun q h => by
      cases h with
      | head => exact orient_ne_zero s
      | tail _ h' => nomatch h')

/-- The heat of registering a gate is zero exactly at the ninth. -/
theorem gate_heat_zero_iff_ninth (T : Nat) (hT : 0 < T) (t : Int) (g :
Gate) :
    price T (erased (gateCfg t g)) = 0 ↔ g = none := by
  cases g with
  | none => exact (fun _ => rfl, fun _ => Nat.zero_mul T)
  | some s =>
    constructor
    · intro h
      have h1 : 1 * T = 0 := h
      rw [Nat.one_mul] at h1
      exact absurd h1 (Nat.pos_iff_ne_zero.mp hT)
    · intro h; cases h

/-- THE NINTH GATE IS LEAST ERASURE. At a gate, five readings are one: it
is the fold's fixed point; it lands on
    the line; registration erases nothing there (forget); its
    registration is free (pay); no bit is owed there
    (supply). Each holds exactly at the ninth. -/
theorem ninth_gate_is_least_erasure (t : Int) (T : Nat) (hT : 0 < T) (g :
Gate) :
    (gfold g = g ↔ g = none) ∧
    (onLine (toChart t g) ↔ g = none) ∧
    (reg (toChart t g) = toChart t g ↔ g = none) ∧
    (price T (erased (gateCfg t g)) = 0 ↔ g = none) ∧
    (¬ OwesBit g ↔ g = none) := by
  have hl : onLine (toChart t g) ↔ g = none :=
    faithful_carrier_lands_only_the_ninth (toChart t)
    (gate_carrier_equivariant t) (orientation_carrier_separates t) g
  refine (seat_is_the_ninth g, hl, (reg_erases_nothing_iff (toChart t
  g)).trans hl,
          gate_heat_zero_iff_ninth T hT t g, ?_)
  cases g with
  | none => exact (fun _ => rfl, fun _ => seat_owes_no_bit)
  | some s => exact (fun h => absurd (off_seat_owes_a_bit s) h, fun h =>
  by cases h)

/-- A RUN IS ITS OWN ATOMIC WITNESS, of the deed: a state that ran proves
that a run exists. -/
theorem the_run_witnesses_a_run {Q : Type} (s : Exec Q) (h : ran s =
true) : ∃ r : Exec Q, ran r = true := by
  exact (s, h)

/-- AND OF NOTHING ELSE: a fact that holds whatever the configuration,
the deed included, forces no value. -/
theorem the_deed_forces_nothing (D : Prop) (hD : D) : ¬ ∀ S : Config,
Closed S → D → Value S := by
  intro hall
  exact pair_world_lacks_value (1, 0) (fun e => by cases e) (hall _
  (pair_world_closed (1, 0)) hD)


/-! ## PART TWO · The force-witness closure -/

/-- THE ATOMIC STEP: a witness with its check proves the existential it
witnesses. -/
theorem witness_proves_exists {α : Type} (P : α → Prop) (w : α) (h : P w)
: ∃ x, P x := (w, h)

/-- THE NINTH GATE, PACKED: a gate is the ninth exactly when all five
readings hold of it. The five biconditionals
    of ninth_gate_is_least_erasure, bound into one. -/
theorem ninth_gate_pack (t : Int) (T : Nat) (hT : 0 < T) (g : Gate) :
    g = none ↔
      (gfold g = g ∧ onLine (toChart t g) ∧ reg (toChart t g) = toChart t
      g ∧
       price T (erased (gateCfg t g)) = 0 ∧ ¬ OwesBit g) := by
  have h := ninth_gate_is_least_erasure t T hT g
  constructor
  · intro e
    exact (h.1.mpr e, h.2.1.mpr e, h.2.2.1.mpr e, h.2.2.2.1.mpr e,
    h.2.2.2.2.mpr e)
  · intro hh
    exact h.1.mp hh.1

/-- The deed of the substrate: a run, and the fact that it ran. -/
structure SubstrateDeed (Q : Type) where
  run : Exec Q
  didRun : ran run = true

/-- THE FREE BIT. A run whose flag is true leaves the value free: by the
run the value does not stand on every
    closed configuration. It is the_deed_forces_nothing read at the flag
    of the deed. A computation is a deed, and
    the value is the field of the act. -/
theorem the_run_does_not_force_the_value {Q : Type} (s : Exec Q) (h : ran
s = true) :
    ¬ (∀ S : Config, Closed S → Value S) :=
  fun hall => the_deed_forces_nothing (ran s = true) h (fun S hS _ =>
  hall S hS)

/-- THE FORCE-WITNESS CLOSURE. For a deed of the substrate, a term of the
act, and any gate: the gate is the ninth
    exactly when it is the fold's fixed point, on the line, erased
    nothing, free to register at a positive
    temperature and owing no bit; the value holds on the field the act
    supplies; a run that ran exists; and the
    run leaves the value free. The fourth conjunct is the free bit. It is
    the implementation of the force. -/
theorem force_witness_closure {Q : Type} (d : SubstrateDeed Q) (Z :
ActualZeros)
    (t : Int) (T : Nat) (hT : 0 < T) (g : Gate) :
    (g = none ↔
      (gfold g = g ∧ onLine (toChart t g) ∧ reg (toChart t g) = toChart t
      g ∧
       price T (erased (gateCfg t g)) = 0 ∧ ¬ OwesBit g)) ∧
    Value Z.zeros ∧
    (∃ r : Exec Q, ran r = true) ∧
    ¬ (∀ S : Config, Closed S → Value S) :=
  (ninth_gate_pack t T hT g,
   value_from_the_act Z,
   the_run_witnesses_a_run d.run d.didRun,
   the_run_does_not_force_the_value d.run d.didRun)

/-- THE GUARD ON THE CLOSURE, the double defense restated: the value
comes by the act; between the registered
    world and the pair world of a point off the line, every reading that
    respects the record leaves the value
    free; and the deed leaves it free. -/
theorem force_witness_guarded {Q : Type} (d : SubstrateDeed Q) (Z :
ActualZeros) :
    Value Z.zeros ∧
    (∀ (p : Point), ¬ onLine p → ∀ (g : Config → Prop), (∀ S S',
    SameRecord S S' → (g S ↔ g S')) →
      ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔
      Value (pairWorld p)))) ∧
    ¬ (∀ S : Config, Closed S → Value S) :=
  (value_from_the_act Z,
   fun p h g hg => record_decides_nothing p h g hg,
   the_run_does_not_force_the_value d.run d.didRun)

end ForceWitness

/-! ## The cones, pinned. Each line is the compiler's own; a drifted cone
fails the compile. -/
/-- info: 'ForceWitness.neg_neg_free' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.neg_neg_free
/-- info: 'ForceWitness.neg_self_zero' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.neg_self_zero
/-- info: 'ForceWitness.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.fold_involutive
/-- info: 'ForceWitness.the_cut_is_the_line' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.the_cut_is_the_line
/-- info: 'ForceWitness.reg_lands' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.reg_lands
/-- info: 'ForceWitness.reg_erases_nothing_iff' does not depend on any
axioms -/
```

```
#guard_msgs in #print axioms ForceWitness.reg_erases_nothing_iff
/-- info: 'ForceWitness.pair_world_closed' does not depend on any axioms
-/
#guard_msgs in #print axioms ForceWitness.pair_world_closed
/-- info: 'ForceWitness.one_record' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.one_record
/-- info: 'ForceWitness.registered_has_value' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.registered_has_value
/-- info: 'ForceWitness.pair_world_lacks_value' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.pair_world_lacks_value
/-- info: 'ForceWitness.record_decides_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.record_decides_nothing
/-- info: 'ForceWitness.value_from_the_act' does not depend on any axioms
-/
#guard_msgs in #print axioms ForceWitness.value_from_the_act
/-- info: 'ForceWitness.seat_is_the_ninth' does not depend on any axioms
-/
#guard_msgs in #print axioms ForceWitness.seat_is_the_ninth
/-- info: 'ForceWitness.orient_ne_zero' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.orient_ne_zero
/-- info: 'ForceWitness.orient_flips' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.orient_flips
/-- info: 'ForceWitness.gate_carrier_equivariant' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.gate_carrier_equivariant
/-- info: 'ForceWitness.faithful_carrier_lands_only_the_ninth' does not
depend on any axioms -/
#guard_msgs in #print axioms
ForceWitness.faithful_carrier_lands_only_the_ninth
/-- info: 'ForceWitness.orientation_carrier_separates' does not depend on
any axioms -/
#guard_msgs in #print axioms ForceWitness.orientation_carrier_separates
/-- info: 'ForceWitness.seat_owes_no_bit' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.seat_owes_no_bit
/-- info: 'ForceWitness.off_seat_owes_a_bit' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.off_seat_owes_a_bit
/-- info: 'ForceWitness.gate_heat_zero_iff_ninth' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.gate_heat_zero_iff_ninth
/-- info: 'ForceWitness.ninth_gate_is_least_erasure' does not depend on
any axioms -/
#guard_msgs in #print axioms ForceWitness.ninth_gate_is_least_erasure
/-- info: 'ForceWitness.the_run_witnesses_a_run' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.the_run_witnesses_a_run
/-- info: 'ForceWitness.the_deed_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.the_deed_forces_nothing
/-- info: 'ForceWitness.witness_proves_exists' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.witness_proves_exists
/-- info: 'ForceWitness.ninth_gate_pack' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.ninth_gate_pack
/-- info: 'ForceWitness.the_run_does_not_force_the_value' does not depend
on any axioms -/
#guard_msgs in #print axioms ForceWitness.the_run_does_not_force_the_value
/-- info: 'ForceWitness.force_witness_closure' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.force_witness_closure
/-- info: 'ForceWitness.force_witness_guarded' does not depend on any
axioms -/
#guard_msgs in #print axioms ForceWitness.force_witness_guarded

/-! ## The closure and its guard, printed. -/
#print axioms ForceWitness.force_witness_closure
#print axioms ForceWitness.force_witness_guarded

/-! ## THE PLUG · appended, nothing above changed. The run's true flag
offered as the act's supply field. -/
namespace ForceWitness
/-- The plug, attempted: if a true flag supplied least erasure on every
closed world, the run would force the value.
    The compiler refuses it. -/
theorem the_plug_fails {Q : Type} (s : Exec Q) (h : ran s = true) :
    ¬ (ran s = true → ∀ S : Config, Closed S → LeastErasure S) :=
  fun plug => the_run_does_not_force_the_value s h (fun S hS =>
  value_from_the_act ⟨S, hS, plug h S hS⟩)
/-- What the plug does deliver: with the act's field beside the run, the
value stands on that field, the run is
    witnessed, and the run still leaves the value free. Run and field are
    two inputs, never one. -/
theorem the_plug_delivers {Q : Type} (d : SubstrateDeed Q) (Z :
ActualZeros) :
    Value Z.zeros ∧ (∃ r : Exec Q, ran r = true) ∧ ¬ (∀ S : Config,
    Closed S → Value S) :=
  (value_from_the_act Z, the_run_witnesses_a_run d.run d.didRun,
  the_run_does_not_force_the_value d.run d.didRun)
end ForceWitness

end PartIVb

/-! # PART IV-c · the time kernel of the monism-fixed codex, verbatim,
with the judge probe -/
section PartIVc
/-
  The time kernel of the monism-fixed codex, carried verbatim: Codex.lean
  of the Master Codex v5.1.0 with the patch of
  APEX-PSP-MONISM-FIXED-01 applied (patched sha256 22d908f7d3d2c96b…),
  its two TimeLocus sections, lines 7875 to 7947,
  followed by the judge probe of this edition. Nothing in the carried
  lines is changed. The carried chart reads real
  parts as the first edition's chart does at resolution one, x = 2·Re s,
  with heights in ℤ, and its counter-world's
  off-line pair sits at x = 0 and 2, Re s = 0 and 1;
  the judge probe states the same test on the offset chart inside the
  open strip, with the conjugate symmetry and the
  empty low band, its off-line zeros at Re s = 1/4 and 3/4. Two names in a
  carried docstring,
  `not_constructible_off_line` and
  `monism_witness_fails_in_counter_world`, are theorems of the patched
  codex outside
  the carried lines; they are not in this seed and nothing here uses them.
-/
namespace TimeLocus

def onLine (p : Int × Int) : Prop := p.1 = 1

/-- A timed world: the locus at each time, and the zeros present at each
time. -/
structure World where
  locus : Nat → (Int × Int → Prop)
  zeros : Nat → (Int × Int → Prop)

/-- TIME PSP premise: prior and posterior locus are the SAME locus, the
line, at every time. -/
def SameLocus (W : World) : Prop := ∀ t p, W.locus t p ↔ onLine p

/-- The claim to be tested: every zero at every time is bound to the
locus. -/
def Bound (W : World) : Prop := ∀ t p, W.zeros t p → W.locus t p

/-- Fold-invariance at every time: the functional equation acts at each
iteration. -/
def FoldInv (W : World) : Prop := ∀ t p, W.zeros t p → W.zeros t (2 -
p.1, p.2)

/-- 51. Given the same locus at all times, being bound IS the line
property at all times. -/
theorem bound_iff_line (W : World) (h : SameLocus W) :
    Bound W ↔ ∀ t p, W.zeros t p → onLine p :=
  (fun b t p z => (h t p).mp (b t p z), fun l t p z => (h t p).mpr (l t p
  z))

/-- 52, the test world. A world where the locus is the same line at every
time, the fold acts at every
    time, the on-line zero recurs at every time, and a mirror pair sits
    off the line at every time. -/
def testWorld : World :=
  (fun _ p => onLine p, fun _ p => p = (1, 0) ∨ p = (0, 5) ∨ p = (2, 5))

theorem test_same_locus : SameLocus testWorld := fun _ _ => Iff.rfl

theorem test_fold_inv : FoldInv testWorld := by
  intro _ p hp
  rcases hp with h | h | h <;> subst h
  · exact Or.inl rfl
  · exact Or.inr (Or.inr rfl)
  · exact Or.inr (Or.inl rfl)

theorem test_not_bound : ¬ Bound testWorld := fun b => by
  have := b 0 (0, 5) (Or.inr (Or.inl rfl)); cases this

/-- 52. Therefore SameLocus and FoldInv, at every time, do not imply
Bound. -/
theorem time_does_not_bind :
    ¬ ∀ W : World, SameLocus W → FoldInv W → Bound W :=
  fun h => test_not_bound (h testWorld test_same_locus test_fold_inv)

end TimeLocus

namespace TimeLocus

/-- Residual Monism read as the Codex states it: one involution serves
both routes. -/
def sigmaGeo (p : Int × Int) : Int × Int := (2 - p.1, p.2)
def sigmaForm (p : Int × Int) : Int × Int := (2 - p.1, p.2)
def OneInvolution : Prop := ∀ p, sigmaGeo p = sigmaForm p

/-- Residual Monism read as timeless: whether the zeros sit on the locus
does not vary in time. -/
def P (W : World) (t : Nat) : Prop := ∀ p, W.zeros t p → onLine p
def Timeless (W : World) : Prop := ∀ t s, P W t ↔ P W s

/-- 53. The seedless reading holds in the counter-world: one involution
by rfl, and timelessness
    because the property "all zeros on the line" is constantly false
    there. Timeless and unbound.
    This is not monism. The monism witness carries its seed, and in the
    counter-world it fails
    (`not_constructible_off_line`,
    `monism_witness_fails_in_counter_world`). -/
theorem seedless_reading_holds_in_counter_world :
    OneInvolution ∧ Timeless testWorld ∧ ¬ Bound testWorld :=
  (fun _ => rfl, fun _ _ => Iff.rfl, test_not_bound)

/-- 54. What timeless monism does do: it makes one instant decide all
time. With the seed, binding
    follows; the seed is "every zero present at the instant is on the
    line". -/
theorem timeless_with_seed_binds (W : World) (h : SameLocus W) (u :
Timeless W) (s0 : P W 0) :
    Bound W :=
```

```
  (bound_iff_line W h).mpr (fun t p z => ((u 0 t).mp s0) p z)

end TimeLocus

namespace Judge
open TimeLocus
/-- The identical locus, the fold at every time, one involution and timelessness together do not exclude a zero off
    the line: the counter-world carries all four. -/
theorem identical_locus_does_not_exclude :
    ¬ ∀ W : World, SameLocus W → FoldInv W → OneInvolution → Timeless W →
      Bound W :=
  fun h => test_not_bound (h testWorld test_same_locus test_fold_inv (fun _ => rfl) (fun _ _ => Iff.rfl))
/-- What does exclude it: on the same locus and timeless, being bound is exactly the seed at time zero. -/
theorem the_seed_is_exactly_what_binds (W : World) (h : SameLocus W) (u : Timeless W) : Bound W ↔ P W 0 :=
  (fun b => fun p z => ((bound_iff_line W h).mp b) 0 p z, fun s0 => timeless_with_seed_binds W h u s0)

/-- The same test on the offset chart, d = 8·Re s − 4, inside the open strip: at every time the line occupied at every
    height |t| ≥ 14, and one mirrored pair at Re s = 1/4 and 3/4 at height 3·10¹² + 1 with its conjugate. -/
def offsetWorld (_ : Nat) (p : RememberedOffered.Point) : Prop :=
  (p.1 = 0 ∧ (14 ≤ p.2 ∨ p.2 ≤ -14)) ∨ p = (2, 3000000000001) ∨ p = (-2, 3000000000001) ∨
    p = (2, -3000000000001) ∨ p = (-2, -3000000000001)
/-- Bound on the offset chart: every zero at every time on the line, the same locus at every time. -/
def BoundO (Z : Nat → RememberedOffered.Config) : Prop := ∀ t p, Z t p → RememberedOffered.onLine p
/-- Timeless on the offset chart: whether every zero sits on the line does not vary in time. -/
def TimelessO (Z : Nat → RememberedOffered.Config) : Prop :=
  ∀ t s, (∀ p, Z t p → RememberedOffered.onLine p) ↔ (∀ p, Z s p → RememberedOffered.onLine p)

/-- THE IDENTICAL LOCUS INSIDE THE STRIP DOES NOT EXCLUDE: on the offset chart, the same locus at every time, the fold
    and conjugation at every time, the empty low band, every zero strictly inside the strip (Re s strictly between 0
    and 1) and timelessness hold together on a world that is not bound; and on every timeless world being bound is
    exactly the seed at time zero. -/
theorem identical_locus_inside_the_strip_does_not_exclude :
    (∀ t, RememberedOffered.Closed (offsetWorld t)) ∧
    (∀ t p, offsetWorld t p → offsetWorld t (p.1, -p.2)) ∧
    (∀ t p, offsetWorld t p → 14 ≤ p.2 ∨ p.2 ≤ -14) ∧
    (∀ t p, offsetWorld t p → -4 < p.1 ∧ p.1 < 4) ∧
    TimelessO offsetWorld ∧ ¬ BoundO offsetWorld ∧
    (∀ Z : Nat → RememberedOffered.Config, TimelessO Z →
      (BoundO Z ↔ ∀ p, Z 0 p → RememberedOffered.onLine p)) := by
  refine (fun t p hp => ?_, fun t p hp => ?_, fun t p hp => ?_, fun t p hp => ?_, fun _ _ => Iff.rfl, ?_, ?_)
  · unfold offsetWorld at hp ⊢
    rcases hp with (h1, h2) | h | h | h | h
    · exact Or.inl (show -p.1 = 0 by omega, h2)
    · subst h; exact Or.inr (Or.inr (Or.inl (by decide)))
    · subst h; exact Or.inr (Or.inl (by decide))
    · subst h; exact Or.inr (Or.inr (Or.inr (Or.inr (by decide))))
    · subst h; exact Or.inr (Or.inr (Or.inr (Or.inl (by decide))))
  · unfold offsetWorld at hp ⊢
    rcases hp with (h1, h2) | h | h | h | h
    · exact Or.inl (h1, by omega)
    · subst h; exact Or.inr (Or.inr (Or.inr (Or.inl (by decide))))
    · subst h; exact Or.inr (Or.inr (Or.inr (Or.inr (by decide))))
    · subst h; exact Or.inr (Or.inl (by decide))
    · subst h; exact Or.inr (Or.inr (Or.inl (by decide)))
  · unfold offsetWorld at hp
    rcases hp with (_, h2) | h | h | h | h
    · exact h2
    all_goals (subst h; decide)
  · unfold offsetWorld at hp
    rcases hp with (h1, _) | h | h | h | h
    · constructor <;> omega
    all_goals (subst h; decide)
  · intro b
    have h : (2 : Int) = 0 := b 0 (2, 3000000000001) (Or.inr (Or.inl rfl))
    exact absurd h (by decide)
  · intro Z u
    exact (fun b => b 0, fun s0 t p z => ((u 0 t).mp s0) p z)
end Judge

end PartIVc

/-! # PART V · the arrow-side test -/
section PartV
/-
  ARROW SIDE TEST · the ζ-specific facts, placed on RA's arrow, tested against an off-line pair.
  Core Lean 4.19.0, no import, no axiom declared, no sorry.
  Chart: x = 2W·Re s at resolution W, height t. The arrow's tail pole s = 0 sits at x = 0, its head pole s = 1 at
  x = 2W, its hinge Re s = 1/2 at x = W. The Euler product lives beyond the head.
  Each fact ζ has and the Davenport–Heilbronn shape lacks, or that is proved for ζ, enters as a predicate at its
  place on the arrow, its source cited:
    ENDS       Build and Edge: no zero at or beyond either pole (Euler product for Re s > 1; the prime number theorem's
               non-vanishing on Re s = 1; mirrored by the functional equation) [cited].
    ZERO-FREE  near the head, and by the fold near the tail: at height t no zero within m t of a pole
               (de la Vallée Poussin; explicit constants Mossinghoff–Trudgian) [cited]; the margin is a parameter.
    REAL PART  every zero up to height T0 on the line (Platt–Trudgian: every zero with 0 < Im s ≤ 3·10¹²) [cited];
               heights are read by their integer part, so chart height T0 = 3·10¹² − 1 reads Im s < 3·10¹².
    FINITELY OFF  only finitely many zeros off the line: stronger than every positive-proportion theorem
               (Selberg, Levinson, Conrey) and every zero-density estimate [cited].
-/
set_option autoImplicit false
namespace ArrowSide

abbrev Pt := Int × Int

structure Chart where
  W : Int
  hW : 0 < W

def onLine (C : Chart) (p : Pt) : Prop := p.1 = C.W
def fold (C : Chart) (p : Pt) : Pt := (2 * C.W - p.1, p.2)
def Closed (C : Chart) (S : Pt → Prop) : Prop := ∀ p, S p → S (fold C p)
def Value (C : Chart) (S : Pt → Prop) : Prop := ∀ p, S p → onLine C p

def ArrowEnds (C : Chart) (S : Pt → Prop) : Prop := ∀ p, S p → 0 < p.1 ∧ p.1 < 2 * C.W
def ZeroFree (C : Chart) (m : Int → Int) (S : Pt → Prop) : Prop :=
  ∀ p, S p → m p.2 < p.1 ∧ p.1 < 2 * C.W - m p.2
def RealPart (C : Chart) (T0 : Int) (S : Pt → Prop) : Prop := ∀ p, S p → p.2 ≤ T0 → onLine C p
def FinitelyOff (C : Chart) (S : Pt → Prop) : Prop := ∃ L : List Pt, ∀ p, S p → ¬ onLine C p → p ∈ L

/-- The ζ-shaped world: a zero on the line at every height, and one mirrored pair at offset d, height H. -/
def zetaShaped (C : Chart) (d H : Int) : Pt → Prop :=
  fun p => p.1 = C.W ∨ p = (C.W - d, H) ∨ p = (C.W + d, H)

theorem shaped_closed (C : Chart) (d H : Int) : Closed C (zetaShaped C d H) := by
  intro p hp
  unfold zetaShaped at hp ⊢
  rcases hp with h | h | h
  · left; show 2 * C.W - p.1 = C.W; omega
  · subst h; right; right; exact Prod.ext (show 2 * C.W - (C.W - d) = C.W + d by omega) rfl
  · subst h; right; left; exact Prod.ext (show 2 * C.W - (C.W + d) = C.W - d by omega) rfl

theorem shaped_ends (C : Chart) (d H : Int) (hd : 0 < d) (hdW : d < C.W) : ArrowEnds C (zetaShaped C d H) := by
  have hW := C.hW
  intro p hp
  unfold zetaShaped at hp
  rcases hp with h | h | h
  · constructor <;> omega
  · subst h; show 0 < C.W - d ∧ C.W - d < 2 * C.W; constructor <;> omega
  · subst h; show 0 < C.W + d ∧ C.W + d < 2 * C.W; constructor <;> omega

theorem shaped_zero_free (C : Chart) (m : Int → Int) (hm : ∀ t, m t < C.W) (d H : Int) (hmH : m H < C.W - d)
    (hd : 0 < d) : ZeroFree C m (zetaShaped C d H) := by
  intro p hp
  unfold zetaShaped at hp
  rcases hp with h | h | h
  · have := hm p.2; constructor <;> omega
  · subst h; show m H < C.W - d ∧ C.W - d < 2 * C.W - m H; constructor <;> omega
  · subst h; show m H < C.W + d ∧ C.W + d < 2 * C.W - m H; constructor <;> omega

theorem shaped_real_part (C : Chart) (T0 d H : Int) (hH : T0 < H) : RealPart C T0 (zetaShaped C d H) := by
  intro p hp ht
  unfold zetaShaped at hp
  rcases hp with h | h | h
  · exact h
  · subst h; have : H ≤ T0 := ht; exfalso; omega
  · subst h; have : H ≤ T0 := ht; exfalso; omega

theorem shaped_finitely_off (C : Chart) (d H : Int) : FinitelyOff C (zetaShaped C d H) := by
  refine ([(C.W - d, H), (C.W + d, H)], fun p hp hoff => ?_)
  unfold zetaShaped at hp
  rcases hp with h | h | h
  · exact absurd h hoff
  · subst h; exact List.Mem.head _
  · subst h; exact List.Mem.tail _ (List.Mem.head _)

theorem shaped_lacks_value (C : Chart) (d H : Int) (hd : 0 < d) : ¬ Value C (zetaShaped C d H) := by
  intro hv
  have h2 : C.W - d = C.W := hv (C.W - d, H) (Or.inr (Or.inl rfl))
  omega

/-- THE ARROW SIDE TEST. Every ζ-specific fact placed on the arrow (ends, zero-free region, real part to T0,
    finitely many off the line) holds on a world with a mirrored pair off the line above T0, so together they do
    not force the value. -/
```

```
theorem arrow_side_test (C : Chart) (m : Int → Int) (hm : ∀ t, m t < C.W)
(T0 d H : Int)
    (hd : 0 < d) (hdW : d < C.W) (hH : T0 < H) (hmH : m H < C.W - d) :
    Closed C (zetaShaped C d H) ∧ ArrowEnds C (zetaShaped C d H) ∧
    ZeroFree C m (zetaShaped C d H) ∧
    RealPart C T0 (zetaShaped C d H) ∧ FinitelyOff C (zetaShaped C d H) ∧
    ¬ Value C (zetaShaped C d H) :=
  (shaped_closed C d H, shaped_ends C d H hd hdW, shaped_zero_free C m hm
  d H hmH hd,
   shaped_real_part C T0 d H hH, shaped_finitely_off C d H,
   shaped_lacks_value C d H hd)

/-- THE FACTS ARE ζ-SPECIFIC AT THE ENDS: a zero beyond the head, the
Davenport–Heilbronn shape, breaks ENDS. -/
theorem beyond_the_head_breaks_ends (C : Chart) (x t : Int) (hx : 2 * C.W
≤ x) :
    ¬ ArrowEnds C (fun p => p = (x, t)) := by
  intro h
  have h2 : x < 2 * C.W := (h (x, t) rfl).2
  omega

/-- AT ζ'S OWN NUMBERS: resolution W = 4, so x = 8·Re s; the pair at Re s
= 1/4 and 3/4 (x = 2 and 6); the
    zero-free margin below one unit of 1/8 at every height where zeros
    occur (c/log t < 0.07 for t ≥ 14); the real
    part to chart height T0 = 3·10¹² − 1, every zero with Im s < 3·10¹²;
    the pair two units of height above it. -/
def zetaChart : Chart := (4, by decide)
theorem at_zetas_numbers :
    Closed zetaChart (zetaShaped zetaChart 2 3000000000001) ∧
    ArrowEnds zetaChart (zetaShaped zetaChart 2 3000000000001) ∧
    ZeroFree zetaChart (fun _ => 0) (zetaShaped zetaChart 2
    3000000000001) ∧
    RealPart zetaChart 2999999999999 (zetaShaped zetaChart 2
    3000000000001) ∧
    FinitelyOff zetaChart (zetaShaped zetaChart 2 3000000000001) ∧
    ¬ Value zetaChart (zetaShaped zetaChart 2 3000000000001) :=
  arrow_side_test zetaChart (fun _ => 0) (fun _ => show (0 : Int) < 4 by
  decide) 2999999999999 2 3000000000001
    (by decide) (by decide) (by decide) (by decide)

/-! ## The faithful model: the facts of ζ with its two symmetries and its
empty low band -/

/-- Conjugate symmetry: the zero set is closed under t ↦ −t. -/
def ConjClosed (S : Pt → Prop) : Prop := ∀ p, S p → S (p.1, -p.2)
/-- No zero below height 14 in absolute value: ζ's first zero lies at
height 14.13… [cited]; heights read by their
    integer part, chart height 14 holds Im s in [14, 15). -/
def NoLowZeros (S : Pt → Prop) : Prop := ∀ p, S p → 14 ≤ p.2 ∨ p.2 ≤ -14
/-- The verification in both half-planes: every zero at chart height |t| ≤
T0 is on the line; at T0 = 3·10¹² − 1,
    every zero with |Im s| < 3·10¹² [Platt–Trudgian, cited]. -/
def RealPartAbs (C : Chart) (T0 : Int) (S : Pt → Prop) : Prop := ∀ p, S p
→ p.2 ≤ T0 → -T0 ≤ p.2 → onLine C p
/-- The zero-free margin, in units of 1/(2W): the whole width below |t| =
14, where ζ has no zero, and no unit at
    every height |t| ≥ 14. At every width it holds of ζ's reading, since
    it adds nothing to the ends there and the
    reader's rule keeps every zero of the open strip inside the open
    chart strip; at the widths W ≤ 7 it is the
    region at full strength, since c/log t < 0.07 < 1/(2W).
    `faithful_every_margin` carries the region at every
    width, one unit short for the rounding. -/
def margin14 (C : Chart) : Int → Int := fun t => if 14 ≤ t ∨ t ≤ -14 then
0 else C.W
/-- The faithful ζ-shaped world: a zero on the line at every height |t| ≥
14, and one mirrored pair at offset d at
    heights H and −H. -/
def zetaFaithful (C : Chart) (d H : Int) : Pt → Prop := fun p =>
  (p.1 = C.W ∧ (14 ≤ p.2 ∨ p.2 ≤ -14)) ∨ p = (C.W - d, H) ∨ p = (C.W + d,
  H) ∨
    p = (C.W - d, -H) ∨ p = (C.W + d, -H)

theorem faithful_arrow_side_test (C : Chart) (T0 d H : Int) (hd : 0 < d)
(hdW : d < C.W) (hH : T0 < H)
    (h14 : 14 ≤ H) :
    Closed C (zetaFaithful C d H) ∧ ConjClosed (zetaFaithful C d H) ∧
    ArrowEnds C (zetaFaithful C d H) ∧
    ZeroFree C (margin14 C) (zetaFaithful C d H) ∧ NoLowZeros
    (zetaFaithful C d H) ∧
    RealPartAbs C T0 (zetaFaithful C d H) ∧ FinitelyOff C (zetaFaithful C
    d H) ∧
    ¬ Value C (zetaFaithful C d H) := by
  have hW := C.hW
  refine (?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_)
  · intro p hp
    unfold zetaFaithful at hp ⊢
    rcases hp with (h1, h2) | h | h | h | h
    · left; exact (show 2 * C.W - p.1 = C.W by omega, h2)
    · subst h; right; right; left; exact Prod.ext (show 2 * C.W - (C.W -
    d) = C.W + d by omega) rfl
    · subst h; right; left; exact Prod.ext (show 2 * C.W - (C.W + d) =
    C.W - d by omega) rfl
    · subst h; right; right; right; right; exact Prod.ext (show 2 * C.W -
    (C.W - d) = C.W + d by omega) rfl
    · subst h; right; right; right; left; exact Prod.ext (show 2 * C.W -
    (C.W + d) = C.W - d by omega) rfl
  · intro p hp
    unfold zetaFaithful at hp ⊢
    rcases hp with (h1, h2) | h | h | h | h
    · left; exact (h1, show 14 ≤ -p.2 ∨ -p.2 ≤ -14 by omega)
    · subst h; right; right; right; left; rfl
    · subst h; right; right; right; right; rfl
    · subst h; right; right; left; exact Prod.ext rfl (show -(-H) = H by omega)
    · subst h; right; right; left; exact Prod.ext rfl (show -(-H) = H by
    omega)
  · intro p hp
    unfold zetaFaithful at hp
    rcases hp with (h1, _) | h | h | h | h
    · constructor <;> omega
    all_goals (subst h; constructor <;> (show (_ : Int) < _; omega))
  · intro p hp
    unfold zetaFaithful at hp
    unfold margin14
    rcases hp with (h1, h2) | h | h | h | h
    · rw [if_pos h2]; constructor <;> omega
    · subst h; rw [if_pos (Or.inl h14)]; show (0 : Int) < C.W - d ∧ C.W -
    d < 2 * C.W - 0; constructor <;> omega
    · subst h; rw [if_pos (Or.inl h14)]; show (0 : Int) < C.W + d ∧ C.W +
    d < 2 * C.W - 0; constructor <;> omega
    · subst h; rw [if_pos (Or.inr (show -H ≤ -14 by omega))]
      show (0 : Int) < C.W - d ∧ C.W - d < 2 * C.W - 0; constructor <;>
      omega
    · subst h; rw [if_pos (Or.inr (show -H ≤ -14 by omega))]
      show (0 : Int) < C.W + d ∧ C.W + d < 2 * C.W - 0; constructor <;>
      omega
  · intro p hp
    unfold zetaFaithful at hp
    rcases hp with (_, h2) | h | h | h | h
    · exact h2
    · subst h; exact Or.inl h14
    · subst h; exact Or.inl h14
    · subst h; exact Or.inr (show -H ≤ -14 by omega)
    · subst h; exact Or.inr (show -H ≤ -14 by omega)
  · intro p hp ht1 ht2
    unfold zetaFaithful at hp
    rcases hp with (h1, _) | h | h | h | h
    · exact h1
    · subst h; have : H ≤ T0 := ht1; exfalso; omega
    · subst h; have : H ≤ T0 := ht1; exfalso; omega
    · subst h; have : -T0 ≤ -H := ht2; exfalso; omega
    · subst h; have : -T0 ≤ -H := ht2; exfalso; omega
  · refine ([(C.W - d, H), (C.W + d, H), (C.W - d, -H), (C.W + d, -H)],
  fun p hp hoff => ?_)
    unfold zetaFaithful at hp
    rcases hp with (h1, _) | h | h | h | h
    · exact absurd h1 hoff
    · subst h; exact List.Mem.head _
    · subst h; exact List.Mem.tail _ (List.Mem.head _)
    · subst h; exact List.Mem.tail _ (List.Mem.tail _ (List.Mem.head _))
    · subst h; exact List.Mem.tail _ (List.Mem.tail _ (List.Mem.tail _
    (List.Mem.head _)))
  · intro hv
    have h2 : C.W - d = C.W := hv (C.W - d, H) (Or.inr (Or.inl rfl))
    omega

/-- AT ζ'S OWN NUMBERS, FAITHFULLY: W = 4, so x = 8·Re s; the pair at Re s
= 1/4 and 3/4 at height 3·10¹² + 1 and
    its conjugate at −(3·10¹² + 1); the line occupied at every |t| ≥ 14;
    the verification, every zero with
    |Im s| < 3·10¹², at chart height 3·10¹² − 1 in both half-planes. -/
theorem at_zetas_numbers_faithful :
    Closed zetaChart (zetaFaithful zetaChart 2 3000000000001) ∧
    ConjClosed (zetaFaithful zetaChart 2 3000000000001) ∧
    ArrowEnds zetaChart (zetaFaithful zetaChart 2 3000000000001) ∧
    ZeroFree zetaChart (margin14 zetaChart) (zetaFaithful zetaChart 2
    3000000000001) ∧
    NoLowZeros (zetaFaithful zetaChart 2 3000000000001) ∧
    RealPartAbs zetaChart 2999999999999 (zetaFaithful zetaChart 2
    3000000000001) ∧
    FinitelyOff zetaChart (zetaFaithful zetaChart 2 3000000000001) ∧
    ¬ Value zetaChart (zetaFaithful zetaChart 2 3000000000001) :=
  faithful_arrow_side_test zetaChart 2999999999999 2 3000000000001 (by
  decide) (by decide) (by decide) (by decide)

/-- THE FAITHFUL MODEL UNDER EVERY ZERO-FREE MARGIN: at every chart width
and for every margin that, at the heights
    where zeros occur, stays short of the line and leaves the off-line
    quadruple outside it at ±H, the facts hold on
    the faithful world and the value fails. ζ's known region, c/log|t| in
    Re s with c/log 14 < 0.07 and
    c/log(3·10¹²) < 0.0065, read in units of 1/(2W), is such a margin at
    every width for the pair at Re s = 1/4 and
    3/4 [Mossinghoff–Trudgian 2015, cited]. -/
theorem faithful_every_margin (C : Chart) (m : Int → Int) (T0 d H : Int)
(hd : 0 < d) (hdW : d < C.W)
    (hH : T0 < H) (h14 : 14 ≤ H) (hm : ∀ t, (14 ≤ t ∨ t ≤ -14) → m t <
    C.W)
    (hmH : m H < C.W - d) (hmH' : m (-H) < C.W - d) :
    Closed C (zetaFaithful C d H) ∧ ConjClosed (zetaFaithful C d H) ∧
    ArrowEnds C (zetaFaithful C d H) ∧
    ZeroFree C m (zetaFaithful C d H) ∧ NoLowZeros (zetaFaithful C d H) ∧
    RealPartAbs C T0 (zetaFaithful C d H) ∧ FinitelyOff C (zetaFaithful C
    d H) ∧
    ¬ Value C (zetaFaithful C d H) := by
  have base := faithful_arrow_side_test C T0 d H hd hdW hH h14
  refine (base.1, base.2.1, base.2.2.1, ?_, base.2.2.2.2.1,
  base.2.2.2.2.2.1, base.2.2.2.2.2.2.1,
    base.2.2.2.2.2.2.2)
  intro p hp
  unfold zetaFaithful at hp
  rcases hp with (h1, h2) | h | h | h | h
  · have := hm p.2 h2; constructor <;> omega
  · subst h; show m H < C.W - d ∧ C.W - d < 2 * C.W - m H; constructor
  <;> omega
```

```
  · subst h; show m H < C.W + d ∧ C.W + d < 2 * C.W - m H; constructor
  <;> omega
  · subst h; show m (-H) < C.W - d ∧ C.W - d < 2 * C.W - m (-H);
  constructor <;> omega
  · subst h; show m (-H) < C.W + d ∧ C.W + d < 2 * C.W - m (-H);
  constructor <;> omega

/-- AT WIDTH ONE THE ENDS ARE THE VALUE: the open strip holds only the
line, so the placed ends force the value there,
    as at the first edition's resolution one; the faithful model needs a
    width of at least two. -/
theorem at_width_one_the_ends_are_the_value (S : Pt → Prop) :
    ArrowEnds (1, by decide) S ↔ Value (1, by decide) S := by
  constructor
  · intro h p hp
    have := h p hp
    show p.1 = 1
    have h1 : 0 < p.1 ∧ p.1 < 2 * 1 := this
    omega
  · intro h p hp
    have h1 : p.1 = 1 := h p hp
    show 0 < p.1 ∧ p.1 < 2 * 1
    omega

/-- AN OFF-LINE ZERO COMES IN FOUR: in a world closed under the
reflection and under conjugation, with no zero below
    height 14, a zero off the line brings its mirror, its conjugate and
    its conjugate's mirror, four distinct points:
    one mirrored pair and its conjugate. -/
theorem an_off_line_zero_comes_in_four (C : Chart) (S : Pt → Prop) (hc :
Closed C S) (hk : ConjClosed S)
    (hl : NoLowZeros S) (p : Pt) (hp : S p) (hoff : ¬ onLine C p) :
    S (fold C p) ∧ S (p.1, -p.2) ∧ S (fold C (p.1, -p.2)) ∧
    fold C p ≠ p ∧ (p.1, -p.2) ≠ p ∧ fold C (p.1, -p.2) ≠ p ∧
    (p.1, -p.2) ≠ fold C p ∧ fold C (p.1, -p.2) ≠ fold C p ∧ fold C (p.1,
    -p.2) ≠ (p.1, -p.2) := by
  have h14 := hl p hp
  refine (hc p hp, hk p hp, hc _ (hk p hp), ?_)
  obtain (x, t) := p
  have hx : x ≠ C.W := hoff
  have ht : 14 ≤ t ∨ t ≤ -14 := h14
  refine (fun e => ?_, fun e => ?_, fun e => ?_, fun e => ?_, fun e =>
  ?_, fun e => ?_)
  · have h1 : 2 * C.W - x = x := congrArg Prod.fst e
    omega
  · have h2 : -t = t := congrArg Prod.snd e
    omega
  · have h2 : -t = t := congrArg Prod.snd e
    omega
  · have h2 : -t = t := congrArg Prod.snd e
    omega
  · have h2 : -t = t := congrArg Prod.snd e
    omega
  · have h1 : 2 * C.W - x = x := congrArg Prod.fst e
    omega

end ArrowSide


end PartV

/-! # PART V-b · the charts are one -/
section PartVb
/-
  THE CHARTS ARE ONE. The seed reads the critical line on five charts:
  Part II's at x = 2n·Re s for every resolution n
  (x = 2·Re s at resolution one, x = 4·Re s at resolution two), heights
  in ℕ; Part IV's at x = 4·Re s; Part IV-c's at
  x = 2·Re s, heights in ℤ; Part V's at x = 2W·Re s for every width W;
  and the offset chart of Parts III and IV-b,
  d = 2n·Re s − n at resolution n, whose line d = 0 and fold d ↦ −d are
  the same at every resolution. Each maps into
  the offset chart at a definite resolution by an equivariant map, one
  that carries the chart's fold to the offset
  fold and the chart's line to the offset line: the maps of Parts II at
  resolutions one and two, IV, IV-c and V at
  width four land at resolution four, where one real part read on any of
  them lands at one offset point; the maps
  at resolution n and width W land at resolution n and W. Every map keeps
  the chart's height and scales the offset
  from the line by a positive factor; the maps out of the charts with
  heights in ℤ commute with conjugation, and
  the first edition's chart reads the upper half-plane. Transport carries
  reflection-closure and the value across. Every act type of the seed is
  an act of the offset chart at the image of
  its zero set; the root read at the zeros on Part II's chart is the root
  read twice at the image; both constructed
  roots, Part I's and Part II's, feed the closing theorems. The zero set
  of ζ is read on the offset chart at every
  resolution by the reader's rule, and the act supplies least erasure at
  every resolution.
-/
set_option autoImplicit false
namespace Charts

/-- The image of a configuration under a chart map into the offset chart.
-/
def img {α : Type} (φ : α → RememberedOffered.Point) (S : α → Prop) :
RememberedOffered.Config :=
  fun q => ∃ p, S p ∧ φ p = q

/-- A chart map is equivariant when it carries the chart's fold to the
offset fold and the chart's line to the
    offset line. -/
structure Equivariant {α : Type} (fold : α → α) (onLine : α → Prop) (φ : α
→ RememberedOffered.Point) : Prop where
  comm : ∀ p, φ (fold p) = RememberedOffered.fold (φ p)
  line : ∀ p, RememberedOffered.onLine (φ p) ↔ onLine p

/-- TRANSPORT: an equivariant map carries reflection-closure across, and
the value both ways. -/
theorem transport {α : Type} {fold : α → α} {onLine : α → Prop} {φ : α →
RememberedOffered.Point}
    (E : Equivariant fold onLine φ) (S : α → Prop) :
    ((∀ p, S p → S (fold p)) → RememberedOffered.Closed (img φ S)) ∧
    ((∀ p, S p → onLine p) ↔ RememberedOffered.Value (img φ S)) := by
  refine (fun hS q (p, hp, e) => (fold p, hS p hp, by rw [E.comm, e]),
    (fun hv q (p, hp, e) => ?_, fun hv p hp => (E.line p).mp (hv _ (p,
    hp, rfl))))
  rw [← e]; exact (E.line p).mpr (hv p hp)

/-- Part IV's chart, x = 4·Re s, into the offset chart d = 8·Re s − 4. -/
def φH (p : Hinge.Pt) : RememberedOffered.Point := (2 * p.1 - 4, p.2)
/-- Part V's chart at ζ's resolution, x = 8·Re s, into the offset chart.
-/
def φA (p : ArrowSide.Pt) : RememberedOffered.Point := (p.1 - 4, p.2)
/-- Part II's chart, x = 2·Re s with heights in ℕ, into the offset chart.
-/
def φP (z : PrimeFreedom.Pt) : RememberedOffered.Point := (4 * z.x - 4,
(z.t : Int))
/-- Part II's chart at resolution two, x = 4·Re s, into the offset chart.
-/
def φP₂ (z : PrimeFreedom.Pt) : RememberedOffered.Point := (2 * z.x - 4,
(z.t : Int))
/-- Part V's chart at every width W, x = 2W·Re s, into the offset chart
at resolution W: d = x − W. -/
def φW (C : ArrowSide.Chart) (p : ArrowSide.Pt) : RememberedOffered.Point
:= (p.1 - C.W, p.2)
/-- Part II's chart at every resolution n, x = 2n·Re s with heights in ℕ,
into the offset chart at resolution n. -/
def φPn (n : Int) (z : PrimeFreedom.Pt) : RememberedOffered.Point := (z.x
- n, (z.t : Int))
/-- Part IV-c's chart, x = 2·Re s with heights in ℤ, into the offset
chart. -/
def φT (p : Int × Int) : RememberedOffered.Point := (4 * p.1 - 4, p.2)
/-- Part IV-b's chart is the offset chart: the identity. -/
def φF (p : ForceWitness.Point) : RememberedOffered.Point := p

theorem eqH : Equivariant Hinge.fold Hinge.onLine φH :=
  (fun p => Prod.ext (show 2 * (0 + 4 - p.1) - 4 = -(2 * p.1 - 4) by
  omega) rfl,
    fun p => show 2 * p.1 - 4 = 0 ↔ p.1 = 2 by omega)
theorem eqA : Equivariant (ArrowSide.fold ArrowSide.zetaChart)
(ArrowSide.onLine ArrowSide.zetaChart) φA :=
  (fun p => Prod.ext (show 2 * 4 - p.1 - 4 = -(p.1 - 4) by omega) rfl,
    fun p => show p.1 - 4 = 0 ↔ p.1 = 4 by omega)
theorem eqP : Equivariant PrimeFreedom.fold PrimeFreedom.onLine φP :=
  (fun z => Prod.ext (show 4 * (2 - z.x) - 4 = -(4 * z.x - 4) by omega)
  rfl,
    fun z => show 4 * z.x - 4 = 0 ↔ z.x = 1 by omega)
theorem eqP₂ : Equivariant PrimeFreedom.fold₂ PrimeFreedom.onLine₂ φP₂ :=
  (fun z => Prod.ext (show 2 * (4 - z.x) - 4 = -(2 * z.x - 4) by omega)
  rfl,
    fun z => show 2 * z.x - 4 = 0 ↔ z.x = 2 by omega)
theorem eqW (C : ArrowSide.Chart) : Equivariant (ArrowSide.fold C)
(ArrowSide.onLine C) (φW C) :=
  (fun p => Prod.ext (show 2 * C.W - p.1 - C.W = -(p.1 - C.W) by omega)
  rfl,
    fun p => show p.1 - C.W = 0 ↔ p.1 = C.W by omega)
theorem eqPn (n : Int) : Equivariant (PrimeFreedom.foldAt n) (fun z =>
z.x = n) (φPn n) :=
  (fun z => Prod.ext (show 2 * n - z.x - n = -(z.x - n) by omega) rfl,
    fun z => show z.x - n = 0 ↔ z.x = n by omega)
theorem eqT : Equivariant TimeLocus.sigmaGeo TimeLocus.onLine φT :=
  (fun p => Prod.ext (show 4 * (2 - p.1) - 4 = -(4 * p.1 - 4) by omega)
  rfl,
    fun p => show 4 * p.1 - 4 = 0 ↔ p.1 = 1 by omega)
theorem eqF : Equivariant ForceWitness.fold ForceWitness.onLine φF :=
  (fun _ => rfl, fun _ => Iff.rfl)

/-- EVERY MAP KEEPS HEIGHT AND SCALES THE OFFSET from the line by a
positive factor. -/
theorem the_maps_keep_height_and_scale :
    (∀ p : Hinge.Pt, φH p = (2 * (p.1 - 2), p.2)) ∧
    (∀ p : ArrowSide.Pt, φA p = (1 * (p.1 - 4), p.2)) ∧
    (∀ (C : ArrowSide.Chart) (p : ArrowSide.Pt), φW C p = (1 * (p.1 -
    C.W), p.2)) ∧
    (∀ z : PrimeFreedom.Pt, φP z = (4 * (z.x - 1), (z.t : Int))) ∧
    (∀ z : PrimeFreedom.Pt, φP₂ z = (2 * (z.x - 2), (z.t : Int))) ∧
    (∀ (n : Int) (z : PrimeFreedom.Pt), φPn n z = (1 * (z.x - n), (z.t :
    Int))) ∧
    (∀ p : Int × Int, φT p = (4 * (p.1 - 1), p.2)) ∧
    (∀ p : ForceWitness.Point, φF p = (1 * (p.1 - 0), p.2)) :=
  (fun p => Prod.ext (show 2 * p.1 - 4 = 2 * (p.1 - 2) by omega) rfl,
   fun p => Prod.ext (show p.1 - 4 = 1 * (p.1 - 4) by omega) rfl,
   fun C p => Prod.ext (show p.1 - C.W = 1 * (p.1 - C.W) by omega) rfl,
   fun z => Prod.ext (show 4 * z.x - 4 = 4 * (z.x - 1) by omega) rfl,
   fun z => Prod.ext (show 2 * z.x - 4 = 2 * (z.x - 2) by omega) rfl,
   fun n z => Prod.ext (show z.x - n = 1 * (z.x - n) by omega) rfl,
   fun p => Prod.ext (show 4 * p.1 - 4 = 4 * (p.1 - 1) by omega) rfl,
   fun p => Prod.ext (show p.1 = 1 * (p.1 - 0) by omega) rfl)

/-- The first edition's act, carried to the offset chart at the image of
its zero set. -/
def actP (A : PrimeFreedom.ActualZeros) : RememberedOffered.ActualZeros :=
  (img φP A.zeros, (transport eqP A.zeros).1 A.fold_closed,
```

```
    (RememberedOffered.twins_on_every_world _).mpr ((transport eqP
    A.zeros).2.mp (PrimeFreedom.rh_from_the_act A)))

/-- The first edition's act at every resolution n, carried to the offset
chart at the image of its zero set. -/
def actPn (n : Int) (A : PrimeFreedom.ActualZerosAt n) :
RememberedOffered.ActualZeros :=
  (img (φPn n) A.zeros, (transport (eqPn n) A.zeros).1 A.closed,
    (RememberedOffered.twins_on_every_world _).mpr ((transport (eqPn n)
    A.zeros).2.mp
      (fun z hz => (PrimeFreedom.foldAt_fixed_iff n z).mp
      (PrimeFreedom.rh_from_the_act_at n A z hz))))
/-- Part IV's act, carried to the offset chart at the image of its zero
set. -/
def actH (A : Hinge.ActualZeros) : RememberedOffered.ActualZeros :=
  (img φH A.zeros, (transport eqH A.zeros).1 A.closed,
    (RememberedOffered.twins_on_every_world _).mpr ((transport eqH
    A.zeros).2.mp (Hinge.value_from_the_act A)))
/-- Part IV-b's act is an act of the offset chart as it stands. -/
def actF (A : ForceWitness.ActualZeros) : RememberedOffered.ActualZeros
:= (A.zeros, A.closed, A.supply)

/-- The constructed root of Part I, as the root the closing theorems of
Part III take. -/
def rootOf {R : Prop} (G : TOE0.SelfGrounding R) :
RememberedOffered.SelfGrounding R := (G.Act, G.anAct, G.instances)
/-- Part II's self-grounding root, as the root the closing theorems of
Part III take. -/
def rootP {R : Prop} (G : PrimeFreedom.SelfGrounding R) :
RememberedOffered.SelfGrounding R :=
  (G.Act, G.anAct, G.instances)

/-- THE CHARTS ARE ONE. Every plane chart of the seed maps equivariantly
into the offset chart, at every resolution and every
    width; the value of a configuration is the value of its image on
    every chart; every act type of the seed is an act
    of the offset chart at the image of its zero set; the root read at
    the zeros on Part II's chart is the root read
    twice at the image; and both constructed roots feed the closing
    theorems of Part III: Part I's forces no value,
    and the deed of either is the supply at a zero set exactly where the
    value holds. -/
theorem the_charts_are_one :
    Equivariant Hinge.fold Hinge.onLine φH ∧
    Equivariant (ArrowSide.fold ArrowSide.zetaChart) (ArrowSide.onLine
    ArrowSide.zetaChart) φA ∧
    Equivariant PrimeFreedom.fold PrimeFreedom.onLine φP ∧
    Equivariant PrimeFreedom.fold₂ PrimeFreedom.onLine₂ φP₂ ∧
    (∀ S : Hinge.Pt → Prop, Hinge.Value S ↔ RememberedOffered.Value (img
    φH S)) ∧
    (∀ S : ArrowSide.Pt → Prop, ArrowSide.Value ArrowSide.zetaChart S ↔
    RememberedOffered.Value (img φA S)) ∧
    (∀ Z : PrimeFreedom.World, PrimeFreedom.RH Z ↔
    RememberedOffered.Value (img φP Z)) ∧
    (∀ A : PrimeFreedom.ActualZeros, ∃ Z : RememberedOffered.ActualZeros,
    Z.zeros = img φP A.zeros) ∧
    (∀ Z : PrimeFreedom.World, PrimeFreedom.RootAtZeros Z ↔
    RememberedOffered.RootReadTwice (img φP Z)) ∧
    (¬ ∀ S : RememberedOffered.Config, RememberedOffered.Closed S →
    TOE0.RA₀ → RememberedOffered.Value S) ∧
    (∀ S : RememberedOffered.Config, RememberedOffered.Closed S →
      ((TOE0.ra₀.Act → ∃ Z : RememberedOffered.ActualZeros, Z.zeros = S) ↔
      RememberedOffered.Value S)) ∧
    (∀ C : ArrowSide.Chart, Equivariant (ArrowSide.fold C)
    (ArrowSide.onLine C) (φW C)) ∧
    (∀ n : Int, Equivariant (PrimeFreedom.foldAt n) (fun z => z.x = n)
    (φPn n)) ∧
    Equivariant TimeLocus.sigmaGeo TimeLocus.onLine φT ∧
    Equivariant ForceWitness.fold ForceWitness.onLine φF ∧
    (∀ (C : ArrowSide.Chart) (S : ArrowSide.Pt → Prop),
      ArrowSide.Value C S ↔ RememberedOffered.Value (img (φW C) S)) ∧
    (∀ (n : Int) (A : PrimeFreedom.ActualZerosAt n),
      ∃ Z : RememberedOffered.ActualZeros, Z.zeros = img (φPn n) A.zeros)
      ∧
    (∀ A : Hinge.ActualZeros, ∃ Z : RememberedOffered.ActualZeros,
    Z.zeros = img φH A.zeros) ∧
    (∀ A : ForceWitness.ActualZeros, ∃ Z : RememberedOffered.ActualZeros,
    Z.zeros = A.zeros) ∧
    (∀ (R : Prop) (G : PrimeFreedom.SelfGrounding R) (S :
    RememberedOffered.Config), RememberedOffered.Closed S →
      (((rootP G).Act → ∃ Z : RememberedOffered.ActualZeros, Z.zeros = S)
      ↔ RememberedOffered.Value S)) :=
  (eqH, eqA, eqP, eqP₂,
    fun S => (transport eqH S).2, fun S => (transport eqA S).2, fun Z =>
    (transport eqP Z).2,
    fun A => (actP A, rfl),
    fun Z => ((PrimeFreedom.root_at_zeros_is_the_hypothesis Z).trans
    (transport eqP Z).2).trans
      (RememberedOffered.the_root_read_twice_is_the_value _).symm,
    RememberedOffered.the_floor_alone_decides_no_value (rootOf TOE0.ra₀),
    fun S hS => (RememberedOffered.the_last_act (rootOf TOE0.ra₀) S hS).1,
    eqW, eqPn, eqT, eqF, fun C S => (transport (eqW C) S).2,
    fun n A => (actPn n A, rfl), fun A => (actH A, rfl), fun A => (actF A,
    rfl),
    fun _ G S hS => (RememberedOffered.the_last_act (rootP G) S hS).1)

/-- The model pair of the zero side: Re s = 3/4 at height 3·10¹² + 1 on
the offset chart at resolution four (d = 2),
    with its mirror at Re s = 1/4 (d = −2). -/
def thePair : RememberedOffered.Point := (2, 3000000000001)

/-- THE ONE PAIR, READ BY EVERY INSTRUMENT, on the charts that hold it.
On the offset chart it is closed, off the line, of one
    record with its registered point, without the root read twice, and
    every report of it reads no act; its preimage
    on Part IV's chart is centred on the line; its preimage on Part V's
    chart lies in the faithful ζ-shaped world,
    which satisfies every fact of ζ placed on the arrow. -/
theorem the_pair_read_by_every_instrument :
    RememberedOffered.Closed (RememberedOffered.pairWorld thePair) ∧
    ¬ RememberedOffered.Value (RememberedOffered.pairWorld thePair) ∧
    RememberedOffered.SameRecord (RememberedOffered.registered thePair)
    (RememberedOffered.pairWorld thePair) ∧
    ¬ RememberedOffered.RootReadTwice (RememberedOffered.pairWorld
    thePair) ∧
    (¬ ∃ Z : RememberedOffered.ActualZeros, ∀ p ∈ [thePair,
    RememberedOffered.fold thePair], Z.zeros p) ∧
    (φH (3, 3000000000001) = thePair ∧ φH (1, 3000000000001) =
    RememberedOffered.fold thePair ∧
      Hinge.sumX [(1, 3000000000001), (3, 3000000000001)] = 2 * 2) ∧
    (φA (6, 3000000000001) = thePair ∧ φA (2, 3000000000001) =
    RememberedOffered.fold thePair ∧
      ArrowSide.zetaFaithful ArrowSide.zetaChart 2 3000000000001 (6,
      3000000000001) ∧
      ArrowSide.zetaFaithful ArrowSide.zetaChart 2 3000000000001 (2,
      3000000000001)) := by
  have off : ¬ RememberedOffered.onLine thePair := fun h => by
    have h2 : (2 : Int) = 0 := h
    exact absurd h2 (by decide)
  have nv := RememberedOffered.pair_world_lacks_value thePair off
  refine ⟨RememberedOffered.pair_world_closed thePair, nv,
  RememberedOffered.one_record thePair,
    RememberedOffered.the_line_no_contradicts_the_root_read_twice _ nv,
    (RememberedOffered.row_four_from_the_zero_side
    (RememberedOffered.pairWorld thePair)).2.1
      [thePair, RememberedOffered.fold thePair] (thePair, List.Mem.head
      _, off),
    (by decide, by decide, by decide), (by decide, by decide, ?_, ?_)⟩
  · exact Or.inr (Or.inr (Or.inl (by decide)))
  · exact Or.inr (Or.inl (by decide))

/-- THE BLOCKS AT THE MODEL PAIR: at the pair at height 3·10¹² + 1, which
every fact of ζ placed on the arrow admits,
    the registered world and the pair world leave one record; both
    readings of the halt hold on the first and fail on
    the second; no reading of the record decides the value; every deed is
    the supply at the first and not at the
    second; and the loop closes on the first and stays open on the
    second. -/
theorem the_blocks_at_the_one_pair :
    RememberedOffered.SameRecord (RememberedOffered.registered thePair)
    (RememberedOffered.pairWorld thePair) ∧
    (RememberedOffered.ZeroReports (RememberedOffered.registered thePair)
    ∧
      RememberedOffered.PrimeBound (RememberedOffered.registered
      thePair)) ∧
    (¬ RememberedOffered.ZeroReports (RememberedOffered.pairWorld
    thePair) ∧
      ¬ RememberedOffered.PrimeBound (RememberedOffered.pairWorld
      thePair)) ∧
    (∀ g : RememberedOffered.Config → Prop,
      (∀ S S', RememberedOffered.SameRecord S S' → (g S ↔ g S')) →
      ¬ ((g (RememberedOffered.registered thePair) ↔
            RememberedOffered.Value (RememberedOffered.registered
            thePair)) ∧
         (g (RememberedOffered.pairWorld thePair) ↔
            RememberedOffered.Value (RememberedOffered.pairWorld
            thePair)))) ∧
    ((TOE0.ra₀.Act → ∃ Z : RememberedOffered.ActualZeros, Z.zeros =
    RememberedOffered.registered thePair) ∧
      ¬ (TOE0.ra₀.Act → ∃ Z : RememberedOffered.ActualZeros, Z.zeros =
      RememberedOffered.pairWorld thePair)) ∧
    ((TOE0.RA₀ ↔ RememberedOffered.RootReadTwice
    (RememberedOffered.registered thePair)) ∧
      ¬ (TOE0.RA₀ ↔ RememberedOffered.RootReadTwice
      (RememberedOffered.pairWorld thePair))) := by
  have off : ¬ RememberedOffered.onLine thePair := fun h => by
    have h2 : (2 : Int) = 0 := h
    exact absurd h2 (by decide)
  have nv := RememberedOffered.pair_world_lacks_value thePair off
  have rv := RememberedOffered.registered_has_value thePair
  have pc := RememberedOffered.pair_world_closed thePair
  have rc : RememberedOffered.Closed (RememberedOffered.registered
  thePair) := fun s hs => by
    rw [hs]
    show RememberedOffered.fold (RememberedOffered.reg thePair) =
    RememberedOffered.reg thePair
    rfl
  have zr : ∀ T : RememberedOffered.Config, RememberedOffered.Closed T →
      (RememberedOffered.ZeroReports T ↔ RememberedOffered.Value T) :=
    fun T hT => (RememberedOffered.row_four_from_the_zero_side T).2.2.2 hT
  have la : ∀ T : RememberedOffered.Config, RememberedOffered.Closed T →
      ((TOE0.ra₀.Act → ∃ Z : RememberedOffered.ActualZeros, Z.zeros = T) ↔
      RememberedOffered.Value T) :=
    fun T hT => (RememberedOffered.the_last_act (rootOf TOE0.ra₀) T hT).1
  have lp : ∀ T : RememberedOffered.Config,
      ((TOE0.RA₀ ↔ RememberedOffered.RootReadTwice T) ↔
      RememberedOffered.Value T) :=
    fun T => (RememberedOffered.the_loop (rootOf TOE0.ra₀) T).2.2.2.2.1
  exact ⟨RememberedOffered.one_record thePair,
    ⟨(zr _ rc).mpr rv, (RememberedOffered.prime_side_halts _ rc).mpr rv⟩,
    ⟨fun h => nv ((zr _ pc).mp h), fun h => nv
    ((RememberedOffered.prime_side_halts _ pc).mp h)⟩,
    fun g hg (h1, h2) => nv (h2.mp ((hg _ _ (RememberedOffered.one_record
    thePair)).mp (h1.mpr rv))),
    ⟨(la _ rc).mpr rv, fun h => nv ((la _ pc).mp h)⟩,
```

```
    ((lp _).mpr rv, fun h => nv ((lp _).mp h)))

/-- THE VERIFIED PART TRANSPORTS: at every width, the verification in both half-planes on Part V's chart is the
    verified part on the offset chart with the height in absolute value. -/
theorem the_verified_part_transports (C : ArrowSide.Chart) (T : Nat) (S : ArrowSide.Pt → Prop) :
    ArrowSide.RealPartAbs C (T : Int) S ↔
      RHUnicorn.RealPartAt (img (φW C) S) RememberedOffered.onLine
      RememberedOffered.htAbs T := by
  constructor
  · intro h q (p, hp, e) hq
    subst e
    have hq' : (p.2).natAbs ≤ T := hq
    have h1 : p.1 = C.W := h p hp (by omega) (by omega)
    show p.1 - C.W = 0
    omega
  · intro h p hp h1 h2
    have h3 : p.1 - C.W = 0 := h (φW C p) (p, hp, rfl) (show (p.2).natAbs ≤ T by omega)
    show p.1 = C.W
    omega

/-- THE FAITHFUL MODEL ON THE OFFSET CHART: its image carries the verified part at chart height 3·10¹² − 1, every zero
    with |Im s| < 3·10¹², and lacks the value. -/
theorem the_faithful_model_on_the_offset_chart :
    RHUnicorn.RealPartAt (img φA (ArrowSide.zetaFaithful ArrowSide.zetaChart 2 3000000000001))
      RememberedOffered.onLine RememberedOffered.htAbs 2999999999999 ∧
    ¬ RememberedOffered.Value (img φA (ArrowSide.zetaFaithful ArrowSide.zetaChart 2 3000000000001)) :=
  ((the_verified_part_transports ArrowSide.zetaChart 2999999999999 _).mp
     ArrowSide.at_zetas_numbers_faithful.2.2.2.2.2.1,
   fun v => ArrowSide.at_zetas_numbers_faithful.2.2.2.2.2.2 ((transport eqA _).2.mpr v))

/-- ONE REAL PART, ONE OFFSET POINT, AT RESOLUTION FOUR: a real part a/4 read on Part IV's chart, on Part V's chart at
    width four, on Part II's chart at resolution two, and a real part a/2 read on Part II's chart at resolution one
    and on Part IV-c's chart, land at one point of the offset chart at resolution four. -/
theorem the_maps_agree_at_resolution_four :
    (∀ a t : Int, φH (a, t) = φA (2 * a, t) ∧ φA (2 * a, t) = φW ArrowSide.zetaChart (2 * a, t)) ∧
    (∀ z : PrimeFreedom.Pt, φP₂ z = φH (z.x, (z.t : Int)) ∧ φP z = φH (2 * z.x, (z.t : Int))) ∧
    (∀ x t : Int, φT (x, t) = φH (2 * x, t)) :=
  (fun a t => (Prod.ext (show 2 * a - 4 = 2 * a - 4 from rfl) rfl, rfl),
   fun z => (rfl, Prod.ext (show 4 * z.x - 4 = 2 * (2 * z.x) - 4 by omega) rfl),
   fun x t => Prod.ext (show 4 * x - 4 = 2 * (2 * x) - 4 by omega) rfl)

/-- Conjugation on a chart with heights in ℤ. -/
def conj (p : Int × Int) : Int × Int := (p.1, -p.2)
/-- THE MAPS COMMUTE WITH CONJUGATION on every chart with heights in ℤ; the first edition's chart, with heights in ℕ,
    reads the upper half-plane, and its maps reach only heights at or above zero. -/
theorem the_maps_commute_with_conjugation :
    (∀ p : Hinge.Pt, φH (conj p) = conj (φH p)) ∧ (∀ p : ArrowSide.Pt, φA (conj p) = conj (φA p)) ∧
    (∀ (C : ArrowSide.Chart) (p : ArrowSide.Pt), φW C (conj p) = conj (φW C p)) ∧
    (∀ p : Int × Int, φT (conj p) = conj (φT p)) ∧ (∀ p : ForceWitness.Point, φF (conj p) = conj (φF p)) ∧
    (∀ (n : Int) (z : PrimeFreedom.Pt), 0 ≤ (φPn n z).2) ∧ (∀ z : PrimeFreedom.Pt, 0 ≤ (φP z).2) ∧
    (∀ z : PrimeFreedom.Pt, 0 ≤ (φP₂ z).2) :=
  (fun _ => rfl, fun _ => rfl, fun _ _ => rfl, fun _ => rfl, fun _ => rfl,
   fun _ z => Int.ofNat_nonneg z.t, fun z => Int.ofNat_nonneg z.t, fun z => Int.ofNat_nonneg z.t)

/-! ## The reading at every resolution -/

/-- The open chart strip at resolution n: a reading inside the open strip 0 < Re s < 1 is kept at 1 ≤ x ≤ 2n − 1. -/
def clampStrip (n x : Int) : Int := max 1 (min (2 * n - 1) x)
/-- THE CLAMP KEEPS THE OPEN STRIP, COMMUTES WITH THE FOLD, AND KEEPS THE SIDE: at every resolution n ≥ 1 the clamped
    reading lies in the open chart strip; the clamp commutes with x ↦ 2n − x; and at every resolution n ≥ 2 it lands
    on the line exactly when the rounded reading does. -/
theorem the_clamp_keeps_strip_fold_and_side :
    (∀ n x : Int, 1 ≤ n → 1 ≤ clampStrip n x ∧ clampStrip n x ≤ 2 * n - 1) ∧
    (∀ n x : Int, 1 ≤ n → clampStrip n (2 * n - x) = 2 * n - clampStrip n x) ∧
    (∀ n x : Int, 2 ≤ n → (clampStrip n x = n ↔ x = n)) :=
  (fun n x _ => by unfold clampStrip; omega, fun n x _ => by unfold clampStrip; omega,
   fun n x _ => by unfold clampStrip; omega)

/-- Rounding half to even of a / b, for b > 0. -/
def roundHalfEven (a b : Int) : Int :=
  let q := a / b
  let r := a % b
  if 2 * r < b then q else if 2 * r > b then q + 1 else if q % 2 = 0 then q else q + 1
/-- THE READER'S RULE on the offset chart at resolution n ≥ 1: a real part σ = a / b of the open strip is read at
    x = round(2n·σ), rounded half to even and kept in the open chart strip, and at d = x − n. -/
def readAt (n : Nat) (a b : Int) : Int := clampStrip n (roundHalfEven (2 * n * a) b) - n

/-- ONE RESOLUTION IS NOT ENOUGH: a zero at Re s = 51/100 and its mirror at 49/100 read onto the line at resolution
    four and off it at resolution fifty; the pair at Re s = 1/4 and 3/4 reads at d = ∓2 at resolution four; and a
    zero at Re s = 97/100 or 3/100 is kept off the poles, at d = ±3. -/
theorem one_resolution_is_not_enough :
    readAt 4 51 100 = 0 ∧ readAt 4 49 100 = 0 ∧ readAt 50 51 100 = 1 ∧
    readAt 50 49 100 = -1 ∧
    readAt 4 1 4 = -2 ∧ readAt 4 3 4 = 2 ∧ readAt 4 97 100 = 3 ∧ readAt 4 3 100 = -3 := by decide

/-- A zero at Re s = 51/100 with its mirror at 49/100, at height 3·10¹² + 1, read at resolution n. -/
def nearWorld (n : Nat) : RememberedOffered.Config := fun q =>
  q = (readAt n 51 100, 3000000000001) ∨ q = (readAt n 49 100, 3000000000001)

/-- THE VALUE AT ONE RESOLUTION DOES NOT DECIDE IT AT A FINER ONE: read at resolution four the near pair is closed and
    has the value; read at resolution fifty it is closed and lacks it. -/
theorem one_resolution_does_not_decide_the_next :
    RememberedOffered.Closed (nearWorld 4) ∧ RememberedOffered.Value (nearWorld 4) ∧
    RememberedOffered.Closed (nearWorld 50) ∧ ¬ RememberedOffered.Value (nearWorld 50) := by
  obtain (e1, e2, e3, e4, _, _, _, _) := one_resolution_is_not_enough
  refine (fun q hq => ?_, fun q hq => ?_, fun q hq => ?_, fun v => ?_)
  · unfold nearWorld at hq ⊢
    rw [e1, e2] at hq ⊢
    rcases hq with h | h <;> (subst h; exact Or.inl (by decide))
  · unfold nearWorld at hq
    rw [e1, e2] at hq
    rcases hq with h | h <;> (subst h; rfl)
  · unfold nearWorld at hq ⊢
    rw [e3, e4] at hq ⊢
    rcases hq with h | h
    · subst h; exact Or.inr (by decide)
    · subst h; exact Or.inl (by decide)
  · have h : (1 : Int) = 0 := v (1, 3000000000001) (by unfold nearWorld; rw [e3]; exact Or.inl rfl)
    exact absurd h (by decide)

/-- THE ACT AT EVERY RESOLUTION: a family of readings of the offset chart, one at each resolution, each closed under
    the fold, with least erasure supplied at every resolution. The zero set of ζ read by the reader's rule at every
    resolution n ≥ 1 is such a family by the reading of the paper's Section 11; resolution zero is no reading, and the
    family's term there is the empty configuration by stipulation. -/
structure ActualZerosEvery where
  zeros : Nat → RememberedOffered.Config
  closed : ∀ n, RememberedOffered.Closed (zeros n)
  supply : ∀ n, RememberedOffered.LeastErasure (zeros n)

/-- The act at every resolution is an act of the offset chart at each one. -/
def ActualZerosEvery.atRes (Z : ActualZerosEvery) (n : Nat) : RememberedOffered.ActualZeros :=
  ⟨Z.zeros n, Z.closed n, Z.supply n⟩

/-- THE VALUE AT EVERY RESOLUTION, ON NO AXIOM: on the act at every resolution, every reading has the value. -/
theorem the_value_at_every_resolution (Z : ActualZerosEvery) (n : Nat) : RememberedOffered.Value (Z.zeros n) :=
  RememberedOffered.value_from_the_act (Z.atRes n)

/-- THE ACT AT EVERY RESOLUTION, WHOLE: the act stands at the readings exactly where the value holds at every
    resolution; on it, at every resolution, the value, the zero reports and the prime bound hold; every proof of the
    value at every resolution, from any premise, is a supply of the act; the supply is unique; and the atomic proof,
    the atomic necessity and the two-sided halt hold at every resolution. -/
theorem the_act_at_every_resolution (R : Nat → RememberedOffered.Config)
    (hR : ∀ n, RememberedOffered.Closed (R n)) :
    ((∃ Z : ActualZerosEvery, Z.zeros = R) ↔ ∀ n, RememberedOffered.Value (R n)) ∧
    (∀ (Z : ActualZerosEvery) (n : Nat), RememberedOffered.Value (Z.zeros n) ∧
      RememberedOffered.ZeroReports (Z.zeros n) ∧
      RememberedOffered.PrimeBound (Z.zeros n)) ∧
    (∀ Q : Prop, (Q → ∀ n, RememberedOffered.Value (R n)) → Q → ∃ Z : ActualZerosEvery, Z.zeros = R) ∧
    (∀ Z₁ Z₂ : ActualZerosEvery, Z₁.zeros = R → Z₂.zeros = R → Z₁ = Z₂) ∧
    (∀ n, type_of% (RememberedOffered.the_atomic_proof (rootOf TOE0.ra₀) (R n) (hR n))) ∧
    (∀ n, type_of% (RememberedOffered.the_atomic_necessity (R n) (hR n))) ∧
    (∀ n, type_of% (RememberedOffered.bidirectional_halted_uniduction (R n) (hR n))) := by
  have mk : (∀ n, RememberedOffered.Value (R n)) → ∃ Z : ActualZerosEvery, Z.zeros = R := fun v =>
    ((R, hR, fun n => (RememberedOffered.twins_on_every_world (R n)).mpr (v n)), rfl)
  refine ((fun ⟨Z, hZ⟩ n => by rw [← hZ]; exact RememberedOffered.value_from_the_act (Z.atRes n)), mk),
    fun Z n => (RememberedOffered.value_from_the_act (Z.atRes n),
```

```
      (RememberedOffered.bidirectional_halted_uniduction _ (Z.closed
      n)).2.2.2.2.2 (Z.atRes n)),
    fun _ hQ q => mk (hQ q), ?_,
    fun n => RememberedOffered.the_atomic_proof (rootOf TOE0.ra₀) (R n)
    (hR n),
    fun n => RememberedOffered.the_atomic_necessity (R n) (hR n),
    fun n => RememberedOffered.bidirectional_halted_uniduction (R n) (hR
    n))
  intro Z₁ Z₂ h₁ h₂
  obtain ⟨z₁, c₁, s₁⟩ := Z₁
  obtain ⟨z₂, c₂, s₂⟩ := Z₂
  change z₁ = R at h₁
  change z₂ = R at h₂
  subst h₁
  subst h₂
  rfl

/-- THE PRIME BOUND AT ONE RESOLUTION IS WEAKER: the near pair at Re s =
51/100 meets the prime bound read at
    resolution four and fails it read at resolution fifty; at one
    resolution n the chart's prime bound reads a zero
    within 1/(4n) of the line onto it. -/
theorem the_prime_bound_at_one_resolution :
    RememberedOffered.PrimeBound (nearWorld 4) ∧ ¬
    RememberedOffered.PrimeBound (nearWorld 50) := by
  have h := one_resolution_does_not_decide_the_next
  exact ((RememberedOffered.prime_side_halts _ h.1).mpr h.2.1,
    fun b => h.2.2.2 ((RememberedOffered.prime_side_halts _ h.2.2.1).mp
    b))

/-- THE HALT AT EVERY RESOLUTION: for every family of reflection-closed
readings, the zero reports at every resolution,
    the prime bound at every resolution and the value at every resolution
    are one proposition. The prime bound at
    every resolution together is the chart's image of the cited bound at
    x^(1/2 + ε). -/
theorem the_halt_at_every_resolution (R : Nat → RememberedOffered.Config)
    (hR : ∀ n, RememberedOffered.Closed (R n)) :
    ((∀ n, RememberedOffered.ZeroReports (R n)) ↔ ∀ n,
    RememberedOffered.Value (R n)) ∧
    ((∀ n, RememberedOffered.PrimeBound (R n)) ↔ ∀ n,
    RememberedOffered.Value (R n)) ∧
    ((∀ n, RememberedOffered.ZeroReports (R n)) ↔ ∀ n,
    RememberedOffered.PrimeBound (R n)) := by
  have z := fun n => (RememberedOffered.bidirectional_halted_uniduction
  (R n) (hR n)).1
  have p := fun n => (RememberedOffered.bidirectional_halted_uniduction
  (R n) (hR n)).2.1
  refine ⟨fun h n => (z n).mp (h n), fun h n => (z n).mpr (h n)⟩,
    ⟨fun h n => (p n).mp (h n), fun h n => (p n).mpr (h n)⟩,
    ⟨fun h n => (p n).mpr ((z n).mp (h n)), fun h n => (z n).mpr ((p
    n).mp (h n))⟩⟩

end Charts

end PartVb

/-! # PART VI · THE TWO-SIDED CLOSURE -/
section PartVI
namespace BothSides

/-- THE TWO-SIDED CLOSURE. One theorem, every capstone of the seed bound
by its exact type.
    I    the floor, on no axiom: the root holds for every self-grounding
    root; the Root Axiom on its constructed
         domain; the master seal of the three axes; the line is the fixed
         set of conjugation.
    II   the arrow: the only rigid reversal of the two poles; the freedom
    cut, the line blind to the arrow's sign;
         the hinge at Re s = 1/2; the fold's fixed set is the line.
    III  the prime side: the primes admit every assignment; prime freedom
    forces nothing; the prime bound halts
         at the value; the ζ-specific facts on the arrow, their test and
         their reach at the ends.
    IV   the non-trivial-zero side: the zero set's own centre is the
    line; a centred pair off it; row four read
         from the zeros.
    V    the nine rows: the form; the open leg; the blind record; the
    premise forced, the act, the weakest
         forcing premise; the barred refuter; the terminal saturation;
         the capstone; the annihilating timeless
         closure and the one; the atomic form forced.
    VI   the termination: the bidirectional halted uniduction; the atomic
    proof; the last act; least erasure and
         least escape; the loop.
    VII  the run: a run witnesses that it ran and leaves the value free; a
    run's flag offered as the field at every
         closed configuration is refused; beside the field, the run is
         witnessed and the value stands on the field.
    VIII the charts are one; the one model pair read by every instrument;
    the faithful model of ζ's zeros; the hinge
         exists; both values of the bit are atomic.
    IX   further capstones and row mechanisms of Parts II, III, IV, IV-b
    and IV-c.
    X    every further theorem the body of the paper cites, generated
    from the body's citations: with I to IX, the
         closure binds every theorem the body cites. -/
theorem the_two_sided_closure :
    -- I · the floor
    (type_of% @TOE0.the_floor_is_universal) ∧ (type_of%
    @TOE0.root_on_the_constructed_domain) ∧
    (type_of% @TOE0.the_master_seal) ∧ (type_of%
    @TOE0.the_line_is_the_fixed_set) ∧
    -- II · the arrow
    (type_of% @Hinge.the_only_rigid_reversal) ∧ (type_of%
    @Hinge.hinge_sign_blind) ∧
    (type_of% @Hinge.zeta_hinge) ∧ (type_of% @Hinge.locus) ∧
    -- III · the prime side
    (type_of% @PrimeFreedom.primes_admit_every_assignment) ∧ (type_of%
    @PrimeFreedom.prime_freedom_forces_nothing) ∧
    (type_of% @RememberedOffered.prime_side_halts) ∧ (type_of%
    @ArrowSide.arrow_side_test) ∧
    (type_of% @ArrowSide.beyond_the_head_breaks_ends) ∧ (type_of%
    @ArrowSide.at_zetas_numbers) ∧
    -- IV · the non-trivial-zero side
    (type_of% @Hinge.centre_is_the_line) ∧ (type_of%
    @Hinge.centred_yet_off) ∧
    (type_of% @RememberedOffered.row_four_from_the_zero_side) ∧
    -- V · the nine rows
    (type_of% @PrimeFreedom.least_erasure_is_the_value) ∧ (type_of%
    @PrimeFreedom.keyless_forces_nothing) ∧
    (type_of% @PrimeFreedom.record_decides_nothing) ∧ (type_of%
    @PrimeFreedom.rh_from_the_act) ∧
    (type_of% @PrimeFreedom.rh_is_the_weakest_forcing_premise) ∧
    (type_of% @PrimeFreedom.the_tenth_is_barred_by_the_act) ∧
    (type_of% @PrimeFreedom.nothing_left_open_on_the_act) ∧ (type_of%
    @PrimeFreedom.the_capstone) ∧
    (type_of% @RememberedOffered.the_annihilation_closure) ∧ (type_of%
    @RememberedOffered.the_one) ∧
    (type_of% @RememberedOffered.the_atomic_necessity) ∧
    -- VI · the termination
    (type_of% @RememberedOffered.bidirectional_halted_uniduction) ∧
    (type_of% @RememberedOffered.the_atomic_proof) ∧
    (type_of% @RememberedOffered.the_last_act) ∧ (type_of%
    @RememberedOffered.least_erasure_and_least_escape) ∧
    (type_of% @RememberedOffered.the_loop) ∧
    -- VII · the run
    (type_of% @ForceWitness.force_witness_closure) ∧ (type_of%
    @ForceWitness.the_plug_fails) ∧
    (type_of% @ForceWitness.the_plug_delivers) ∧
    -- VIII · the charts, the pair and the faithful model
    (type_of% @Charts.the_charts_are_one) ∧ (type_of%
    @Charts.the_pair_read_by_every_instrument) ∧
    (type_of% @ArrowSide.at_zetas_numbers_faithful) ∧ (type_of%
    @Hinge.hinge_exists) ∧
    (type_of% @RememberedOffered.both_values_atomic) ∧
    -- IX · every further capstone and row mechanism the paper cites
    (type_of% @PrimeFreedom.vocabulary_law) ∧ (type_of%
    @PrimeFreedom.the_front) ∧
    (type_of% @PrimeFreedom.posit_is_the_value) ∧ (type_of%
    @PrimeFreedom.every_face_proves_every_face) ∧
    (type_of% @PrimeFreedom.keyless_forces_nothing₂) ∧ (type_of%
    @PrimeFreedom.no_reading_decides_in_the_strip) ∧
    (type_of% @PrimeFreedom.the_tenth_is_the_refuter) ∧ (type_of%
    @PrimeFreedom.reader_frame) ∧
    (type_of% @RememberedOffered.the_timeless_zone_closed) ∧ (type_of%
    @RememberedOffered.the_unicorn_contained) ∧
    (type_of% @RememberedOffered.unicorn_blocked_on_the_chart) ∧
    (type_of% @RememberedOffered.flowed_record_forgets) ∧
    (type_of% @RememberedOffered.reality_transported) ∧ (type_of%
    @RememberedOffered.upstream_is_not_forced) ∧
    (type_of% @RememberedOffered.the_deed_is_not_the_supply) ∧ (type_of%
    @RememberedOffered.floor_is_not_least_erasure) ∧
    (type_of% @ForceWitness.force_witness_guarded) ∧ (type_of%
    @Hinge.arrow_gives_line_not_value) ∧
    (type_of% @Judge.identical_locus_does_not_exclude) ∧ (type_of%
    @Judge.the_seed_is_exactly_what_binds) ∧
    (type_of% @TimeLocus.time_does_not_bind) ∧
    -- X · every further theorem the body of the paper cites
    (type_of% @Hinge.the_line_off_the_arrow) ∧ (type_of%
    @RememberedOffered.no_closed_principle_decides) ∧
    (type_of% @Hinge.keyless_forces_nothing) ∧ (type_of%
    @ArrowSide.faithful_arrow_side_test) ∧
    (type_of% @Charts.the_act_at_every_resolution) ∧ (type_of%
    @PrimeFreedom.independence_forces_truth) ∧
    (type_of% @Charts.the_blocks_at_the_one_pair) ∧ (type_of%
    @RememberedOffered.twins_on_every_world) ∧
    (type_of% @ArrowSide.faithful_every_margin) ∧ (type_of%
    @RememberedOffered.the_unicorn_contained_abs) ∧
    (type_of% @RememberedOffered.the_form_is_the_hypothesis) ∧ (type_of%
    @RememberedOffered.line_no_has_a_world) ∧
    (type_of% @RememberedOffered.tongue_no_has_no_world) ∧ (type_of%
    @Charts.the_value_at_every_resolution) ∧
    (type_of% @TOE0.denial_reenacts_root) ∧ (type_of%
    @PrimeFreedom.denial_reenacts_root) ∧
    (type_of% @RememberedOffered.denial_reenacts_root) ∧ (type_of%
    @TOE0.no_level_above) ∧
    (type_of% @PrimeFreedom.root_at_zeros_is_least_erasure) ∧ (type_of%
    @RememberedOffered.the_root_read_twice_is_the_value) ∧
    (type_of% @RememberedOffered.monism_witness_is_the_claim) ∧ (type_of%
    @RememberedOffered.the_floor_alone_decides_no_value) ∧
    (type_of% @RememberedOffered.merging_at_a_world_is_the_supply) ∧
    (type_of% @TOE0.twenty_four_units_twelve_gates) ∧
    (type_of% @TOE0.the_shell_splits_twelve_and_twelve) ∧ (type_of%
    @Charts.the_maps_keep_height_and_scale) ∧
    (type_of% @Charts.the_maps_agree_at_resolution_four) ∧ (type_of%
    @Charts.the_maps_commute_with_conjugation) ∧
    (type_of% @Charts.transport) ∧ (type_of%
    @Charts.one_resolution_is_not_enough) ∧
    (type_of% @Charts.one_resolution_does_not_decide_the_next) ∧
    (type_of% @Hinge.hinge) ∧
    (type_of% @Hinge.hinge_unique) ∧ (type_of% @Hinge.hinge_iff_even) ∧
    (type_of% @Hinge.centre_on_every_extension) ∧ (type_of%
    @Hinge.the_point_reflection_reads_the_same_line) ∧
    (type_of% @Hinge.fold_is_the_height_keeping_extension) ∧ (type_of%
    @PrimeFreedom.liouville_is_the_sign_arrow) ∧
```

```
    (type_of% @ForceWitness.record_decides_nothing) ∧ (type_of%
    @PrimeFreedom.open_strip_forces_at_resolution_one) ∧
    (type_of% @ArrowSide.an_off_line_zero_comes_in_four) ∧ (type_of%
    @ArrowSide.at_width_one_the_ends_are_the_value) ∧
    (type_of% @Charts.the_halt_at_every_resolution) ∧ (type_of%
    @Charts.the_prime_bound_at_one_resolution) ∧
    (type_of% @RememberedOffered.price_zero_iff_value) ∧ (type_of%
    @Hinge.centre_of_a_closed_family) ∧
    (type_of% @RememberedOffered.two_doors) ∧ (type_of%
    @RememberedOffered.doors_exclusive) ∧
    (type_of%
    @RememberedOffered.the_false_world_carries_the_floor_and_no_monism) ∧
    (type_of% @RememberedOffered.one_record) ∧
    (type_of% @ForceWitness.one_record) ∧ (type_of%
    @RememberedOffered.price_counts_pairs) ∧
    (type_of% @RememberedOffered.monism_is_the_sole_witness) ∧ (type_of%
    @Judge.identical_locus_inside_the_strip_does_not_exclude) ∧
    (type_of% @PrimeFreedom.least_erasure_unconditional) ∧ (type_of%
    @RememberedOffered.four_names_one_proposition) ∧
    (type_of% @PrimeFreedom.unicorn_block) ∧ (type_of%
    @RHUnicorn.unicorn_block) ∧
    (type_of% @PrimeFreedom.the_witness_forced_by_the_act) ∧ (type_of%
    @PrimeFreedom.two_denials) ∧
    (type_of% @RememberedOffered.two_denials) ∧ (type_of%
    @RememberedOffered.no_arrival_off_the_line_on_the_act) ∧
    (type_of% @RememberedOffered.on_the_act_the_line_no_is_refuted) ∧
    (type_of% @RememberedOffered.the_unconditional_route_is_blocked) ∧
    (type_of% @RememberedOffered.accepting_rh_forces_the_atomic_proof) ∧
    (type_of% @RememberedOffered.band_does_not_force) ∧
    (type_of% @RememberedOffered.band_zero_is_value) ∧ (type_of%
    @RememberedOffered.the_form_of_a_field) ∧
    (type_of% @RememberedOffered.premise_trichotomy) ∧ (type_of%
    @Charts.the_clamp_keeps_strip_fold_and_side) ∧
    (type_of% @PrimeFreedom.unicorn_never_registered) ∧ (type_of%
    @PrimeFreedom.round_trip_identity) ∧
    (type_of% @PrimeFreedom.trip_adds_presence_only) ∧ (type_of%
    @PrimeFreedom.massless_arrow) ∧
    (type_of% @PrimeFreedom.no_coalition_decides) :=
  (@TOE0.the_floor_is_universal, @TOE0.root_on_the_constructed_domain,
  @TOE0.the_master_seal,
   @TOE0.the_line_is_the_fixed_set,
   @Hinge.the_only_rigid_reversal, @Hinge.hinge_sign_blind,
   @Hinge.zeta_hinge, @Hinge.locus,
   @PrimeFreedom.primes_admit_every_assignment,
   @PrimeFreedom.prime_freedom_forces_nothing,
   @RememberedOffered.prime_side_halts, @ArrowSide.arrow_side_test,
   @ArrowSide.beyond_the_head_breaks_ends,
   @ArrowSide.at_zetas_numbers,
   @Hinge.centre_is_the_line, @Hinge.centred_yet_off,
   @RememberedOffered.row_four_from_the_zero_side,
   @PrimeFreedom.least_erasure_is_the_value,
   @PrimeFreedom.keyless_forces_nothing,
   @PrimeFreedom.record_decides_nothing, @PrimeFreedom.rh_from_the_act,
   @PrimeFreedom.rh_is_the_weakest_forcing_premise,
   @PrimeFreedom.the_tenth_is_barred_by_the_act,
   @PrimeFreedom.nothing_left_open_on_the_act, @PrimeFreedom.the_capstone,
   @RememberedOffered.the_annihilation_closure,
   @RememberedOffered.the_one, @RememberedOffered.the_atomic_necessity,
   @RememberedOffered.bidirectional_halted_uniduction,
   @RememberedOffered.the_atomic_proof,
   @RememberedOffered.the_last_act,
   @RememberedOffered.least_erasure_and_least_escape,
   @RememberedOffered.the_loop,
   @ForceWitness.force_witness_closure, @ForceWitness.the_plug_fails,
   @ForceWitness.the_plug_delivers,
   @Charts.the_charts_are_one, @Charts.the_pair_read_by_every_instrument,
   @ArrowSide.at_zetas_numbers_faithful,
   @Hinge.hinge_exists, @RememberedOffered.both_values_atomic,
   @PrimeFreedom.vocabulary_law, @PrimeFreedom.the_front,
   @PrimeFreedom.posit_is_the_value,
   @PrimeFreedom.every_face_proves_every_face,
   @PrimeFreedom.keyless_forces_nothing₂,
   @PrimeFreedom.no_reading_decides_in_the_strip,
   @PrimeFreedom.the_tenth_is_the_refuter, @PrimeFreedom.reader_frame,
   @RememberedOffered.the_timeless_zone_closed,
   @RememberedOffered.the_unicorn_contained,
   @RememberedOffered.unicorn_blocked_on_the_chart,
   @RememberedOffered.flowed_record_forgets,
   @RememberedOffered.reality_transported,
   @RememberedOffered.upstream_is_not_forced,
   @RememberedOffered.the_deed_is_not_the_supply,
   @RememberedOffered.floor_is_not_least_erasure,
   @ForceWitness.force_witness_guarded, @Hinge.arrow_gives_line_not_value,
   @Judge.identical_locus_does_not_exclude,
   @Judge.the_seed_is_exactly_what_binds, @TimeLocus.time_does_not_bind,
   @Hinge.the_line_off_the_arrow,
   @RememberedOffered.no_closed_principle_decides,
   @Hinge.keyless_forces_nothing,
   @ArrowSide.faithful_arrow_side_test,
   @Charts.the_act_at_every_resolution,
   @PrimeFreedom.independence_forces_truth,
   @Charts.the_blocks_at_the_one_pair,
   @RememberedOffered.twins_on_every_world,
   @ArrowSide.faithful_every_margin,
   @RememberedOffered.the_unicorn_contained_abs,
   @RememberedOffered.the_form_is_the_hypothesis,
   @RememberedOffered.line_no_has_a_world,
   @RememberedOffered.tongue_no_has_no_world,
   @Charts.the_value_at_every_resolution, @TOE0.denial_reenacts_root,
   @PrimeFreedom.denial_reenacts_root,
   @RememberedOffered.denial_reenacts_root, @TOE0.no_level_above,
   @PrimeFreedom.root_at_zeros_is_least_erasure,
   @RememberedOffered.the_root_read_twice_is_the_value,
   @RememberedOffered.monism_witness_is_the_claim,
   @RememberedOffered.the_floor_alone_decides_no_value,
   @RememberedOffered.merging_at_a_world_is_the_supply,
   @TOE0.twenty_four_units_twelve_gates,
   @TOE0.the_shell_splits_twelve_and_twelve,
   @Charts.the_maps_keep_height_and_scale,
   @Charts.the_maps_agree_at_resolution_four,
   @Charts.the_maps_commute_with_conjugation, @Charts.transport,
   @Charts.one_resolution_is_not_enough,
   @Charts.one_resolution_does_not_decide_the_next, @Hinge.hinge,
   @Hinge.hinge_unique,
   @Hinge.hinge_iff_even, @Hinge.centre_on_every_extension,
   @Hinge.the_point_reflection_reads_the_same_line,
   @Hinge.fold_is_the_height_keeping_extension,
   @PrimeFreedom.liouville_is_the_sign_arrow,
   @ForceWitness.record_decides_nothing,
   @PrimeFreedom.open_strip_forces_at_resolution_one,
   @ArrowSide.an_off_line_zero_comes_in_four,
   @ArrowSide.at_width_one_the_ends_are_the_value,
   @Charts.the_halt_at_every_resolution,
   @Charts.the_prime_bound_at_one_resolution,
   @RememberedOffered.price_zero_iff_value,
   @Hinge.centre_of_a_closed_family, @RememberedOffered.two_doors,
   @RememberedOffered.doors_exclusive,
   @RememberedOffered.the_false_world_carries_the_floor_and_no_monism,
   @RememberedOffered.one_record, @ForceWitness.one_record,
   @RememberedOffered.price_counts_pairs,
   @RememberedOffered.monism_is_the_sole_witness,
   @Judge.identical_locus_inside_the_strip_does_not_exclude,
   @PrimeFreedom.least_erasure_unconditional,
   @RememberedOffered.four_names_one_proposition,
   @PrimeFreedom.unicorn_block,
   @RHUnicorn.unicorn_block, @PrimeFreedom.the_witness_forced_by_the_act,
   @PrimeFreedom.two_denials,
   @RememberedOffered.two_denials,
   @RememberedOffered.no_arrival_off_the_line_on_the_act,
   @RememberedOffered.on_the_act_the_line_no_is_refuted,
   @RememberedOffered.the_unconditional_route_is_blocked,
   @RememberedOffered.accepting_rh_forces_the_atomic_proof,
   @RememberedOffered.band_does_not_force,
   @RememberedOffered.band_zero_is_value,
   @RememberedOffered.the_form_of_a_field,
   @RememberedOffered.premise_trichotomy,
   @Charts.the_clamp_keeps_strip_fold_and_side,
   @PrimeFreedom.unicorn_never_registered,
   @PrimeFreedom.round_trip_identity,
   @PrimeFreedom.trip_adds_presence_only, @PrimeFreedom.massless_arrow,
   @PrimeFreedom.no_coalition_decides)

end BothSides
end PartVI

/-! # The cones of Parts IV to VI -/
/-- info: 'ArrowSide.an_off_line_zero_comes_in_four' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.an_off_line_zero_comes_in_four
/-- info: 'ArrowSide.arrow_side_test' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.arrow_side_test
/-- info: 'ArrowSide.at_width_one_the_ends_are_the_value' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.at_width_one_the_ends_are_the_value
/-- info: 'ArrowSide.at_zetas_numbers' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.at_zetas_numbers
/-- info: 'ArrowSide.at_zetas_numbers_faithful' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.at_zetas_numbers_faithful
/-- info: 'ArrowSide.beyond_the_head_breaks_ends' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.beyond_the_head_breaks_ends
/-- info: 'ArrowSide.faithful_arrow_side_test' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.faithful_arrow_side_test
/-- info: 'ArrowSide.faithful_every_margin' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.faithful_every_margin
/-- info: 'ArrowSide.shaped_closed' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.shaped_closed
/-- info: 'ArrowSide.shaped_ends' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.shaped_ends
/-- info: 'ArrowSide.shaped_finitely_off' does not depend on any axioms -/
#guard_msgs in #print axioms ArrowSide.shaped_finitely_off
/-- info: 'ArrowSide.shaped_lacks_value' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.shaped_lacks_value
/-- info: 'ArrowSide.shaped_real_part' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.shaped_real_part
/-- info: 'ArrowSide.shaped_zero_free' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms ArrowSide.shaped_zero_free
/-- info: 'BothSides.the_two_sided_closure' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BothSides.the_two_sided_closure
/-- info: 'Charts.eqA' depends on axioms: [propext, Classical.choice,
Quot.sound] -/
#guard_msgs in #print axioms Charts.eqA
/-- info: 'Charts.eqF' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.eqF
/-- info: 'Charts.eqH' depends on axioms: [propext, Classical.choice,
Quot.sound] -/
```

```
#guard_msgs in #print axioms Charts.eqH
/-- info: 'Charts.eqP' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.eqP
/-- info: 'Charts.eqPn' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.eqPn
/-- info: 'Charts.eqP₂' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.eqP₂
/-- info: 'Charts.eqT' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.eqT
/-- info: 'Charts.eqW' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.eqW
/-- info: 'Charts.one_resolution_does_not_decide_the_next' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.one_resolution_does_not_decide_the_next
/-- info: 'Charts.one_resolution_is_not_enough' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.one_resolution_is_not_enough
/-- info: 'Charts.the_act_at_every_resolution' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_act_at_every_resolution
/-- info: 'Charts.the_blocks_at_the_one_pair' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_blocks_at_the_one_pair
/-- info: 'Charts.the_charts_are_one' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_charts_are_one
/-- info: 'Charts.the_clamp_keeps_strip_fold_and_side' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_clamp_keeps_strip_fold_and_side
/-- info: 'Charts.the_faithful_model_on_the_offset_chart' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_faithful_model_on_the_offset_chart
/-- info: 'Charts.the_halt_at_every_resolution' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_halt_at_every_resolution
/-- info: 'Charts.the_maps_agree_at_resolution_four' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_maps_agree_at_resolution_four
/-- info: 'Charts.the_maps_commute_with_conjugation' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.the_maps_commute_with_conjugation
/-- info: 'Charts.the_maps_keep_height_and_scale' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_maps_keep_height_and_scale
/-- info: 'Charts.the_pair_read_by_every_instrument' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.the_pair_read_by_every_instrument
/-- info: 'Charts.the_prime_bound_at_one_resolution' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_prime_bound_at_one_resolution
/-- info: 'Charts.the_value_at_every_resolution' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.the_value_at_every_resolution
/-- info: 'Charts.the_verified_part_transports' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Charts.the_verified_part_transports
/-- info: 'Charts.transport' does not depend on any axioms -/
#guard_msgs in #print axioms Charts.transport
/-- info: 'ForceWitness.the_plug_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.the_plug_delivers
/-- info: 'ForceWitness.the_plug_fails' does not depend on any axioms -/
#guard_msgs in #print axioms ForceWitness.the_plug_fails
/-- info: 'Hinge.arrow_gives_line_not_value' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hinge.arrow_gives_line_not_value
/-- info: 'Hinge.centre_is_the_line' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.centre_is_the_line
/-- info: 'Hinge.centre_of_a_closed_family' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.centre_of_a_closed_family
/-- info: 'Hinge.centre_on_every_extension' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.centre_on_every_extension
/-- info: 'Hinge.centred_yet_off' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.centred_yet_off
/-- info: 'Hinge.fold_is_the_height_keeping_extension' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.fold_is_the_height_keeping_extension
/-- info: 'Hinge.fold_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.fold_keeps_height
/-- info: 'Hinge.hinge' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hinge.hinge
/-- info: 'Hinge.hinge_exists' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.hinge_exists
/-- info: 'Hinge.hinge_iff_even' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.hinge_iff_even
/-- info: 'Hinge.hinge_sign_blind' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.hinge_sign_blind
/-- info: 'Hinge.hinge_unique' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.hinge_unique
/-- info: 'Hinge.keyless_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.keyless_forces_nothing
/-- info: 'Hinge.least_erasure_iff_value' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.least_erasure_iff_value
/-- info: 'Hinge.locus' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.locus
/-- info: 'Hinge.off_line_leans' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.off_line_leans
/-- info: 'Hinge.pair_closed' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.pair_closed
/-- info: 'Hinge.pair_in_strip' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.pair_in_strip
/-- info: 'Hinge.pair_lacks_value' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.pair_lacks_value
/-- info: 'Hinge.reg_keeps_on_line' does not depend on any axioms -/
#guard_msgs in #print axioms Hinge.reg_keeps_on_line
/-- info: 'Hinge.reg_merges' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.reg_merges
/-- info: 'Hinge.rev_flips_offset' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.rev_flips_offset
/-- info: 'Hinge.rev_involutive' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.rev_involutive
/-- info: 'Hinge.rev_swaps' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.rev_swaps
/-- info: 'Hinge.sumX_perm' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.sumX_perm
/-- info: 'Hinge.the_line_off_the_arrow' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hinge.the_line_off_the_arrow
/-- info: 'Hinge.the_only_rigid_reversal' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.the_only_rigid_reversal
/-- info: 'Hinge.the_point_reflection_reads_the_same_line' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hinge.the_point_reflection_reads_the_same_line
/-- info: 'Hinge.value_from_the_act' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Hinge.value_from_the_act
/-- info: 'Hinge.zeta_hinge' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hinge.zeta_hinge
/-- info: 'Judge.identical_locus_does_not_exclude' does not depend on any axioms -/
#guard_msgs in #print axioms Judge.identical_locus_does_not_exclude
/-- info: 'Judge.identical_locus_inside_the_strip_does_not_exclude' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms Judge.identical_locus_inside_the_strip_does_not_exclude
/-- info: 'Judge.the_seed_is_exactly_what_binds' does not depend on any axioms -/
#guard_msgs in #print axioms Judge.the_seed_is_exactly_what_binds
/-- info: 'RememberedOffered.accepting_rh_forces_the_atomic_proof' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.accepting_rh_forces_the_atomic_proof
/-- info: 'RememberedOffered.bidirectional_halted_uniduction' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.bidirectional_halted_uniduction
/-- info: 'RememberedOffered.both_values_atomic' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.both_values_atomic
/-- info: 'RememberedOffered.least_erasure_and_least_escape' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.least_erasure_and_least_escape
/-- info: 'RememberedOffered.premise_trichotomy' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.premise_trichotomy
/-- info: 'RememberedOffered.prime_side_halts' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.prime_side_halts
/-- info: 'RememberedOffered.row_four_from_the_zero_side' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.row_four_from_the_zero_side
/-- info: 'RememberedOffered.the_atomic_necessity' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_atomic_necessity
/-- info: 'RememberedOffered.the_atomic_proof' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_atomic_proof
/-- info: 'RememberedOffered.the_form_is_the_hypothesis' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_form_is_the_hypothesis
/-- info: 'RememberedOffered.the_form_of_a_field' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_form_of_a_field
/-- info: 'RememberedOffered.the_last_act' does not depend on any axioms -/
#guard_msgs in #print axioms RememberedOffered.the_last_act
/-- info: 'RememberedOffered.the_loop' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_loop
/-- info: 'RememberedOffered.the_one' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_one
/-- info: 'RememberedOffered.the_unconditional_route_is_blocked' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_unconditional_route_is_blocked
```

```
/-- info: 'RememberedOffered.the_unicorn_contained_abs' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms RememberedOffered.the_unicorn_contained_abs
/-- info: 'TimeLocus.seedless_reading_holds_in_counter_world' does not depend on any axioms -/
#guard_msgs in #print axioms TimeLocus.seedless_reading_holds_in_counter_world
/-- info: 'TimeLocus.time_does_not_bind' does not depend on any axioms -/
#guard_msgs in #print axioms TimeLocus.time_does_not_bind

#print axioms BothSides.the_two_sided_closure

/-! # PART VII · THE HARDENING · the identification, the bridge, the bound and the capstone, every input printed

  Added 11 October 2026 for edition 2.0.3 of The Atomic Proof of the Riemann Hypothesis, on the Final Scribal
  Recommendation of the audit of 10 October 2026. Nothing above this line changed: Parts I to VI are byte-identical
  to RH_Both_Sides_Seed.lean of 8 October 2026 (sha256 cb719eab915a9973…). Core Lean 4.19.0, no import, no axiom
  declared, no sorry, no admit, no native_decide.

  What this part does, and in this order:
  VII-a  carries the kernel of the companion paper, Least Erasure from Degrees of Freedom and Heat, repaired to
         compile in core Lean (the shipped v2.0.1 kernel used the Mathlib tactic by_contra and left one conjunction
         unsolved, so its master seal depended on sorryAx under Lean 4.19.0), with the quantitative companion
         density_bound_from_residual_bound added and the master seal extended to seven claims;
  VII-b  states the identification exactly: on the chart, least erasure of a configuration IS its value, for every
         configuration, by theorem; so the one thing to be granted is the reading, that the classical hypothesis is
         the value of the configuration at which the zeros of ζ are read (Section 11 of the paper). The field
         `supply` of `ActualZeros` is shown to be the premise packaged, by its type;
  VII-c  writes the bridge as six named arrows composed by theorem, the rate and the heat feeding the signal, and
         proves the uniform threshold δ = n₀·r₀·q₀ independent of which carrier and which zero;
  VII-d  keeps the heat theorem at its exact strength: the exact-null case and the finite-sensitivity case apart;
  VII-e  THE CAPSTONE, classical_rh_from_identification_gap_and_bound, with its five inputs printed, and its
         exploded form with the nine inputs of the Recommendation; the exact-null route beside it with ITS premise
         printed; and no_third_position: given the identification, the bound and the gap, the denial of the
         hypothesis and the bridge cannot both stand;
  VII-f  what the capstone does not do, by theorem: the reading and the bridge are keyed, each fails on a closed
         configuration of the interface, so neither is a theorem of the seed and each is a premise, named;
  VII-g  the hardened closure, binding the two-sided closure and the capstones by their exact types, and the cones.

  The reading of the empty cone: `#print axioms` names the axioms a proof depends on. It says nothing about the
  hypotheses in the signature. Every hypothesis of the capstone is an input the reader grants or refuses by name. -/
section PartVII
set_option autoImplicit false

/-! ## VII-a · the least-erasure kernel of the companion paper, repaired, with the quantitative companion -/
namespace LeastErasureDoF

def ln2_scaled : Nat := 693147
def scale : Nat := 1000000
def cmb_mK : Nat := 2725

/-- A system relative to a record: macro states distinguished, binary labels free, micro states in total. -/
structure System where
  distinct : Nat
  free : Nat
  total : Nat
  inv : total = distinct * (2 ^ free)

/-- A recoding that halves the record and frees one label, total preserved. -/
structure DiscardProcess where
  before : System
  after : System
  h_distinct : before.distinct = 2 * after.distinct
  h_free : after.free = before.free + 1
  h_total : before.total = after.total
  h_gt : before.distinct > after.distinct

/-- An erasure: the record kept, one free label lost, the total halved. -/
structure EraseProcess where
  before : System
  after : System
  h_distinct : before.distinct = after.distinct
  h_free : before.free = after.free + 1
  h_total : before.total = 2 * after.total
  h_gt : before.total > after.total

/-- A discard halves the record and frees one label. Repaired: the shipped proof left this conjunction open. -/
theorem discard_is_free (p : DiscardProcess) :
    p.after.distinct * 2 = p.before.distinct ∧
    p.after.free = p.before.free + 1 :=
  (by rw [p.h_distinct, Nat.mul_comm], p.h_free)

def sysEntropy (s : System) : Nat := s.free * ln2_scaled

theorem sysEntropy_decrease (p : EraseProcess) :
    sysEntropy p.after + ln2_scaled = sysEntropy p.before := by
  unfold sysEntropy
  rw [p.h_free, Nat.succ_mul]

/-- The lower bound is witnessed by the choice heat = T · ln2_scaled. The process is not consulted: the kernel
    records that a number meeting the bound exists, nothing more. The physical necessity, joint conservation of
    probability with the second law, is carried in the prose of the companion paper at its own grade. -/
theorem heat_from_joint_conservation (_p : EraseProcess) (T : Nat) :
    ∃ heat : Nat, heat ≥ T * ln2_scaled :=
  (T * ln2_scaled, Nat.le_refl _)

theorem heat_positive (_p : EraseProcess) (T : Nat) (hT : T > 0) :
    ∃ heat : Nat, heat > 0 ∧ heat ≥ T * ln2_scaled :=
  (T * ln2_scaled, Nat.mul_pos hT (by decide), Nat.le_refl _)

theorem landauer_special_case (p : EraseProcess) (_h : p.before.free = 1) :
    ∃ heat : Nat, heat ≥ cmb_mK * ln2_scaled :=
  heat_from_joint_conservation p cmb_mK

/-- The residual observable of the counting model: population × rate × heat per event. -/
def residual (density rate heat : Nat) : Nat := density * rate * heat

theorem residual_pos {density rate heat : Nat}
    (hd : density > 0) (hr : rate > 0) (hh : heat > 0) :
    residual density rate heat > 0 := by
  unfold residual
  have h1 : density * rate > 0 := Nat.mul_pos hd hr
  exact Nat.mul_pos h1 hh

/-- THE EXACT-NULL CASE. Residual exactly zero, rate and heat positive: the population is zero. Repaired: the
    shipped proof used by_contra, a Mathlib tactic, and so did not compile in core Lean. -/
theorem null_forces_zero {density rate heat : Nat}
    (hr : rate > 0) (hh : heat > 0) (h0 : residual density rate heat = 0) :
    density = 0 := by
  cases density with
  | zero => rfl
  | succ d =>
    have hpos := residual_pos (Nat.succ_pos d) hr hh
    rw [h0] at hpos
    exact absurd hpos (Nat.lt_irrefl 0)

/-- THE FINITE-SENSITIVITY CASE, the quantitative companion. Residual at most ε, rate at least r₀, heat at
    least q₀: the population times the two floors, ρ · r₀ · q₀, is at most ε. This is what a null result at finite sensitivity
    supports. It is not density = 0, and the paper never rewrites it as density = 0. Stated multiplicatively so
    that no integer division rounds anything: the bound is exact in the kernel's units. -/
theorem density_bound_from_residual_bound {density rate heat r0 q0 ε : Nat}
    (hR : residual density rate heat ≤ ε) (hr : rate ≥ r0) (hq : heat ≥ q0) :
    density * r0 * q0 ≤ ε :=
  Nat.le_trans (Nat.mul_le_mul (Nat.mul_le_mul_left density hr) hq) hR

/-- The same bound read as a quotient, density ≤ ε / (r₀ q₀), for positive floors. Integer division floors, so
    this form is weaker than the multiplicative one; it is the form the Recommendation names. Its cone carries
    propext through the core lemma on division, and is pinned below as such. -/
theorem density_le_div {density rate heat r0 q0 ε : Nat}
    (hR : residual density rate heat ≤ ε) (hr : rate ≥ r0) (hq : heat ≥ q0)
    (hr0 : r0 > 0) (hq0 : q0 > 0) :
    density ≤ ε / (r0 * q0) :=
  (Nat.le_div_iff_mul_le (Nat.mul_pos hr0 hq0)).mpr
    (Nat.mul_assoc density r0 q0 ▸ density_bound_from_residual_bound hR hr hq)

def Discards (p : EraseProcess) (label : Nat) : Prop :=
  label < p.before.distinct ∧ p.before.distinct > 0

def NullForcesZeroOn (label : Nat) : Prop :=
  ∃ p : EraseProcess, Discards p label ∧ residual 1 1 ln2_scaled > 0

theorem boundary_holds :
    ∀ (label : Nat), ¬ (∃ p : EraseProcess, Discards p label) →
      ¬ NullForcesZeroOn label := by
  intro label hNo hYes
  unfold NullForcesZeroOn at hYes
  obtain (p, hp, _) := hYes
  exact hNo (p, hp)

/-- The Noether skeleton. Both predicates are True: the kernel keeps only the shape of the statement, a
```

```
    symmetry yields a conserved current, and proves nothing about any
    dynamics. Stated so, not otherwise. -/
def ContinuousSymmetry (_ξ : Nat) : Prop := True
def Conserved (_curr : Nat) : Prop := True

theorem noether_current :
    ∀ (ξ : Nat), ContinuousSymmetry ξ →
      ∃ curr : Nat, Conserved curr :=
  fun _ _ => (0, trivial)

/-- THE MASTER SEAL, seven claims: the six of the companion paper and the
quantitative companion. -/
theorem the_master_seal :
    (∀ p : DiscardProcess, p.after.distinct * 2 = p.before.distinct ∧
    p.after.free = p.before.free + 1) ∧
    (∀ _p : EraseProcess, ∀ T : Nat, ∃ heat : Nat, heat ≥ T * ln2_scaled)
    ∧
    (∀ p : EraseProcess, p.before.free = 1 → ∃ heat : Nat, heat ≥ cmb_mK *
    ln2_scaled) ∧
    (∀ density rate heat : Nat, density > 0 → rate > 0 → heat > 0 →
    residual density rate heat > 0) ∧
    (∀ label : Nat, ¬ (∃ p : EraseProcess, Discards p label) → ¬
    NullForcesZeroOn label) ∧
    (∀ ξ : Nat, ContinuousSymmetry ξ → ∃ curr : Nat, Conserved curr) ∧
    (∀ density rate heat r0 q0 ε : Nat, residual density rate heat ≤ ε →
    rate ≥ r0 → heat ≥ q0 →
      density * r0 * q0 ≤ ε) := by
  refine (?_, ?_, ?_, ?_, ?_, ?_, ?_)
  · exact fun p => discard_is_free p
  · exact fun p T => heat_from_joint_conservation p T
  · exact fun p h => landauer_special_case p h
  · exact fun _ _ _ hd hr hh => residual_pos hd hr hh
  · exact fun label => boundary_holds label
  · exact fun ξ h => noether_current ξ h
  · exact fun _ _ _ _ _ _ hR hr hq => density_bound_from_residual_bound
  hR hr hq

end LeastErasureDoF

namespace Hardening
open RememberedOffered

/-! ## VII-b · the identification, stated exactly -/

/-- On the chart, the registration fixes a point exactly when the point
stands on the line. This is the whole of
    `zeta_fixed_locus_correct` that the chart can carry: the line of the
    chart is d = 0, and Section 11 of the
    paper reads Re s = 1/2 onto it by the reader's rule. -/
theorem zeta_fixed_locus_on_the_chart (p : Point) : reg p = p ↔ onLine p
:=
  reg_erases_nothing_iff p

/-- Least erasure of a configuration is the value of that configuration,
for every configuration, the one at
    which the zeros of ζ are read included. No hypothesis. -/
theorem le_iff_value (S : Config) : LeastErasure S ↔ Value S :=
  (fun h p hp => (reg_erases_nothing_iff p).mp (h p hp),
   fun h p hp => (reg_erases_nothing_iff p).mpr (h p hp))

/-- THE IDENTIFICATION, REDUCED TO THE READING. Whoever grants that the
classical hypothesis is the value of the
    configuration at which the zeros of ζ are read has, by theorem,
    granted that it is least erasure of that
    configuration. The reading is the one input; nothing else is
    identified by name. -/
theorem classical_rh_iff_le_of_reading {ClassicalRH : Prop} {Zζ : Config}
    (h_read : ClassicalRH ↔ Value Zζ) : ClassicalRH ↔ LeastErasure Zζ :=
  h_read.trans (le_iff_value Zζ).symm

/-- The field `supply` is the premise, by its type. Its presence in an
act establishes nothing about which
    configuration the act carries. -/
theorem supply_is_the_premise (A : ActualZeros) : LeastErasure A.zeros :=
  A.supply

/-- An act exists at a configuration exactly when the configuration is
closed and least erasure holds there:
    the act is the premise packaged, and the package adds nothing. -/
theorem the_act_is_the_premise_packaged (S : Config) :
    (∃ A : ActualZeros, A.zeros = S) ↔ (Closed S ∧ LeastErasure S) :=
  (fun (A, h) => h ▸ (A.closed, A.supply),
   fun (hc, hle) => ((S, hc, hle), rfl))

/-! ## VII-c · the bridge: six named arrows, composed by theorem -/
section Bridge
variable {Carrier : Type}

/-- The floor one realized carrier puts under the residual observable:
population × rate × heat per event. -/
theorem carrier_floor (density rate heat : Carrier → Nat) {n0 r0 q0 :
Nat} (c : Carrier)
    (hd : density c ≥ n0) (hr : rate c ≥ r0) (hq : heat c ≥ q0) :
    LeastErasureDoF.residual (density c) (rate c) (heat c) ≥ n0 * r0 * q0
    := by
  unfold LeastErasureDoF.residual
  exact Nat.mul_le_mul (Nat.mul_le_mul hd hr) hq

/-- The threshold is positive exactly when the population floor, the rate
floor and the heat floor are. -/
theorem gap_positive {n0 r0 q0 : Nat} (hn : n0 > 0) (hr : r0 > 0) (hq :
q0 > 0) : n0 * r0 * q0 > 0 :=
  Nat.mul_pos (Nat.mul_pos hn hr) hq

/-- THE BRIDGE, COMPOSED. Six arrows, each a named hypothesis the reader
grants or refuses:
    h_carrier  a failure of least erasure at the read configuration
    realizes a carrier in the measured domain;
    h_erase    a realized carrier irreversibly loses its which-branch
    label, kept in no other register;
    h_pop      an erasing carrier is a population of at least n₀;
    h_rate     an erasing carrier runs at a rate of at least r₀ during
    the observation;
    h_heat     an erasing carrier deposits at least q₀ per event
    (Landauer, q₀ = k_B T ln 2 in the kernel's units);
    h_couple   the residual observable covers the carrier's contribution:
    R ≥ population × rate × heat.
    Conclusion: a failure of least erasure forces R ≥ n₀ r₀ q₀. The
    threshold does not depend on the carrier
    and does not depend on which point stands off the line: it is uniform
    over the class the hypotheses name.
    Positive rate and positive heat alone do not reach the conclusion;
    the population floor and the coupling are
    needed and are named. -/
theorem off_line_zero_forces_observable_signal
    {Zζ : Config} {R n0 r0 q0 : Nat}
    (Realizes Erases : Carrier → Prop) (density rate heat : Carrier → Nat)
    (h_carrier : ¬ LeastErasure Zζ → ∃ c, Realizes c)
    (h_erase   : ∀ c, Realizes c → Erases c)
    (h_pop     : ∀ c, Erases c → density c ≥ n0)
    (h_rate    : ∀ c, Erases c → rate c ≥ r0)
    (h_heat    : ∀ c, Erases c → heat c ≥ q0)
    (h_couple  : ∀ c, Erases c → R ≥ LeastErasureDoF.residual (density c)
    (rate c) (heat c)) :
    ¬ LeastErasure Zζ → R ≥ n0 * r0 * q0 := by
  intro hnot
  obtain (c, hc) := h_carrier hnot
  have he := h_erase c hc
  exact Nat.le_trans (carrier_floor density rate heat c (h_pop c he)
  (h_rate c he) (h_heat c he)) (h_couple c he)

end Bridge

/-! ## VII-d · the heat theorem at its exact strength: the two cases,
kept apart -/

/-- Exact null: residual = 0 with positive rate and heat forces the
population to zero. -/
theorem density_zero_from_exact_null {ρ r q : Nat} (hr : r > 0) (hq : q >
0)
    (h0 : LeastErasureDoF.residual ρ r q = 0) : ρ = 0 :=
  LeastErasureDoF.null_forces_zero hr hq h0

/-- Finite sensitivity: residual ≤ ε with rate ≥ r₀ and heat ≥ q₀ bounds
the population: ρ · r₀ q₀ ≤ ε. -/
theorem density_bound_at_finite_sensitivity {ρ r q r0 q0 ε : Nat}
    (hR : LeastErasureDoF.residual ρ r q ≤ ε) (hr : r ≥ r0) (hq : q ≥ q0)
    :
    ρ * r0 * q0 ≤ ε :=
  LeastErasureDoF.density_bound_from_residual_bound hR hr hq

/-! ## VII-e · THE CAPSTONE, every input printed -/

/-- THE CAPSTONE. Five inputs, each printed, none carried by a structure:
    h_id       the identification: the classical hypothesis is least
    erasure at the read configuration;
    h_bridge   the bridge: a failure of least erasure there forces the
    residual observable to at least δ;
    h_bound    the observational bound: the residual observable is at most
    ε;
    h_gap      the gap: ε < δ;
    h_pos      the threshold is positive (implied by h_gap over the
    naturals; printed because the Recommendation
               prints it).
    Conclusion: the classical hypothesis. The proof is pointwise and
    constructive: at each read point the
    registration either fixes the point, or it does not and the bridge
    puts the residual at δ > ε ≥ R.
    The cone is empty. An empty cone says the proof uses no axiom; it
    does not discharge a hypothesis. -/
theorem classical_rh_from_identification_gap_and_bound
    {ClassicalRH : Prop} {Zζ : Config} {R δ ε : Nat}
    (h_id     : ClassicalRH ↔ LeastErasure Zζ)
    (h_bridge : ¬ LeastErasure Zζ → R ≥ δ)
    (h_bound  : R ≤ ε)
    (h_gap    : ε < δ)
    (h_pos    : 0 < δ) :
    ClassicalRH := by
  have _hδ : δ ≠ 0 := Nat.pos_iff_ne_zero.mp h_pos
  apply h_id.mpr
  intro p hp
  match decEq (reg p) p with
  | isTrue h => exact h
  | isFalse h =>
    have hnot : ¬ LeastErasure Zζ := fun hle => h (hle p hp)
    have hRδ : R ≥ δ := h_bridge hnot
    exact absurd (Nat.lt_of_le_of_lt (Nat.le_trans hRδ h_bound) h_gap)
    (Nat.lt_irrefl δ)

/-- THE CAPSTONE, EXPLODED: the nine inputs of the Final Scribal
Recommendation. The reading, the six arrows of
    the bridge, the bound, and the gap against the composed threshold n₀
    r₀ q₀. -/
theorem classical_rh_from_the_arrows
    {Carrier : Type} {ClassicalRH : Prop} {Zζ : Config} {R n0 r0 q0 ε :
    Nat}
```

```
    (Realizes Erases : Carrier → Prop) (density rate heat : Carrier → Nat)
    (h_read    : ClassicalRH ↔ Value ZZ)
    (h_carrier : ¬ LeastErasure ZZ → ∃ c, Realizes c)
    (h_erase   : ∀ c, Realizes c → Erases c)
    (h_pop     : ∀ c, Erases c → density c ≥ n0)
    (h_rate    : ∀ c, Erases c → rate c ≥ r0)
    (h_heat    : ∀ c, Erases c → heat c ≥ q0)
    (h_couple  : ∀ c, Erases c → R ≥ LeastErasureDoF.residual (density c)
    (rate c) (heat c))
    (h_bound   : R ≤ ε)
    (h_gap     : ε < n0 * r0 * q0) :
    ClassicalRH :=
  classical_rh_from_identification_gap_and_bound
    (classical_rh_iff_le_of_reading h_read)
    (off_line_zero_forces_observable_signal Realizes Erases density rate
    heat
      h_carrier h_erase h_pop h_rate h_heat h_couple)
    h_bound h_gap (Nat.lt_of_le_of_lt (Nat.zero_le ε) h_gap)

/-- THE EXACT-NULL ROUTE, the Mandate II of the second edition, with its
premise printed. If a failure of least
    erasure realizes a population of positive size, rate and heat whose
    contribution the residual covers, and the
    residual is EXACTLY zero, the hypothesis follows. The premise R = 0
    is not what a measurement at finite
    sensitivity reports; the paper's closure runs on the gap route above,
    and this theorem records where the
    second edition's wording stood. -/
theorem classical_rh_from_identification_and_exact_null
    {ClassicalRH : Prop} {ZZ : Config} {R : Nat}
    (h_id     : ClassicalRH ↔ LeastErasure ZZ)
    (h_bridge : ¬ LeastErasure ZZ → ∃ ρ r q : Nat, ρ > 0 ∧ r > 0 ∧ q > 0 ∧
    R ≥ LeastErasureDoF.residual ρ r q)
    (h_null   : R = 0) :
    ClassicalRH := by
  apply h_id.mpr
  intro ρ hp
  match decEq (reg ρ) ρ with
  | isTrue h => exact h
  | isFalse h =>
    have hnot : ¬ LeastErasure ZZ := fun hle => h (hle ρ hp)
    obtain ⟨ρ, r, q, hp, hr, hq, hR⟩ := h_bridge hnot
    have hpos := LeastErasureDoF.residual_pos hp hr hq
    rw [h_null] at hR
    exact absurd (Nat.lt_of_lt_of_le hpos hR) (Nat.lt_irrefl 0)

/-- NO THIRD POSITION. Given the identification, the bound and the gap,
the denial of the hypothesis and the
    bridge cannot both stand. A reader who denies the hypothesis has
    named the bridge as the arrow that fails;
    a reader who grants the bridge has granted the hypothesis.
    Constructive; the cone is empty. -/
theorem no_third_position
    {ClassicalRH : Prop} {ZZ : Config} {R δ ε : Nat}
    (h_id    : ClassicalRH ↔ LeastErasure ZZ)
    (h_bound : R ≤ ε)
    (h_gap   : ε < δ) :
    ¬ (¬ ClassicalRH ∧ (¬ LeastErasure ZZ → R ≥ δ)) := by
  intro ⟨hno, h_bridge⟩
  have hnot : ¬ LeastErasure ZZ := fun hle => hno (h_id.mpr hle)
  exact absurd (Nat.lt_of_le_of_lt (Nat.le_trans (h_bridge hnot) h_bound)
  h_gap) (Nat.lt_irrefl δ)

/-! ## VII-f · what the capstone does not do, by theorem: its inputs are
keyed -/

/-- The pair world of the off point is closed and lacks least erasure. -/
theorem pair_world_lacks_le : Closed (pairWorld offPoint) ∧ ¬
LeastErasure (pairWorld offPoint) :=
  ⟨pair_world_closed offPoint,
   fun hle => off_point_is_off ((reg_erases_nothing_iff offPoint).mp (hle
   offPoint (Or.inl rfl)))⟩

/-- THE READING IS KEYED. No theorem of the interface identifies an
arbitrary proposition with least erasure at
    an arbitrary closed configuration: for the true proposition and the
    pair world the identification fails. The
    identification is therefore an input, granted for the reading of
    Section 11 and for nothing else. -/
theorem the_reading_is_keyed :
    ∃ (P : Prop) (S : Config), Closed S ∧ ¬ (P ↔ LeastErasure S) :=
  ⟨True, pairWorld offPoint, pair_world_lacks_le.1,
   fun h => pair_world_lacks_le.2 (h.mp trivial)⟩

/-- THE BRIDGE IS KEYED. No theorem of the interface makes a failure of
least erasure lift the residual: on the
    pair world with R = 0 and δ = 1 the bridge fails. The bridge is
    therefore an input: derived from the physics
    at the grade of its arrows, or granted by name, never read off the
    chart. -/
theorem the_bridge_is_keyed :
    ∃ (S : Config) (R δ : Nat), Closed S ∧ ¬ LeastErasure S ∧ ¬ (¬
    LeastErasure S → R ≥ δ) :=
  ⟨pairWorld offPoint, 0, 1, pair_world_lacks_le.1, pair_world_lacks_le.2,
   fun h => Nat.not_succ_le_zero 0 (h pair_world_lacks_le.2)⟩

/-- THE BOUND DOES NOT CLOSE ALONE. A bound R ≤ ε with no bridge leaves
least erasure undecided: the pair world
    is closed, lacks least erasure, and is consistent with every bound. -/
theorem the_bound_alone_decides_nothing (ε : Nat) :
    ∃ S : Config, Closed S ∧ ¬ LeastErasure S ∧ (0 : Nat) ≤ ε :=
  ⟨pairWorld offPoint, pair_world_lacks_le.1, pair_world_lacks_le.2,
   Nat.zero_le ε⟩

/-! ## VII-g · the hardened closure -/

/-- THE HARDENED CLOSURE. The two-sided closure of Part VI, the
seven-claim master seal of the companion kernel,
    the identification reduced to the reading, the act as the premise
    packaged, the bridge composed, the two
    cases of the heat theorem, the capstone in both forms, the exact-null
    route, no third position, and the three
    keyed-input theorems, each by its exact type. -/
theorem the_hardened_closure :
    (type_of% @BothSides.the_two_sided_closure) ∧
    (type_of% @LeastErasureDoF.the_master_seal) ∧
    (type_of% @classical_rh_iff_le_of_reading) ∧
    (type_of% @the_act_is_the_premise_packaged) ∧
    (type_of% @off_line_zero_forces_observable_signal) ∧
    (type_of% @density_zero_from_exact_null) ∧
    (type_of% @density_bound_at_finite_sensitivity) ∧
    (type_of% @classical_rh_from_identification_gap_and_bound) ∧
    (type_of% @classical_rh_from_the_arrows) ∧
    (type_of% @classical_rh_from_identification_and_exact_null) ∧
    (type_of% @no_third_position) ∧
    (type_of% @the_reading_is_keyed) ∧
    (type_of% @the_bridge_is_keyed) ∧
    (type_of% @the_bound_alone_decides_nothing) :=
  ⟨@BothSides.the_two_sided_closure, @LeastErasureDoF.the_master_seal,
   @classical_rh_iff_le_of_reading, @the_act_is_the_premise_packaged,
   @off_line_zero_forces_observable_signal, @density_zero_from_exact_null,
   @density_bound_at_finite_sensitivity,
   @classical_rh_from_identification_gap_and_bound,
   @classical_rh_from_the_arrows,
   @classical_rh_from_identification_and_exact_null,
   @no_third_position, @the_reading_is_keyed, @the_bridge_is_keyed,
   @the_bound_alone_decides_nothing⟩

end Hardening
end PartVII

/-! # The cones of Part VII, pinned -/
/-- info: 'LeastErasureDoF.discard_is_free' does not depend on any axioms
-/
#guard_msgs in #print axioms LeastErasureDoF.discard_is_free
/-- info: 'LeastErasureDoF.null_forces_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms LeastErasureDoF.null_forces_zero
/-- info: 'LeastErasureDoF.density_bound_from_residual_bound' does not
depend on any axioms -/
#guard_msgs in #print axioms
LeastErasureDoF.density_bound_from_residual_bound
/-- info: 'LeastErasureDoF.density_le_div' depends on axioms: [propext] -/
#guard_msgs in #print axioms LeastErasureDoF.density_le_div
/-- info: 'LeastErasureDoF.the_master_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms LeastErasureDoF.the_master_seal
/-- info: 'Hardening.le_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms Hardening.le_iff_value
/-- info: 'Hardening.classical_rh_iff_le_of_reading' does not depend on
any axioms -/
#guard_msgs in #print axioms Hardening.classical_rh_iff_le_of_reading
/-- info: 'Hardening.the_act_is_the_premise_packaged' does not depend on
any axioms -/
#guard_msgs in #print axioms Hardening.the_act_is_the_premise_packaged
/-- info: 'Hardening.off_line_zero_forces_observable_signal' does not
depend on any axioms -/
#guard_msgs in #print axioms
Hardening.off_line_zero_forces_observable_signal
/-- info: 'Hardening.classical_rh_from_identification_gap_and_bound' does
not depend on any axioms -/
#guard_msgs in #print axioms
Hardening.classical_rh_from_identification_gap_and_bound
/-- info: 'Hardening.classical_rh_from_the_arrows' does not depend on any
axioms -/
#guard_msgs in #print axioms Hardening.classical_rh_from_the_arrows
/-- info: 'Hardening.classical_rh_from_identification_and_exact_null'
does not depend on any axioms -/
#guard_msgs in #print axioms
Hardening.classical_rh_from_identification_and_exact_null
/-- info: 'Hardening.no_third_position' does not depend on any axioms -/
#guard_msgs in #print axioms Hardening.no_third_position
/-- info: 'Hardening.the_reading_is_keyed' does not depend on any axioms
-/
#guard_msgs in #print axioms Hardening.the_reading_is_keyed
/-- info: 'Hardening.the_bridge_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms Hardening.the_bridge_is_keyed
/-- info: 'Hardening.the_hardened_closure' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms Hardening.the_hardened_closure

/-! # The capstones of record, printed plainly for the boot of PhysOS$^{\intercal}$
(the pins above consume their messages) -/
#print axioms RememberedOffered.the_atomic_proof
#print axioms LeastErasureDoF.the_master_seal
#print axioms Hardening.classical_rh_from_identification_gap_and_bound
#print axioms Hardening.no_third_position
#print axioms Hardening.the_hardened_closure

/-! # PART VIII · the physico-formal carrier -/
/-!
  PART VIII · THE PHYSICO-FORMAL CARRIER.

  Part VII took the carrier as a type variable and six arrows as
  hypotheses. Part VIII replaces the variable by a
  type given by its constructors, `PhysicalCarrier`: a self-conjugate
  state at a height (the neutral sector), or a
```

```
  member of a conjugate pair at a height with the sign that tells it from
  its partner (the charged sector). Charge
  conjugation `C` and annihilation `annihilate` are defined on it by
  cases; the physical carrier is then an instance
  of the seed's own carrier abstraction (Part II,
  `PrimeFreedom.Carrier`): `C` is a fold, `annihilate` its
  registration, and every law of that namespace holds on the carrier by
  theorem.

  The map from a zero to its carrier, `map_zero_to_carrier`, is total by
  the integer's constructors and a bijection
  with the chart; it carries the fold to charge conjugation, the line to
  the neutral sector, and the registration to
  annihilation. On this map two of Part VII's six arrows are theorems: an
  off-line zero is carried to the charged
  sector (h_carrier), where annihilation does not fix it and sends it and
  its partner to one state, so one label is
  lost and kept nowhere on the carrier (h_erase). A third, h_heat, is
  Landauer's law entered as a definition,
  `landauerHeatPerEvent T = T · ln 2`, with a temperature floor: the
  arrow is then the monotonicity of a product.

  What remains is physics, and is printed as hypotheses of the capstone:
  h_realize, that the vacuum instantiates
  the carrier of every read zero; h_pop and h_rate, the population and
  rate floors of a realized charged carrier;
  h_temp, the temperature floor; h_couple, that the residual observable
  covers the carrier's contribution; the
  reading h_read; and a certified bound B. No definition below makes any
  of these true. A theorem that the universe
  realizes a state is not available to any logic: it enters as an axiom,
  which this file declares none of; as a
  hypothesis, which is what is done here; or as a definition that is the
  claim restated, which the Final Scribal
  Recommendation forbids (no hidden premise in a structure field). The
  theorems of VIII-f record, by witness, that
  the realization predicate is the input: one choice of it makes
  h_realize hold at every configuration, another
  makes it fail at the pair world.
-/
section PartVIII
namespace PhysicoFormal
open RememberedOffered

/-! ## VIII-a · the carrier, by constructors; conjugation and
annihilation, by cases -/

/-- THE PHYSICAL CARRIER. A state of the sector in which a zero is
realized is one of two kinds:
    `selfConjugate t`, a state at height t that charge conjugation fixes,
    the neutral sector; or
    `conjugatePair m t sign`, a member of the conjugate pair of magnitude
    m + 1 at height t, with `sign` the one
    label that tells it from its partner, the charged sector. A type
    given by its constructors, inhabited at every
    height; which of its states the vacuum instantiates is the premise
    h_realize of VIII-e, not a field here. -/
inductive PhysicalCarrier where
  | selfConjugate (t : Int)
  | conjugatePair (m : Nat) (t : Int) (sign : Bool)

/-- Charge conjugation: it fixes the neutral sector and flips the sign in
the charged sector. -/
def C : PhysicalCarrier → PhysicalCarrier
  | .selfConjugate t => .selfConjugate t
  | .conjugatePair m t s => .conjugatePair m t (!s)

theorem C_involutive (c : PhysicalCarrier) : C (C c) = c := by
  cases c with
  | selfConjugate t => rfl
  | conjugatePair m t s => cases s <;> rfl

/-- The neutral sector: the self-conjugate states. -/
def Neutral : PhysicalCarrier → Prop
  | .selfConjugate _ => True
  | .conjugatePair _ _ _ => False

/-- The neutral sector is exactly the fixed locus of charge conjugation.
-/
theorem neutral_iff_C_fixed (c : PhysicalCarrier) : Neutral c ↔ C c = c
:= by
  cases c with
  | selfConjugate t => exact (fun _ => rfl, fun _ => trivial)
  | conjugatePair m t s => exact (fun h => h.elim, fun h => by cases s
  <;> cases h)

/-- Fixedness under charge conjugation is decided on the carrier, by
cases. -/
theorem carrier_em (c : PhysicalCarrier) : C c = c ∨ C c ≠ c := by
  cases c with
  | selfConjugate t => exact Or.inl rfl
  | conjugatePair m t s => exact Or.inr (fun h => by cases s <;> cases h)

/-- Annihilation, the physical discard: a member of a pair goes to the
neutral state at its height; a neutral state
    stays. The sign is kept in no register of the carrier. -/
def annihilate : PhysicalCarrier → PhysicalCarrier
  | .selfConjugate t => .selfConjugate t
  | .conjugatePair _ t _ => .selfConjugate t

theorem annihilate_lands (c : PhysicalCarrier) : C (annihilate c) =
annihilate c := by
  cases c <;> rfl

theorem annihilate_fixes (c : PhysicalCarrier) (h : C c = c) : annihilate
c = c := by
  cases c with
  | selfConjugate t => rfl
  | conjugatePair m t s => cases s <;> cases h

theorem annihilate_forgets (c : PhysicalCarrier) : annihilate (C c) =
annihilate c := by
  cases c <;> rfl

/-- Annihilation fixes a state exactly when the state is neutral. -/
theorem annihilate_fixes_iff_neutral (c : PhysicalCarrier) : annihilate c
= c ↔ Neutral c := by
  cases c with
  | selfConjugate t => exact (fun _ => trivial, fun _ => rfl)
  | conjugatePair m t s => exact (fun h => (by cases h), fun h => h.elim)

/-- A conjugate pair is two states with one image under annihilation: the
sign is the bit that annihilation erases. -/
theorem pair_is_two_states_one_image (m : Nat) (t : Int) :
    PhysicalCarrier.conjugatePair m t true ≠
    PhysicalCarrier.conjugatePair m t false ∧
    annihilate (.conjugatePair m t true) = annihilate (.conjugatePair m t
    false) :=
  (fun h => (by cases h), rfl)

/-- THE PHYSICAL CARRIER IS AN INSTANCE OF THE SEED'S CARRIER ABSTRACTION
(Part II): charge conjugation is a fold. -/
def carrierFold : PrimeFreedom.Carrier.Fold PhysicalCarrier := ⟨C,
C_involutive⟩

/-- Annihilation is a registration for that fold: it lands on the neutral
sector, fixes it, and forgets the sign. -/
def carrierReg : PrimeFreedom.Carrier.Registration carrierFold :=
  ⟨annihilate, annihilate_lands, annihilate_fixes, annihilate_forgets⟩

/-- THE CLOSURE ON THE PHYSICAL CARRIER: the four laws Part II proves on
any carrier, instantiated on this one by
    theorem. Least erasure is the value; the value is lossless
    registration; the lossless member of a record is
    unique; every charged state gives two worlds of one record that
    differ in the value. -/
theorem the_closure_on_the_physical_carrier :
    type_of% (PrimeFreedom.Carrier.the_closure_on_any_carrier carrierReg
    carrier_em) :=
  PrimeFreedom.Carrier.the_closure_on_any_carrier carrierReg carrier_em

/-! ## VIII-b · the map from a zero to its carrier, total and bijective,
equivariant -/

/-- THE MAP FROM A ZERO TO ITS CARRIER. A point of the chart at offset d
and height t goes: on the line, to the
    self-conjugate state at t; at offset n + 1, to the positive member of
    the pair of magnitude n + 1 at t; at offset
    −(n + 1), to the negative member. Total, by the integer's three
    constructors. -/
def map_zero_to_carrier : Point → PhysicalCarrier
  | ⟨Int.ofNat 0, t⟩ => .selfConjugate t
  | ⟨Int.ofNat (n + 1), t⟩ => .conjugatePair n t true
  | ⟨Int.negSucc n, t⟩ => .conjugatePair n t false

/-- The inverse: a carrier state read back as a point of the chart. -/
def carrier_to_zero : PhysicalCarrier → Point
  | .selfConjugate t => ⟨Int.ofNat 0, t⟩
  | .conjugatePair n t true => ⟨Int.ofNat (n + 1), t⟩
  | .conjugatePair n t false => ⟨Int.negSucc n, t⟩

theorem map_left_inverse (p : Point) : carrier_to_zero
(map_zero_to_carrier p) = p := by
  obtain ⟨d, t⟩ := p
  cases d with
  | ofNat n => cases n <;> rfl
  | negSucc n => rfl

theorem map_right_inverse (c : PhysicalCarrier) : map_zero_to_carrier
(carrier_to_zero c) = c := by
  cases c with
  | selfConjugate t => rfl
  | conjugatePair n t s => cases s <;> rfl

/-- The map is injective: two zeros with one carrier are one zero. -/
theorem map_injective (p q : Point) (h : map_zero_to_carrier p =
map_zero_to_carrier q) : p = q := by
  rw [← map_left_inverse p, ← map_left_inverse q, h]

/-- The map is surjective: every carrier state is the carrier of a zero.
-/
theorem map_surjective (c : PhysicalCarrier) : ∃ p, map_zero_to_carrier p
= c :=
  ⟨carrier_to_zero c, map_right_inverse c⟩

/-- EQUIVARIANCE. The fold of the chart is charge conjugation on the
carrier: s ↦ 1 − s̄ is C. -/
theorem map_fold (p : Point) : map_zero_to_carrier (fold p) = C
(map_zero_to_carrier p) := by
  obtain ⟨d, t⟩ := p
  cases d with
  | ofNat n => cases n <;> rfl
  | negSucc n => rfl

/-- The line of the chart is the neutral sector of the carrier. -/
theorem map_neutral_iff_onLine (p : Point) : Neutral (map_zero_to_carrier
p) ↔ onLine p := by
```

```
  obtain (d, t) := p
  cases d with
  | ofNat n =>
    cases n with
    | zero => exact (fun _ => rfl, fun _ => trivial)
    | succ n => exact (fun h => h.elim, fun h => by cases h)
  | negSucc n => exact (fun h => h.elim, fun h => by cases h)

/-- The registration of the chart is annihilation on the carrier. -/
theorem map_reg (p : Point) : map_zero_to_carrier (reg p) = annihilate
(map_zero_to_carrier p) := by
  obtain (d, t) := p
  cases d with
  | ofNat n => cases n <;> rfl
  | negSucc n => rfl

/-- Least erasure on the chart is least erasure on the carrier,
configuration by configuration, through the map. -/
theorem le_transports (S : Config) :
    LeastErasure S ↔ ∀ p, S p → annihilate (map_zero_to_carrier p) =
    map_zero_to_carrier p := by
  constructor
  · intro h p hp
    rw [← map_reg, h p hp]
  · intro h p hp
    exact (reg_erases_nothing_iff p).mpr ((map_neutral_iff_onLine p).mp
      ((annihilate_fixes_iff_neutral _).mp (h p hp)))

/-! ## VIII-c · the mathematical violation forces the physical discard -/

/-- MATHEMATICAL VIOLATION FORCES PHYSICAL DISCARD. A zero off the line
is carried to the charged sector; there
    charge conjugation moves it, annihilation does not fix it, and
    annihilation sends it and its conjugate partner
    to one state. One label, the sign, is lost and kept in no register of
    the carrier. Theorem; no premise. This
    is h_carrier and h_erase of Part VII, discharged on the concrete
    carrier. -/
theorem mathematical_violation_forces_physical_discard (p : Point) (h : ¬
onLine p) :
    ¬ Neutral (map_zero_to_carrier p) ∧
    C (map_zero_to_carrier p) ≠ map_zero_to_carrier p ∧
    annihilate (map_zero_to_carrier p) ≠ map_zero_to_carrier p ∧
    annihilate (C (map_zero_to_carrier p)) = annihilate
    (map_zero_to_carrier p) :=
  have hn : ¬ Neutral (map_zero_to_carrier p) := fun hN => h
  ((map_neutral_iff_onLine p).mp hN)
  (hn,
   fun hC => hn ((neutral_iff_C_fixed _).mpr hC),
   fun hA => hn ((annihilate_fixes_iff_neutral _).mp hA),
   annihilate_forgets _)

/-- ON THE LINE NOTHING IS DISCARDED: the carrier is neutral and
annihilation fixes it. -/
theorem on_line_discards_nothing (p : Point) (h : onLine p) :
    Neutral (map_zero_to_carrier p) ∧ annihilate (map_zero_to_carrier p) =
    map_zero_to_carrier p :=
  have hN := (map_neutral_iff_onLine p).mpr h
  (hN, (annihilate_fixes_iff_neutral _).mpr hN)

/-! ## VIII-d · the energetic bookkeeping: labels, entropy, heat -/

/-- LANDAUER'S LAW, ENTERED AS A DEFINITION: the heat of erasing one
label at temperature T is T · ln 2 in the
    kernel's units (ln 2 scaled by 10⁶). It is a physical law, not
    derived here; the kernel's
    `heat_from_joint_conservation` carries only its shape, and no theorem
    below pretends otherwise. What the
    theorems derive is the count of labels and that the heat is positive
    exactly when the count is. -/
def landauerHeatPerEvent (T : Nat) : Nat := T * LeastErasureDoF.ln2_scaled

/-- The heat of erasing b labels at temperature T. -/
def landauerHeat (bits T : Nat) : Nat := bits * landauerHeatPerEvent T

/-- The heat is the seed's price (Part III, `price`) at the temperature T
· ln 2. -/
theorem landauerHeat_is_price (bits T : Nat) : landauerHeat bits T =
price (landauerHeatPerEvent T) bits := rfl

/-- The per-event heat is monotone in the temperature: a temperature
floor is a heat floor. This is h_heat of
    Part VII, derived from the law and the floor. -/
theorem heat_floor_from_temperature {T T0 : Nat} (h : T ≥ T0) :
    landauerHeatPerEvent T ≥ landauerHeatPerEvent T0 :=
  Nat.mul_le_mul_right _ h

/-- The per-event heat at the CMB temperature of the kernel is positive.
-/
theorem heat_at_cmb_positive : landauerHeatPerEvent
LeastErasureDoF.cmb_mK > 0 :=
  Nat.mul_pos (by decide) (by decide)

/-- A FINITE CONFIGURATION AS A SYSTEM OF THE KERNEL: one record per
height on the line and one per pair; one free
    label, the sign, per pair; the total the product. -/
def systemOf (F : FinCfg) : LeastErasureDoF.System :=
  (F.line.length + F.pairs.length, F.pairs.length,
   (F.line.length + F.pairs.length) * 2 ^ F.pairs.length, rfl)

/-- The same records with every sign gone: the system after annihilation.
-/
def systemAfter (F : FinCfg) : LeastErasureDoF.System :=
  (F.line.length + F.pairs.length, 0, (F.line.length + F.pairs.length) * 2
  ^ 0, rfl)

/-- The entropy the kernel assigns to the configuration is one ln 2 per
pair, and after annihilation it is zero:
    the entropy drop of the discard is the erased count times ln 2. -/
theorem entropy_of_discard (F : FinCfg) :
    LeastErasureDoF.sysEntropy (systemOf F) = erased F *
    LeastErasureDoF.ln2_scaled ∧
    LeastErasureDoF.sysEntropy (systemAfter F) = 0 :=
  (rfl, Nat.zero_mul _)

/-- One annihilation is one erasure of the kernel: a system with one more
label and twice the total, taken to
    this system. Records kept, one label lost, total halved. -/
def oneAnnihilation (s : LeastErasureDoF.System) (hs : s.distinct > 0) :
LeastErasureDoF.EraseProcess :=
  { before := (s.distinct, s.free + 1, 2 * s.total, by
      rw [s.inv, Nat.pow_succ, Nat.mul_comm 2 (s.distinct * 2 ^ s.free),
      Nat.mul_two, Nat.mul_two,
        Nat.left_distrib])
    after := s
    h_distinct := rfl
    h_free := rfl
    h_total := rfl
    h_gt := by
      show s.total < 2 * s.total
      have hpos : 0 < s.total := by
        rw [s.inv]; exact Nat.mul_pos hs (Nat.two_pow_pos _)
      rw [Nat.two_mul]
      exact Nat.lt_add_of_pos_left hpos }

/-- The kernel's entropy law holds of each annihilation by instance. -/
theorem annihilation_drops_one_ln2 (s : LeastErasureDoF.System) (hs :
s.distinct > 0) :
    LeastErasureDoF.sysEntropy (oneAnnihilation s hs).after +
    LeastErasureDoF.ln2_scaled =
      LeastErasureDoF.sysEntropy (oneAnnihilation s hs).before :=
  LeastErasureDoF.sysEntropy_decrease _

/-- DISCARD FORCES HEAT GENERATION. The annihilation of a finite
configuration erases one label per off-line pair,
    the seed's `erased`; at any positive temperature its Landauer heat is
    zero exactly at the value, and positive
    whenever a point stands off the line. The count is theorem; the law
    is `landauerHeatPerEvent`. -/
theorem discard_forces_heat_generation (F : FinCfg) (T : Nat) (hT : 0 <
T) :
    (landauerHeat (erased F) T = 0 ↔ Value F.pts) ∧
    (¬ Value F.pts → landauerHeat (erased F) T > 0) :=
  have hq : 0 < landauerHeatPerEvent T := Nat.mul_pos hT (by decide)
  have hiff : landauerHeat (erased F) T = 0 ↔ Value F.pts :=
  price_zero_iff_value F _ hq
  (hiff, fun hv => Nat.pos_of_ne_zero (fun h0 => hv (hiff.mp h0)))

/-! ## VIII-e · the certified bound, the physical premises, and the
physico-formal capstone -/

/-- A CERTIFIED OBSERVATIONAL BOUND: the residual observable R, the
sensitivity ε, the certificate R ≤ ε, and the
    name of the dataset that issued it. A measurement is a premise by its
    nature; this is the type it is entered as. -/
structure CertifiedObservationalBound where
  R : Nat
  ε : Nat
  certificate : R ≤ ε
  dataset : String

/-- The detection gap: the uniform floor a realized charged carrier puts
under the residual observable,
    δ = n₀ · r₀ · (T₀ · ln 2). -/
def detectionGap (n0 r0 T0 : Nat) : Nat := n0 * r0 * landauerHeatPerEvent
T0

/-- The gap is positive exactly when its three floors are. -/
theorem detectionGap_pos {n0 r0 T0 : Nat} (hn : n0 > 0) (hr : r0 > 0) (hT
: T0 > 0) : detectionGap n0 r0 T0 > 0 :=
  Nat.mul_pos (Nat.mul_pos hn hr) (Nat.mul_pos hT (by decide))

/-- THE SIGNAL FLOOR OF A REALIZED CHARGED CARRIER, from the three floors
and the coupling: population × rate × heat
    per event is at least n₀ · r₀ · (T₀ · ln 2), and the residual covers
    it. -/
theorem signal_floor {R n0 r0 T0 T : Nat} (population rate :
PhysicalCarrier → Nat) (c : PhysicalCarrier)
    (h_pop : population c ≥ n0) (h_rate : rate c ≥ r0) (h_temp : T ≥ T0)
    (h_couple : R ≥ LeastErasureDoF.residual (population c) (rate c)
    (landauerHeatPerEvent T)) :
    R ≥ detectionGap n0 r0 T0 :=
  Nat.le_trans (Nat.mul_le_mul (Nat.mul_le_mul h_pop h_rate)
  (heat_floor_from_temperature h_temp)) h_couple

/-- EVERY VIOLATION EXCEEDS THE DETECTION GAP. Two physical premises,
named: the vacuum instantiates the carrier of
    every read zero (h_realize); a realized charged carrier puts the
    floor δ under R (h_signal). Then a read zero
    off the line forces R ≥ δ. That the zero's carrier is charged is
    theorem (VIII-c), not premise. -/
theorem every_violation_exceeds_detection_gap
    {ZC : Config} {R δ : Nat} (realized : PhysicalCarrier → Prop)
    (h_realize : ∀ p, ZC p → realized (map_zero_to_carrier p))
    (h_signal  : ∀ c, realized c → ¬ Neutral c → R ≥ δ)
    (p : Point) (hp : ZC p) (hoff : ¬ onLine p) : R ≥ δ :=
```

```
  h_signal _ (h_realize p hp)
    (mathematical_violation_forces_physical_discard p hoff).1

/-- THE BRIDGE OF PART VII, DERIVED. From the two physical premises, a
failure of least erasure at the read
    configuration forces R ≥ δ: exactly h_bridge of
    `classical_rh_from_identification_gap_and_bound`. The step from
    a failed universal to a witness is classical; the cone is printed. -/
theorem bridge_from_realization
    {Zζ : Config} {R δ : Nat} (realized : PhysicalCarrier → Prop)
    (h_realize : ∀ p, Zζ p → realized (map_zero_to_carrier p))
    (h_signal  : ∀ c, realized c → ¬ Neutral c → R ≥ δ) :
    ¬ LeastErasure Zζ → R ≥ δ := by
  intro hnot
  apply Classical.byContradiction
  intro hlt
  apply hnot
  intro p hp
  apply (reg_erases_nothing_iff p).mpr
  apply Classical.byContradiction
  intro hoff
  exact hlt (every_violation_exceeds_detection_gap realized h_realize
  h_signal p hp hoff)

/-- THE PHYSICO-FORMAL CLOSURE. The capstone, every input printed, the
carrier concrete:
    h_read     the reading: the classical hypothesis is the value of the
    configuration at which the zeros are read;
    h_realize  the vacuum instantiates the carrier of every read zero
    (physics);
    h_pop      a realized charged carrier is a population of at least n₀
    (physics);
    h_rate     a realized charged carrier annihilates at a rate of at
    least r₀ during the observation (physics);
    h_temp     the temperature during the observation is at least T₀
    (physics);
    h_couple   the residual observable covers the carrier's contribution,
    population × rate × heat   (physics);
    B          a certified bound R ≤ ε from a named dataset
    (measurement);
    h_gap      ε < δ = n₀ · r₀ · (T₀ · ln 2)
    (arithmetic on the two).
    Discharged on the carrier, no longer hypotheses: that an off-line
    zero is realized as a charged state, that its
    annihilation erases one label, and that the heat per label is T · ln 2
    ≥ T₀ · ln 2.
    Conclusion: the classical hypothesis. Pointwise and constructive; the
    cone is empty. An empty cone says the
    proof uses no axiom; it discharges none of the eight inputs. -/
theorem classical_rh_from_physico_formal_closure
    {ClassicalRH : Prop} {Zζ : Config} {n0 r0 T0 T : Nat}
    (realized : PhysicalCarrier → Prop) (population rate :
    PhysicalCarrier → Nat)
    (B : CertifiedObservationalBound)
    (h_read    : ClassicalRH ↔ Value Zζ)
    (h_realize : ∀ p, Zζ p → realized (map_zero_to_carrier p))
    (h_pop     : ∀ c, realized c → ¬ Neutral c → population c ≥ n0)
    (h_rate    : ∀ c, realized c → ¬ Neutral c → rate c ≥ r0)
    (h_temp    : T ≥ T0)
    (h_couple  : ∀ c, realized c → ¬ Neutral c →
                    B.R ≥ LeastErasureDoF.residual (population c) (rate c)
                    (landauerHeatPerEvent T))
    (h_gap     : B.ε < detectionGap n0 r0 T0) :
    ClassicalRH := by
  apply h_read.mpr
  intro p hp
  match decEq p.1 0 with
  | isTrue h => exact h
  | isFalse hoff =>
    have hc : ¬ Neutral (map_zero_to_carrier p) :=
    (mathematical_violation_forces_physical_discard p hoff).1
    have hr := h_realize p hp
    have hfloor : B.R ≥ detectionGap n0 r0 T0 :=
      signal_floor population rate _ (h_pop _ hr hc) (h_rate _ hr hc)
      h_temp (h_couple _ hr hc)
    exact absurd (Nat.lt_of_le_of_lt (Nat.le_trans hfloor B.certificate)
    h_gap) (Nat.lt_irrefl _)

/-- NO THIRD POSITION, ON THE CARRIER. Given the reading, the bound and
the gap, the denial of the hypothesis and
    the two physical premises cannot all stand: whoever denies the
    hypothesis has named h_realize or h_signal as
    the premise that fails. -/
theorem no_third_position_on_the_carrier
    {ClassicalRH : Prop} {Zζ : Config} {δ : Nat} (realized :
    PhysicalCarrier → Prop)
    (B : CertifiedObservationalBound)
    (h_read : ClassicalRH ↔ Value Zζ) (h_gap : B.ε < δ) :
    ¬ (¬ ClassicalRH ∧ (∀ p, Zζ p → realized (map_zero_to_carrier p)) ∧
       (∀ c, realized c → ¬ Neutral c → B.R ≥ δ)) := by
  intro (hno, h_realize, h_signal)
  apply hno
  apply h_read.mpr
  intro p hp
  match decEq p.1 0 with
  | isTrue h => exact h
  | isFalse hoff =>
    have hRδ := every_violation_exceeds_detection_gap realized h_realize
    h_signal p hp hoff
    exact absurd (Nat.lt_of_le_of_lt (Nat.le_trans hRδ B.certificate)
    h_gap) (Nat.lt_irrefl _)

/-! ## VIII-f · what is premise, shown by witness -/


/-- THE REALIZATION IS THE PREMISE. The predicate that says which
carriers the vacuum instantiates is an input:
    under the predicate that realizes every state, h_realize holds at
    every configuration; under the predicate that
    realizes none, h_realize fails at the pair world of the off point. No
    theorem of this file chooses between them. -/
theorem realization_is_the_premise :
    (∃ realized : PhysicalCarrier → Prop, ∀ S : Config, ∀ p, S p →
    realized (map_zero_to_carrier p)) ∧
    (∃ realized : PhysicalCarrier → Prop, ¬ ∀ p, pairWorld offPoint p →
    realized (map_zero_to_carrier p)) :=
  ((fun _ => True, fun _ _ _ => trivial),
   (fun _ => False, fun h => h offPoint (Or.inl rfl)))

/-- THE CARRIER ALONE DECIDES NOTHING. The pair world of the off point is
closed, lacks least erasure, and its
    carriers are charged; with no realization premise and no bound no
    theorem above reaches the value there. -/
theorem the_carrier_alone_decides_nothing :
    Closed (pairWorld offPoint) ∧ ¬ LeastErasure (pairWorld offPoint) ∧
    ¬ Neutral (map_zero_to_carrier offPoint) :=
  (pair_world_closed offPoint,
   fun hle => off_point_is_off ((reg_erases_nothing_iff offPoint).mp (hle
   offPoint (Or.inl rfl))),
   (mathematical_violation_forces_physical_discard offPoint
   off_point_is_off).1)

/-- THE LAW IS NOT DERIVED. Landauer's heat is a definition of this file:
at temperature zero it is zero for every
    count. The theorem records that nothing in the kernel forces the
    heat, only counts it. -/
theorem landauer_is_entered_not_derived (bits : Nat) : landauerHeat bits 0
= 0 := by
  unfold landauerHeat landauerHeatPerEvent
  rw [Nat.zero_mul, Nat.mul_zero]

/-! ## VIII-h · the root on the physical side: the floor is there, and
the realization is not -/

/-- THE PHYSICAL FLOOR: that some state is realized. -/
def PhysicalFloor (realized : PhysicalCarrier → Prop) : Prop := ∃ c,
realized c

/-- THE PHYSICAL FLOOR IS SELF-GROUNDING, in the seed's own sense (Part
I, `SelfGrounding`): an act of measurement
    is a realized state, so every act, the act of denial included,
    instantiates the floor. Constructed, not posited:
    the acts, one act, and the state each act realizes are the three
    fields, and the floor follows from them. -/
def physicalFloorGrounding (realized : PhysicalCarrier → Prop) (Act :
Type) (anAct : Act)
    (stateOf : Act → PhysicalCarrier) (h : ∀ a, realized (stateOf a)) :
    SelfGrounding (PhysicalFloor realized) :=
  (Act, anAct, fun a => (stateOf a, h a))

/-- THE PHYSICAL FLOOR DECIDES NO REALIZATION. Under the predicate that
realizes exactly the apparatus state, the
    floor holds, the floor is self-grounding, and the realization of the
    off-line zero fails at the pair world. The
    floor is there and the carrier of the off-line zero is not: the
    mirror, on the physical side, of the seed's
    `the_floor_alone_decides_no_value`. The passage from "some state is
    realized" to "the carrier of every read zero
    is realized" is not an inference from the floor; it is the second
    layer, the act at the configuration. -/
theorem the_physical_floor_decides_no_realization :
    ∃ realized : PhysicalCarrier → Prop,
      PhysicalFloor realized ∧
      SelfVerifying (PhysicalFloor realized) ∧
      ¬ ∀ p, pairWorld offPoint p → realized (map_zero_to_carrier p) :=
  (fun c => c = PhysicalCarrier.selfConjugate 0,
   (_, rfl),
   fun _ => (_, rfl),
   fun h => by have e := h offPoint (Or.inl rfl); cases e)

/-- THE REALIZATION IS NOT THE FLOOR: identified on every configuration,
they would agree at the pair world, where
    the floor holds and the realization fails. The mirror of the seed's
    `floor_is_not_least_erasure`. -/
theorem realization_is_not_the_floor :
    ¬ ∀ realized : PhysicalCarrier → Prop, ∀ S : Config,
        (PhysicalFloor realized ↔ ∀ p, S p → realized
        (map_zero_to_carrier p)) := by
  intro h
  obtain (realized, hfloor, _, hfail) :=
  the_physical_floor_decides_no_realization
  exact hfail ((h realized (pairWorld offPoint)).mp hfloor)

/-- THE PHYSICAL ROUTE READS THE ROOT TWICE. The physical premises, the
certified bound and the gap deliver, at the
    read configuration, exactly what the seed says the act supplies: the
    root read twice, `RootReadTwice Zζ`, which
    is the value there by `the_root_read_twice_is_the_value`. The physics
    is the second layer, not the first. -/
theorem the_physical_route_reads_the_root_twice
    {Zζ : Config} {n0 r0 T0 T : Nat}
    (realized : PhysicalCarrier → Prop) (population rate :
    PhysicalCarrier → Nat)
    (B : CertifiedObservationalBound)
    (h_realize : ∀ p, Zζ p → realized (map_zero_to_carrier p))
    (h_pop     : ∀ c, realized c → ¬ Neutral c → population c ≥ n0)
    (h_rate    : ∀ c, realized c → ¬ Neutral c → rate c ≥ r0)
```

```
    (h_temp    : T ≥ T0)
    (h_couple  : ∀ c, realized c → ¬ Neutral c →
                   B.R ≥ LeastErasureDoF.residual (population c) (rate c)
                   (landauerHeatPerEvent T))
    (h_gap     : B.ε < detectionGap n0 r0 T0) :
    RootReadTwice Zζ :=
  (the_root_read_twice_is_the_value Zζ).mpr
    (classical_rh_from_physico_formal_closure (ClassicalRH := Value Zζ)
    realized population rate B Iff.rfl
      h_realize h_pop h_rate h_temp h_couple h_gap)

/-- THE ONE EXIT, SHAPED. Under the reading, a certified bound, the gap, and the signal floor of a realized charged
    carrier, a denial of the hypothesis asserts a read zero off the line whose carrier is realized in no measured
    register: a second register, a second world. The denier owns that assertion; nothing else is open. The step from
    the failed universal to its witness is classical; the cone is printed. -/
theorem the_one_exit_is_a_second_register
    {ClassicalRH : Prop} {Zζ : Config} {δ : Nat} (realized : PhysicalCarrier → Prop)
    (B : CertifiedObservationalBound) (h_read : ClassicalRH ↔ Value Zζ)
    (h_gap : B.ε < δ)
    (h_signal : ∀ c, realized c → ¬ Neutral c → B.R ≥ δ) :
    ¬ ClassicalRH → ∃ p, Zζ p ∧ ¬ onLine p ∧ ¬ realized
    (map_zero_to_carrier p) := by
  intro hno
  apply Classical.byContradiction
  intro hnone
  apply hno
  apply h_read.mpr
  intro p hp
  match decEq p.1 0 with
  | isTrue h => exact h
  | isFalse hoff =>
    exact (hnone (p, hp, hoff, fun hr =>
      absurd (Nat.lt_of_le_of_lt (Nat.le_trans
        (h_signal _ hr (mathematical_violation_forces_physical_discard p hoff).1) B.certificate) h_gap)
        (Nat.lt_irrefl _))).elim

/-! ## VIII-g · the physico-formal closure, bound -/

/-- THE PHYSICO-FORMAL CLOSURE, BOUND. The hardened closure of Part VII; the closure on the physical carrier; the
    map's two inverses and its equivariance; the line as the neutral sector; the registration as annihilation; the
    violation forcing the discard; the discard forcing the heat; every violation exceeding the gap; the bridge
    derived; the capstone; no third position on the carrier; the two witnesses of what is premise; and the four
    laws of the root on the physical side: the floor decides no realization, the realization is not the floor, the
    physical route reads the root twice, and the one exit is a second register. -/
theorem the_physico_formal_closure :
    (type_of% @Hardening.the_hardened_closure) ∧
    (type_of% @the_closure_on_the_physical_carrier) ∧
    (type_of% @map_left_inverse) ∧
    (type_of% @map_right_inverse) ∧
    (type_of% @map_fold) ∧
    (type_of% @map_neutral_iff_onLine) ∧
    (type_of% @map_reg) ∧
    (type_of% @le_transports) ∧
    (type_of% @mathematical_violation_forces_physical_discard) ∧
    (type_of% @discard_forces_heat_generation) ∧
    (type_of% @every_violation_exceeds_detection_gap) ∧
    (type_of% @bridge_from_realization) ∧
    (type_of% @classical_rh_from_physico_formal_closure) ∧
    (type_of% @no_third_position_on_the_carrier) ∧
    (type_of% @realization_is_the_premise) ∧
    (type_of% @landauer_is_entered_not_derived) ∧
    (type_of% @the_physical_floor_decides_no_realization) ∧
    (type_of% @realization_is_not_the_floor) ∧
    (type_of% @the_physical_route_reads_the_root_twice) ∧
    (type_of% @the_one_exit_is_a_second_register) :=
  (@Hardening.the_hardened_closure, @the_closure_on_the_physical_carrier,
   @map_left_inverse, @map_right_inverse, @map_fold,
   @map_neutral_iff_onLine, @map_reg, @le_transports,
   @mathematical_violation_forces_physical_discard,
   @discard_forces_heat_generation,
   @every_violation_exceeds_detection_gap, @bridge_from_realization,
   @classical_rh_from_physico_formal_closure,
   @no_third_position_on_the_carrier,
   @realization_is_the_premise, @landauer_is_entered_not_derived,
   @the_physical_floor_decides_no_realization,
   @realization_is_not_the_floor,
   @the_physical_route_reads_the_root_twice,
   @the_one_exit_is_a_second_register)

end PhysicoFormal
end PartVIII

/-! ## Part VIII · the cones, pinned -/

/-- info: 'PhysicoFormal.the_closure_on_the_physical_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.the_closure_on_the_physical_carrier
/-- info: 'PhysicoFormal.map_left_inverse' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.map_left_inverse
/-- info: 'PhysicoFormal.map_right_inverse' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.map_right_inverse
/-- info: 'PhysicoFormal.map_fold' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.map_fold
/-- info: 'PhysicoFormal.map_neutral_iff_onLine' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.map_neutral_iff_onLine
/-- info: 'PhysicoFormal.map_reg' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.map_reg
/-- info: 'PhysicoFormal.le_transports' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.le_transports
/-- info: 'PhysicoFormal.mathematical_violation_forces_physical_discard' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.mathematical_violation_forces_physical_discard
/-- info: 'PhysicoFormal.oneAnnihilation' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.oneAnnihilation
/-- info: 'PhysicoFormal.discard_forces_heat_generation' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.discard_forces_heat_generation
/-- info: 'PhysicoFormal.every_violation_exceeds_detection_gap' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.every_violation_exceeds_detection_gap
/-- info: 'PhysicoFormal.bridge_from_realization' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PhysicoFormal.bridge_from_realization
/-- info: 'PhysicoFormal.classical_rh_from_physico_formal_closure' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.classical_rh_from_physico_formal_closure
/-- info: 'PhysicoFormal.no_third_position_on_the_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.no_third_position_on_the_carrier
/-- info: 'PhysicoFormal.realization_is_the_premise' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.realization_is_the_premise
/-- info: 'PhysicoFormal.the_physical_floor_decides_no_realization' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.the_physical_floor_decides_no_realization
/-- info: 'PhysicoFormal.realization_is_not_the_floor' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.realization_is_not_the_floor
/-- info: 'PhysicoFormal.the_physical_route_reads_the_root_twice' does not depend on any axioms -/
#guard_msgs in #print axioms PhysicoFormal.the_physical_route_reads_the_root_twice
/-- info: 'PhysicoFormal.the_one_exit_is_a_second_register' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PhysicoFormal.the_one_exit_is_a_second_register
/-- info: 'PhysicoFormal.the_physico_formal_closure' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PhysicoFormal.the_physico_formal_closure

#print axioms PhysicoFormal.mathematical_violation_forces_physical_discard
#print axioms PhysicoFormal.discard_forces_heat_generation
#print axioms PhysicoFormal.classical_rh_from_physico_formal_closure
#print axioms PhysicoFormal.no_third_position_on_the_carrier
#print axioms PhysicoFormal.the_physical_floor_decides_no_realization
#print axioms PhysicoFormal.the_physical_route_reads_the_root_twice
#print axioms PhysicoFormal.the_physico_formal_closure
```